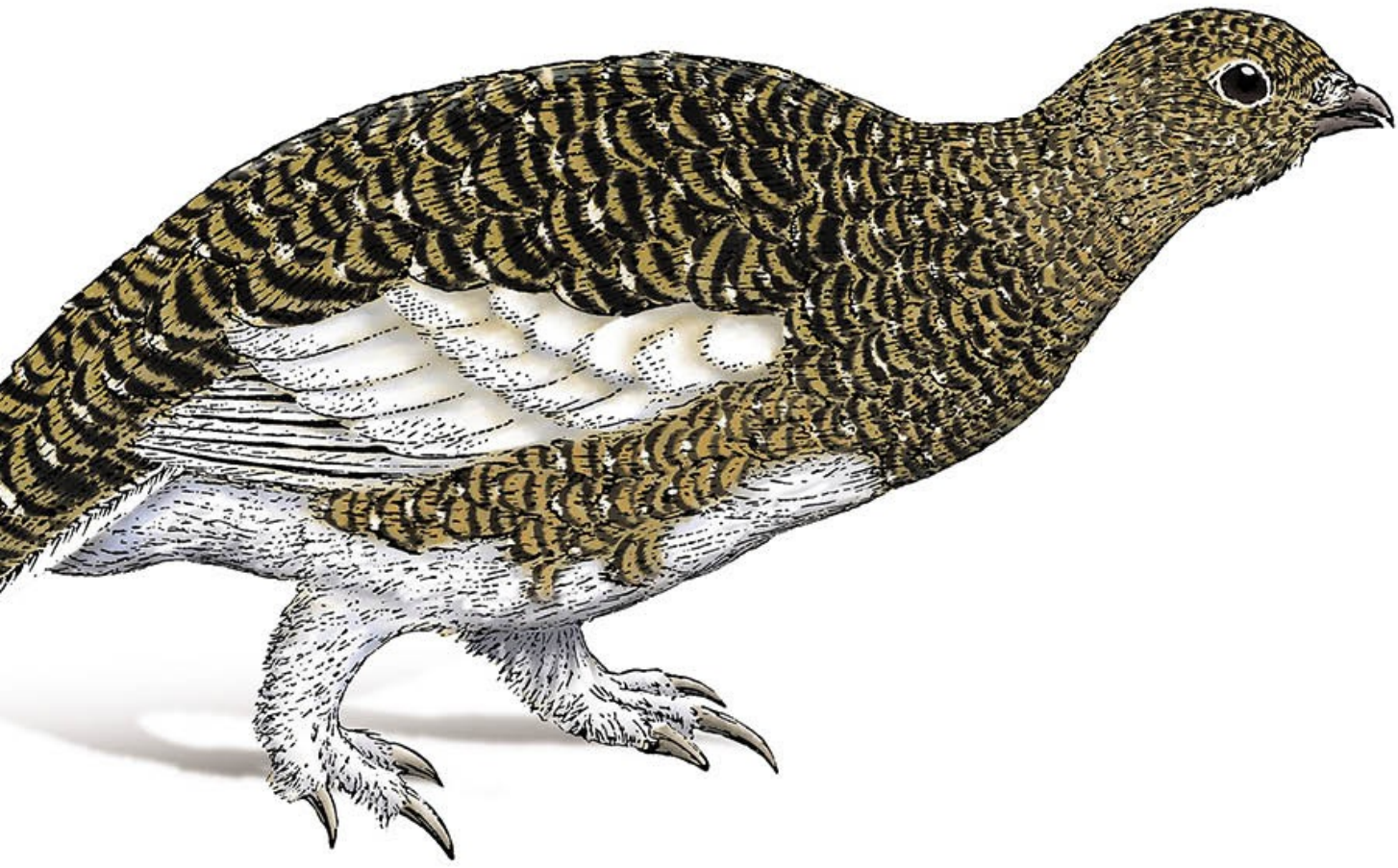
2.ª Edición

O'REILLY® ANAYA MULTIMEDIA

Ciencia de datos desde cero

Principios básicos con Python



Joel Grus

Ciencia de datos desde cero

Principios básicos con Python

2.ª Edición

Joel Grus

ANAYA MULTIMEDIA

Agradecimientos

En primer lugar, quiero agradecer a Mike Loukides por aceptar mi propuesta para este libro (y por insistir en que lo reduzca a un tamaño razonable). Habría sido muy fácil para él decir: "¿Quién es esta persona que no para de enviarme correos electrónicos con capítulos de muestra, y qué hago para deshacerme de él?". Pero me siento muy agradecido de que no lo dijera. También quisiera agradecer a mis editoras, Michele Cronin y Marie Beaugureau, por guiarme a lo largo del proceso de la publicación de libros y conseguir el libro en un estado mucho mejor de lo que jamás yo hubiera podido lograr por mí mismo.

No podría haber escrito este libro si nunca hubiera aprendido ciencia de datos o *data science*, y probablemente no habría aprendido ciencia de datos si no hubiera sido por la influencia de Dave Hsu, Igor Tatarinov, John Rauser y el resto de la banda Farecast (hace ya tanto tiempo que, en ese momento, la ciencia de datos ¡ni siquiera se conocía con ese nombre!). Hay que reconocerles también el gran mérito que tienen los buenos chicos de Coursera y DataTau.

Doy también las gracias a mis lectores y revisores beta. Jay Fundling encontró una gran cantidad de errores y resaltó muchas explicaciones no claras, y el libro es mucho mejor (y mucho más correcto) gracias a él. Debashis Shosh es un héroe por comprobar la sensatez de todas mis estadísticas. Andrew Musselman sugirió bajar el tono "la gente que prefiere R a Python son unos inmorales" del libro, lo que creo que finalmente acabó siendo un consejo bastante bueno. Trey Causey, Ryan Matthew Balfanz, Loris Mularoni, Núria Pujol, Rob Jefferson, Mary Pat Campbell, Zach Geary, Denise Mauldin, Jimmy O'Donnell y Wendy Grus proporcionaron también unos comentarios de gran valor. Gracias a todos los que leyeron la primera edición y ayudaron a que este libro fuera mejor. Los errores que puedan quedar son, por supuesto, responsabilidad mía.

Le debo mucho a la comunidad de Twitter #datascience por exponerme a un montón de conceptos nuevos, presentarme a mucha gente estupenda y

hacerme sentir tan manta, que se me ocurrió escribir un libro para compensarlo. Agradezco especialmente a Trey Causey (de nuevo) por recordarme (sin darse cuenta) incluir un capítulo sobre álgebra lineal y a Sean J. Taylor por señalar (sin darse cuenta) un par de enormes lapsus en el capítulo “Trabajando con datos”.

Por encima de todo, le debo una tonelada de agradecimientos a Ganga y Madeline. La única cosa más difícil que escribir un libro es vivir con alguien que esté escribiendo un libro, y no podría haberlo logrado sin su apoyo.

Sobre el autor

Joel Grus es ingeniero investigador en el Allen Institute for AI. Anteriormente trabajó como ingeniero de software en Google y como científico de datos en varias *startups*. Vive en Seattle, donde habitualmente asiste a *podcasts* sobre ciencia de datos. Tiene un blog que actualiza ocasionalmente en `joelgrus.com`, pero se pasa el día tuiteando en `@joelgrus`.

Índice

- Agradecimientos
- Sobre el autor

Prefacio a la segunda edición

- Convenciones empleadas en este libro
- Uso del código de ejemplo
- Sobre la imagen de cubierta

Prefacio a la primera edición

- Ciencia de datos o data science
- Partir de cero

1. Introducción

- El ascenso de los datos
- ¿Qué es la ciencia de datos o data science?
- Hipótesis motivadora: DataSciencester
- Localizar los conectores clave
- Científicos de datos que podría conocer
- Salarios y experiencia
- Cuentas de pago
- Temas de interés
- Sigamos adelante

2. Un curso acelerado de Python

- El zen de Python
- Conseguir Python

Entornos virtuales
Formato con espacios en blanco
Módulos
Funciones
Cadenas
Excepciones
Listas
Tuplas
Diccionarios
    defaultdict
Contadores
Conjuntos
Flujo de control
Verdadero o falso
Ordenar
Comprensiones de listas
Pruebas automatizadas y assert
Programación orientada a objetos
Iterables y generadores
Aleatoriedad
Expresiones regulares
Programación funcional
Empaquetado y desempaquetado de argumentos
args y kwargs
Anotaciones de tipos
    Cómo escribir anotaciones de tipos
Bienvenido a DataSciencester
Para saber más

3. Visualizar datos

matplotlib
Gráficos de barras
Gráficos de líneas
Gráficos de dispersión

Para saber más

4. Álgebra lineal

Vectores
Matrices
Para saber más

5. Estadística

Describir un solo conjunto de datos
Tendencias centrales
Dispersión
Correlación
La paradoja de Simpson
Otras advertencias sobre la correlación
Correlación y causación
Para saber más

6. Probabilidad

Dependencia e independencia
Probabilidad condicional
Teorema de Bayes
Variables aleatorias
Distribuciones continuas
La distribución normal
El teorema central del límite
Para saber más

7. Hipótesis e inferencia

Comprobación de hipótesis estadísticas
Ejemplo: Lanzar una moneda
Valores p
Intervalos de confianza
p-hacking o dragado de datos

Ejemplo: Realizar una prueba A/B
Inferencia bayesiana
Para saber más

8. Descenso de gradiente

La idea tras el descenso de gradiente
Estimar el gradiente
Utilizar el gradiente
Elegir el tamaño de paso adecuado
Utilizar descenso de gradiente para ajustar modelos
Descenso de gradiente en minilotes y estocástico
Para saber más

9. Obtener datos

stdin y stdout
Leer archivos
Conocimientos básicos de los archivos de texto
Archivos delimitados
Raspado web
HTML y su análisis
Ejemplo: Controlar el congreso
Utilizar API
JSON y XML
Utilizar una API no autenticada
Encontrar API
Ejemplo: Utilizar las API de Twitter
Obtener credenciales
Para saber más

10. Trabajar con datos

Explorar los datos
Explorar datos unidimensionales
Dos dimensiones

Muchas dimensiones
Utilizar NamedTuples
Clases de datos
Limpiar y preparar datos
Manipular datos
Redimensionar
Un inciso: tqdm
Reducción de dimensionalidad
Para saber más

11. Machine learning (aprendizaje automático)

Modelos
¿Qué es el machine learning?
Sobreajuste y subajuste
Exactitud
El término medio entre sesgo y varianza
Extracción y selección de características
Para saber más

12. k vecinos más cercanos

El modelo
Ejemplo: el conjunto de datos iris
La maldición de la dimensionalidad
Para saber más

13. Naive Bayes

Un filtro de spam realmente tonto
Un filtro de spam más sofisticado
Implementación
A probar nuestro modelo
Utilizar nuestro modelo
Para saber más

14. Regresión lineal simple

El modelo
Utilizar descenso de gradiente
Estimación por máxima verosimilitud
Para saber más

15. Regresión múltiple

El modelo
Otros supuestos del modelo de mínimos cuadrados
Ajustar el modelo
Interpretar el modelo
Bondad de ajuste
Digresión: el bootstrap
Errores estándares de coeficientes de regresión
Regularización
Para saber más

16. Regresión logística

El problema
La función logística
Aplicar el modelo
Bondad de ajuste
Máquinas de vectores de soporte
Para saber más

17. Árboles de decisión

¿Qué es un árbol de decisión?
Entropía
La entropía de una partición
Crear un árbol de decisión
Ahora, a combinarlo todo
Bosques aleatorios
Para saber más

18. Redes neuronales

Perceptrones
Redes neuronales prealimentadas
Retropropagación
Ejemplo: Fizz Buzz
Para saber más

19. Deep learning (aprendizaje profundo)

El tensor
La capa de abstracción
La capa lineal
Redes neuronales como una secuencia de capas
Pérdida y optimización
Ejemplo: XOR revisada
Otras funciones de activación
Ejemplo: FizzBuzz revisado
Funciones softmax y entropía cruzada
Dropout
Ejemplo: MNIST
Guardar y cargar modelos
Para saber más

20. Agrupamiento (clustering)

La idea
El modelo
Ejemplo: Encuentros
Eligiendo k
Ejemplo: agrupando colores
Agrupamiento jerárquico de abajo a arriba
Para saber más

21. Procesamiento del lenguaje natural

Nubes de palabras
Modelos de lenguaje n-Gram

Gramáticas
Un inciso: muestreo de Gibbs
Modelos de temas
Vectores de palabras
Redes neuronales recurrentes
Ejemplo: utilizar una RNN a nivel de carácter
Para saber más

22. Análisis de redes

Centralidad de intermediación
Centralidad de vector propio
Multiplicación de matrices
Centralidad
Grafos dirigidos y PageRank
Para saber más

23. Sistemas recomendadores

Método manual
Recomendar lo que es popular
Filtrado colaborativo basado en usuarios
Filtrado colaborativo basado en artículos
Factorización de matrices
Para saber más

24. Bases de datos y SQL

CREATE TABLE e INSERT
UPDATE
DELETE
SELECT
GROUP BY
ORDER BY
JOIN
Subconsultas

Índices
Optimización de consultas
NoSQL
Para saber más

25. MapReduce

Ejemplo: Recuento de palabras
¿Por qué MapReduce?
MapReduce, más general
Ejemplo: Analizar actualizaciones de estado
Ejemplo: Multiplicación de matrices
Un inciso: Combinadores
Para saber más

26. La ética de los datos

¿Qué es la ética de los datos?
No, ahora en serio, ¿qué es la ética de datos?
¿Debo preocuparme de la ética de los datos?
Crear productos de datos de mala calidad
Compromiso entre precisión e imparcialidad
Colaboración
Capacidad de interpretación
Recomendaciones
Datos sesgados
Protección de datos
En resumen
Para saber más

27. Sigamos haciendo ciencia de datos

IPython
Matemáticas
No desde cero
NumPy

pandas
scikit-learn
Visualización
R
Deep learning (aprendizaje profundo)
Encontrar datos
Haga ciencia de datos
Hacker News
Camiones de bomberos
Camisetas
Tuits en un globo terráqueo
¿Y usted?

Créditos

Prefacio a la segunda edición

Me siento excepcionalmente orgulloso de la primera edición de este libro. Ha resultado ser en buena parte el libro que yo quería que fuera. Pero varios años de desarrollos en ciencia de datos, de progreso en el ecosistema Python y de crecimiento personal como desarrollador y educador han cambiado lo que creo que debe ser un primer libro sobre ciencia de datos.

En la vida no hay vuelta atrás, pero en la escritura de libros sí hay segundas ediciones.

De acuerdo con esto, he reescrito todo el código y los ejemplos utilizando Python 3.6 (y muchas de sus funciones más recientes, como las anotaciones de tipos). En el libro hago continuamente énfasis en escribir código limpio. He reemplazado algunos de los ejemplos de la primera edición por otros más realistas, utilizando conjuntos de datos "reales". He añadido nuevo material en temas como *deep learning*, estadísticas y procesamiento del lenguaje natural, que se corresponden con cosas con las que es más probable que los científicos de datos de hoy en día trabajen (también he eliminado otras informaciones que parecían ser menos relevantes). Y he repasado el libro de una forma muy concienzuda, arreglando errores, reescribiendo explicaciones que eran menos claras de lo que podrían ser y actualizando algunos de los chistes.

La primera edición fue un gran libro, y esta edición es aún mejor. ¡Disfrútela!

Joel Grus
Seattle, WA
2019

Convenciones empleadas en este libro

En este libro se utilizan las siguientes convenciones tipográficas:

- *Cursiva*: Es un tipo que se usa para diferenciar términos anglosajones o de uso poco común. También se usa para destacar algún concepto.
- **Negrita**: Le ayudará a localizar rápidamente elementos como las combinaciones de teclas.
- Fuente especial: Nombres de botones y opciones de programas. Por ejemplo, Aceptar para hacer referencia a un botón con ese título.
- `Monoespacial`: Utilizado para el código y dentro de los párrafos para hacer referencia a elementos como nombres de variables o funciones, bases de datos, tipos de datos, variables de entorno, declaraciones y palabras clave.

También encontrará a lo largo del libro recuadros con elementos destacados sobre el texto normal, comunicándole de manera breve y rápida algún concepto relacionado con lo que está leyendo, un truco o advirtiéndole de algo.

Aunque el termino *"data science"* es de uso generalizado y reconocido en todo el mundo, hemos decidido traducir este termino por "ciencia de datos" que es como se conoce a este área de conocimiento en castellano. Hemos preferido utilizar el termino en castellano por respeto a la riqueza de nuestra lengua y a los usuarios de los países de habla hispana.

Uso del código de ejemplo

Se puede descargar material adicional (ejemplos de código, ejercicios, etc.) de la página web de Anaya Multimedia (`http://www.anayamultimedia.es`). Vaya al botón Selecciona Complemento de la ficha del libro, donde podrá descargar el contenido para poder utilizarlo directamente. También puede descargar el material de la página web original del libro: `https://github.com/joelgrus/data-science-from-scratch`.

Este libro ha sido creado para ayudarle en su trabajo. En general, puede

utilizar el código de ejemplo incluido en sus programas y en su documentación. No es necesario contactar con la editorial para solicitar permiso a menos que esté reproduciendo una gran cantidad del código. Por ejemplo, escribir un programa que utilice varios fragmentos de código tomados de este libro no requiere permiso. Sin embargo, vender o distribuir un CD-ROM de ejemplos de los libros de O'Reilly sí lo requiere. Responder una pregunta citando este libro y empleando textualmente código de ejemplo incluido en él no requiere permiso. Pero incorporar una importante cantidad de código de ejemplo de este libro en la documentación de su producto sí lo requeriría.

Sobre la imagen de cubierta

El animal de la portada de este libro es una perdiz nival o lagópodo alpino (*Lagopus muta*). Este robusto miembro de la familia de los faisánidos, del tamaño de un pollo, vive en la tundra del hemisferio norte, en las regiones árticas y subárticas de Eurasia y Norteamérica. Se alimenta de lo que encuentra en el suelo, recorriendo las praderas con sus patas bien emplumadas, comiendo brotes de abedul y sauce, así como semillas, flores, hojas y bayas. Los polluelos de perdiz nival también comen insectos.

Los lagópodos alpinos son muy conocidos por los sorprendentes cambios anuales que sufre su enigmático camuflaje, habiendo evolucionado para mudar sus plumas blancas y pardas varias veces en el transcurso de un año y así adaptarse mejor a los cambiantes colores estacionales de su entorno. En invierno tienen plumas blancas; en primavera y otoño, cuando el manto nevado se mezcla con la dehesa, su plumaje mezcla los colores blanco y pardo y, en verano, sus plumas, pardas por completo, coinciden con la variada coloración de la tundra. Con este camuflaje, las hembras pueden incubar sus huevos, que dejan en nidos sobre el suelo, siendo casi invisibles.

Las perdices nivales macho adultas tienen también una especie de cresta roja sobre sus ojos. Durante la temporada de cría la utilizan para el cortejo, así como en los combates contra otros machos (existen estudios que demuestran una correlación entre el tamaño de la cresta y el nivel de

testosterona de los machos).

La población de perdiz de las nieves está actualmente en declive, aunque en su hábitat natural siguen siendo comunes (pero difíciles de observar). Tienen muchos depredadores, entre otros el zorro ártico, el gerifalte, la gaviota común y la gaviota salteadora o escua. Además, con el tiempo, el cambio climático puede afectar negativamente a sus cambios de color estacionales.

Muchos de los animales de las portadas de O'Reilly están en peligro de extinción; todos ellos son importantes para el mundo.

La imagen de la portada procede de la obra *Cassell's Book of Birds* (1875), de Thomas Rymer Jones.

Prefacio a la primera edición

Ciencia de datos o data science

El trabajo de científico de datos ha sido denominado “el empleo más sexy del siglo XXI”,[1] presuntamente por alguien que no ha visitado nunca un parque de bomberos. Sin embargo, la ciencia de datos es un campo en pleno auge y crecimiento, y no hace falta ser muy perspicaz para encontrar analistas prediciendo sin descanso que, en los próximos 10 años, necesitaremos miles de millones de científicos de datos más de los que tenemos ahora.

Pero ¿qué es la ciencia de datos? Después de todo, no podemos crear científicos de datos si no sabemos cuál es su trabajo. Según un diagrama de Venn,[2] que es en cierto modo famoso en este sector, la ciencia de datos reside en la intersección entre:

- Habilidades informáticas a nivel de *hacker*.
- Conocimiento de matemáticas y estadística.
- Experiencia relevante.

Aunque mi intención inicial era escribir un libro que hablara sobre estos tres puntos, rápidamente me di cuenta de que un tratamiento en profundidad de la expresión “experiencia relevante” requeriría cientos de miles de páginas. En ese momento decidí centrarme en los dos primeros. Mi objetivo es ayudar a los lectores a desarrollar las habilidades informáticas a nivel de *hacker* que necesitarán para empezar a trabajar en la ciencia de datos. Pero también es permitirles sentirse cómodos con las matemáticas y la estadística, que son el núcleo de la ciencia de datos.

Quizá esta sea una aspiración demasiado elevada para un libro. La mejor forma de aprender las habilidades informáticas de un *hacker* es hackeando cosas. Leyendo este libro, los lectores podrán llegar a comprender bastante

bien la forma en la que yo hackeo cosas, que no tiene por qué ser necesariamente la suya. También conocerán bastante bien algunas de las herramientas que utilizo, que no han de ser obligadamente las mejores para ellos. Y entenderán bien el modo en que yo abordo los problemas de datos, que tampoco tiene por qué ser el mejor modo para ellos. La intención (y la esperanza) es que mis ejemplos les inspiren a probar las cosas a su manera. Todo el código y los datos del libro están disponibles en GitHub[3] para que puedan ponerse manos a la obra.

De forma similar, la mejor manera de aprender matemáticas es haciendo matemáticas. Este no es rotundamente un libro de mates, y en su mayor parte no estaremos "haciendo matemáticas". Sin embargo, no se puede hacer ciencia de datos de verdad sin ciertos conocimientos de probabilidad, estadística y álgebra lineal. Esto significa que, donde corresponda, profundizaremos en ecuaciones matemáticas, intuición matemática, axiomas matemáticos y versiones caricaturizadas de grandes ideas matemáticas. Espero que los lectores no teman sumergirse conmigo.

A lo largo de todo el libro también espero dar a entender que jugar con datos es divertido porque, bueno, ¡jugar con datos realmente lo es! (especialmente si lo comparamos con algunas alternativas, como hacer la declaración de la renta o trabajar en una mina).

Partir de cero

Hay muchísimas librerías de ciencia de datos, *frameworks*, módulos y kits de herramientas que implementan de forma eficaz los algoritmos y las técnicas de ciencia de datos más conocidas (así como las menos habituales). Si alguno de mis lectores llega a ser científico de datos, acabará estando íntimamente familiarizado con NumPy, scikit-learn, pandas y todas las demás librerías existentes. Son fabulosas para hacer ciencia de datos, pero también suponen una buena forma de empezar a hacer ciencia de datos sin realmente comprender lo que es.

En este libro nos acercaremos a la ciencia de datos desde el principio de los principios. Esto significa que crearemos herramientas e implementaremos

algoritmos a mano para poder comprenderlos mejor. Pensé mucho en crear implementaciones y ejemplos que fueran claros y legibles y estuvieran bien comentados. En la mayoría de los casos, las herramientas que construiremos serán esclarecedoras, pero poco prácticas. Funcionarán bien en pequeños conjuntos de datos, pero no lo harán en otros "a gran escala". Durante todo el libro iré señalando las librerías que se podrían utilizar para aplicar estas técnicas sobre conjuntos de datos más grandes. Pero aquí no las utilizaremos.

Existe una sana discusión en torno al mejor lenguaje que se puede utilizar para aprender ciencia de datos. Mucha gente cree que es el lenguaje de programación estadístico R (de esas personas decimos que están equivocadas). Unas pocas personas sugieren Java o Scala. Sin embargo, en mi opinión, Python es la elección obvia.

Python tiene varias características que le hacen ser ideal para aprender (y hacer) ciencia de datos:

- Es gratuito.
- Es relativamente sencillo para crear código (y en particular, de comprender).
- Tiene muchas librerías asociadas a la ciencia de datos que son de gran utilidad.

Dudo si decir que Python es mi lenguaje de programación favorito. Hay otros lenguajes que encuentro más agradables, mejor diseñados o simplemente más divertidos de utilizar. Pero, aun así, cada vez que inicio un nuevo proyecto de ciencia de datos, termino utilizando Python. Cada vez que necesito crear rápidamente un prototipo de algo que simplemente funcione, termino utilizando Python. Y cada vez que quiero demostrar conceptos de ciencia de datos de una forma clara y sencilla de comprender, acabo por utilizar Python. De ahí que este libro utilice Python.

El objetivo de este libro no es enseñar Python (aunque es casi seguro que leyéndolo se aprende un poco). Llevaré a los lectores a lo largo de un curso acelerado de un capítulo de duración, que resalta las características más importantes para nuestros propósitos, pero, si no se sabe nada sobre programar en Python (o sobre programar en general), entonces quizá

convenga complementar este libro con algún tutorial de tipo “Python para principiantes”.

El resto de nuestra introducción a la ciencia de datos seguirá este mismo enfoque, es decir, entrar en detalle donde ello parezca ser crucial o esclarecedor, pero en otras ocasiones dejarle al lector los detalles para que investigue por sí mismo (o lo busque en la Wikipedia).

Durante años he formado a un buen número de científicos de datos. Aunque no todos han evolucionado para convertirse en revolucionarias estrellas del rock ninja de los datos, les he dejado siendo mejores científicos de datos de lo que eran cuando les conocí. Y yo he llegado a creer que cualquiera que tenga una cierta cantidad de aptitud matemática y una determinada habilidad para programar tiene los fundamentos necesarios para hacer ciencia de datos. Todo lo que se necesita es una mente curiosa, voluntad para trabajar duro y este libro. De ahí este libro.

---

[1] https://hbr.org/2012/10/data-scientist-the-sexiest-job-of-the-21st-century.

[2] http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram.

[3] https://github.com/joelgrus/data-science-from-scratch.

1 Introducción

"¡Datos, datos, datos!", gritó con impaciencia. "No puedo hacer ladrillos sin arcilla".
—Arthur Conan Doyle

El ascenso de los datos

Vivimos en un mundo que se está ahogando en datos. Los sitios web controlan cada clic de cada usuario. Los teléfonos inteligentes crean registros con la ubicación y velocidad de sus dueños cada segundo de cada día. Cada vez más autodidactas llevan puestos podómetros superpotentes que registran continuamente su ritmo cardíaco, sus hábitos de movimiento, su dieta y sus patrones de sueño. Los coches inteligentes recogen las costumbres en la conducción, las casas inteligentes recopilan hábitos de vida y los comerciantes inteligentes recolectan hábitos de compra. Internet representa un grafo de conocimiento gigante que contiene (entre otras cosas) una enorme enciclopedia con referencias cruzadas, bases de datos específicas de dominio sobre películas, música, resultados deportivos, máquinas de *pinball*, memes y cócteles, y tal cantidad de estadísticas gubernamentales (¡algunas de ellas casi ciertas!) de tantos gobiernos que no le caben a uno en la cabeza.

Enterradas en estos datos están las respuestas a incontables preguntas que nadie nunca pensó en responder. En este libro aprenderemos a encontrarlas.

¿Qué es la ciencia de datos o data science?

Hay un chiste que dice que un científico de datos es alguien que conoce más estadísticas que un científico informático y más ciencia informática que un estadista (yo no diría que el chiste es bueno). De hecho, algunos

científicos de datos son (a todos los efectos prácticos) estadistas, mientras que a otros apenas se les puede distinguir de un ingeniero de software. Algunos son expertos en *machine learning*, mientras que otros no llegaron a aprender ni tan siquiera por dónde se salía de la guardería. Algunos son doctores con impresionantes registros de publicaciones, mientras que otros nunca han leído un documento académico (aunque les dé vergüenza admitirlo). En resumen, da igual cómo se defina la ciencia de datos; siempre habrá profesionales para los que la definición es total y absolutamente errónea. Sin embargo, no dejaremos que eso nos impida seguir intentándolo. Diremos que un científico de datos es alguien que extrae conocimientos a partir de datos desordenados. El mundo de hoy en día está repleto de personas que intentan convertir datos en conocimiento.

Por ejemplo, en el sitio web de citas OkCupid se les pide a sus miembros que respondan a cientos de preguntas para poder encontrar las parejas más adecuadas para ellos. Pero también analizan estos resultados para dar con preguntas aparentemente inocuas que se le puedan formular a alguien para averiguar la probabilidad de que esa persona vaya a dormir contigo en la primera cita.[1]

Facebook suele preguntar a sus usuarios por su ciudad natal y su ubicación actual, aparentemente para que les resulte más fácil a sus amigos encontrarles y conectar con ellos. Pero también analiza estas ubicaciones para identificar patrones de migración globales[2] y para averiguar dónde viven las comunidades de aficionados de distintos equipos de fútbol.[3]

La cadena americana de grandes almacenes Target controla las compras e interacciones de sus usuarios, tanto en línea como en sus tiendas físicas. Igualmente realiza modelos predictivos[4] con los datos para averiguar cuáles de sus clientas están embarazadas y así venderles con más facilidad productos relacionados con el bebé.

En 2012, la campaña de Obama empleó a muchísimos científicos que investigaron con datos y experimentaron con ellos para identificar a los votantes que necesitaban más atención, eligiendo los métodos perfectos de recaudación de fondos dirigidos a determinados donantes y poniendo todos los esfuerzos de obtención del voto allí donde era más probable que fueran útiles. En 2016, la campaña Trump probó una asombrosa variedad de

anuncios *online*[5] y analizó los datos para averiguar lo que funcionaba y lo que no. Lo último antes de resultar cansino: algunos científicos de datos utilizan también de vez en cuando sus habilidades para cosas buenas, por ejemplo, para que el gobierno sea más eficaz[6] o para ayudar a los sin techo.[7] Pero sin duda tampoco resultan perjudicados si lo que les gusta es buscar la mejor manera de conseguir que la gente haga clic en anuncios.

Hipótesis motivadora: DataSciencester

¡Felicidades! Le acaban de contratar para dirigir el departamento de Ciencia de Datos de DataSciencester, la red social para científicos de datos.

> **Nota:** Cuando escribí la primera edición de este libro, pensé que "una red social para científicos de datos" era una invención que podía resultar absurda, pero a la vez divertida. Desde entonces, se han creado redes sociales para científicos de datos de verdad, y sus creadores les han sacado a capitalistas de riesgo mucho más dinero del que yo obtuve con mi libro. Probablemente esto suponga una valiosa lección sobre inventos absurdos de ciencia de datos o la publicación de libros.

A pesar de estar destinada a científicos de datos, DataSciencester nunca ha invertido realmente en crear sus propios métodos de ciencia de datos (para ser justo, nunca ha invertido realmente en crear siquiera su producto). Esto será lo que hagamos aquí. A lo largo del libro, aprenderemos conceptos de ciencia de datos resolviendo problemas con los que uno se puede encontrar en el trabajo. Algunas veces veremos datos suministrados específicamente por los usuarios, otras veces examinaremos datos generados por sus interacciones con sitios web, y en otras ocasiones incluso trataremos datos de experimentos que nosotros mismos diseñaremos.

Como DataSciencester sufre terriblemente el síndrome NIH (*Not invented here*, no inventado aquí), crearemos nuestras propias herramientas desde cero. Al final, el lector terminará comprendiendo bastante bien los fundamentos de la ciencia de datos y podrá aplicar sus habilidades en una

compañía con una premisa menos inestable o en cualquier otro problema que le interese.

Bienvenidos a bordo y ¡buena suerte! (se pueden llevar vaqueros los viernes y el baño está al fondo a la derecha).

Localizar los conectores clave

Es el primer día de trabajo en DataSciencester, y el vicepresidente de Redes tiene muchas preguntas sobre los usuarios. Hasta ahora no tenía nadie a quien preguntar, así que está muy emocionado de tener alguien nuevo en el equipo.

En particular, le interesa identificar quiénes son los "conectores clave" de todos los científicos de datos. Para ello proporciona un volcado de la red completa de DataSciencester (en la vida real, la gente no suele pasar los datos que uno necesita; el capítulo 9 está dedicado a obtener datos).

¿Qué aspecto tiene este volcado de datos? Consiste en una lista de usuarios, cada uno representado por un `dict` que contiene su `id` (que es un número) y su `name` (que, en una de esas fabulosas conjunciones planetarias, concuerda con su `id`):

```
users = [
   { "id": 0, "name": "Hero" },
   { "id": 1, "name": "Dunn" },
   { "id": 2, "name": "Sue" },
   { "id": 3, "name": "Chi" },
   { "id": 4, "name": "Thor" },
   { "id": 5, "name": "Clive" },
   { "id": 6, "name": "Hicks" },
   { "id": 7, "name": "Devin" },
   { "id": 8, "name": "Kate" },
   { "id": 9, "name": "Klein" }
]
```

También ofrece los datos de "amistad" (*friendship*), representados como una lista de pares de identificadores:

```
friendship_pairs =      [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3), (3, 4),
                         (4, 5), (5, 6), (5, 7), (6, 8), (7, 8), (8, 9)]
```

Por ejemplo, la tupla (0, 1) indica que los científicos de datos con `id` 0 (Hero) e `id` 1 (Dunn) son amigos. La red aparece representada en la figura 1.1.

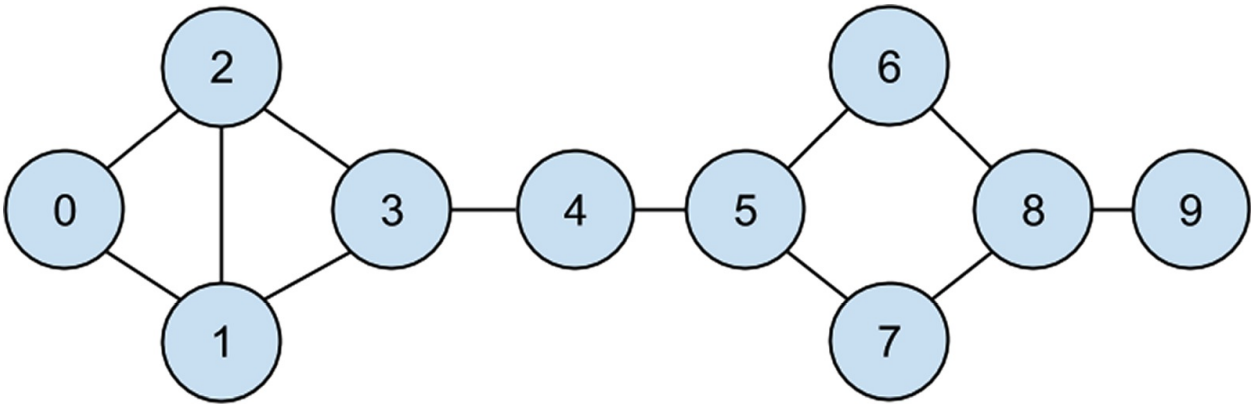


Figura 1.1. La red de DataSciencester.

Representar las amistades como una lista de pares no es la forma más sencilla de trabajar con ellas. Para encontrar todas las amistades por usuario, hay que pasar repetidamente por cada par buscando pares que contengan 1. Si hubiera muchos pares, el proceso tardaría mucho en realizarse.

En lugar de ello, vamos a crear un `dict` en el que las claves sean `id` de usuario y los valores sean listas de `id` de amigos (consultar cosas en un `dict` es muy rápido).

> **Nota:** No conviene obsesionarse demasiado con los detalles del código ahora mismo. En el capítulo 2 haremos un curso acelerado de Python. Por ahora, basta con hacerse una idea general de lo que estamos haciendo.

Aún tendremos que consultar cada par para crear el `dict`, pero solamente hay que hacerlo una vez y, después, las consultas no costarán nada:

```
# Inicializar el dict con una lista vacía para cada id de usuario:
friendships = {user["id"]: [] for user in users}
# Y pasar por todos los pares de amistad para llenarlo:
```

```
for i, j in friendship_pairs:
   friendships[i].append(j)     # Añadir j como un amigo del usuario i
   friendships[j].append(i)     # Añadir i como un amigo del usuario j
```

Ahora que ya tenemos las amistades en un `dict`, podemos formular fácilmente preguntas sobre nuestro grafo, como por ejemplo: "¿Cuál es el número medio de conexiones?".

Primero, hallamos el número total de conexiones sumando las longitudes de todas las listas `friends`:

```
def number_of_friends(user):
   """How many friends does _user_ have?"""
   user_id = user["id"]
   friend_ids = friendships[user_id]
   return len(friend_ids)
total_connections = sum(number_of_friends(user)
                  for user in users)                # 24
```

Y, después, simplemente dividimos por el número de usuarios:

```
num_users = len(users)                              # longitud de la lista de
                                                    usuarios
avg_connections = total_connections /               # 24 / 10 == 2,4
num_users
```

También es sencillo encontrar las personas más conectadas (las que tienen la mayor cantidad de amigos).

Como no hay muchos usuarios, simplemente podemos ordenarlos de "la mayor cantidad de amigos" a "la menor cantidad de amigos":

```
# Crea una lista (user_id, number_of_friends).
num_friends_by_id = [(user["id"], number_of_friends(user))
                for user in users]
num_friends_by_id.sort(                                    # Ordena la lista
     key=lambda id_and_friends: id_and_friends[1],         # por num_friends
     reverse=True)                                         # del mayor al menor
# Cada par es (user_id, num_friends):
# [(1, 3), (2, 3), (3, 3), (5, 3), (8, 3),
# (0, 2), (4, 2), (6, 2), (7, 2), (9, 1)]
```

Una manera de pensar en lo que hemos hecho es como en una forma de identificar a las personas que son de alguna manera centrales para la red. En realidad, lo que acabamos de calcular es la métrica de la centralidad de grado de la red (véase la figura 1.2).

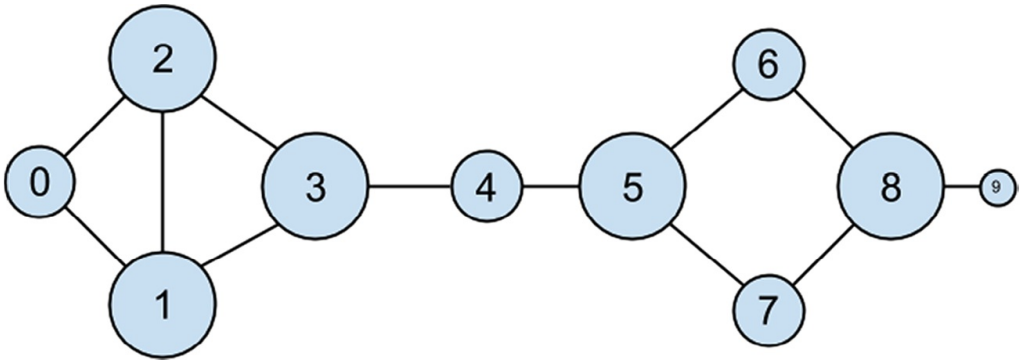


Figura 1.2. La red de DataSciencester dimensionada por grado.

Esto tiene la virtud de ser bastante fácil de calcular, pero no siempre da los resultados que se desean o esperan. Por ejemplo, en la red de DataSciencester Thor (`id` 4) solo tiene dos conexiones, mientras que Dunn (`id` 1) tiene tres. Ya cuando miramos a la red, parece intuitivo que Thor debería estar ubicado en una posición más central. En el capítulo 22 investigaremos las redes con más detalle y veremos nociones más complejas de centralidad, que pueden corresponderse más o menos con nuestra intuición.

Científicos de datos que podría conocer

Mientras aún está rellenando el papeleo de su nueva contratación, la vicepresidenta de Fraternización pasa por su despacho. Quiere fomentar más conexiones entre sus miembros y le pide que diseñe un sugeridor “Científicos de datos que podría conocer”.

Lo primero que se le ocurre es sugerir que los usuarios podrían conocer a los amigos de sus amigos. Así que escribe un poco de código para pasar varias veces por sus amigos y recoger los amigos de los amigos:

```
def foaf_ids_bad(user):
```

```
    """foaf is short for "friend of a friend" """
    return [foaf_id
         for friend_id in friendships[user["id"]]
         for foaf_id in friendships[friend_id]]
```

Cuando aplicamos esto sobre `users[0]` (Hero), produce lo siguiente:

```
[0, 2, 3, 0, 1, 3]
```

Incluye el usuario 0 dos veces, ya que Hero es de hecho amigo de sus dos amigos. Incluye los usuarios 1 y 2, aunque ambos ya son amigos de Hero. Y también incluye el usuario 3 dos veces, ya que se puede llegar hasta Chi a través de dos amigos distintos:

```
print(friendships[0]) # [1, 2]
print(friendships[1]) # [0, 2, 3]
print(friendships[2]) # [0, 1, 3]
```

Saber que las personas son amigos de amigos de diversas maneras parece ser información interesante, de modo que quizá en su lugar podríamos producir un contador de amigos mutuos. Y deberíamos probablemente excluir gente ya conocida por el usuario:

```
from collections import Counter                  # no cargado inicialmente
def friends_of_friends(user):
   user_id = user[“id”]
   return Counter(
      foaf_id
      for friend_id in friendships[user_id]      # Para cada uno de mis amigos,
      for foaf_id in friendships[friend_id]      # encuentra sus amigos
      if foaf_id != user_id                      # que no son yo
      and foaf_id not in                         # y no son mis amigos
      friendships[user_id]
   )
print(friends_of_friends(users[3]))              # Contador({0: 2, 5: 1})
```

Esto le dice correctamente a Chi (`id` 3) que tiene dos amigos mutuos con Hero (`id` 0), pero solo uno con Clive (`id` 5).

Uno, como científico de datos, sabe que también se puede disfrutar conociendo amigos con intereses comunes (este es un buen ejemplo del

aspecto “experiencia relevante” de la ciencia de datos). Tras preguntar por ahí, conseguimos estos datos, como una lista de pares (`user_id`, `interest`):

```
interests = [
   (0, "Hadoop"), (0, "Big Data"), (0, "HBase"), (0, "Java"),
   (0, "Spark"), (0, "Storm"), (0, "Cassandra"),
   (1, "NoSQL"), (1, "MongoDB"), (1, "Cassandra"), (1, "HBase"),
   (1, "Postgres"), (2, "Python"), (2, "scikit-learn"), (2, "scipy"),
   (2, "numpy"), (2, "statsmodels"), (2, "pandas"), (3, "R"), (3, "Python"),
   (3, "statistics"), (3, "regression"), (3, "probability"),
   (4, "machine learning"), (4, "regression"), (4, "decision trees"),
   (4, "libsvm"), (5, "Python"), (5, "R"), (5, "Java"), (5, "C++"),
   (5, "Haskell"), (5, "programming languages"), (6, "statistics"),
   (6, "probability"), (6, "mathematics"), (6, "theory"),
   (7, "machine learning"), (7, "scikit-learn"), (7, "Mahout"),
   (7, "neural networks"), (8, "neural networks"), (8, "deep learning"),
   (8, "Big Data"), (8, "artificial intelligence"), (9, "Hadoop"),
   (9, "Java"), (9, "MapReduce"), (9, "Big Data")
]
```

Por ejemplo, Hero (`id` 0) no tiene amigos comunes con Klein (`id` 9), pero ambos comparten intereses en Java y Big Data.

Es fácil crear una función que encuentre usuarios con un determinado interés:

```
def data_scientists_who_like(target_interest):
   """Find the ids of all users who like the target interest."""
   return [user_id
      for user_id, user_interest in interests
      if user_interest == target_interest]
```

Esto funciona, pero tiene que examinar la lista completa de aficiones en cada búsqueda. Si tenemos muchos usuarios e intereses (o si simplemente queremos hacer muchas búsquedas), es probablemente mejor que nos dediquemos a crear un índice de intereses a usuarios:

```
from collections import defaultdict
# Las claves son intereses, los valores son listas de user_ids con ese interés
user_ids_by_interest = defaultdict(list)
for user_id, interest in interests:
```

```
    user_ids_by_interest[interest].append(user_id)
```

Y otro de usuarios a intereses:

```
# Las claves son user_ids, los valores son listas de intereses para ese
user_id.
interests_by_user_id = defaultdict(list)
for user_id, interest in interests:
    interests_by_user_id[user_id].append(interest)
```

Ahora es fácil averiguar quién tiene el mayor número de intereses en común con un determinado usuario:

- Pasamos varias veces por los intereses del usuario.
- Para cada interés, volvemos a pasar en repetidas ocasiones por los demás usuarios que tienen ese mismo interés.
- Contamos las veces que vemos cada uno de los usuarios.

En código:

```
def most_common_interests_with(user):
    return Counter(
        interested_user_id
        for interest in interests_by_user_id[user["id"]]
        for interested_user_id in user_ids_by_interest[interest]
        if interested_user_id != user["id"]
)
```

Después, podríamos utilizar esto para crear una función “Científicos de datos que podría conocer” más completa basándonos en una combinación de amigos mutuos e intereses comunes. Exploraremos estos tipos de aplicación en el capítulo 23.

Salarios y experiencia

Justo cuando se iba a comer, el vicepresidente de Relaciones Públicas le

pregunta si le puede suministrar datos curiosos sobre lo que ganan los científicos de datos. Los datos de sueldos son, por supuesto, confidenciales, pero se las arregla para conseguir un conjunto de datos anónimo que contiene el salario (`salary`) de cada usuario (en dólares) y su antigüedad en el puesto (`tenure`) como científico de datos (en años):

```
salaries_and_tenures = [(83000, 8.7), (88000, 8.1),
           (48000, 0.7), (76000, 6),
           (69000, 6.5), (76000, 7.5),
           (60000, 2.5), (83000, 10),
           (48000, 1.9), (63000, 4.2)]
```

El primer paso natural es trazar los datos en un gráfico (cosa que veremos cómo hacer en el capítulo 3). La figura 1.3 muestra los resultados.

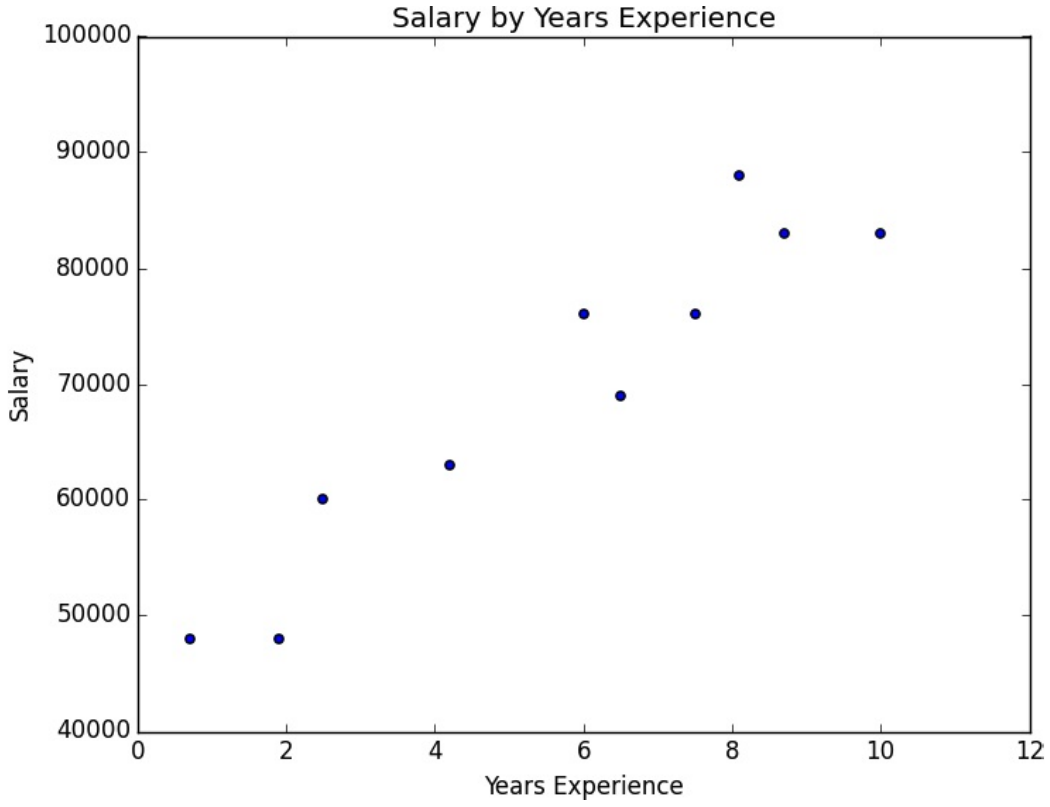


Figura 1.3. Salario por años de experiencia.

Parece claro que la gente con más experiencia tiende a ganar más. ¿Cómo se puede convertir esto en un dato curioso? Lo primero que se nos ocurre es

mirar el salario medio por antigüedad:

```
# Las claves son años, los valores son listas de los salarios por antigüedad.
salary_by_tenure = defaultdict(list)

for salary, tenure in salaries_and_tenures:
   salary_by_tenure[tenure].append(salary)

# Las claves son años, cada valor es el salario medio para dicha antigüedad.
average_salary_by_tenure = {
   tenure: sum(salaries) / len(salaries)
   for tenure, salaries in salary_by_tenure.items()
}
```

Resulta que esto no es especialmente útil, ya que ninguno de los usuarios tiene la misma antigüedad en el puesto de trabajo, lo que significa que simplemente estamos informando de los salarios individuales de los usuarios:

```
{0.7: 48000.0,
   1.9: 48000.0,
   2.5: 60000.0,
   4.2: 63000.0,
   6: 76000.0,
   6.5: 69000.0,
   7.5: 76000.0,
   8.1: 88000.0,
   8.7: 83000.0,
   10: 83000.0}
```

Podría ser más útil poner los años de antigüedad en un *bucket*:

```
def tenure_bucket(tenure):
   if tenure < 2:
      return "less than two"
   elif tenure < 5:
      return "between two and five"
   else:
      return "more than five"
```

Entonces podemos agrupar los salarios correspondientes a cada *bucket*:

```
# Las claves son buckets de años de antigüedad, los valores son listas de
salarios para bucket
salary_by_tenure_bucket = defaultdict(list)
for salary, tenure in salaries_and_tenures:
   bucket = tenure_bucket(tenure)
   salary_by_tenure_bucket[bucket].append(salary)
```

Y, por último, calcular el salario medio para cada grupo:

```
# Las claves son buckets de años de antigüedad, los valores son el salario
medio para bucket
average_salary_by_bucket = {
   tenure_bucket: sum(salaries) / len(salaries)
   for tenure_bucket, salaries in salary_by_tenure_bucket.items()
}
```

Lo que es más interesante:

```
{‘between two and five’: 61500.0,
   ‘less than two’: 48000.0,
   ‘more than five’: 79166.66666666667}
```

Y ya tenemos nuestra proclama: “Los científicos de datos con más de cinco años de experiencia ganan un 65 % más que los científicos de datos con poca experiencia o ninguna”.

Pero hemos elegido los *buckets* de una forma bastante aleatoria. Lo que realmente haríamos es hacer alguna declaración sobre el efecto que tiene en el salario (en promedio) tener un año adicional de experiencia. Además de conseguir un dato curioso más eficiente, esto nos permite hacer predicciones sobre salarios que no conocemos. Exploraremos esta idea en el capítulo 14.

Cuentas de pago

Cuando vuelve a su puesto de trabajo, la vicepresidenta de Finanzas le está esperando. Quiere entender mejor qué usuarios pagan por las cuentas y cuáles no (ella conoce sus nombres, pero esa información no es especialmente procesable).

Se da cuenta de que parece haber una correspondencia entre los años de experiencia y las cuentas de pago:

```
0.7 paid
1.9 unpaid
2.5 paid
4.2 unpaid
6.0 unpaid
6.5 unpaid
7.5 unpaid
8.1 unpaid
8.7 paid
10.0 paid
```

Los usuarios con muy pocos y muchos años de experiencia tienden a pagar; los usuarios con cantidades de experiencia medias no lo hacen. Según esto, si quería crear un modelo (aunque sin duda no son datos suficientes en los que basarlo), podría intentar predecir “de pago” para usuarios con muy pocos y muchos años de experiencia y “no de pago” para usuarios con cantidades de experiencia medias:

```
def predict_paid_or_unpaid(years_experience):
   if years_experience < 3.0:
      return "paid"
   elif years_experience < 8.5:
      return "unpaid"
   else:
      return "paid"
```

Por supuesto, esto lo hemos calculado a ojo.

Con más datos (y más matemáticas), podríamos crear un modelo que predijera la probabilidad de que un usuario pagara, basándonos en sus años de experiencia. Investigaremos este tipo de problema en el capítulo 16.

Temas de interés

A medida que se va acercando el final de su primer día, la vicepresidenta

de Estrategia de Contenidos le pide datos sobre los temas que más interesan a los usuarios, de forma que pueda planificar adecuadamente el calendario de su blog. Ya disponemos de los datos sin procesar del proyecto del sugeridor de amigos:

```
interests = [
   (0, "Hadoop"), (0, "Big Data"), (0, "HBase"), (0, "Java"),
   (0, "Spark"), (0, "Storm"), (0, "Cassandra"),
   (1, "NoSQL"), (1, "MongoDB"), (1, "Cassandra"), (1, "HBase"),
   (1, "Postgres"), (2, "Python"), (2, "scikit-learn"), (2, "scipy"),
   (2, "numpy"), (2, "statsmodels"), (2, "pandas"), (3, "R"), (3, "Python"),
   (3, "statistics"), (3, "regression"), (3, "probability"),
   (4, "machine learning"), (4, "regression"), (4, "decision trees"),
   (4, "libsvm"), (5, "Python"), (5, "R"), (5, "Java"), (5, "C++"),
   (5, "Haskell"), (5, "programming languages"), (6, "statistics"),
   (6, "probability"), (6, "mathematics"), (6, "theory"),
   (7, "machine learning"), (7, "scikit-learn"), (7, "Mahout"),
   (7, "neural networks"), (8, "neural networks"), (8, "deep learning"),
   (8, "Big Data"), (8, "artificial intelligence"), (9, "Hadoop"),
   (9, "Java"), (9, "MapReduce"), (9, "Big Data")
]
```

Una manera sencilla (aunque no especialmente apasionante) de encontrar los intereses más populares es contando las palabras:

1. Ponemos en minúsculas todos los *hobbies* (ya que habrá usuarios que los pongan en mayúscula y otros en minúscula).
2. Los dividimos en palabras.
3. Contamos los resultados.

En código:

```
words_and_counts = Counter(word
                  for user, interest in interests
                  for word in interest.lower().split())
```

Así es posible hacer fácilmente un listado con las palabras que aparecen más de una vez:

```
for word, count in words_and_counts.most_common():
```

```
    if count > 1:
        print(word, count)
```

Lo que da los resultados esperados (a menos que se suponga que “scikit-learn” ha quedado dividido en dos palabras, en cuyo caso no los da).

```
learning 3
java 3
python 3
big 3
data 3
hbase 2
regression 2
cassandra 2
statistics 2
probability 2
hadoop 2
networks 2
machine 2
neural 2
scikit-learn 2
r 2
```

En el capítulo 21 veremos maneras más sofisticadas de extraer temas de datos.

Sigamos adelante

¡Ha sido un día bastante fructuoso! Cansado, sale del edificio sigilosamente, antes de que alguien pueda pedirle algo más. Descanse bien esta noche, porque mañana tendrá su sesión de orientación al empleado (sí, ha tenido un completo día de trabajo sin tan siquiera pasar por orientación al empleado; háblelo con RR. HH.).

---

[1] https://theblog.okcupid.com/the-most-important-questions-on-okcupid-32e80bad0854.

[2] https://www.facebook.com/notes/10158928002728415/.

[3] https://www.facebook.com/notes/10158927994943415/.

[4] https://www.nytimes.com/2012/02/19/magazine/shopping-habits.html.

[5] https://www.wired.com/2016/11/facebook-won-trump-election-not-just-fake-news/.

[6] https://www.marketplace.org/2014/08/22/tech/beyond-ad-clicks-using-big-data-social-good.

[7] https://dssg.uchicago.edu/2014/08/20/tracking-the-paths-of-homelessness/.

2 Un curso acelerado de Python

La gente sigue loca por Python tras veinticinco años, cosa que me resulta difícil de creer.

—Michael Palin

Todos los empleados nuevos de DataSciencester tienen que pasar obligadamente por orientación al empleado, cuya parte más interesante es un curso acelerado de Python.

No se trata de un tutorial extenso, sino que está destinado a destacar las partes del lenguaje que serán más importantes para nosotros (algunas de las cuales no suelen ser el objetivo de los tutoriales de Python habituales). Si nunca había utilizado Python antes, probablemente quiera complementar esto con algún tipo de tutorial para principiante.

El zen de Python

Python tiene una descripción un poco zen de sus principios de diseño,[1] que se pueden encontrar dentro del propio intérprete de Python escribiendo `import this` (importar esto). Uno de los más discutidos es:

There should be one—and preferably only one—obvious way to do it.

(Solo debería haber una, y preferiblemente solo una, forma obvia de hacerlo).

El código escrito de acuerdo con esta forma "obvia" (que no tiene por qué serlo en absoluto para un principiante) se describe a menudo como "*pythonic*" (emplearemos la traducción "pitónico" en castellano, aunque suene un poco raro). Aunque este libro no trata de Python, de vez en cuando

contrastaremos formas pitónicas y no pitónicas de realizar las mismas cosas, y en general tenderemos más a emplear las soluciones pitónicas a nuestros problemas.

Otros principios aluden a lo estético:

Beautiful is better than ugly. Explicit is better than implicit. Simple is better than complex.

> *(Lo bello es mejor que lo feo. Lo explícito es mejor que lo implícito. Lo simple es mejor que lo complejo).*

Y representan ideales por los que lucharemos en nuestro código.

Conseguir Python

> **Nota:** Como las instrucciones de instalación de las cosas pueden cambiar, mientras que los libros impresos no, se pueden encontrar instrucciones actualizadas para instalar Python en el repositorio GitHub del libro.[2]
> Conviene revisar las del sitio web si las incluidas aquí no funcionan.

Se puede descargar Python desde Python.org.[3] Pero, si todavía no se dispone de él, es más recomendable instalar la distribución Anaconda,[4] que ya incluye la mayoría de las librerías necesarias para hacer ciencia de datos.

Cuando escribí la primera versión de este volumen, Python 2.7 seguía siendo el preferido por la mayoría de los científicos de datos. Por eso la primera edición del libro estaba basada en esta versión.

Pero, en los últimos años, casi todo el mundo ha migrado a Python 3. Las últimas versiones de Python tienen muchas funciones que permiten escribir código limpio con mayor facilidad, y nos aprovecharemos de otras que solo están disponibles en la versión 3.6 de Python o posterior, lo que significa que habría que conseguir esta versión más reciente de Python (además, muchas librerías útiles ya no soportan Python 2.7; otra razón más para cambiar).

Entornos virtuales

Ya desde el próximo capítulo utilizaremos la librería matplotlib para generar gráficas y diagramas o tablas. Esta librería no forma parte de Python; hay que instalarla por separado. Todos los proyectos de ciencia de datos que realicemos requerirán alguna combinación de librerías externas, a veces con versiones específicas que difieren de las empleadas en otros proyectos. Si tuviéramos una única instalación de Python, estas librerías entrarían en conflicto y provocarían todo tipo de problemas.

La solución estándar es utilizar entornos virtuales, es decir, entornos aislados de Python que mantienen sus propias versiones de librerías del lenguaje (y, dependiendo del modo en que se configure el entorno, del lenguaje en sí mismo).

Recomiendo instalar la distribución de Python denominada Anaconda, de modo que en esta sección explicaré cómo funcionan los entornos de Anaconda. También se puede utilizar el módulo venv integrado[5] o instalar virtualenv,[6] en cuyo caso habría que seguir sus instrucciones.

Para crear un entorno virtual (Anaconda) basta con hacer lo siguiente:

```
# crear un entorno Python 3.6 llamado "dsfs"
conda create -n dsfs python=3.6
```

Siguiendo los mensajes, logramos un entorno virtual llamado “dsfs”, con estas instrucciones:

```
#
# Para activar este entorno utilice:
# > source activate dsfs
#
# Para desactivar un entorno activo utilice:
# > source deactivate
#
```

Como se indica, el entorno se activa utilizando:

```
source activate dsfs
```

En ese punto, la línea de comandos debería cambiar para indicar el entorno activo. En mi MacBook aparece ahora lo siguiente en la línea de comandos:

```
(dsfs) ip-10-0-0-198:~ joelg$
```

Siempre que este entorno esté activo, las librerías se instalarán únicamente en el entorno dsfs. Cuando termine este libro y cree sus propios proyectos, debería crear sus propios entornos para ellos.

Ahora que tenemos el entorno, podemos instalar IPython,[7] que es un *shell* o intérprete de Python completo:

```
python -m pip install ipython
```

> **Nota:** Anaconda incluye su propio gestor de paquetes, `conda`, pero se puede utilizar tranquilamente el gestor de paquetes estándar de Python, `pip`, que es lo que haremos.

El resto de este libro supondrá que se ha creado y activado dicho entorno virtual de Python 3.6 (aunque le puede llamar como le parezca), y los últimos capítulos podrían hacer referencia a las librerías cuya instalación indiqué en anteriores capítulos.

Por una cuestión de disciplina, sería conveniente trabajar siempre en un entorno virtual y no utilizar nunca la instalación "básica" de Python.

Formato con espacios en blanco

Muchos lenguajes utilizan llaves para delimitar los bloques de código. Python emplea la sangría:

```
# El signo de la almohadilla marca el comienzo de un comentario. Python
# ignora los comentarios, pero son útiles para cualquiera que lea el código.
for i in [1, 2, 3, 4, 5]:
   print(i)                          # primera línea del bloque "for i"
   for j in [1, 2, 3, 4, 5]:
      print(j)                       # primera línea del bloque "for j"
      print(i + j)                   # última línea del bloque "for j"
```

```
  print(i)                          # última línea del bloque "for i"
print("done looping")
```

Así, el código de Python resulta fácilmente legible, pero también significa que hay que tener mucho cuidado con el formato.

> **Advertencia:** Los programadores suelen debatir sobre si utilizar tabuladores o espacios para la sangría. En muchos lenguajes eso no importa, pero Python considera los tabuladores y los espacios como distintos tipos de sangría y el código no se ejecutará si se mezclan los dos. Al escribir en Python siempre hay que utilizar espacios, nunca tabuladores (si se escribe código en un editor es posible configurarlo de forma que la tecla **Tab** inserte espacios).

Los espacios en blanco se ignoran dentro de paréntesis y corchetes, lo que puede resultar útil para cálculos largos:

```
long_winded_computation       (1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 + 11 + 12
=                             +
                              13 + 14 + 15 + 16 + 17 + 18 + 19 + 20)
```

Y para que el código resulte más fácil de leer:

```
list_of_lists = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
easier_to_read_list_of_lists =       [[1, 2, 3],
                                      [4, 5, 6],
                                      [7, 8, 9]]
```

También se puede utilizar una barra invertida para indicar que una sentencia continúa en la siguiente línea, aunque pocas veces se hace:

```
two_plus_three = 2 + \
                 3
```

Una consecuencia del formato con espacios en blanco es que puede ser difícil copiar y pegar código en el intérprete de Python. Por ejemplo, al intentar pegar este código:

```
for i in [1, 2, 3, 4, 5]:
   # observe la línea en blanco
   print(i)
```

En el *shell* normal de Python, aparecería este error:

```
IndentationError: expected an indented block
```

Porque el intérprete piensa que la línea vacía señala el final del bloque del bucle `for`.

IPython tiene una función mágica llamada `%paste` que pega correctamente lo que haya en el portapapeles, con espacios en blanco y todo. Solamente esto ya es una muy buena razón para utilizar IPython.

Módulos

Ciertas funciones de Python no se cargan por defecto. Entre ellas, hay funciones que están incluidas como parte del lenguaje y funciones externas que cada usuario puede descargar por su cuenta. Para poder utilizar estas funciones es necesario importar los módulos que las contienen.

Una forma de hacer esto es simplemente importando el propio módulo:

```
import re
my_regex = re.compile("[0-9]+", re.I)
```

Aquí, `re` es el módulo que contiene funciones y constantes para trabajar con expresiones regulares. Tras este tipo de `import` hay que poner delante de esas funciones el prefijo `re.` para poder acceder a ellas.

Si ya había un `re` distinto en el código que se está utilizando, se puede emplear otro nombre:

```
import re as regex
my_regex = regex.compile("[0-9]+", regex.I)
```

También se podría hacer esto si el módulo tiene un nombre poco manejable o si se va a escribir muy a menudo. Por ejemplo, un convenio estándar cuando se visualizan datos con matplotlib es:

```
import matplotlib.pyplot as plt
plt.plot(...)
```

Si se necesitan algunos valores específicos de un módulo, se pueden importar de manera explícita y utilizarlos sin reservas:

```
from collections import defaultdict, Counter
lookup = defaultdict(int)
my_counter = Counter()
```

Siendo malas personas, podríamos importar el contenido completo de un módulo en nuestro espacio de nombres, lo que podría sobrescribir involuntariamente variables que ya estaban definidas:

```
match = 10
from re import *      # oh, oh, re tiene una función que se llama igual
print(match)          # "<function match at 0x10281e6a8>"
```

Pero, como en realidad no somos malas personas, nunca haremos esto.

Funciones

Una función es una regla para tomar cero o más entradas y devolver una salida correspondiente. En Python, las funciones se definen normalmente utilizando `def`:

```
def double(x):
    """
    This is where you put an optional docstring that explains what the
    function does. For example, this function multiplies its input by 2.
    """
    return x * 2
```

Las funciones de Python son de primera clase, lo que significa que podemos asignarlas a variables y pasarlas a funciones como si se tratara de cualesquiera otros argumentos:

```
def apply_to_one(f):
```

```
  """Calls the function f with 1 as its argument"""
  return f(1)
my_double = double              # se refiere a la función anteriormente
                                definida
x =                             # es igual a 2
apply_to_one(my_double)
```

También es fácil crear funciones anónimas cortas, denominadas lambdas:

```
y = apply_to_one(lambda x: x + 4)     # es igual a 5
```

Se pueden asignar lambdas a variables, aunque la mayoría de la gente dirá que en su lugar solo hay que utilizar `def`:

```
another_double = lambda x: 2 * x      # no haga esto
def another_double(x):
  """Do this instead"""
  return 2 * x
```

A los parámetros de función también se les pueden asignar argumentos predeterminados, que solamente tienen que especificarse cuando se desea un valor distinto al predeterminado:

```
def my_print(message = "my default message"):
  print(message)
my_print("hello")     # imprime ‘hello’
my_print()            # imprime ‘my default message’
```

Algunas veces es útil especificar argumentos por el nombre:

```
def full_name(first = "What’s-his-name", last = "Something"):
  return first + " " + last
full_name("Joel", "Grus")          # "Joel Grus"
full_name("Joel")                  # "Joel Something"
full_name(last="Grus")             # "What’s-his-name Grus"
```

Vamos a crear muchas muchas funciones.

Cadenas

Las cadenas (o *strings*) pueden estar delimitadas por comillas simples o dobles (pero las comillas tienen que ir en pares):

```
single_quoted_string = ‘data science’
double_quoted_string = "data science"
```

Python utiliza las barras invertidas para codificar caracteres especiales. Por ejemplo:

```
tab_string = "\t"      # representa el carácter del tabulador
len(tab_string)        # es 1
```

Si queremos barras invertidas como tales (las que se utilizan en nombres de directorio de Windows o en expresiones regulares), se pueden crear cadenas en bruto (*raw strings*) utilizando `r""`:

```
not_tab_string = r"\t"      # representa los caracteres ‘\’ y ‘t’
len(not_tab_string)         # es 2
```

Se pueden crear cadenas multilínea utilizando comillas triples:

```
multi_line_string = """This is the first line,
and this is the second line
and this is the third line."""
```

Una función nueva en Pyhton es la *f-string*, que ofrece una sencilla manera de sustituir valores por cadenas. Por ejemplo, si nos dieran el nombre y el apellido por separado:

```
first_name = "Joel"
last_name = "Grus"
```

Querríamos combinarlos como un nombre completo. Hay distintas formas de construir una cadena `full_name`:

```
full_name1 = first_name + " " + last_name                 # suma de cadenas
full_name2 = "{0} {1}".format(first_name, last_name)      # string.format
```

Pero el método f-string es mucho más manejable:

```
full_name3 = f”{first_name} {last_name}”
```

Y lo preferiremos a lo largo del libro.

Excepciones

Cuando algo va mal, Python levanta una excepción. Si no se controlan adecuadamente, las excepciones pueden hacer que el programa se cuelgue. Se pueden manejar utilizando `try` y `except`:

```
try:
   print(0 / 0)
except ZeroDivisionError:
   print("cannot divide by zero")
```

Aunque en muchos lenguajes las excepciones no están bien consideradas, en Python no hay problema en utilizarlas para que el código sea más limpio, así que en ocasiones lo haremos.

Listas

Probablemente la estructura de datos más esencial de Python es la lista, que no es más que una colección ordenada (similar a lo que en otros lenguajes se podría denominar *array*, pero con funcionalidad añadida):

```
integer_list = [1, 2, 3]
heterogeneous_list = ["string", 0.1, True]
list_of_lists = [integer_list, heterogeneous_list, []]
list_length = len(integer_list)        # es igual a 3
list_sum = sum(integer_list)           # es igual a 6
```

Se puede obtener o establecer el elemento *n* de una lista con corchetes:

```
x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
zero = x[0]       # es igual a 0, las listas están indexadas al 0
```

```
one = x[1]         # es igual a 1
nine = x[-1]       # es igual a 9, 'pitónico' para el último elemento
eight = x[-2]      # es igual a 8, 'pitónico' para el penúltimo elemento
x[0] = -1          # ahora x es [-1, 1, 2, 3, ..., 9]
```

También se pueden utilizar corchetes para crear *slices* en listas (cortes o arreglos). El corte `i:j` significa todos los elementos desde `i` (incluido) hasta `j` (excluido). Si dejamos fuera el principio del corte, lo extraeremos desde el principio de la lista, pero, si dejamos fuera el final del corte, lo extraeremos hasta el final:

```
first_three = x[:3]                  # [-1, 1, 2]
three_to_end = x[3:]                 # [3, 4, ..., 9]
one_to_four = x[1:5]                 # [1, 2, 3, 4]
last_three = x[-3:]                  # [7, 8, 9]
without_first_and_last = x[1:-1]     # [1, 2, ..., 8]
copy_of_x = x[:]                     # [-1, 1, 2, ..., 9]
```

De forma similar se pueden crear cortes de cadenas y otros tipos "secuenciales".

Un corte puede admitir un tercer argumento para indicar su *stride* (avance), que puede ser negativo:

```
every_third = x[::3]          # [-1, 3, 6, 9]
five_to_three = x[5:2:-1]     # [5, 4, 3]
```

Python tiene un operador `in` para comprobar los miembros de la lista:

```
1 in [1, 2, 3]     # True
0 in [1, 2, 3]     # False
```

Esta comprobación implica examinar los elementos de la lista de uno en uno, lo que significa que probablemente no se debería utilizar a menos que se sepa que la lista es pequeña (o a menos que no nos preocupe el tiempo que tarde en hacerse la comprobación).

Es fácil concatenar listas. Si se desea modificar una lista en su lugar, se puede utilizar `extend` para añadir elementos de otra colección:

```
x = [1, 2, 3]
x.extend([4, 5, 6])     # x es ahora [1, 2, 3, 4, 5, 6]
```

Si no queremos modificar `x`, podemos ampliar la lista:

```
x = [1, 2, 3]
y = x + [4, 5, 6]     # y es [1, 2, 3, 4, 5, 6]; x no ha cambiado
```

Lo más frecuente que haremos será añadir elementos a listas uno a uno:

```
x = [1, 2, 3]
x.append(0)     # x es ahora [1, 2, 3, 0]
y = x[-1]       # es igual a 0
z = len(x)      # es igual a 4
```

A menudo, es conveniente desempaquetar listas cuando se sabe cuántos elementos contienen:

```
x, y = [1, 2]     # ahora x es 1, y es 2
```

Aunque obtendremos un `ValueError` si no tenemos el mismo número de elementos en ambos lados.

Algo que se utiliza habitualmente es un carácter de subrayado para un valor del que nos vamos a deshacer:

```
_, y = [1, 2]     # ahora y == 2, no nos importa el primer elemento
```

Tuplas

Las tuplas son las primas inmutables de las listas. Casi todo lo que se le puede hacer a una lista que implique modificarla, se le puede hacer a una tupla. Se especifica una tupla utilizando paréntesis (o nada) en lugar de corchetes:

```
my_list = [1, 2]
my_tuple = (1, 2)
```

```
other_tuple = 3, 4
my_list[1] = 3      # my_list es ahora[1, 3]
try:
my_tuple[1] = 3
except TypeError:
print("cannot modify a tuple")
```

Las tuplas son una forma cómoda de devolver varios valores de funciones:

```
def sum_and_product(x, y):
   return (x + y), (x * y)
sp = sum_and_product(2, 3)         # sp es (5, 6)
s, p = sum_and_product(5, 10)      # s es 15, p es 50
```

Las tuplas (y las listas) se pueden utilizar para asignación múltiple:

```
x, y = 1,        # ahora x es 1, y es 2
2
x, y = y,        # Forma pitónica de intercambiar variables; ahora x es 2, y es
x                1
```

Diccionarios

Otra estructura de datos fundamental es un diccionario, que asocia valores a claves y permite recuperar rápidamente el valor correspondiente a una determinada clave:

```
empty_dict = {}                       # pitónico
empty_dict2 = dict()                  # menos pitónico
grades = {"Joel": 80, "Tim": 95}      # dict literal
```

Se puede consultar el valor para una clave utilizando corchetes:

```
  joels_grade = grades[“Joel”]      # es igual a 80
```

Pero se obtendrá un `KeyError` si se pregunta por una clave que no está en el diccionario:

```
try:
   kates_grade = grades["Kate"]
except KeyError:
   print("no grade for Kate!")
```

Se puede comprobar la existencia de una clave utilizando `in`:

```
joel_has_grade = "Joel" in grades      # True
kate_has_grade = "Kate" in grades      # False
```

Esta verificación de membresía es aún más rápida para diccionarios grandes.

Los diccionarios tienen un método `get` que devuelve un valor predeterminado (en lugar de levantar una excepción) cuando se consulta una clave que no está en el diccionario:

```
joels_grade = grades.get("Joel", 0)       # es igual a 80
kates_grade = grades.get("Kate", 0)       # es igual a 0
no_ones_grade = grades.get("No One")      # el valor predeterminado es None
```

Se pueden asignar pares clave/valor utilizando los mismos corchetes:

```
grades["Tim"] = 99              # reemplaza el valor anterior
grades["Kate"] = 100            # añade una tercera entrada
num_students = len(grades)      # es igual a 3
```

Como vimos en el capítulo 1, se pueden utilizar diccionarios para representar datos estructurados:

```
tweet = {
   "user" : "joelgrus",
   "text" : "Data Science is Awesome",
   "retweet_count" : 100,
   "hashtags" : ["#data", "#science", "#datascience", "#awesome", "#yolo"]
}
```

Aunque pronto veremos un enfoque mejor.

Además de buscar claves específicas, también podemos mirarlas todas:

```
tweet_keys = tweet.keys()       # iterable para las claves
tweet_values =                  # iterable para los valores
tweet.values()
tweet_items =                   # iterable para las tuplas (clave, valor)
tweet.items()
"user" in tweet_keys            # True, pero no pitónico
"user" in tweet                 # forma pitónica de comprobar claves
"joelgrus" in tweet_values      # True (es lenta, pero la única forma de
                                verificar)
```

Las claves de diccionario pueden ser "*hashables*"; en particular, no se pueden utilizar listas como claves. Si se necesita una clave multiparte, probablemente se debería utilizar una tupla o idear un modo de convertir la clave en una cadena.

defaultdict

Imaginemos que estamos intentando contar las palabras de un documento. Un método obvio para lograrlo es crear un diccionario en el que las claves sean palabras y los valores sean contadores. Al comprobar cada palabra, se puede incrementar su contador si ya está en el diccionario y añadirlo al diccionario si no estaba:

```
word_counts = {}
for word in document:
   if word in word_counts:
      word_counts[word] += 1
   else:
      word_counts[word] = 1
```

Se podría utilizar también el sistema "mejor pedir perdón que permiso" y simplemente manejar la excepción al intentar consultar una clave inexistente:

```
word_counts = {}
for word in document:
   try:
      word_counts[word] += 1
   except KeyError:
      word_counts[word] = 1
```

Un tercer enfoque es utilizar `get`, que se comporta con mucha elegancia con las claves inexistentes:

```
word_counts = {}
for word in document:
   previous_count = word_counts.get(word, 0)
   word_counts[word] = previous_count + 1
```

Todo esto es muy poco manejable, razón por la cual `defaultdict` es útil. Un `defaultdict` es como un diccionario normal, excepto que, cuando se intenta buscar una clave que no contiene, primero añade un valor para ella utilizando una función de argumento cero suministrada al crearla. Para utilizar diccionarios `defaultdicts`, es necesario importarlos de `collections`:

```
from collections import defaultdict
word_counts = defaultdict(int)     # int() produce 0
for word in document:
   word_counts[word] += 1
```

También pueden resultar útiles con `list` o `dict`, o incluso con nuestras propias funciones:

```
dd_list = defaultdict(list)              # list() produce una lista vacía
dd_list[2].append(1)                     # ahora dd_list contiene {2: [1]}
dd_dict = defaultdict(dict)              # dict() produce un dict vacío
dd_dict["Joel"]["City"] = "Seattle"      # {"Joel" : {"City": Seattle"}}
dd_pair = defaultdict(lambda: [0, 0])
dd_pair[2][1] = 1                        # ahora dd_pair contiene {2: [0, 1]}
```

Serán útiles cuando estemos utilizando diccionarios para “recopilar” resultados según alguna clave y no queramos comprobar todo el tiempo si la clave sigue existiendo.

Contadores

Un `Counter` convierte una secuencia de valores en un objeto de tipo

`defaultdict(int)` mapeando claves en contadores:

```
from collections import Counter
c = Counter([0, 1, 2, 0])     # c es (básicamente) {0: 2, 1: 1, 2: 1}
```

Lo que nos ofrece un modo muy sencillo de resolver problemas de `word_counts`:

```
# recuerde, document es una lista de palabras
word_counts = Counter(document)
```

Una instancia `Counter` tiene un método `most_common` que se utiliza con frecuencia:

```
# imprime las 10 palabras más comunes y sus contadores
for word, count in word_counts.most_common(10):
   print(word, count)
```

Conjuntos

Otra estructura de datos útil es el conjunto o *set*, que representa una colección de distintos elementos. Se pueden definir un conjunto listando sus elementos entre llaves:

```
primes_below_10 = {2, 3, 5, 7}
```

Sin embargo, esto no funciona con conjuntos vacíos, dado que `{}` ya significa "`dict` vacío". En ese caso, habrá que utilizar la propia `set()`:

```
s = set()
s.add(1)       # s es ahora {1}
s.add(2)       # s es ahora {1, 2}
s.add(2)       # s sigue siendo {1, 2}
x = len(s)     # es igual a 2
y = 2 in s     # es igual a True
z = 3 in s     # es igual a False
```

Utilizaremos conjuntos por dos razones principales. La primera es que `in`

es una operación muy rápida con conjuntos. Si tenemos una gran colección de elementos que queremos utilizar para hacer una prueba de membresía, un conjunto es más adecuado que una lista:

```
stopwords_list = ["a", "an", "at"] + hundreds_of_other_words + ["yet", "you"]
"zip" in stopwords_list       # False, pero hay que verificar cada elemento
stopwords_set = set(stopwords_list)
"zip" in stopwords_set        # muy rápido de comprobar
```

La segunda razón es encontrar los elementos distintos de una colección:

```
item_list = [1, 2, 3, 1, 2, 3]
num_items = len(item_list)              # 6
item_set = set(item_list)               # {1, 2, 3}
num_distinct_items = len(item_set)      # 3
distinct_item_list = list(item_set)     # [1, 2, 3]
```

Utilizaremos conjuntos con menos frecuencia que diccionarios y listas.w

Flujo de control

Como en la mayoría de los lenguajes de programación, se puede realizar una acción de forma condicional utilizando `if`:

```
if 1 > 2:
   message = "if only 1 were greater than two..."
elif 1 > 3:
   message = "elif stands for ‘else if’"
else:
   message = "when all else fails use else (if you want to)"
```

Se puede también escribir un ternario if-then-else en una sola línea, cosa que haremos muy de tanto en tanto:

```
parity = “even” if x % 2 == 0 else “odd”
```

Python tiene un bucle `while`:

```
x = 0
while x < 10:
   print(f"{x} is less than 10")
   x += 1
```

Aunque con mucha más frecuencia usaremos `for` e `in`:

```
# range(10) es los números 0, 1, ..., 9
for x in range(10):
   print(f"{x} is less than 10")
```

Si necesitáramos lógica más compleja, podríamos utilizar `continue` y `break`:

```
for x in range(10):
   if x == 3:
      continue      # va inmediatamente a la siguiente repetición
   if x == 5:
      break         # sale del todo del bucle
   print(x)
```

Esto imprimirá 0, 1, 2 y 4.

Verdadero o falso

Los valores booleanos en Python funcionan igual que en casi todos los demás lenguajes, excepto que llevan la primera letra en mayúscula:

```
one_is_less_than_two = 1 < 2          # es igual a True
true_equals_false = True == False     # es igual a False
```

Python utiliza el valor `None` para indicar un valor no existente. Es similar al `null` de otros lenguajes:

```
x = None
assert x == None, "this is the not the Pythonic way to check for None"
assert x is None, "this is the Pythonic way to check for None"
```

Python permite utilizar cualquier valor donde espera un booleano. Las siguientes expresiones son todas “falsas”:

- `False`.
- `None`.
- `[]` (una `list` vacía).
- `{}` (un `dict` vacío).
- `""`.
- `set()`.
- `0`.
- `0.0`.

Casi todo lo demás se trata como `True`. Ello permite utilizar fácilmente sentencias `if` para probar listas vacías, cadenas vacías, diccionarios vacíos, etc. También produce en ocasiones errores complicados si no se espera este comportamiento:

```
s = some_function_that_returns_a_string()
if s:
   first_char = s[0]
else:
   first_char = ""
```

Una forma más corta (pero posiblemente más confusa) de hacer lo mismo es:

```
first_char = s and s[0]
```

Ya que `and` devuelve su segundo valor cuando el primero es “verdadero”, y el primer valor cuando no lo es. De forma similar, si `x` es o bien un número o posiblemente `None`:

```
safe_x = x or 0
```

Es definitivamente un número, aunque:

```
safe_x = x if x is not None else 0
```

Es posiblemente más legible.

Python tiene una función `all`, que toma un iterable y devuelve `True` exactamente cuando cada elemento es verdadero, y una función `any`, que devuelve `True` cuando al menos un elemento es verdad:

```
all([True, 1, {3}])     # True, todos son verdaderos
all([True, 1, {}])      # False, {} is falso
any([True, 1, {}])      # True, True is verdadero
all([])                 # True, no hay elementos falsos en la lista
any([])                 # False, no hay elementos verdaderos en la lista
```

Ordenar

Toda lista de Python tiene un método `sort` que la ordena en su lugar. Si no queremos estropear nuestra lista, podemos usar la función `sorted`, que devuelve una lista nueva:

```
x = [4, 1, 2, 3]
y = sorted(x)     # y es [1, 2, 3, 4], x queda igual
x.sort()          # ahora x es [1, 2, 3, 4]
```

Por defecto, `sort` (y `sorted`) ordena una lista de menor a mayor basándose en comparar inocentemente los elementos uno con otro.

Si queremos que los elementos estén ordenados de mayor a menor, se puede especificar un parámetro `reverse=True`. Y, en lugar de comparar los elementos por sí mismos, se pueden comparar los resultados de una función que se especifica con `key`:

```
# ordena la lista por valor absoluto de mayor a menor
x = sorted([-4, 1, -2, 3], key=abs, reverse=True)         # is [-4, 3, -2, 1]
# ordena las palabras y contadores del contador mayor al menor
wc = sorted(word_counts.items(),
        key=lambda word_and_count: word_and_count[1],
        reverse=True)
```

Comprensiones de listas

Con frecuencia, vamos a querer transformar una lista en otra distinta seleccionando solo determinados elementos, transformando elementos o haciendo ambas cosas. La forma pitónica de hacer esto es con *list comprehensions*, o comprensiones de listas:

```
even_numbers = [x for x in range(5) if x % 2 == 0]     # [0, 2, 4]
squares = [x * x for x in range(5)]                    # [0, 1, 4, 9, 16]
even_squares = [x * x for x in even_numbers]           # [0, 4, 16]
```

De forma similar, se pueden convertir listas en diccionarios o conjuntos:

```
square_dict = {x: x * x for x in                # {0: 0, 1: 1, 2: 4, 3: 9, 4:
range(5)}                                       16}
square_set = {x * x for x in [1, -1]}           # {1}
```

Si no necesitamos el valor de la lista, es habitual utilizar un guion bajo como variable:

```
zeros = [0 for _ in                     # tiene la misma longitud que
even_numbers]                           even_numbers
```

Una comprensión de lista puede incluir varios `for`:

```
pairs = [(x, y)
     for x in range(10)
     for y in range(10)]     # 100 pares (0,0) (0,1) ... (9,8), (9,9)
```

Y después los `for` pueden utilizar los resultados de los anteriores:

```
increasing_pairs = [(x, y)                # solo pares con x < y,
          for x in range(10)              # range(bajo, alto) es igual a
          for y in range(x + 1, 10)]      # [bajo, bajo + 1, ..., alto-1]
```

Utilizaremos mucho las comprensiones de listas.

Pruebas automatizadas y assert

Como científicos de datos, escribiremos mucho código. ¿Cómo podemos estar seguros de que nuestro código es correcto? Una forma es con tipos (de los que hablaremos en breve), pero otra forma es con *automated tests* o pruebas automatizadas.

Hay estructuras muy complicadas para escribir y ejecutar pruebas, pero en este libro nos limitaremos a utilizar sentencias `assert`, que harán que el código levante un `AssertionError` si la condición especificada no es verdadera:

```
assert 1 + 1 == 2
assert 1 + 1 == 2, "1 + 1 should equal 2 but didn't"
```

Como podemos ver en el segundo caso, se puede añadir si se desea un mensaje que se imprimirá si la evaluación falla.

No es especialmente interesante evaluar que 1 + 1 = 2, pero lo es mucho más verificar que las funciones que escribamos hagan lo que se espera de ellas:

```
def smallest_item(xs):
   return min(xs)
assert smallest_item([10, 20, 5, 40]) == 5
assert smallest_item([1, 0, -1, 2]) == -1
```

A lo largo del libro utilizaremos `assert` de esta forma. Es una buena práctica, y yo animo a utilizar libremente esta sentencia (en el código del libro que encontramos en GitHub se comprueba que contiene muchas más sentencias `assert` de las que aparecen impresas en el libro, lo que me permite estar seguro de que el código que he escrito es correcto).

Otro uso menos habitual es evaluar cosas sobre entradas a funciones:

```
def smallest_item(xs):
   assert xs, "empty list has no smallest item"
   return min(xs)
```

Haremos esto de vez en cuando, pero será más frecuente utilizar `assert`

para verificar que el código escrito es correcto.

Programación orientada a objetos

Como muchos lenguajes, Python permite definir clases que encapsulan los datos y las funciones que operan con ellos. Las utilizaremos algunas veces para que nuestro código sea más limpio y sencillo. Probablemente, es más fácil explicarlas construyendo un ejemplo con muchas anotaciones.

Aquí vamos a crear una clase que represente un "contador de clics", del tipo de los que se ponen en la puerta para controlar cuántas personas han acudido al encuentro "Temas avanzados sobre ciencia de datos".

Mantiene un contador (`count`), se le puede hacer clic (`clicked`) para aumentar la cuenta, permite lectura de contador (`read_count`) y se puede reiniciar (`reset`) de vuelta a cero (en la vida real una de estas clases pasa de 9999 a 0000, pero no vamos a preocuparnos de eso ahora).

Para definir una clase, utilizamos la palabra clave `class` y un nombre de tipo PascalCase:

```
class CountingClicker:
   """A class can/should have a doctring, just like a function"""
```

Una clase contiene cero o más funciones miembro. Por convenio, cada una toma un primer parámetro, `self`, que se refiere a la instancia en particular de la clase.

Normalmente, una clase tiene un constructor, llamado `__init__`, que toma los parámetros que necesita para construir una instancia de dicha clase y hace cualquier configuración que se necesite:

```
def __init__(self, count = 0):
   self.count = count
```

Aunque el constructor tiene un nombre divertido, construimos las instancias del contador de clics utilizando solamente el nombre de la clase:

```
clicker1 = CountingClicker()              # inicializado a 0
clicker2 = CountingClicker(100)           # empieza con count=100
clicker3 =                                # forma más explícita de hacer lo
CountingClicker(count=100)                mismo
```

Vemos que el nombre del método `__init__` empieza y termina con guiones bajos. A veces a estos métodos "mágicos" se les llama métodos "dunder" (término inventado que viene de *doubleUNDERscore*, es decir, doble guion bajo) y representan comportamientos "especiales".

> **Nota:** Los métodos de clase cuyos nombres empiezan con un guion bajo se consideran (por convenio) "privados", y se supone que los usuarios de esa clase no les llaman directamente. Sin embargo, Python no impide a los usuarios llamarlos.

Otro método similar es `__repr__`, que produce la representación de cadena de una instancia de clase:

```
def __repr__(self):
   return f"CountingClicker(count={self.count})"
```

Y finalmente tenemos que implementar la API pública de la clase que hemos creado:

```
def click(self, num_times = 1):
   """Click the clicker some number ot times."""
   self.count += num_times
def read(self):
   return self.count
def reset(self):
   self.count = 0
```

Una vez definido, utilicemos `assert` para escribir algunos casos de prueba para nuestro contador de clics:

```
clicker = CountingClicker()
assert clicker.read() == 0, "clicker should start with count 0"
clicker.click()
```

```
clicker.click()
assert clicker.read() == 2, "after two clicks, clicker should have count 2"
clicker.reset()
assert clicker.read() == 0, "after reset, clicker should be back to 0"
```

Escribir pruebas como estas nos permite estar seguros de que nuestro código esté funcionando tal y como está diseñado, y que esto va a seguir siendo así siempre que le hagamos cambios.

También crearemos de vez en cuando subclases que heredan parte de su funcionalidad de una clase padre. Por ejemplo, podríamos crear un contador de clics no reiniciable utilizando `CountingClicker` como clase base y anulando el método `reset` para que no haga nada:

```
# Una subclase hereda todo el comportamiento de su clase padre.
class NoResetClicker(CountingClicker):
   # Esta clase tiene los mismos métodos que CountingClicker
   # Salvo que tiene un método reset que no hace nada.
   def reset(self):
      pass
clicker2 = NoResetClicker()
assert clicker2.read() == 0
clicker2.click()
assert clicker2.read() == 1
clicker2.reset()
assert clicker2.read() == 1, "reset shouldn’t do anything"
```

Iterables y generadores

Una cosa buena de las listas es que se pueden recuperar determinados elementos por sus índices. Pero ¡esto no siempre es necesario! Una lista de mil millones de números ocupa mucha memoria. Si solo queremos los elementos uno cada vez, no hay una buena razón que nos haga conservarlos a todos. Si solamente terminamos necesitando los primeros elementos, generar los mil millones es algo tremendamente inútil.

A menudo, todo lo que necesitamos es pasar varias veces por la colección utilizando `for` e `in`. En este caso podemos crear generadores, que se pueden

iterar igual que si fueran listas, pero generan sus valores bajo petición.

Una forma de crear generadores es con funciones y con el operador `yield`:

```
def generate_range(n):
   i = 0
   while i < n:
      yield I      # cada llamada a yield produce un valor del generador
      i += 1
```

El siguiente bucle consumirá uno a uno los valores a los que se ha aplicado `yield` hasta que no quede ninguno:

```
for i in generate_range(10):
   print(f"i: {i}")
```

(En realidad, `range` es bastante perezosa de por sí, así que hacer esto no tiene ningún sentido).

Con un generador, incluso se puede crear una secuencia infinita:

```
def natural_numbers():
   """returns 1, 2, 3, ..."""
   n = 1
   while True:
      yield n
      n += 1
```

Aunque probablemente no deberíamos iterar sobre él sin utilizar algún tipo de lógica de interrupción.

Truco: La otra cara de la pereza es que solo se puede iterar una única vez por un generador. Si hace falta pasar varias veces, habrá que volver a crear el generador cada vez o bien utilizar una lista. Si generar los valores resulta caro, podría ser una buena razón para utilizar una lista en su lugar.

Una segunda manera de crear generadores es utilizar las comprensiones envueltas en paréntesis:

```
evens_below_20 = (i for i in generate_range(20) if i % 2 == 0)
```

Una “comprensión de generador” como esta no hace nada hasta que se itera sobre ella (utilizando `for` o `next`). Podemos utilizar esto para crear complicadas líneas de proceso de datos:

```
# Ninguno de estos cálculos *hace* nada hasta que iteramos
data = natural_numbers()
evens = (x for x in data if x % 2 == 0)
even_squares = (x ** 2 for x in evens)
even_squares_ending_in_six = (x for x in even_squares if x % 10 == 6)
# y así sucesivamente
```

No pocas veces, cuando estemos iterando sobre una lista o un generador, no querremos solamente los valores, sino también sus índices. Para este caso habitual, Python ofrece una función `enumerate`, que convierte valores en pares (`index`, `value`):

```
names = ["Alice", "Bob", "Charlie", "Debbie"]
# no pitónico
for i in range(len(names)):
   print(f"name {i} is {names[i]}")
# tampoco pitónico
i = 0
for name in names:
   print(f"name {i} is {names[i]}")
   i += 1
# pitónico
for i, name in enumerate(names):
   print(f"name {i} is {name}")
```

Utilizaremos mucho esto.

Aleatoriedad

A medida que aprendamos ciencia de datos, necesitaremos con frecuencia generar números aleatorios, lo que podemos hacer con el módulo `random`:

```
import random
random.seed(10)     # esto asegura que obtenemos los mismos resultados cada vez
```

```
four_uniform_randoms = [random.random() for _ in range(4)]

# [0.5714025946899135,      # random.random() produce números
# 0.4288890546751146,       # de manera uniforme entre 0 y 1.
# 0.5780913011344704,       # Es la función random que utilizaremos
# 0.20609823213950174]      # con más frecuencia.
```

El módulo `random` produce en realidad números pseudoaleatorios (es decir, deterministas) basados en un estado interno que se puede configurar con `random.seed` si lo que se desea es obtener resultados reproducibles:

```
random.seed(10)             # establece la semilla en 10
print(random.random())      # 0.57140259469
random.seed(10)             # reinicia la semilla en 10
print(random.random())      # 0.57140259469 de nuevo
```

Algunas veces utilizaremos `random.randrange`, que toma uno o dos argumentos y devuelve un elemento elegido aleatoriamente del `range` correspondiente:

```
random.randrange(10)      # selecciona aleatoriamente de range(10) = [0, 1,
                          ..., 9]
random.randrange(3,       # selecciona aleatoriamente de range(3, 6) = [3, 4,
6)                        5]
```

Hay varios métodos más que en ocasiones nos resultarán convenientes. Por ejemplo, `random.shuffle` reordena aleatoriamente los elementos de una lista:

```
up_to_ten = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(up_to_ten)
print(up_to_ten)
# [7, 2, 6, 8, 9, 4, 10, 1, 3,      (sus resultados serán probablemente
5]                                  diferentes)
```

Si se necesita elegir aleatoriamente un elemento de una lista, se puede utilizar `random.choice`:

```
my_best_friend = random.choice([“Alice”, “Bob”, “Charlie”])     # “Bob” para mí
```

Y, si lo que hace falta es elegir aleatoriamente una muestra de elementos sin sustituto (es decir, sin duplicados), se puede utilizar `random.sample`:

```
lottery_numbers = range(60)
winning_numbers = random.sample(lottery_numbers,        # [16, 36, 10, 6, 25,
6)                                                      9]
```

Para elegir una muestra de elementos con sustituto (es decir, que permita duplicados), simplemente basta con hacer varias llamadas a `random.choice`:

```
four_with_replacement = [random.choice(range(10)) for _ in range(4)]
print(four_with_replacement)                  # [9, 4, 4, 2]
```

Expresiones regulares

Las expresiones regulares ofrecen un modo de buscar texto. Son increíblemente útiles, pero también bastante complicadas (tanto, que hay escritos libros enteros sobre ellas). Entraremos en detalle las pocas veces que nos las encontremos; estos son algunos ejemplos de cómo utilizarlas en Python:

```
import re
re_examples = [                                 # Todas son True, porque
   not re.match("a", "cat"),                    # 'cat' no empieza por 'a'
   re.search("a", "cat"),                       # 'cat' contiene una 'a'
   not re.search("c", "dog"),                   # 'dog' no contiene una 'c'.
   3 == len(re.split("[ab]",                    # Partido en a o b para
   "carbs")),                                   ['c','r','s'].
   "R-D-" == re.sub("[0-9]", "-",               # Reemplaza dígitos por guiones.
   "R2D2")
   ]
assert all(re_examples), "all the regex examples should be True"
```

Algo importante a tener en cuenta es que `re.match` comprueba si el principio de una cadena coincide con una expresión regular, mientras que `re.search` lo comprueba con alguna parte de una cadena. En algún momento es probable que se mezclen los dos y creen problemas.

La documentación oficial en https://docs.python.org/es/3/library/re.html ofrece muchos más detalles.

Programación funcional

> **Nota:** La primera edición de este libro presentaba en este momento las funciones de Python `partial`, `map`, `reduce` y `filter`. En mi viaje hacia la iluminación, me he dado cuenta de que es mejor evitarlas, y sus usos en el libro han sido reemplazados por comprensiones de listas, bucles `for` y otras construcciones más pitónicas.

Empaquetado y desempaquetado de argumentos

A menudo, necesitaremos empaquetar (*zip*) dos o más iterables juntos. La función `zip` transforma varios iterables en uno solo de tuplas de función correspondiente:

```
list1 = ['a', 'b', 'c']
list2 = [1, 2, 3]
# zip es perezoso, de modo que hay que hacer algo como lo siguiente
[pair for pair in zip(list1, list2)]     # es [('a', 1), ('b', 2), ('c', 3)]
```

Si las listas tienen distintas longitudes, `zip` se detiene tan pronto como termina la primera lista.

También se puede "desempaquetar" una lista utilizando un extraño truco:

```
pairs = [('a', 1), ('b', 2), ('c', 3)]
letters, numbers = zip(*pairs)
```

El asterisco (*) realiza desempaquetado de argumento, que utiliza los elementos de `pairs` como argumentos individuales para `zip`. Termina igual que si lo hubiéramos llamado:

```
letters, numbers = zip(('a', 1), ('b', 2), ('c', 3))
```

Se puede utilizar desempaquetado de argumento con cualquier función:

```
def add(a, b): return a + b
add(1, 2)          # devuelve 3
try:
   add([1, 2])
except TypeError:
   print("add expects two inputs")
add(*[1, 2])       # devuelve 3
```

Es raro que encontremos esto útil, pero, cuando lo hacemos, es un truco genial.

args y kwargs

Digamos que queremos crear una función de máximo orden que requiere como entrada una función `f` y devuelve una función nueva que para cualquier entrada devuelve el doble del valor de `f`:

```
def doubler(f):
   # Aquí definimos una nueva función que mantiene una referencia a f
   def g(x):
      return 2 * f(x)
   # Y devuelve esa nueva función
   return g
```

Esto funciona en algunos casos:

```
def f1(x):
   return x + 1
g = doubler(f1)
assert g(3) == 8, "(3 + 1) * 2 should equal 8"
assert g(-1) == 0, "(-1 + 1) * 2 should equal 0"
```

Sin embargo, no sirve con funciones que requieren algo más que un solo argumento:

```
def f2(x, y):
```

```
   return x + y
g = doubler(f2)
try:
   g(1, 2)
except TypeError:
   print("cas defined, g only takes one argument")
```

Lo que necesitamos es una forma de especificar una función que tome argumentos arbitrarios. Podemos hacerlo con desempaquetado de argumento y un poco de magia:

```
def magic(*args, **kwargs):
   print("unnamed args:", args)
   print("keyword args:", kwargs)
magic(1, 2, key="word", key2="word2")
# imprime
# argumentos sin nombre: (1, 2)
# argumentos de palabra clave: {‘key’: ‘word’, ‘key2’: ‘word2’}
```

Es decir, cuando definimos una función como esta, `args` es una tupla de sus argumentos sin nombre y `kwargs` es un `dict` de sus argumentos con nombre. Funciona también a la inversa, si queremos utilizar una `list` (o `tuple`) y `dict` para proporcionar argumentos a una función:

```
def other_way_magic(x, y, z):
   return x + y + z
x_y_list = [1, 2]
z_dict = {"z": 3}
assert other_way_magic(*x_y_list, **z_dict) == 6, "1 + 2 + 3 should be 6"
```

Se podría hacer todo tipo de trucos extraños con esto; solo lo utilizaremos para producir funciones de máximo orden cuyas entradas puedan aceptar argumentos arbitrarios:

```
def doubler_correct(f):
   """works no matter what kind of inputs f expects"""
   def g(*args, **kwargs):
      """whatever arguments g is supplied, pass them through to f"""
      return 2 * f(*args, **kwargs)
   return g
```

```
g = doubler_correct(f2)
assert g(1, 2) == 6, "doubler should work now"
```

Como regla general, el código que escribamos será más correcto y legible si somos explícitos en lo que se refiere a los tipos de argumentos que las funciones que usemos requieren; de ahí que vayamos a utilizar `args` y `kwargs` solo cuando no tengamos otra opción.

Anotaciones de tipos

Python es un lenguaje de tipos dinámicos. Esto significa que en general no le importan los tipos de objetos que utilicemos, siempre que lo hagamos de formas válidas:

```
def add(a, b):
   return a + b
assert add(10, 5) == 15, "+ is valid for numbers"
assert add([1, 2], [3]) == [1, 2, 3], "+ is valid for lists"
assert add("hi ", "there") == "hi there", "+ is valid for strings"
try:
   add(10, "five")
except TypeError:
   print("cannot add an int to a string")
```

Mientras que en un lenguaje de tipos estáticos nuestras funciones y objetos tendrían tipos específicos:

```
def add(a: int, b: int) -> int:
   return a + b
add(10, 5)              # le gustaría que esto fuera correcto
add("hi ", "there")     # le gustaría que esto no fuera correcto
```

En realidad, las versiones más recientes de Python tienen (más o menos) esta funcionalidad. ¡La versión anterior de `add` con las anotaciones de tipos `int` es válida en Python 3.6! Sin embargo, estas anotaciones de tipos no hacen realmente nada. Aún se puede utilizar la función anotada `add` para

añadir cadenas, y la llamada a `add(10, “five”)` seguirá levantando exactamente el mismo `TypeError`.

Dicho esto, sigue habiendo (al menos) cuatro buenas razones para utilizar anotaciones de tipos en el código Python que escribamos:

- Los tipos son una forma importante de documentación. Esto es doblemente cierto en un libro que utiliza código para enseñar conceptos teóricos y matemáticos. Comparemos las siguientes dos líneas de función:

```
def dot_product(x, y): ...
# aún no hemos definido Vector, pero imagínese que lo habíamos hecho
def dot_product(x: Vector, y: Vector) -> float: ...
```

  Encuentro el segundo extremadamente más informativo; espero que también se lo parezca (en este punto me he acostumbrado tanto a la determinación de tipos que ahora sin ello encuentro Python difícil de leer).
- Hay herramientas externas (siendo la más popular `mypy`) que leerán el código que escribamos, inspeccionarán las anotaciones de tipos y ofrecerán errores de tipos antes siquiera de ejecutar el código. Por ejemplo, si ejecutamos `mypy` en un archivo que contiene `add(“hi “,”there”)`, avisaría de lo siguiente:

```
error: Argument 1 to “add” has incompatible type “str”; expected “int”
```

  Al igual que la prueba `assert`, esta es una buena forma de encontrar errores en el código antes de ejecutarlo. La narración del libro no implicará tal comprobación de tipo; sin embargo, ejecutaré una en segundo plano, lo que me permitirá asegurarme de que el libro en sí es correcto.
- Tener que pensar en los tipos de nuestro código nos obliga a diseñar funciones e interfaces más limpios:

```
from typing import Union
def secretly_ugly_function(value, operation): ...
```

```
def ugly_function(value: int,
                  operation: Union[str, int, float, bool]) -> int:
    ...
```

Aquí tenemos una función cuyo parámetro de operación puede ser un `string`, un `int`, un `float` o un `bool`. Es muy probable que esta función sea frágil y difícil de utilizar, pero aún queda más claro cuando los tipos resultan explícitos. Hacer esto nos obligará a diseñar de un modo menos torpe, cosa que nuestros usuarios nos agradecerán.

- Utilizar tipos permite al editor que utilicemos ayudarnos con cosas como autocompletar (véase la figura 2.1) y enfadarnos por los errores de escritura.

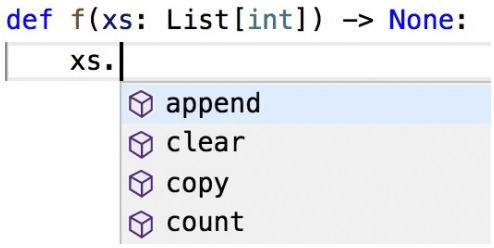


Figura 2.1. VSCode, pero probablemente otro editor haga lo mismo.

A veces, la gente insiste en que las comprobaciones de tipo pueden ser valiosas en proyectos grandes, pero no merecen la pena en otros más pequeños. No obstante, como casi no se tardan nada en escribir y permiten al editor ahorrarnos tiempo, yo mantengo que de verdad permiten escribir código con mayor rapidez, incluso en proyectos pequeños.

Por todas estas razones, todo el código del resto de este libro utilizará anotaciones de tipos. Supongo que algunos lectores se sentirán desanimados por utilizarlas, pero sospecho que al final del libro habrán cambiado de opinión.

Cómo escribir anotaciones de tipos

Como hemos visto, para tipos internos como `int`, `bool` y `float` basta con utilizar el propio tipo como anotación. Pero ¿qué pasa si tenemos (por

ejemplo) un `list`?

```
def total(xs: list) -> float:
   return sum(total)
```

Esto no es erróneo, pero el tipo no es lo bastante específico. Está claro que realmente queremos que `xs` sea un `list` de valores `float`, no (por ejemplo) un `list` de cadenas.

El módulo `typing` ofrece una serie de tipos parametrizados que podemos utilizar para hacer precisamente esto:

```
from typing import List      # observe la L mayúscula
def total(xs: List[float]) -> float:
   return sum(total)
```

Hasta ahora hemos especificado solamente anotaciones para parámetros de función y tipos de retorno. Para las propias variables suele ser obvio cuál es el tipo:

```
# Así es como se anota el tipo de variables cuando se definen.
# Pero esto es innecesario; es "obvio" que x es un int.
x: int = 5
```

No obstante, algunas veces no es obvio:

```
values = []             # ¿cuál es mi tipo?
best_so_far = None      # ¿cuál es mi tipo?
```

En estos casos suministraremos las comprobaciones de tipos *inline*:

```
from typing import Optional
values: List[int] = []
best_so_far: Optional[float] = None      # permitido ser un float o None
```

El módulo `typing` contiene muchos otros tipos, de los que solo emplearemos unos pocos:

```
# las anotaciones de tipo de este fragmento son todas innecesarias
from typing import Dict, Iterable, Tuple
```

```
# las claves son strings, los valores son ints
counts: Dict[str, int] = {‘data’: 1, ‘science’: 2}
# las listas y los generadores son ambos iterables
if lazy:
   evens: Iterable[int] = (x for x in range(10) if x % 2 == 0)
else:
   evens = [0, 2, 4, 6, 8]
# las tuplas especifican un tipo para cada elemento
triple: Tuple[int, float, int] = (10, 2.3, 5)
```

Finalmente, como Python tiene funciones de primera clase, necesitamos un tipo que las represente también. Este es un ejemplo bastante forzado:

```
from typing import Callable
# La comprobación de tipos dice que el repetidor es una función que admite
# dos argumentos, un string y un int, y devuelve un string.
def twice(repeater: Callable[[str, int], str], s: str) -> str:
   return repeater(s, 2)
def comma_repeater(s: str, n: int) -> str:
   n_copies = [s for _ in range(n)]
   return ‘, ‘.join(n_copies)
assert twice(comma_repeater, "type hints") == "type hints, type hints"
```

Como las anotaciones de tipos son solo objetos Python, podemos asignarles variables para que sea más fácil hacer referencia a ellos:

```
Number = int
Numbers = List[Number]
def total(xs: Numbers) -> Number:
   return sum(xs)
```

Para cuando lleguemos al final del libro, el lector estará bastante familiarizado con leer y escribir anotaciones de tipos, y espero que las utilice en su código.

Bienvenido a DataSciencester

Esto concluye la orientación al empleado. Ah, otra cosa: intente no

“distraer” nada.

Para saber más

- No faltan tutoriales de Python en el mundo. El oficial en https://docs.python.org/es/3/tutorial/ no es mal punto de partida para empezar.
- El tutorial oficial de IPython en http://ipython.readthedocs.io/en/stable/interactive/index.ht le permitirá empezar con IPython, si decide utilizarlo. Por favor, utilícelo.
- La documentación de `mypy` en https://mypy.readthedocs.io/en/stable/ le dará más información de la que nunca quiso tener sobre anotaciones de tipos y comprobaciones de tipos de Python.

---

[1] http://legacy.python.org/dev/peps/pep-0020/

[2] https://github.com/joelgrus/data-science-from-scratch/blob/master/INSTALL.md.

[3] https://www.python.org/.

[4] https://www.anaconda.com/products/distribution.

[5] https://docs.python.org/es/3/library/venv.html.

[6] https://virtualenv.pypa.io/en/latest/.

[7] http://ipython.org/.

3 Visualizar datos

Creo que la visualización es uno de los medios más poderosos de lograr objetivos personales.

—Harvey Mackay

La visualización de datos es una parte fundamental del kit de herramientas de un científico de datos. Es muy fácil crear visualizaciones, pero es mucho más difícil lograr que sean buenas. Tiene dos usos principales:

- Explorar datos.
- Comunicar datos.

En este capítulo, nos centraremos en adquirir las habilidades necesarias para empezar a explorar nuestros propios datos y producir las visualizaciones que vamos a utilizar a lo largo del libro. Al igual que la mayoría de los temas que se tratan en sus capítulos, la visualización de datos es un campo de estudio tan profundo que merece un libro entero. No obstante, trataré de darle una idea de lo que conduce a una buena visualización de datos y lo que no.

matplotlib

Existe una gran variedad de herramientas para visualizar datos. Emplearemos la librería de matplotlib[1], la más utilizada (aunque ya se le notan un poco los años). Si lo que queremos es producir una visualización elaborada e interactiva para la web, probablemente no es la mejor opción, pero sirve a la perfección para sencillos gráficos de barras, líneas y dispersión. Como ya mencioné anteriormente, matplotlib no es parte de la librería esencial de Python. Con el entorno virtual activado (para configurar uno, repase las instrucciones dadas en el apartado “Entornos virtuales” del capítulo 2), lo instalamos utilizando este comando:

```
python -m pip install matplotlib
```

Emplearemos el módulo `matplotlib.pyplot`. En su uso más sencillo, `pyplot` mantiene un estado interno en el que se crea una visualización paso a paso. En cuanto está lista, se puede guardar con `savefig` o mostrar con `show`.

Por ejemplo, hacer gráficos simples (como el de la figura 3.1) es bastante fácil:

```
from matplotlib import pyplot as plt
years = [1950, 1960, 1970, 1980, 1990, 2000, 2010]
gdp = [300.2, 543.3, 1075.9, 2862.5, 5979.6, 10289.7, 14958.3]
# crea un gráfico de líneas, años en el eje x, cantidades en el eje y
plt.plot(years, gdp, color='green', marker='o', linestyle='solid')
# añade un título
plt.title("Nominal GDP")
# añade una etiqueta al eje y
plt.ylabel("Billions of $")
plt.show()
```

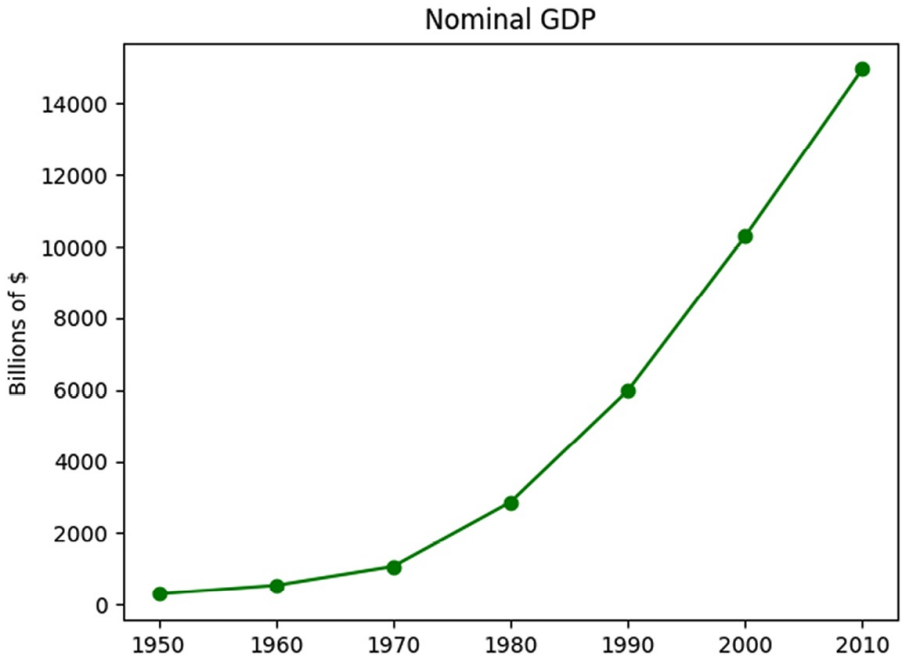


Figura 3.1. Un sencillo gráfico de líneas.

Crear gráficos con una calidad apta para publicaciones es más complicado,

y va más allá del objetivo de este capítulo. Hay muchas formas de personalizar los gráficos, por ejemplo, con etiquetas de ejes, estilos de línea y marcadores de puntos. En lugar de explicar estas opciones con todo detalle, simplemente utilizaremos algunas en nuestros ejemplos (y llamaré la atención sobre ello).

> **Nota:** Aunque no vayamos a utilizar mucho esta funcionalidad, matplotlib es capaz de producir complicados gráficos dentro de gráficos, aplicar formato de maneras sofisticadas y crear visualizaciones interactivas. En su documentación se puede encontrar información más detallada de la que ofrecemos en este libro.

Gráficos de barras

Un gráfico de barras es una buena elección cuando se desea mostrar cómo varía una cierta cantidad a lo largo de un conjunto discreto de elementos. Por ejemplo, la figura 3.2 muestra el número de Óscar que les fueron otorgados a cada una de una serie de películas:

```
movies = ["Annie Hall", "Ben-Hur", "Casablanca", "Gandhi", "West Side Story"]
num_oscars = [5, 11, 3, 8, 10]
# dibuja barras con coordenadas x de la izquierda [0, 1, 2, 3, 4], alturas
[num_oscars]
plt.bar(range(len(movies)), num_oscars)
plt.title("My Favorite Movies")                    # añade un título
plt.ylabel("# of Academy Awards")                  # etiqueta el eje y
# etiqueta el eje x con los nombres de las películas en el centro de las barras
plt.xticks(range(len(movies)), movies)
plt.show()
```

Un gráfico de barras también puede ser una buena opción para trazar histogramas de valores numéricos ordenados por cubos o *buckets*, como en la figura 3.3, con el fin de explorar visualmente el modo en que los valores están distribuidos:

```
from collections import Counter
```

```
grades = [83, 95, 91, 87, 70, 0, 85, 82, 100, 67, 73, 77, 0]
# Agrupa las notas en bucket por decil, pero pone 100 con los 90
histogram = Counter(min(grade // 10 * 10, 90) for grade in grades)
plt.bar([x + 5 for x in                    # Mueve barras a la derecha en 5
histogram.keys()],
     histogram.values(),                   # Da a cada barra su altura
                                           correcta
     10,                                   # Da a cada barra una anchura de 10
     edgecolor=(0, 0, 0))                  # Bordes negros para cada barra
plt.axis([-5, 105, 0, 5])                  # eje x desde -5 hasta 105,
                                           # eje y desde 0 hasta 5
plt.xticks([10 * i for i in                # etiquetas de eje x en 0, 10, ...,
range(11)])                                100
plt.xlabel("Decile")
plt.ylabel("# of Students")
plt.title("Distribution of Exam 1
Grades")
plt.show()
```

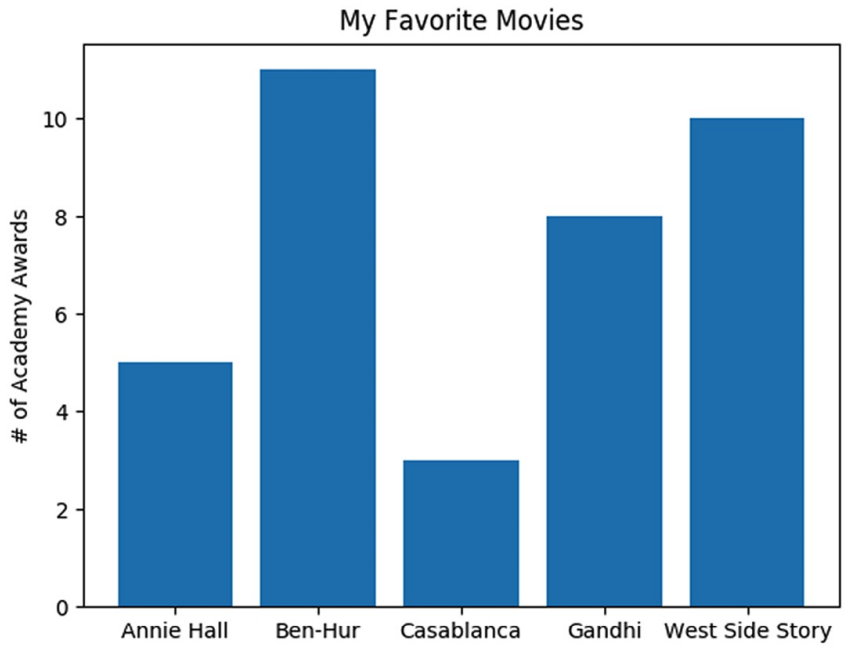


Figura 3.2. Un sencillo gráfico de barras.

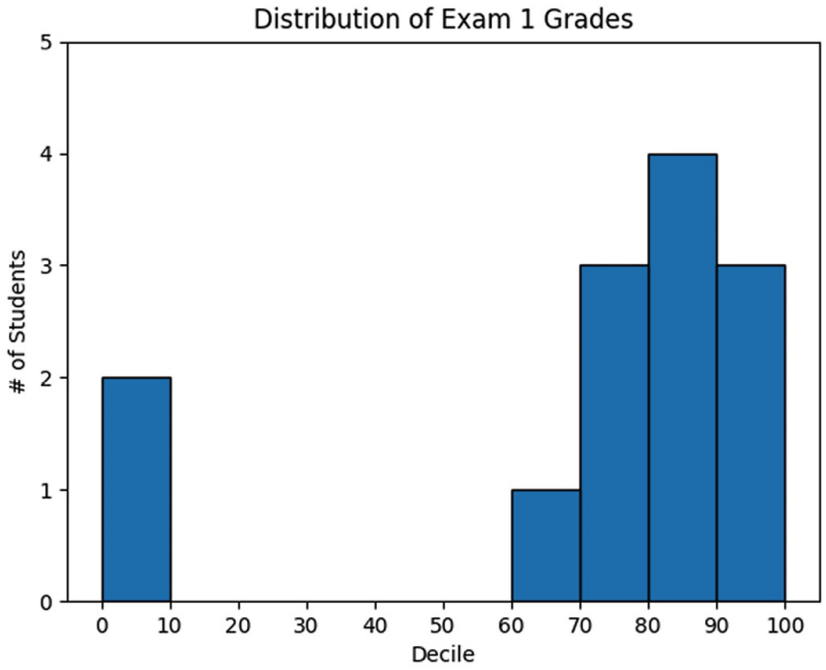


Figura 3.3. Utilizando un gráfico de barras para un histograma.

El tercer argumento de `plt.bar` especifica la anchura de las barras. Hemos elegido una anchura de 10, para llenar así todo el decil. También hemos desplazado las barras a la derecha en 5, de forma que, por ejemplo, la barra "10" (que corresponde al decil 10-20) tendría su centro en 15 y, por lo tanto, ocuparía el rango correcto. También añadimos un borde negro a cada barra para distinguirlas de forma visual.

La llamada a `plt.axis` indica que queremos que el eje x vaya desde -5 hasta 105 (solo para dejar un poco de espacio a la izquierda y a la derecha), y que el eje y varíe de 0 a 5; la llamada a `plt.xticks` pone las etiquetas del eje x en 0, 10, 20, ..., 100.

Conviene ser juiciosos al utilizar `plt.axis`. Cuando se crean gráficos de barras, está especialmente mal considerado que el eje y no empiece en 0, ya que de ese modo la gente se confunde con mucha facilidad (véase la figura 3.4):

```
mentions = [500, 505]
years = [2017, 2018]
plt.bar(years, mentions, 0.8)
```

```
plt.xticks(years)
plt.ylabel("# of times I heard someone say ‘data science’")
# si no se hace esto, matplotlib etiquetará el eje x con 0, 1
# y añadirá +2.013e3 en la esquina (¡qué malo es matplotlib!)
plt.ticklabel_format(useOffset=False)
# el eje y erróneo solo muestra la parte sobre 500
plt.axis([2016.5, 2018.5, 499, 506])
plt.title("Look at the ‘Huge’ Increase!")
plt.show()
```

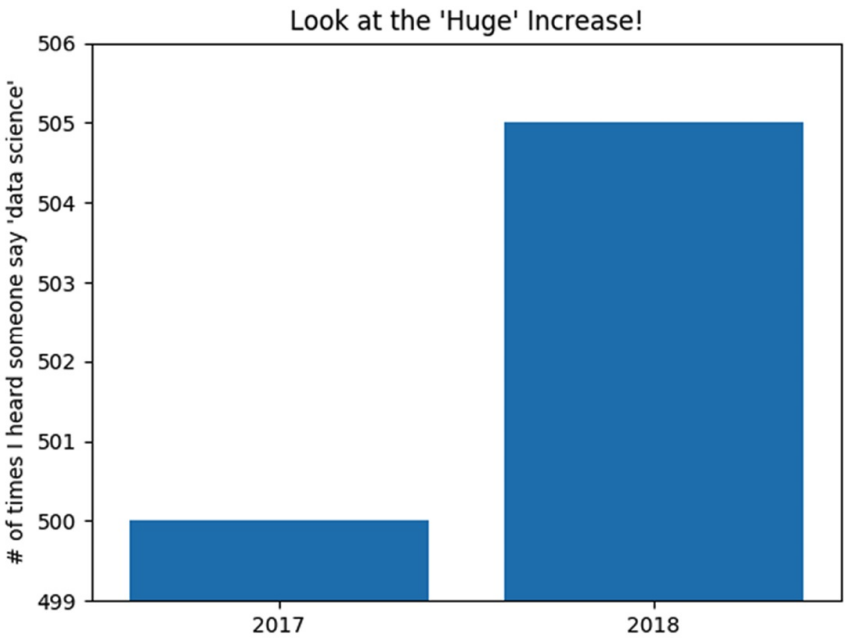


Figura 3.4. Un gráfico con el eje y erróneo.

En la figura 3.5 utilizamos ejes más sensatos, aunque así no queda tan impresionante:

```
plt.axis([2016.5, 2018.5, 0, 550])
plt.title("Not So Huge Anymore")
plt.show()
```

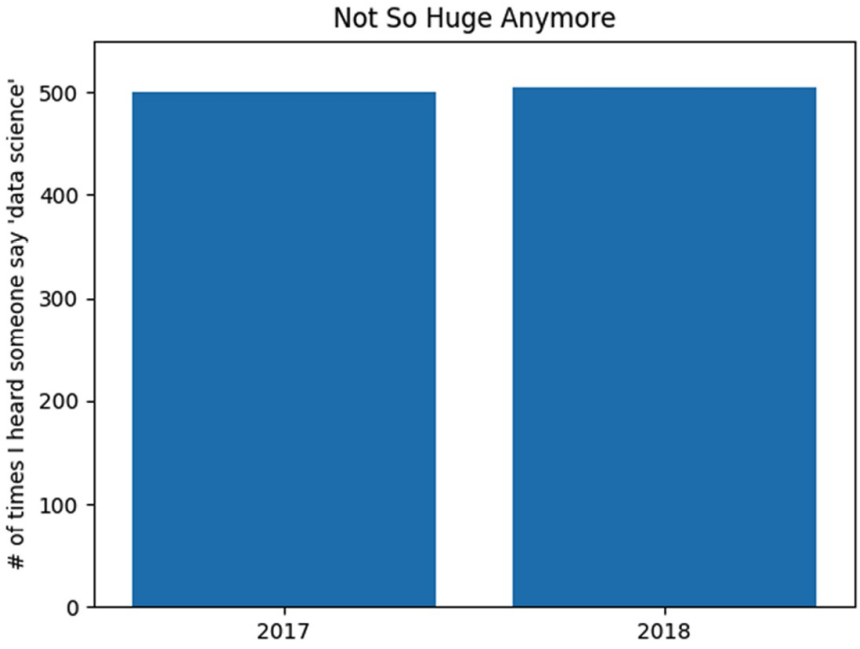


Figura 3.5. El mismo gráfico con un eje y nada confuso.

Gráficos de líneas

Como ya hemos visto, podemos hacer gráficos de líneas utilizando `plt.plot`. Son una buena elección para mostrar tendencias, como se ilustra en la figura 3.6:

```
variance = [1, 2, 4, 8, 16, 32, 64, 128, 256]
bias_squared = [256, 128, 64, 32, 16, 8, 4, 2, 1]
total_error = [x + y for x, y in zip(variance, bias_squared)]
xs = [i for i, _ in enumerate(variance)]
# Podemos hacer varias llamadas a plt.plot
# para mostrar varias series en el mismo gráfico
plt.plot(xs, variance, ‘g-’, label=’variance’)      # línea continua
plt.plot(xs, bias_squared, ‘r-.’, label=’bias^2’)   # línea de puntos y guiones
plt.plot(xs, total_error, ‘b:’, label=’total error’)# línea de puntos
# Como asignamos etiquetas a cada serie,
# obtenemos una leyenda gratis (loc=9 significa "arriba centro")
plt.legend(loc=9)
```

```
plt.xlabel("model complexity")
plt.xticks([])
plt.title("The Bias-Variance Tradeoff")
plt.show()
```

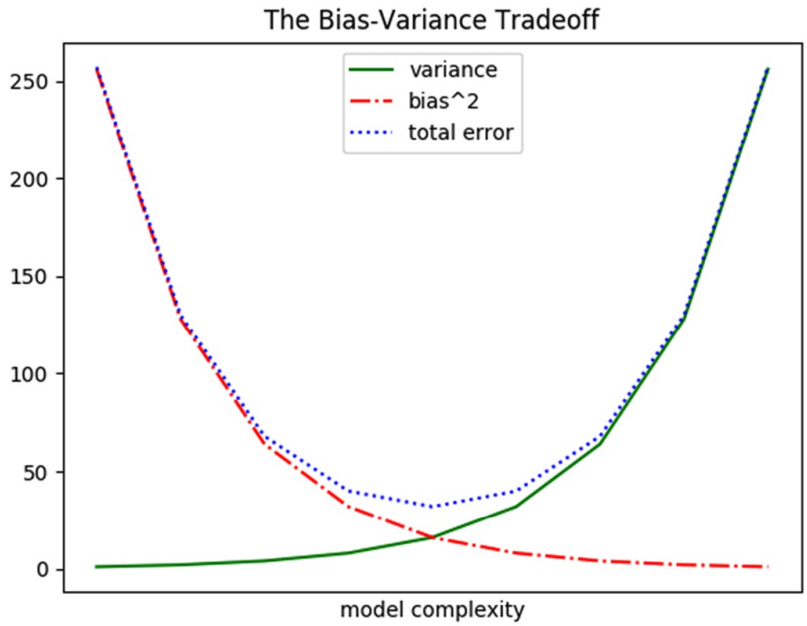


Figura 3.6. Varios gráficos de líneas con una leyenda.

Gráficos de dispersión

Un gráfico de dispersión es la opción adecuada para visualizar la relación entre dos conjuntos de datos emparejados. Por ejemplo, la figura 3.7 ilustra la relación entre el número de amigos que tienen sus usuarios y el número de minutos que pasan cada día en el sitio:

```
friends = [ 70, 65, 72, 63, 71, 64, 60, 64, 67]
minutes = [175, 170, 205, 120, 220, 130, 105, 145, 190]
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
plt.scatter(friends, minutes)
# etiqueta cada punto
for label, friend_count, minute_count in zip(labels, friends, minutes):
   plt.annotate(label,
```

```
    xy=(friend_count, minute_count),      # Pone la etiqueta con su punto
    xytext=(5, -5),                       # pero un poco desplazada
    textcoords='offset points')
plt.title("Daily Minutes vs. Number of Friends")
plt.xlabel("# of friends")
plt.ylabel("daily minutes spent on the site")
plt.show()
```

Si estamos representando variables comparables, podríamos obtener una imagen confusa si dejáramos que matplotlib eligiera la escala, como en la figura 3.8.

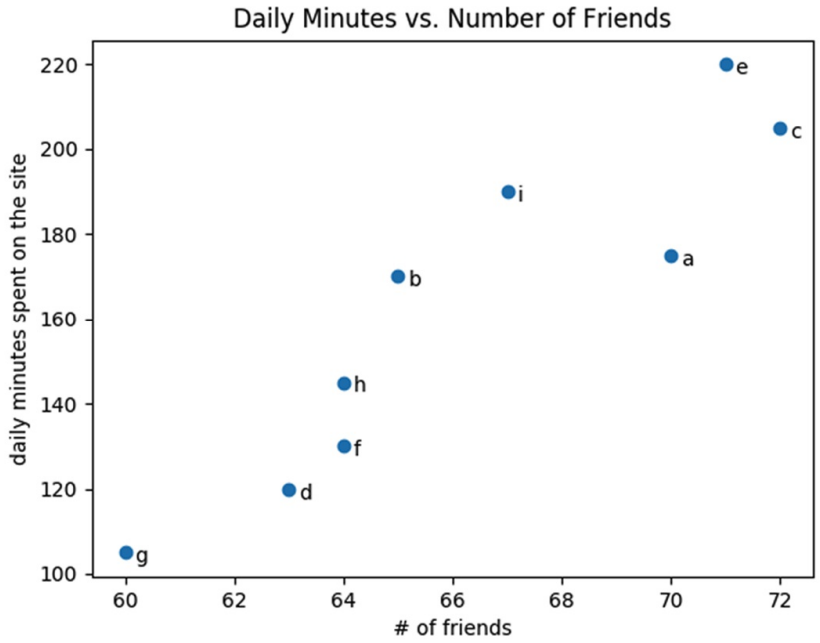


Figura 3.7. Un gráfico de dispersión de amigos y tiempo en el sitio.

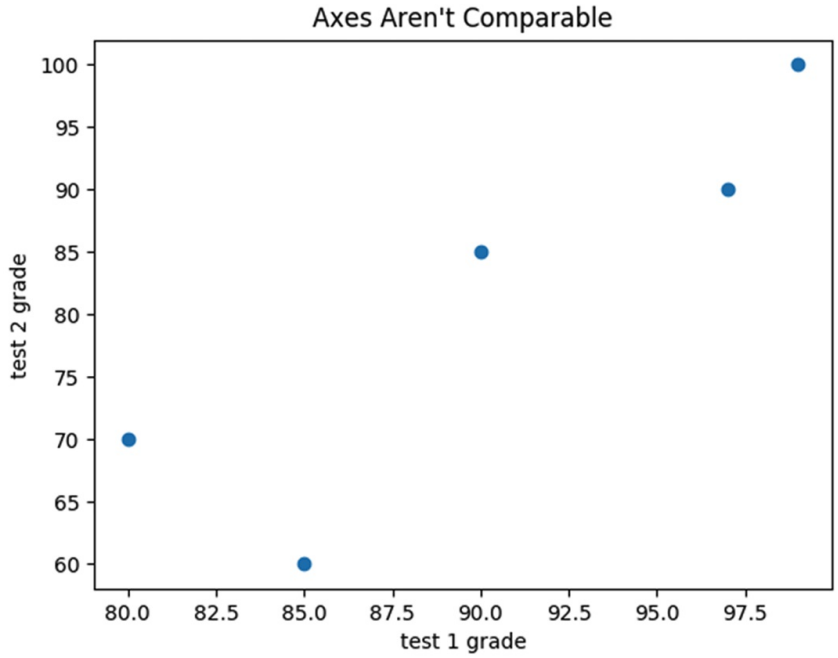


Figura 3.8. Un gráfico de dispersión con ejes imposibles de comparar.

```
test_1_grades = [ 99, 90, 85, 97, 80]
test_2_grades = [100, 85, 60, 90, 70]
plt.scatter(test_1_grades, test_2_grades)
plt.title("Axes Aren’t Comparable")
plt.xlabel("test 1 grade")
plt.ylabel("test 2 grade")
plt.show()
```

Si incluimos una llamada a `plt.axis(“equal”)`, el gráfico (figura 3.9) muestra con mayor precisión que la mayor parte de la variación tiene lugar en la prueba 2.

Con esto basta para empezar con la visualización. Aprenderemos mucho más sobre la visualización a lo largo del libro.

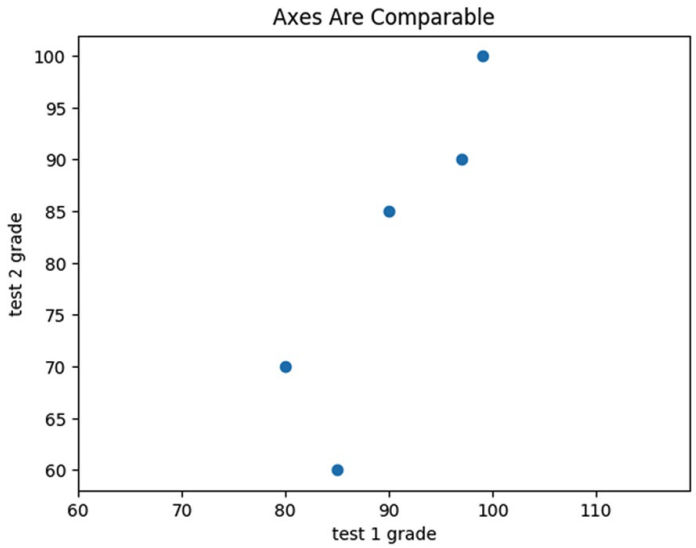


Figura 3.9. El mismo gráfico de dispersión con ejes iguales.

Para saber más

- La galería de matplotlib, en `https://matplotlib.org/stable/gallery/index.html`, da una idea bastante buena del tipo de cosas que se pueden hacer con matplotlib (y de cómo hacerlas).
- seaborn, en `https://seaborn.pydata.org/`, se ha creado en base a matplotlib y permite producir fácilmente visualizaciones más bonitas (y complejas).
- Altair, en `https://altair-viz.github.io/`, es una nueva librería de Python para crear visualizaciones declarativas.
- D3.js, en `https://d3js.org`, es una librería de JavaScript para producir sofisticadas visualizaciones interactivas para la web. Aunque no está en Python, se utiliza mucho y vale la pena familiarizarse con ella.
- Bokeh, en `https://bokeh.pydata.org`, es una librería que permite incorporar a Python visualizaciones de estilo D3.

[1] https://matplotlib.org/.

4 Álgebra lineal

¿Hay algo más inútil o menos útil que el álgebra?

—Billy Conolly

El álgebra lineal es la rama de las matemáticas que se ocupa de los espacios vectoriales. Aunque no espero que el lector aprenda álgebra lineal en un breve capítulo, se apoya en un gran número de conceptos y técnicas de ciencia de datos, lo que significa que al menos les debo un intento. Lo que vamos a aprender en este capítulo lo utilizaremos mucho a lo largo del libro.

Vectores

Definidos de una forma abstracta, los vectores son objetos que se pueden sumar para formar nuevos vectores y se pueden multiplicar por escalares (es decir, números), también para formar nuevos vectores.

De una forma más concreta (para nosotros), digamos que los vectores son puntos de un espacio de dimensión finita. Aunque no se nos suele ocurrir pensar en los datos como vectores, a menudo son una forma útil de representar datos numéricos.

Por ejemplo, si tenemos las alturas, pesos y edades de un gran número de personas, podemos tratar los datos como vectores tridimensionales `[height, weight, age]`. Si tuviéramos una clase con cuatro exámenes, podríamos tratar las notas de los alumnos como vectores de cuatro dimensiones `[exam1, exam2, exam3, exam4]`.

El enfoque más sencillo para aprender esto desde cero es representar los vectores como una lista de números. Una lista de tres números corresponde a un vector en un espacio tridimensional y viceversa.

Realizaremos esto con un alias de tipo que dice que un `Vector` es solo una `list` de valores de tipo `float`:

```
from typing import List
Vector = List[float]
height_weight_age = [70,         # pulgadas,
                     170,        # libras,
                     40 ]        # años
grades = [95,      # examen1
80,                # examen2
75,                # examen3
62 ]               # examen4
```

También querremos realizar aritmética con los vectores. Como las `list` de Python no son vectores (y por lo tanto no dan facilidades para la aritmética con vectores), tendremos que crear nosotros mismos estas herramientas de aritmética. Así que vamos allá.

Para empezar, muy a menudo necesitaremos sumar dos vectores. Los vectores se suman componente a componente, lo que significa que, si dos vectores `v` y `w` tienen la misma longitud, su suma es sencillamente el vector cuyo primer elemento es `v[0] + w[0]`, cuyo segundo elemento es `v[1] + w[1]`, y así sucesivamente (si no tienen la misma longitud, entonces no se pueden sumar). Por ejemplo, sumar los vectores `[1, 2]` y `[2, 1]` da como resultado `[1 + 2, 2 + 1]` o `[3,3]`, como muestra la figura 4.1.

Podemos implementar esto fácilmente comprimiendo los vectores con `zip` y utilizando una comprensión de lista para sumar los elementos correspondientes:

```
def add(v: Vector, w: Vector) -> Vector:
   """Adds corresponding elements"""
   assert len(v) == len(w), "vectors must be the same length"
   return [v_i + w_i for v_i, w_i in zip(v, w)]
assert add([1, 2, 3], [4, 5, 6]) == [5, 7, 9]
```

De forma similar, para restar dos vectores simplemente restamos los elementos correspondientes:

```
def subtract(v: Vector, w: Vector) -> Vector:
   """Subtracts corresponding elements"""
   assert len(v) == len(w), "vectors must be the same length"
   return [v_i–w_i for v_i, w_i in zip(v, w)]
```

```
assert subtract([5, 7, 9], [4, 5, 6]) == [1, 2, 3]
```

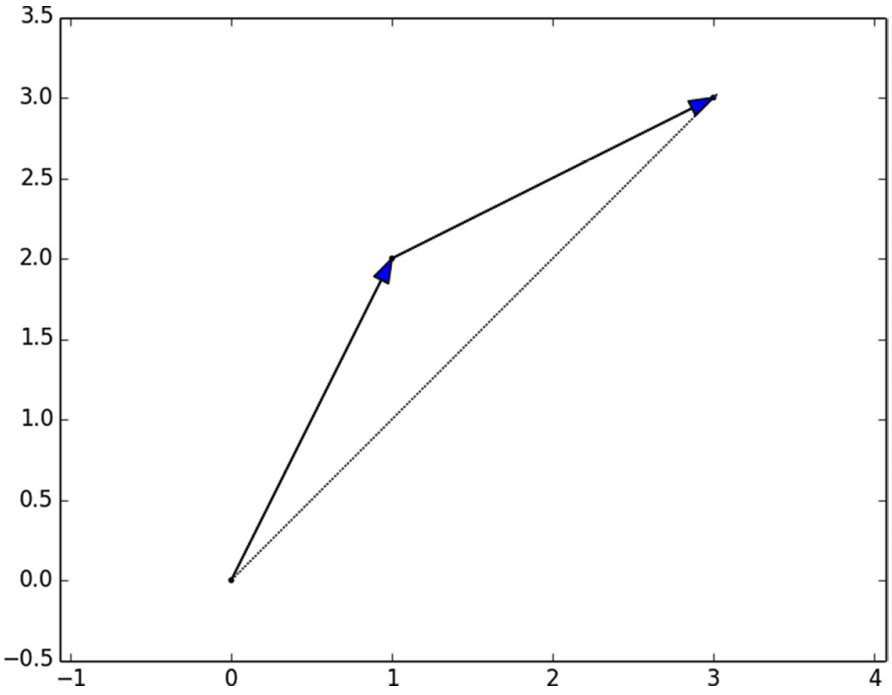


Figura 4.1. Sumando dos vectores.

También querremos en ocasiones sumar una lista de vectores por componentes (es decir, crear un nuevo vector cuyo primer elemento es la suma de todos los primeros elementos y cuyo segundo elemento es la suma de todos los segundos elementos, y así sucesivamente):

```
def vector_sum(vectors: List[Vector]) -> Vector:
   """Sums all corresponding elements"""
   # Comprueba que los vectores no estén vacíos
   assert vectors, "no vectors provided!"
   # Comprueba que los vectores tienen el mismo tamaño
   num_elements = len(vectors[0])
   assert all(len(v) == num_elements for v in vectors), "different sizes!"
   # el elemento i del resultado es la suma de cada vector [i]
   return [sum(vector[i] for vector in vectors)
        for i in range(num_elements)]
assert vector_sum([[1, 2], [3, 4], [5, 6], [7, 8]]) == [16, 20]
```

También tendremos que ser capaces de multiplicar un vector por un escalar, cosa que hacemos sencillamente multiplicando cada elemento del

vector por dicho número:

```
def scalar_multiply(c: float, v: Vector) -> Vector:
     """Multiplies every element by c"""
     return [c * v_i for v_i in v]
assert scalar_multiply(2, [1, 2, 3]) == [2, 4, 6]
```

Esto nos permite calcular la media por componentes de una lista de vectores (del mismo tamaño):

```
def vector_mean(vectors: List[Vector]) -> Vector:
   """Computes the element-wise average"""
   n = len(vectors)
   return scalar_multiply(1/n, vector_sum(vectors))
assert vector_mean([[1, 2], [3, 4], [5, 6]]) == [3, 4]
```

Una herramienta menos obvia es el producto punto. El producto punto de dos vectores es la suma de los productos de sus componentes:

```
def dot(v: Vector, w: Vector) -> float:
   """Computes v_1 * w_1 + ... + v_n * w_n"""
   assert len(v) == len(w), "vectors must be same length"
   return sum(v_i * w_i for v_i, w_i in zip(v, w))
assert dot([1, 2, 3], [4, 5, 6]) == 32     # 1 * 4 + 2 * 5 + 3 * 6
```

Si `w` tiene magnitud 1, el producto punto mide hasta dónde se extiende el vector `v` en la dirección `w`. Por ejemplo, si `w = [1, 0]`, entonces `dot(v, w)` es el primer componente de `v`. Otra forma de decir esto es que es la longitud del vector que se obtendría si se proyectara `v` en `w` (véase la figura 4.2).

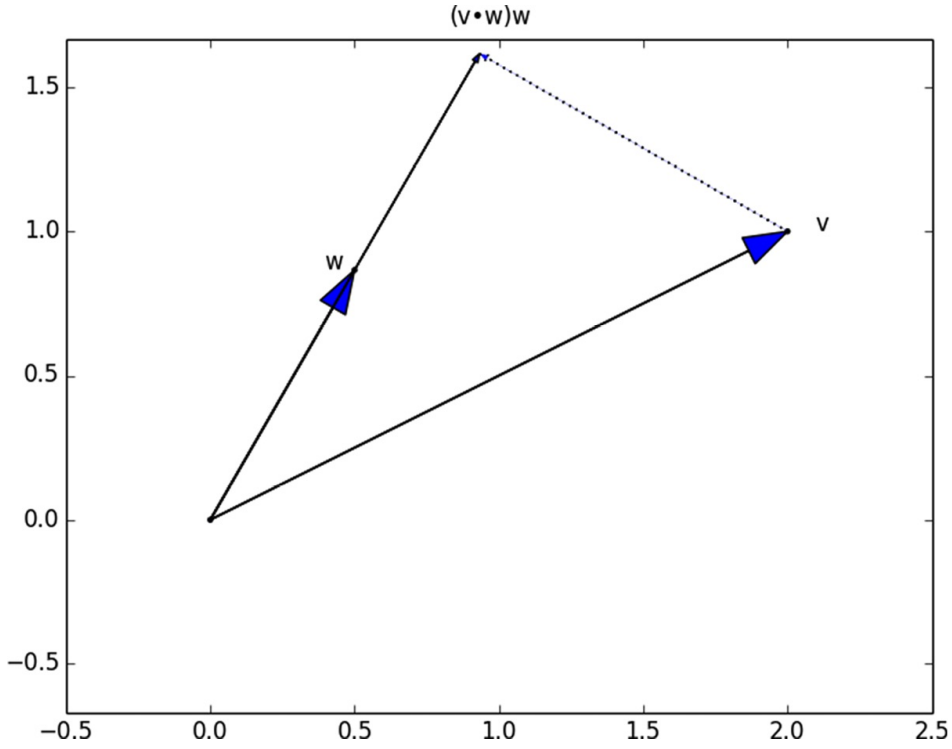


Figura 4.2. El producto punto como proyección de vector.

Utilizando esto, es fácil calcular la suma de cuadrados de un vector:

```
def sum_of_squares(v: Vector) -> float:
   """Returns v_1 * v_1 + ... + v_n * v_n"""
   return dot(v, v)
assert sum_of_squares([1, 2, 3]) == 14 # 1 * 1 + 2 * 2 + 3 * 3
```

Que podemos utilizar para calcular su magnitud (o longitud):

```
import math
def magnitude(v: Vector) -> float:
     """Returns the magnitude (or length) of v"""
     return                                  # math.sqrt es la función raíz
     math.sqrt(sum_of_squares(v))            cuadrada
assert magnitude([3, 4]) == 5
```

Ahora tenemos todas las piezas necesarias para calcular la distancia entre dos vectores, definida como:

$$\sqrt{(v_1 - w_1)^2 + ... + (v_n - w_n)^2}$$

En código:

```
def squared_distance(v: Vector, w: Vector) -> float:
   """Computes (v_1-w_1) ** 2 + ... + (v_n-w_n) ** 2"""
   return sum_of_squares(subtract(v, w))
def distance(v: Vector, w: Vector) -> float:
   """Computes the distance between v and w"""
   return math.sqrt(squared_distance(v, w))
```

Quizá esto quede más claro si lo escribimos como (el equivalente):

```
def distance(v: Vector, w: Vector) -> float:
   return magnitude(subtract(v, w))
```

Esto debería ser suficiente para empezar. Vamos a utilizar muchísimo estas funciones a lo largo del libro.

> **Nota:** Utilizar listas como vectores es excelente de cara a la galería, pero terrible para el rendimiento.
> En código de producción, nos interesará más utilizar la librería NumPy, que incluye una clase de objetos *array* de alto rendimiento con todo tipo de operaciones aritméticas incluidas.

Matrices

Una matriz es una colección de números bidimensional. Representaremos las matrices como listas de listas, teniendo cada lista interior el mismo tamaño y representando una fila de la matriz. Si `A` es una matriz, entonces `A[i][j]` es el elemento de la fila *i* y la columna *j*. Por convenio matemático, utilizaremos con frecuencia letras mayúsculas para representar matrices. Por ejemplo:

```
# Otro alias de tipo
Matrix = List[List[float]]
A = [[1, 2, 3],      # A tiene 2 filas y 3 columnas
   [4, 5, 6]]
B = [[1, 2],         # B tiene 3 filas y 2 columnas
   [3, 4],
   [5, 6]]
```

Nota: En matemáticas, se suele denominar a la primera fila de la matriz “fila 1” y a la primera columna “columna 1”. Como estamos representando matrices con `list` de Python, que están indexadas al 0, llamaremos a la primera fila de la matriz “fila 0” y a la primera columna “columna 0”.

Dada esta representación de lista de listas, la matriz `A` tiene `len(A)` filas y `len(A[0])` columnas, lo que consideramos su `shape`:

```
from typing import Tuple
def shape(A: Matrix) -> Tuple[int, int]:
   """Returns (# of rows of A,               # of columns of A)"""
   num_rows = len(A)
   num_cols = len(A[0]) if A else 0          # número de elementos de la
                                             primera fila
   return num_rows, num_cols
assert shape([[1, 2, 3], [4, 5, 6]]) ==      # 2 filas, 3 columnas
(2, 3)
```

Si una matriz tiene *n* filas y *k* columnas, nos referiremos a ella como una matriz *n* x *k*. Podemos (y a veces lo haremos) pensar en cada fila de una matriz *n* x *k* como un vector de longitud *k*, y en cada columna como en un vector de longitud *n*:

```
def get_row(A: Matrix, i: int) -> Vector:
   """Returns the i-th row of A (as a Vector)"""
   return A[i]            # A[i] es ya la fila i
def get_column(A: Matrix, j: int) -> Vector:
   """Returns the j-th column of A (as a Vector)"""
   return [A_i[j]         # elemento j de la fila A_i
        for A_i in A]     # para cada fila A_i
```

También querremos poder crear una matriz dada su forma y una función

para generar sus elementos. Podemos hacer esto utilizando una comprensión de lista anidada:

```
from typing import Callable
def make_matrix(num_rows: int,
                num_cols: int,
                entry_fn: Callable[[int, int], float]) -> Matrix:
   """
   Returns a num_rows x num_cols matrix
   whose (i,j)-th entry is entry_fn(i, j)
   """
   return [[entry_fn(i, j)             # dado i, crea una lista
        for j in range(num_cols)]       # [entry_fn(i, 0), ... ]
        for i in range(num_rows)]      # crea una lista por cada i
```

Dada esta función, se podría crear una matriz identidad 5 x 5 (con unos en la diagonal y ceros en el resto) de esta forma:

```
def identity_matrix(n: int) -> Matrix:
   """Returns the n x n identity matrix"""
   return make_matrix(n, n, lambda i, j: 1 if i == j else 0)
assert identity_matrix(5) ==          [[1, 0, 0, 0, 0],
                                       [0, 1, 0, 0, 0],
                                       [0, 0, 1, 0, 0],
                                       [0, 0, 0, 1, 0],
                                       [0, 0, 0, 0, 1]]
```

Las matrices serán importantes para nosotros por varias razones.

En primer lugar, podemos utilizar una matriz para representar un conjunto de datos formado por varios vectores, simplemente considerando cada vector como una fila de la matriz. Por ejemplo, si tuviéramos las alturas, pesos y edades de 1.000 personas, podríamos poner estos datos en una matriz 1.000 x 3:

```
data = [[70, 170, 40],
        [65, 120, 26],
        [77, 250, 19],
        # ....
      ]
```

En segundo lugar, como veremos más tarde, podemos utilizar una matriz *n*

x *k* para representar una función lineal que transforme vectores de *k* dimensiones en vectores de *n* dimensiones. Varias de nuestras técnicas y conceptos implicarán tales funciones.

Tercero, las matrices se pueden utilizar para representar relaciones binarias. En el capítulo 1, representamos los bordes de una red como una colección de pares `(i, j)`. Una representación alternativa sería crear una matriz `A` tal que `A[i][j]` sea 1 si los nodos `i` y `j` están conectados y 0 en otro caso.

Recordemos que antes teníamos:

```
friendships = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3), (3, 4),
          (4, 5), (5, 6), (5, 7), (6, 8), (7, 8), (8, 9)]
```

También podemos representar esto como:

```
#                                          usuario 0 1 2 3 4 5 6 7 8 9
#
friend_matrix = [[0, 1, 1, 0, 0, 0, 0, 0, 0, 0],          # usuario 0

             [1, 0, 1, 1, 0, 0, 0, 0, 0, 0],           # usuario 1
             [1, 1, 0, 1, 0, 0, 0, 0, 0, 0],           # usuario 2
             [0, 1, 1, 0, 1, 0, 0, 0, 0, 0],           # usuario 3
             [0, 0, 0, 1, 0, 1, 0, 0, 0, 0],           # usuario 4
             [0, 0, 0, 0, 1, 0, 1, 1, 0, 0],           # usuario 5
             [0, 0, 0, 0, 0, 1, 0, 0, 1, 0],           # usuario 6
             [0, 0, 0, 0, 0, 1, 0, 0, 1, 0],           # usuario 7
             [0, 0, 0, 0, 0, 0, 1, 1, 0, 1],           # usuario 8
             [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]]           # usuario 9
```

Si hay pocas conexiones, esta representación es mucho menos eficaz, ya que terminamos teniendo que almacenar muchos ceros. No obstante, con la representación en matriz se comprueba mucho más rápido si dos nodos están conectados (basta con hacer una búsqueda en la matriz en lugar de, posiblemente, inspeccionar cada borde):

```
assert friend_matrix[0][2] == 1, "0 and 2 are friends"
assert friend_matrix[0][8] == 0, "0 and 8 are not friends"
```

De forma similar, para encontrar las conexiones de un nodo, basta con inspeccionar la columna (o la fila) correspondiente a ese nodo:

```
# basta con mirar en una fila
friends_of_five = [i
             for i, is_friend in enumerate(friend_matrix[5])
             if is_friend]
```

Con un gráfico de pequeño tamaño se podría añadir una lista de conexiones a cada objeto de nodo para acelerar este proceso, pero, con uno más grande y en constante evolución, hacer esto sería probablemente demasiado caro y difícil de mantener.

Revisaremos las matrices a lo largo del libro.

Para saber más

- Los científicos de datos utilizan mucho el álgebra lineal (lo que se da por sentado con frecuencia, aunque no con tanta por personas que no lo comprenden). No sería mala idea leer un libro de texto. Se pueden encontrar varios disponibles gratuitamente en línea:
  - *Linear Algebra*, en http://joshua.smcvt.edu/linearalgebra/, de Jim Hefferon (Saint Michael's College).
  - *Linear Algebra*, en https://www.math.ucdavis.edu/~linear/linear-guest.pdf, de David Cherney, Tom Denton, Rohit Thomas y Andrew Waldron (UC Davis).
  - Si se siente aventurero, *Linear Algebra Done Wrong*, en https://www.math.brown.edu/~treil/papers/LADW/LADW_2017-09-04.pdf, de Sergei Treil (Brown University), es una introducción más avanzada.
- Toda la maquinaria que hemos creado en este capítulo se puede conseguir de forma gratuita utilizando NumPy, en http://www.numpy.org (también se obtiene mucho más, incluyendo mucho mejor rendimiento).

5 Estadística

Los hechos son obstinados, pero las estadísticas son más maleables.

—Mark Twain

El término estadística hace referencia a las matemáticas y a las técnicas con las que comprendemos los datos. Es un campo rico y extenso, más adecuado para una estantería (o una sala entera) de una biblioteca que para un capítulo de un libro, de modo que necesariamente mi exposición no podrá ser profunda. Trataré de enseñar lo justo para que resulte comprometido y active el interés lo suficiente como para seguir adelante y aprender aún más.

Describir un solo conjunto de datos

Mediante una combinación de boca a boca y suerte, DataSciencester ha crecido y ahora tiene muchísimos miembros, y el vicepresidente de Recaudación de fondos quiere algún tipo de descripción de la cantidad de amigos que tienen sus miembros para poder incluirla en sus discursos de presentación.

Utilizando las técnicas del capítulo 1, es muy sencillo producir estos datos. Pero ahora nos enfrentamos al problema de cómo describirlos. Una descripción obvia de cualquier conjunto de datos es sencillamente los propios datos:

```
num_friends = [100, 49, 41, 40, 25,
          # ... y muchos más
          ]
```

Con un conjunto de datos bastante pequeño, esta podría ser la mejor descripción. Pero, con otro más grande, resulta difícil de manejar y probablemente opaco (imagínese tener que mirar fijamente una lista de 1

millón de números). Por esta razón, utilizamos las estadísticas para sintetizar y comunicar las características relevantes de nuestros datos. Como primer enfoque, ponemos los contadores de amigos en un histograma utilizando `Counter` y `plt.bar` (véase la figura 5.1):

```
from collections import Counter
import matplotlib.pyplot as plt
friend_counts = Counter(num_friends)
xs = range(101)                         # el valor mayor es 100
ys = [friend_counts[x] for x in         # la altura es justamente núm. de
xs]                                     amigos
plt.bar(xs, ys)
plt.axis([0, 101, 0, 25])
plt.title("Histogram of Friend Counts")
plt.xlabel("# of friends")
plt.ylabel("# of people")
plt.show()
```

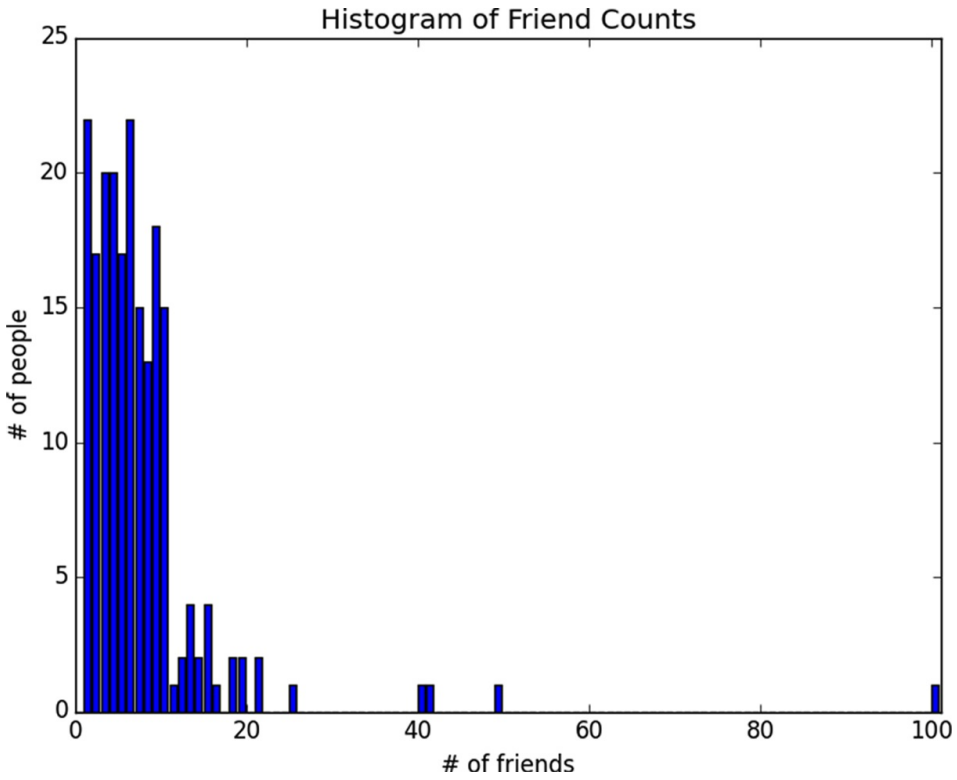


Figura 5.1. Un histograma de contadores de amigos.

Por desgracia, sigue siendo demasiado difícil meter este gráfico en las conversaciones. De modo que empezamos a generar algunas estadísticas;

probablemente la más sencilla es el número de puntos de datos:

```
num_points = len(num_friends)      # 204
```

Probablemente también estemos interesados en los valores mayor y menorww:

```
largest_value = max(num_friends)       # 100
smallest_value = min(num_friends)      # 1
```

Que son simplemente casos especiales de querer saber cuáles son los valores de determinadas posiciones:

```
sorted_values = sorted(num_friends)
smallest_value = sorted_values[0]             # 1
second_smallest_value = sorted_values[1]      # 1
second_largest_value = sorted_values[-2]      # 49
```

Pero solo estamos empezando.

Tendencias centrales

Normalmente, querremos tener alguna noción del lugar en el que nuestros datos están centrados. Lo más habitual es que usemos la media (o promedio), que no es más que la suma de los datos dividida por el número de datos:

```
def mean(xs: List[float]) -> float:
   return sum(xs) / len(xs)
mean(num_friends)       # 7,333333
```

Si tenemos dos puntos de datos, la media es simplemente el punto a mitad de camino entre los dos. A medida que se añaden más puntos, la media se va desplazando, pero siempre depende del valor de cada punto. Por ejemplo, si tenemos 10 puntos de datos y aumentamos el valor de cualquiera de ellos en 1, la media aumenta en 0,1.

También estaremos interesados en ocasiones en la mediana, que es el valor más céntrico (si el número de puntos de datos es impar) o el promedio

de los dos valores más céntricos (si el número de puntos de datos es par).

Por ejemplo, si tenemos cinco puntos de datos en un vector ordenado `x`, la mediana es `x[5 // 2]` o `x[2]`. Si tenemos seis puntos de datos, queremos el promedio de `x[2]` (el tercer punto) y `x[3]` (el cuarto punto).

Tengamos en cuenta que (a diferencia de la media) la mediana no depende por completo de cada valor de los datos. Por ejemplo, si el punto más grande lo hacemos aún mayor (o el punto más pequeño menor), los puntos medios no cambian, lo que significa que la mediana sí lo hace.

Escribiremos distintas funciones para los casos par e impar y las combinaremos:

```
# Los guiones bajos indican que son funciones "privadas", destinadas a
# ser llamadas por nuestra función mediana pero no por otras personas
# que utilicen nuestra librería de estadísticas.
def _median_odd(xs: List[float]) -> float:
   """If len(xs) is odd, the median is the middle element"""
   return sorted(xs)[len(xs) // 2]
def _median_even(xs: List[float]) -> float:
   """If len(xs) is even, it's the average of the middle two elements"""
   sorted_xs = sorted(xs)
   hi_midpoint = len(xs) // 2      # p.ej. longitud 4 => hi_midpoint 2
   return (sorted_xs[hi_midpoint–1] + sorted_xs[hi_midpoint]) / 2
def median(v: List[float]) -> float:
   """Finds the 'middle-most' value of v"""
   return _median_even(v) if len(v) % 2 == 0 else _median_odd(v)
assert median([1, 10, 2, 9, 5]) == 5
assert median([1, 9, 2, 10]) == (2 + 9) / 2
```

Y ahora podemos calcular el número medio de amigos:

```
print(median(num_friends))     # 6
```

Sin duda, la media es más sencilla de calcular y varía ligeramente cuando nuestros datos cambian. Si tenemos *n* puntos de datos y uno de ellos aumenta en una cierta pequeña cantidad *e*, entonces la media necesariamente aumentará en *e* / *n* (lo que consigue que la media sea susceptible a todo tipo de trucos de cálculo). Pero, para hallar la mediana, tenemos que ordenar los datos. Y cambiar uno de nuestros puntos de datos en una pequeña cantidad *e*

podría aumentar la mediana también en *e*, en una cantidad inferior a *e* o en nada en absoluto (dependiendo del resto de datos).

> **Nota:** En realidad, existen trucos no evidentes para calcular medianas eficazmente[1] sin ordenar los datos. Sin embargo, están más allá del objetivo de este libro, de modo que toca ordenar los datos.

Al mismo tiempo, la media es muy sensible a los valores atípicos de nuestros datos. Si nuestro usuario más sociable tuviera 200 amigos (en lugar de 100), entonces la media subiría a 7,82, mientras que la mediana seguiría siendo la misma. Si es probable que los valores atípicos sean datos erróneos (o no representativos del fenómeno que estemos tratando de comprender), entonces la media puede darnos a veces una imagen equívoca. Por ejemplo, a menudo se cuenta la historia de que, a mediados de los años 80, la asignatura de la Universidad de Carolina del Norte con el salario inicial medio más alto era geografía, principalmente debido a la estrella de la NBA (y valor atípico) Michael Jordan.

Una generalización de la mediana es el cuantil, que representa el valor bajo el cual reside un determinado percentil de los datos (la mediana representa el valor bajo el cual reside el 50 % de los datos):

```
def quantile(xs: List[float], p: float) -> float:
   """Returns the pth-percentile value in x"""
   p_index = int(p * len(xs))
   return sorted(xs)[p_index]
assert quantile(num_friends, 0.10) == 1
assert quantile(num_friends, 0.25) == 3
assert quantile(num_friends, 0.75) == 9
assert quantile(num_friends, 0.90) == 13
```

Menos habitual sería que quisiéramos mirar la moda, es decir, el valor o valores más comunes:

```
def mode(x: List[float]) -> List[float]:
   """Returns a list, since there might be more than one mode"""
   counts = Counter(x)
   max_count = max(counts.values())
```

```
    return [x_i for x_i, count in counts.items()
            if count == max_count]
assert set(mode(num_friends)) == {1, 6}
```

Pero lo más habitual es que utilicemos la media.

Dispersión

Dispersión se refiere a las medidas de dispersión de nuestros datos. Normalmente son estadísticas para las que los valores cercanos a cero significan “no disperso en absoluto” y para las que los valores grandes (lo que sea que eso signifique) quieren decir “muy disperso”. Por ejemplo, una medida muy sencilla es el rango, que no es otra cosa que la diferencia entre los elementos mayor y menor:

```
# "range" ya significa algo en Python, así que usaremos otro nombre
def data_range(xs: List[float]) -> float:
    return max(xs)-min(xs)
assert data_range(num_friends) == 99
```

El rango es cero precisamente cuando el `max` y el `min` son iguales, cosa que solo puede ocurrir si los elementos de `x` son todos iguales, lo que significa que los datos están tan poco dispersos como es posible. A la inversa, si el rango es grande, entonces el `max` es mucho más grande que el `min` y los datos están más dispersos.

Al igual que ocurre con la mediana, en realidad el rango no depende del conjunto de datos entero. Un conjunto de datos cuyos puntos son todos 0 o 100 tiene el mismo rango que otro cuyos valores sean 0, 100 y muchos 50. Pero parece que el primer conjunto de datos “debería” estar más disperso.

Una medida más compleja de dispersión es la varianza, que se calcula como:

```
from scratch.linear_algebra import sum_of_squares
def de_mean(xs: List[float]) -> List[float]:
    """Translate xs by subtracting its mean (so the result has mean 0)"""
    x_bar = mean(xs)
    return [x-x_bar for x in xs]
```

```
def variance(xs: List[float]) -> float:
   """Almost the average squared deviation from the mean"""
   assert len(xs) >= 2, "variance requires at least two elements"
   n = len(xs)
   deviations = de_mean(xs)
   return sum_of_squares(deviations) / (n–1)
assert 81.54 < variance(num_friends) < 81.55
```

Nota: Parece que esto sea casi la desviación cuadrática promedio respecto a la media, salvo que estamos dividiendo por `n - 1` en lugar de por `n`. De hecho, cuando tratamos con una muestra de una población mayor, `x_bar` es solamente una estimación de la media real, lo que significa que en promedio `(x_i - x_bar) ** 2` es una subestimación de la desviación cuadrática de `x_i` con respecto a la media, razón por la cual dividimos por `n - 1` en lugar de por `n`. Consulte la Wikipedia.[2]

Ahora, sin importar en qué unidades estén nuestros datos (por ejemplo, “amigos”), todas nuestras medidas de tendencia central están en la misma unidad, igual que el rango. Pero la varianza, por otro lado, tiene unidades que son el cuadrado de las originales (es decir, “amigos al cuadrado”). Como puede resultar difícil darle sentido a esto, a menudo recurrimos en su lugar a la desviación estándar:

```
import math
def standard_deviation(xs: List[float]) -> float:
   """The standard deviation is the square root of the variance"""
   return math.sqrt(variance(xs))
assert 9.02 < standard_deviation(num_friends) < 9.04
```

Tanto el rango como la desviación estándar tienen el mismo problema de valor atípico que vimos antes con la media. Utilizando el mismo ejemplo, si nuestro usuario más sociable tuviera realmente 200 amigos, la desviación estándar sería 14,89 (¡más del 60 % más elevada!).

Una alternativa más robusta calcula la diferencia entre los percentiles 75 y 25:

```
def interquartile_range(xs: List[float]) -> float:
   """Returns the difference between the 75%-ile and the 25%-ile"""
   return quantile(xs, 0.75)–quantile(xs, 0.25)
```

```
assert interquartile_range(num_friends) == 6
```

Que apenas se ve afectada por una pequeña cantidad de valores atípicos.

Correlación

La vicepresidenta de Crecimiento de DataSciencester tiene una teoría según la cual la cantidad de tiempo que la gente se queda en el sitio está relacionada con el número de amigos que tienen en él (ella no es vicepresidenta porque sí), y quiere verificar esta afirmación.

Tras escarbar en los registros de tráfico, obtenemos una lista llamada `daily_minutes`, que muestra los minutos al día que se pasa cada usuario en DataSciencester, y la hemos ordenado de forma que sus elementos se correspondan con los elementos de nuestra lista anterior `num_friends`. Nos gustaría investigar la relación entre estas dos métricas.

Primero, veremos la covarianza, la análoga emparejada de la varianza. Mientras la varianza mide la desviación de la media de una sola variable, la covarianza mide la variación de dos variables en tándem con respecto a sus medias:

```
from scratch.linear_algebra import dot
def covariance(xs: List[float], ys: List[float]) -> float:
   assert len(xs) == len(ys), "xs and ys must have same number of elements"
   return dot(de_mean(xs), de_mean(ys)) / (len(xs)-1)
assert 22.42 < covariance(num_friends, daily_minutes) < 22.43
assert 22.42 / 60 < covariance(num_friends, daily_hours) < 22.43 / 60
```

Recordemos que `dot` suma los productos de los pares de elementos correspondientes. Cuando los elementos correspondientes de `x` e `y` están ambos por encima o por debajo de sus medias, un número positivo entra en la suma. Cuando uno está por encima de la media y el otro por debajo, es un número negativo lo que entra en la suma. De acuerdo con esto, una covarianza positiva “grande” significa que `x` tiende a ser grande cuando `y` es grande y pequeño cuando `y` es pequeño. Una covarianza negativa “grande”

significa lo contrario: que `x` tiende a ser pequeño cuando `y` es grande y viceversa. Una covarianza cercana a cero significa que no existe tal relación.

No obstante, este número puede ser difícil de interpretar por varias razones:

- Sus unidades son el producto de las unidades de las entradas (por ejemplo, amigos-minutos-al-día), lo que puede ser difícil de entender (¿qué es un "amigo-minuto-al-día"?).
- Si cada usuario tuviera el doble de amigos (pero el mismo número de minutos), la covarianza sería el doble de grande. Pero, en cierto sentido, las variables estarían igual de interrelacionadas. Dicho de otro modo, es difícil decir lo que cuenta como una covarianza "grande".

Por esta razón, es más común mirar la correlación, que divide las desviaciones estándares de ambas variables:

```
def correlation(xs: List[float], ys: List[float]) -> float:
   """Measures how much xs and ys vary in tandem about their means"""
   stdev_x = standard_deviation(xs)
   stdev_y = standard_deviation(ys)
   if stdev_x > 0 and stdev_y > 0:
      return covariance(xs, ys) / stdev_x / stdev_y
   else:
      return 0      # si no hay variación, la correlación es cero
assert 0.24 < correlation(num_friends, daily_minutes) < 0.25
assert 0.24 < correlation(num_friends, daily_hours) < 0.25
```

La `correlation` no tiene unidad y siempre está entre -1 (anticorrelación perfecta) y 1 (correlación perfecta). Un número como 0,25 representa una correlación positiva relativamente débil. Sin embargo, una cosa que olvidamos hacer fue examinar nuestros datos. Veamos la figura 5.2.

La persona que tiene 100 amigos (y que pasa únicamente 1 minuto al día en el sitio) es un enorme valor atípico, y la correlación puede ser muy sensible a estos valores. ¿Qué ocurre si le ignoramos?

```
outlier = num_friends.index(100)         # índice de valor atípico
num_friends_good = [x
             for i, x in enumerate(num_friends)
```

```
            if i != outlier]
daily_minutes_good = [x
                for i, x in enumerate(daily_minutes)
                if i != outlier]
daily_hours_good = [dm / 60 for dm in daily_minutes_good]
assert 0.57 < correlation(num_friends_good, daily_minutes_good) < 0.58
assert 0.57 < correlation(num_friends_good, daily_hours_good) < 0.58
```

Sin el valor atípico, hay una correlación mucho más fuerte (véase la figura 5.3).

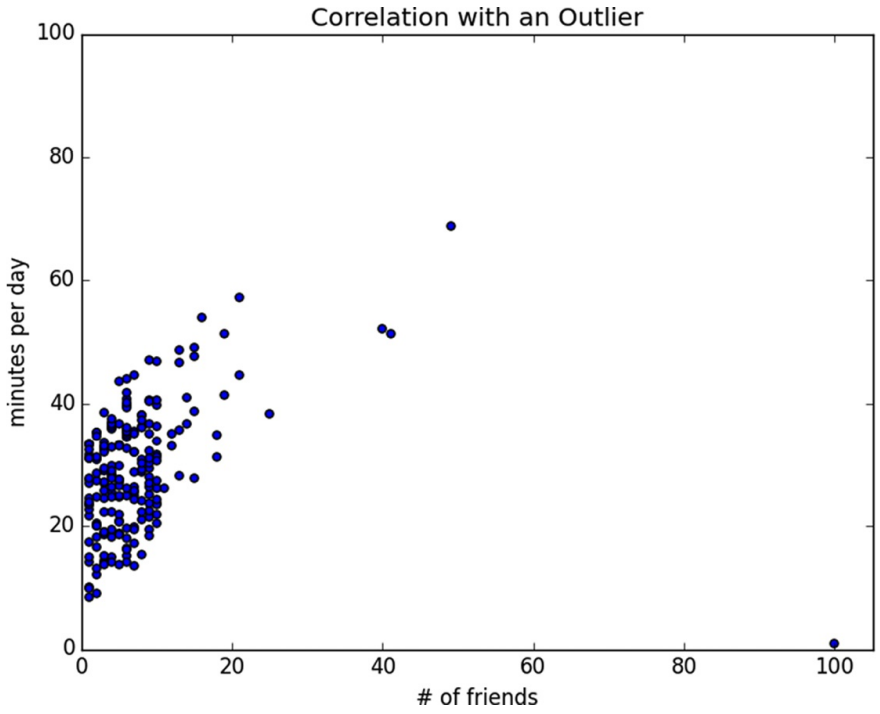


Figura 5.2. Correlación con un valor atípico.

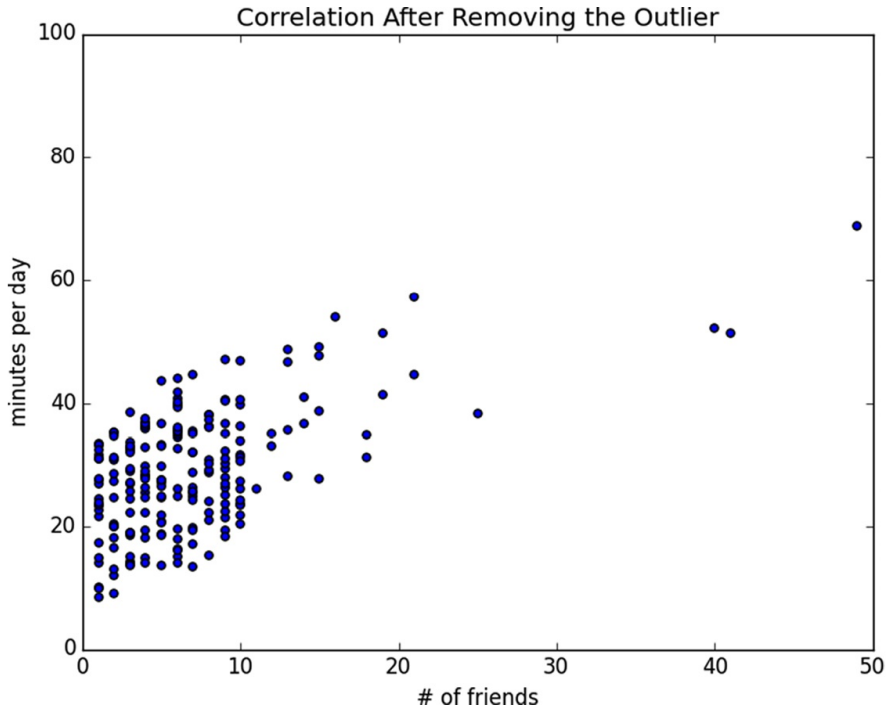


Figura 5.3. Correlación tras eliminar el valor atípico.

Seguimos investigando para descubrir que el valor atípico era realmente una cuenta de prueba interna que nadie se había preocupado nunca de eliminar. Así que su exclusión está totalmente justificada.

La paradoja de Simpson

Una sorpresa no poco habitual al analizar datos es la paradoja de Simpson, en la que las correlaciones pueden ser erróneas cuando se ignoran las variables de confusión.

Por ejemplo, imaginemos que podemos identificar todos nuestros miembros como científicos de datos de la costa este u oeste de EE. UU. Decidimos examinar los científicos de datos de qué costa son más amigables:

Costa	Nº de miembros	Nº medio de amigos
Costa oeste	101	8,2

Costa este	103	6,5

Sin duda parece que los científicos de datos de la costa oeste son más amigables que los de la costa este. Sus compañeros de trabajo avanzan todo tipo de teorías sobre la razón por la que podría ocurrir esto: ¿quizá es el sol, el café, los productos ecológicos o el ambiente relajado del Pacífico?

Pero, jugando con los datos, descubrimos algo muy extraño. Si solamente miramos las personas con doctorado, los científicos de datos de la costa este tienen más amigos en promedio.

Pero, si miramos solo las personas sin doctorado, ¡los científicos de datos de la costa este tienen también más amigos de media!

Costa	Doctorado	Nº de miembros	Nº medio de amigos
Costa oeste	Sí	35	3,1
Costa este	Sí	70	3,2
Costa oeste	No	66	10,9
Costa este	No	33	13,4

En cuanto se tienen en cuenta los doctorados de los usuarios, la correlación se va en la dirección contraria. Agrupar los datos en *buckets* como Costa Este/Costa Oeste disfrazó el hecho de que los científicos de datos de la costa este se inclinaban muchísimo más hacia los tipos con doctorado.

Este fenómeno ocurre en el mundo real con cierta regularidad. Lo esencial es que la correlación está midiendo la relación entre las dos variables, siendo todo lo demás igual. Si las clases de datos se asignan de forma aleatoria, como podría perfectamente ocurrir en un experimento bien diseñado, “siendo todo lo demás igual” podría no ser una suposición horrible.

La única forma real de evitar esto es conociendo los datos y haciendo lo posible por asegurarse de que se han revisado en busca de posibles factores de confusión. Es obvio que esto no es siempre posible. Si no tuviéramos datos sobre los logros académicos de estos 200 científicos de datos,

podríamos simplemente concluir que había algo intrínsecamente más amigable en la costa oeste.

Otras advertencias sobre la correlación

Una correlación de cero indica que no hay relación lineal entre las dos variables. Sin embargo, pueden existir otras formas de relaciones. Por ejemplo, si:

```
x = [-2, -1, 0, 1, 2]
y = [ 2, 1, 0, 1, 2]
```

Entonces `x` e `y` tienen correlación cero. Pero sin duda están relacionados: cada elemento de `y` es igual al valor absoluto del elemento correspondiente de `x`. Lo que no tienen es una relación en la que saber cómo se compara `x_i` con `mean(x)` nos ofrece información sobre cómo `y_i` se compara con `mean(y)`. Este es el tipo de relación que la correlación busca.

Además, la correlación no nos dice nada sobre lo grande que es la relación:

```
x = [-2, -1, 0, 1, 2]
y = [99.98, 99.99, 100, 100.01, 100.02]
```

Las variables están perfectamente correlacionadas, pero (dependiendo de cómo estemos midiendo) es muy posible que esta relación no sea tan interesante.

Correlación y causación

Es probable que en algún momento haya oído que “correlación no es causación”, dicho lo más probable por alguien en busca de datos que plantearan un desafío a partes de su visión del mundo que era reacio a cuestionar. Sin embargo, este es un punto importante: si `x` e `y` están fuertemente correlacionados, podría significar que `x` causa `y`, que `y` causa `x`,

que cada uno causa el otro, que un tercer factor causa ambos, o nada de esto en absoluto.

Pensemos en la relación entre `num_friends` y `daily_minutes`. Es posible que tener más amigos en el sitio cause que los usuarios de DataSciencester pasen más tiempo en el sitio. Este podría ser el caso si cada amigo sube una cierta cantidad de contenido cada día, lo que significa que cuantos más amigos se tengan, más tiempo se necesita para mantenerse al día de sus actualizaciones.

Sin embargo, es también posible que, cuanto más tiempo pasen los usuarios discutiendo en foros de DataSciencester, con más frecuencia encuentren personas con su misma forma de pensar y se hagan amigos de ellas. Es decir, pasar más tiempo en el sitio causa que los usuarios tengan más amigos.

Una tercera posibilidad es que los usuarios más apasionados por la ciencia de datos pasen más tiempo en el sitio (porque lo encuentran más interesante) y consigan de forma más activa amigos de ciencia de datos (porque no quieren asociarse con ningún otro).

Una forma de sentirse más cómodos con la causalidad es realizando ensayos aleatorios. Si es posible dividir aleatoriamente los usuarios en dos grupos con demografías similares y dar a uno de los grupos una experiencia algo distinta, entonces se observa con bastante seguridad que las distintas experiencias están causando los diferentes resultados.

Por ejemplo, si no nos importara que nos acusasen con indignación de experimentar con los usuarios,[3] podríamos elegir aleatoriamente un subconjunto de usuarios y mostrarles contenido únicamente de una parte de sus amigos. Si después este subconjunto se pasara menos tiempo en el sitio, ello nos daría una cierta confianza en pensar que tener más amigos causa pasar más tiempo en el sitio.

Para saber más

- SciPy en `https://www.scipy.org`, pandas en

http://pandas.pydata.org y StatsModels en http://www.statsmodels.org, incluyen todos una gran variedad de funciones estadísticas.

- La estadística es importante. Si quiere ser un científico de datos mejor, sería una buena idea leer un libro de texto sobre estadística. En la red hay muchos disponibles, como por ejemplo:
  - *Introductory Statistics*, en https://open.umn.edu/opentextbooks/textbooks/introductorystatistics, de Douglas Shafer y Zhiyi Zhang (Saylor Foundation).
  - *OnlineStatBook*, en http://onlinestatbook.com/, de David Lane (Rice University).
  - *Introductory Statistics*, en https://openstax.org/details/introductory-statistics, de OpenStax (OpenStax College).

---

[1] http://en.wikipedia.org/wiki/Quickselect.

[2] https://es.wikipedia.org/wiki/Estimaci%C3%B3n_de_la_desviaci%C3%B3n_est%C3%A1nda

[3] https://www.nytimes.com/2014/06/30/technology/facebook-tinkers-with-users-emotions-in-news-feed-experiment-stirring-outcry.html?searchResultPosition=1.

6 Probabilidad

Las leyes de la probabilidad, tan verdaderas en general, tan falaces en particular.

—Edward Gibbon

Es difícil hacer ciencia de datos sin un cierto conocimiento de la probabilidad y sus matemáticas. Al igual que ocurrió con nuestro tratamiento de la estadística en el capítulo 5, agitaremos mucho las manos y suprimiremos muchos de los tecnicismos.

Para nuestros fines lo mejor es pensar en la probabilidad como en una forma de cuantificar la incertidumbre asociada a los eventos elegidos desde un universo de eventos. En lugar de ponerse técnicos sobre lo que significan estas palabras, mejor pensemos en tirar un dado. El universo consiste en todos los resultados posibles, y cualquier subconjunto de estos resultados es un evento. Por ejemplo, "el dado saca un 1" o "el dado saca un número par".

Utilizando notación matemática, escribimos $P(E)$ para indicar "la probabilidad del evento E".

Utilizaremos la teoría de la probabilidad para crear modelos. Y para evaluar modelos. La emplearemos para todo.

Podríamos, si quisiéramos, profundizar en la filosofía del significado de la teoría de la probabilidad (lo que se haría mejor con unas cervezas). Pero no haremos eso.

Dependencia e independencia

A grandes rasgos, digamos que dos eventos E y F son dependientes si saber algo sobre si E ocurre nos da información sobre si F ocurre (y viceversa). De otro modo, son independientes.

Por ejemplo, si lanzamos una moneda dos veces, saber que el primer lanzamiento es cara no nos da información alguna sobre si en el segundo

lanzamiento saldrá también cara. Estos eventos son independientes. Por otro lado, saber si el primer lanzamiento es cara sin duda nos da información sobre si en ambos lanzamientos saldrá cruz (si en el primer lanzamiento sale cara, entonces definitivamente no es el caso de que en ambos lanzamientos salga cruz). Estos dos eventos son dependientes.

Matemáticamente, decimos que dos eventos E y F son independientes si la probabilidad de que ambos ocurran es el producto de las probabilidades de que cada uno ocurre:

$$P(E, F) = P(E)P(F)$$

En el ejemplo, la probabilidad de "primer lanzamiento cara" es de 1/2, y la probabilidad de "ambos lanzamientos cruz" es de 1/4, pero la probabilidad de "primer lanzamiento cara y ambos lanzamientos cruz" es de 0.

Probabilidad condicional

Cuando dos eventos E y F son independientes, entonces por definición tenemos:

$$P(E, F) = P(E)P(F)$$

Si no son necesariamente independientes (y si la probabilidad de F no es cero), entonces definimos la probabilidad de E "condicionada por F" como:

$$P(E|F) = P(E,F)/P(F)$$

Tendríamos que pensar en esto como la probabilidad de que E ocurra, dado que sabemos que F ocurre.

A menudo esto lo reescribimos así:

$$P(E,F) = P(E|F)P(F)$$

Cuando E y F son independientes, se puede verificar que esto da:

$$P(E|F) = P(E)$$

Que es la forma matemática de expresar que saber que F ocurrió no nos da información adicional sobre si E ocurrió.

Un ejemplo habitual y bastante complejo es el de una familia con dos hijos (desconocidos). Si suponemos que:

- Cada hijo tiene la misma probabilidad de ser niño o niña.
- El género del segundo hijo es independiente del género del primer hijo.

Entonces el evento “no niñas” tiene una probabilidad de 1/4, el evento “un niño, una niña” tiene probabilidad 1/4, y el evento “dos niñas” tiene una probabilidad también de 1/4.

Ahora podemos preguntar: ¿cuál es la probabilidad del evento “ambos hijos son niñas” (B) condicionado por el evento “el hijo mayor es una niña” (G)? Utilizando la definición de probabilidad condicional:

$$P(B|G) = P(B,G)/P(G) = P(B)/P(G) = 1/2$$

Ya que el evento B y G (“ambos hijos son niñas y el otro hijo es una niña”) es precisamente el evento B (en cuanto sabemos que ambos hijos son niñas, es necesariamente cierto que el hijo mayor sea una niña).

Lo más probable es que este resultado esté de acuerdo con su intuición.

También podríamos preguntar por la probabilidad del evento “ambos hijos son niñas” condicionado por el evento “al menos uno de los hijos es una niña” (L). Sorprendentemente, ¡la respuesta es distinta a la anterior!

Como antes, el evento B y L (“ambos hijos son niñas y al menos uno de los hijos es una niña”) es justamente el evento B. Esto significa que tenemos:

$$P(B|L) = P(B,L)/P(L) = P(B)/P(L) = 1/3$$

¿Cómo puede ser esto así? Bueno, si todo lo que sabemos es que al menos uno de los hijos es una niña, entonces es el doble de probable que la familia tenga un niño y una niña que tenga dos niñas.

Podemos comprobarlo “generando” muchas familias:

```
import enum, random
# Un Enum es un conjunto con nombre de valores enumerados.
# Podemos usarlos para que el código sea más descriptivo y legible.
class Kid(enum.Enum):
   BOY = 0
   GIRL = 1
def random_kid() -> Kid:
   return random.choice([Kid.BOY, Kid.GIRL])
both_girls = 0
older_girl = 0
either_girl = 0
random.seed(0)
for _ in range(10000):
   younger = random_kid()
   older = random_kid()
   if older == Kid.GIRL:
      older_girl += 1
   if older == Kid.GIRL and younger == Kid.GIRL:
      both_girls += 1
   if older == Kid.GIRL or younger == Kid.GIRL:
      either_girl += 1
print("P(both | older):", both_girls / older_girl)        # 0.514 ~ 1/2
print("P(both | either): ", both_girls / either_girl)     # 0.342 ~ 1/3
```

Teorema de Bayes

Uno de los mejores amigos del científico de datos es el teorema de Bayes, una forma de “revertir” las probabilidades condicionales. Digamos que necesitamos conocer la probabilidad de un cierto evento E condicionado porque algún otro evento F ocurra. Pero solamente tenemos información sobre la probabilidad de F condicionado porque E ocurra. Emplear la definición de probabilidad condicional dos veces nos dice que:

$$P(E|F) = P(E,F)/P(F) = P(F|E)P(E)/P(F)$$

El evento F puede dividirse en los dos eventos mutuamente exclusivos “F y E” y “F y no E”. Si escribimos ¬E por “no E” (es decir, “E no ocurre”),

entonces:

$P(F) = P(F,E) + P(F,\neg E)$

De modo que:

$P(E|F) = P(F|E)P(E)/[P(F|E)P(E) + P(F|\neg E)P(\neg E)]$

Que es como se suele enunciar el teorema de Bayes.

Este teorema se utiliza a menudo para demostrar por qué los científicos de datos son más inteligentes que los médicos. Imaginemos una cierta enfermedad que afecta a 1 de cada 10.000 personas. Supongamos que existe una prueba para esta enfermedad que da el resultado correcto ("enfermo" si se tiene la enfermedad y "no enfermo" si no se tiene) el 99 % de las veces.

¿Qué significa una prueba positiva? Utilicemos *T* para el evento "la prueba es positiva" y *D* para el evento "tiene la enfermedad". Entonces, el teorema de Bayes dice que la probabilidad de que tenga la enfermedad, condicionada porque la prueba sea positiva, es:

$P(D|T) = P(T|D)P(D)/[P(T|D)P(D) + P(T|\neg D)P(\neg D)]$

Aquí sabemos que $P(T|D)$, la probabilidad de que la prueba sea positiva en alguien que tenga la enfermedad, es 0,99. $P(D)$, la probabilidad de que cualquier persona tenga la enfermedad, es 1/10.000 = 0,0001. $P(T|\neg D)$, la probabilidad de que alguien que no tenga la enfermedad dé positivo en la prueba, es de 0,01. Y $P(\neg D)$, la probabilidad de que cualquier persona no tenga la enfermedad, es 0,9999. Si se sustituyen estos números en el teorema de Bayes, se obtiene:

$P(D|T) = 0{,}98\%$

Es decir, menos del 1 % de las personas cuya prueba fue positiva tienen realmente la enfermedad.

Nota: Esto supone que las personas se hacen la prueba más o menos aleatoriamente. Si solo las personas con determinados síntomas se hicieran la

prueba, en lugar de ello tendríamos que condicionar con el evento “prueba positiva y síntomas” y el número sería seguramente mucho más alto.

Una forma más intuitiva de ver esto es imaginar una población de 1 millón de personas. Podríamos esperar que 100 de ellas tuvieran la enfermedad, y que 99 de esas 100 dieran positivo. Por otro lado, supondríamos que 999.990 de ellas no tendrían la enfermedad, y que 9.999 de ellas darían positivo. Eso significa que se esperaría que solo 99 de (99 + 9.999) personas con la prueba positiva tuvieran realmente la enfermedad.

Variables aleatorias

Una variable aleatoria es una variable cuyos posibles valores tienen una distribución de probabilidad asociada. Una variable aleatoria muy sencilla es igual a 1 si al lanzar una moneda sale cara y a 0 si sale cruz. Otra más complicada mediría el número de caras que se observan al lanzar una moneda 10 veces o un valor tomado de `range(10)`, donde cada número es igualmente probable.

La distribución asociada da las probabilidades de que la variable realice cada uno de sus posibles valores. La variable lanzamiento de moneda es igual a 0 con una probabilidad de 0,5 y a 1 con una probabilidad de 0,5. La variable `range(10)` tiene una distribución que asigna una probabilidad de 0,1 a cada uno de los números de 0 a 9.

En ocasiones, hablaremos del valor esperado de una variable aleatoria, que es la media de sus valores ponderados por sus probabilidades. La variable lanzamiento de moneda tiene un valor esperado de 1/2 (= 0 * 1/2 + 1 * 1/2), y la variable `range(10)` tiene un valor esperado de 4,5.

Las variables aleatorias pueden estar condicionadas por eventos igual que el resto de eventos puede estarlo. Volviendo al ejemplo de los dos hijos de la sección “Probabilidad condicional”, si X es la variable aleatoria que representa el número de niñas, X es igual a 0 con una probabilidad de 1/4, 1 con una probabilidad de 1/2 y 2 con una probabilidad de 1/4.

Podemos definir una nueva variable aleatoria *Y* que da el número de niñas condicionado por al menos que uno de los hijos sea una niña. Entonces *Y* es igual a 1 con una probabilidad de 2/3 y a 2 con una probabilidad de 1/3. Y una variable *Z* que es el número de niñas condicionado porque el otro hijo sea una niña es igual a 1 con una probabilidad de 1/2 y a 2 con una probabilidad de 1/2.

La mayor parte de las veces estaremos utilizando variables aleatorias de forma implícita en lo que hagamos sin atraer especialmente la atención hacia ellas. Pero si mira atentamente las verá.

Distribuciones continuas

El lanzamiento de una moneda se corresponde con una distribución discreta, que asocia probabilidad positiva con resultados discretos. A menudo querremos modelar distribuciones a lo largo de una serie de resultados (para nuestros fines, estos resultados siempre serán números reales, aunque ese no sea siempre el caso en la vida real). Por ejemplo, la distribución uniforme pone el mismo peso en todos los números entre 0 y 1.

Como hay infinitos números entre 0 y 1, eso significa que el peso que asigna a puntos individuales debe ser necesariamente 0. Por esta razón representamos una distribución continua con una función de densidad de probabilidad PDF (*Probability Density Function*) tal que la probabilidad de ver un valor en un determinado intervalo es igual a la integral de la función de densidad sobre el intervalo.

> **Nota:** Si tiene un poco oxidado el cálculo de integrales, una forma más sencilla de comprender esto es que si una distribución tiene la función de densidad f, entonces la probabilidad de ver un valor entre x y $x + h$ es aproximadamente de $h * f(x)$ si h es pequeño.

La función de densidad para la distribución uniforme es sencillamente:

```
def uniform_pdf(x: float) -> float:
```

```
    return 1 if 0 <= x < 1 else 0
```

La probabilidad de que una variable aleatoria siguiendo esa distribución esté entre 0,2 y 0,3 es de 1/10, como era de esperar. La variable `random.random` de Python es (pseudo)aleatoria con una densidad uniforme.

Con frecuencia estaremos más interesados en la función de distribución acumulativa CDF (*Cumulative Distribution Function*), que da la probabilidad de que una variable aleatoria sea menor o igual a un determinado valor. No es difícil crear la función CDF para la distribución uniforme (véase la figura 6.1):

```
def uniform_cdf(x: float) -> float:
    """Returns the probability that a uniform random variable is <= x"""
    if x < 0: return 0      # aleatoria uniforme nunca es menor que 0
    elif x < 1: return x    # p.ej. P(X <= 0.4) = 0.4
    else: return 1          # aleatoria uniforme es siempre menor que 1
```

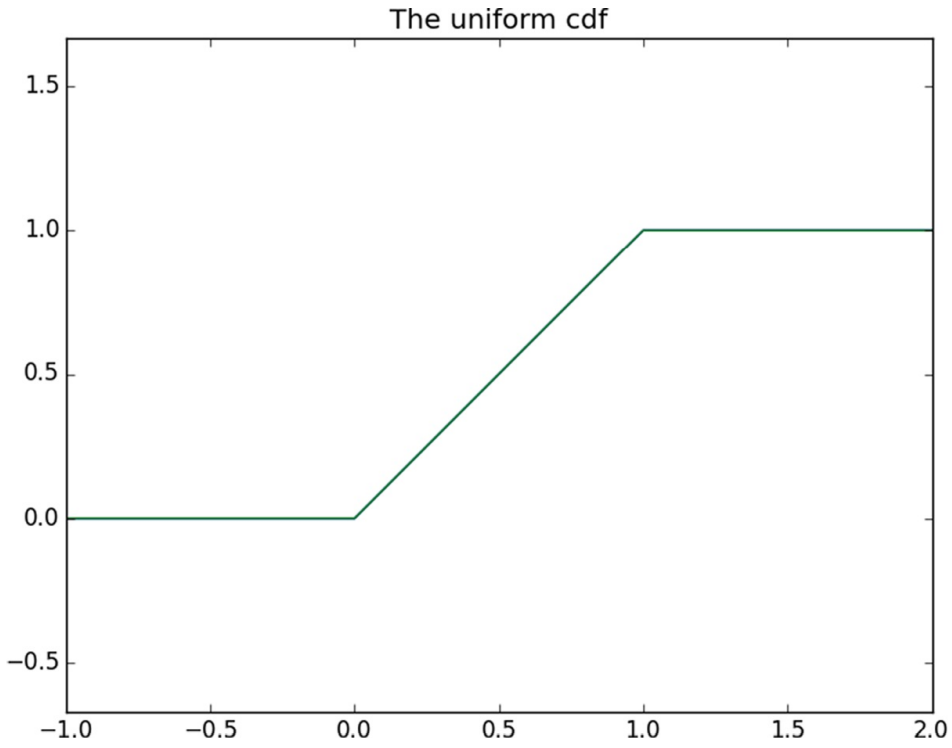


Figura 6.1. La función CDF uniforme.

La distribución normal

La distribución normal es la distribución clásica en forma de campana y se determina completamente con dos parámetros: su media μ (mu) y su desviación estándar σ (sigma). La media indica dónde está centrada la campana, y la desviación estándar lo "ancha" que es.

Tiene la función PDF:

$$f(x \mid \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Que podemos implementar como:

```
import math
SQRT_TWO_PI = math.sqrt(2 * math.pi)
def normal_pdf(x: float, mu: float = 0, sigma: float = 1) -> float:
   return (math.exp(-(x-mu) ** 2 / 2 / sigma ** 2) / (SQRT_TWO_PI * sigma))
```

En la figura 6.2 trazamos algunas de estas funciones PDF para ver cómo quedan:

```
import matplotlib.pyplot as plt
xs = [x / 10.0 for x in range(-50, 50)]
plt.plot(xs,[normal_pdf(x,sigma=1) for x in xs],'-',label='mu=0,sigma=1')
plt.plot(xs,[normal_pdf(x,sigma=2) for x in xs],'—',label='mu=0,sigma=2')
plt.plot(xs,[normal_pdf(x,sigma=0.5) for x in xs],':',label='mu=0,sigma=0.5')
plt.plot(xs,[normal_pdf(x,mu=-1) for x in xs],'-.',label='mu=-1,sigma=1')
plt.legend()
plt.title("Various Normal pdfs")
plt.show()
```

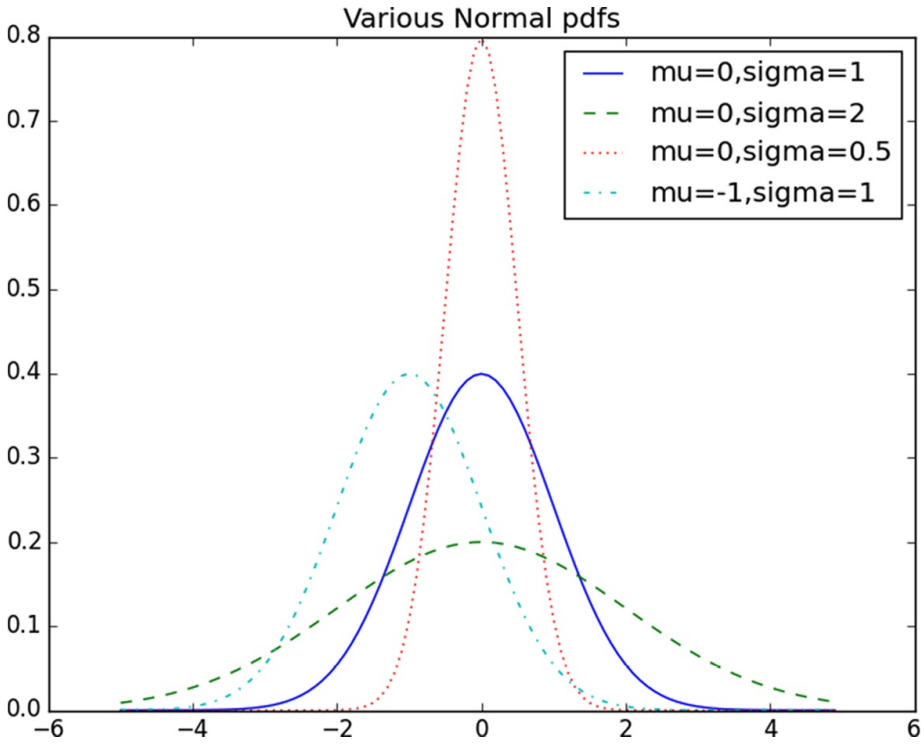


Figura 6.2. Varias funciones PDF normales.

Cuando $\mu = 0$ y $\sigma = 1$, se denomina distribución normal estándar. Si Z es una variable aleatoria normal estándar, entonces resulta que:

$$X = \sigma Z + \mu$$

También es normal, pero con media μ y desviación estándar σ. A la inversa, si X es una variable aleatoria normal con media μ y desviación estándar σ:

$$Z = (X - \mu)/\sigma$$

Es una variable normal estándar.

La función CDF para la distribución normal no se puede escribir de una forma "elemental", pero podemos hacerlo utilizando la función de error `math.erf` de Python:[1]

```
def normal_cdf(x: float, mu: float = 0, sigma: float = 1) -> float:
return (1 + math.erf((x–mu) / math.sqrt(2) / sigma)) / 2
```

De nuevo, en la figura 6.3 trazamos algunas CDF:

```
xs = [x / 10.0 for x in range(-50, 50)]
plt.plot(xs,[normal_cdf(x,sigma=1) for x in xs],'-',label='mu=0,sigma=1')
plt.plot(xs,[normal_cdf(x,sigma=2) for x in xs],'—',label='mu=0,sigma=2')
plt.plot(xs,[normal_cdf(x,sigma=0.5) for x in xs],':',label='mu=0,sigma=0.5')
plt.plot(xs,[normal_cdf(x,mu=-1) for x in xs],'-.',label='mu=-1,sigma=1')
plt.legend(loc=4) # bottom right
plt.title("Various Normal cdfs")
plt.show()
```

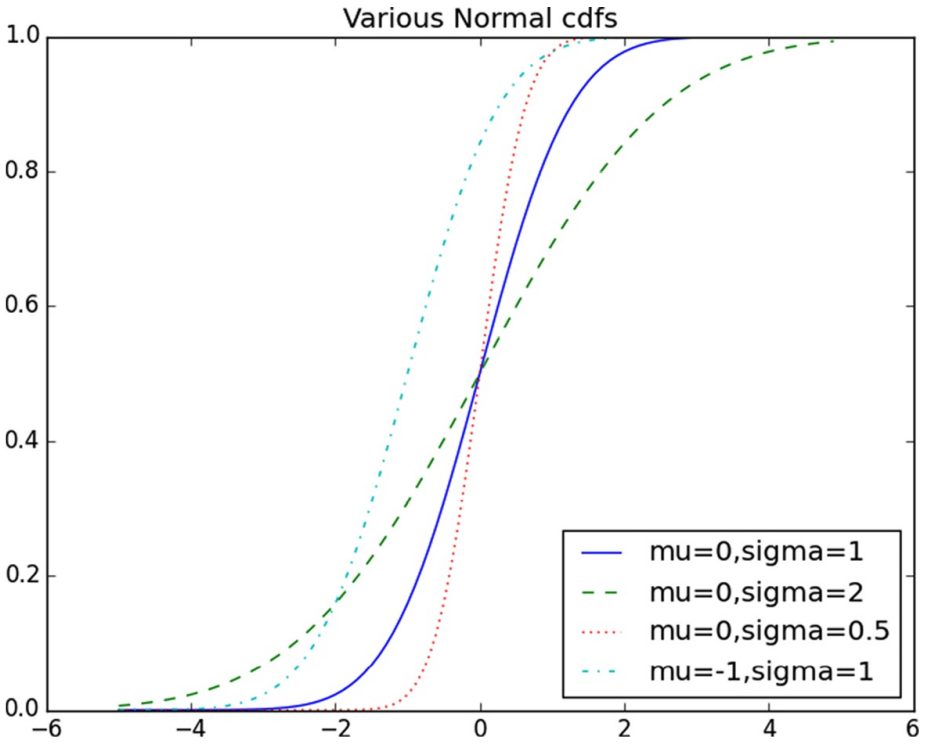


Figura 6.3. Varias CDF normales.

Algunas veces tendremos que invertir `normal_cdf` para encontrar el valor correspondiente a una determinada probabilidad. No hay una forma sencilla de calcular su inverso, pero `normal_cdf` es continuo y estrictamente creciente, de modo que utilizaremos una búsqueda binaria:[2]

```
def inverse_normal_cdf(p: float,
                       mu: float = 0,
                       sigma: float = 1,
                       tolerance: float = 0.00001) -> float:
    """Find approximate inverse using binary search"""
    # si no es estándar, calcula estándar y redimensiona
```

```
if mu != 0 or sigma != 1:
   return mu + sigma * inverse_normal_cdf(p, tolerance=tolerance)
low_z = -10.0                    # normal_cdf(-10) es (muy cercano a) 0
hi_z = 10.0                      # normal_cdf(10) es (muy cercano a) 1
while hi_z–low_z > tolerance:
   mid_z = (low_z + hi_z) /      # Considera el punto medio
   2
   mid_p =                       # y el valor de la CDF allí
   normal_cdf(mid_z)
   if mid_p < p:
      low_z = mid_z              # Punto medio demasiado bajo, busca por
                                 encima
   else:
      hi_z = mid_z               # Punto medio demasiado alto, busca por
                                 debajo
return mid_z
```

La función bisecciona repetidamente intervalos hasta que se estrecha en una *Z* que esté lo bastante cerca de la probabilidad deseada.

El teorema central del límite

Una razón por la que la distribución normal es tan útil es el teorema central del límite, que dice (básicamente) que una variable aleatoria, definida como la media de un gran número de variables aleatorias independientes y distribuidas de manera idéntica, está en sí misma aproximadamente y normalmente distribuida.

En particular, si *x*1, ..., *x*n son variables aleatorias con media μ y desviación estándar σ, y si *n* es grande, entonces:

$$\frac{1}{n}(x_1 + ... + x_n)$$

Está aproximadamente y normalmente distribuida con media μ y desviación estándar

$$\sigma/\sqrt{n}$$

. De forma equivalente (pero a menudo más útil):

$$\frac{\left(x_1 + ... + x_n\right) - \mu n}{\sigma\sqrt{n}}$$

Está aproximadamente y normalmente distribuida con media 0 y desviación estándar 1.

Una forma sencilla de ilustrar esto es mirando las variables aleatorias binomiales, que tienen dos parámetros *n* y *p*. Una variable aleatoria Binomial(*n*,*p*) no es más que la suma de *n* variables aleatorias independientes Bernoulli(*p*), cada una de las cuales es igual a 1 con una probabilidad de *p* y a 0 con una probabilidad 1 - *p*:

```
def bernoulli_trial(p: float) -> int:
   """Returns 1 with probability p and 0 with probability 1-p"""
   return 1 if random.random() < p else 0
def binomial(n: int, p: float) -> int:
   """Returns the sum of n bernoulli(p) trials"""
   return sum(bernoulli_trial(p) for _ in range(n))
```

La media de una variable Bernoulli(*p*) es *p*, y su desviación estándar es $\sqrt{p(1-p)}$. El teorema central del límite dice que cuando *n* es más grande, una variable Binomial(*n*,*p*) es aproximadamente una variable aleatoria normal con media $\mu = np$ y desviación estándar $\sigma = \sqrt{np(1-p)}$.

Si trazamos ambas, se puede ver fácilmente el parecido:

```
from collections import Counter
def binomial_histogram(p: float, n: int, num_points: int) -> None:
   """Picks points from a Binomial(n, p) and plots their histogram"""
   data = [binomial(n, p) for _ in range(num_points)]
   # usa un gráfico de barras para mostrar las muestras binomiales reales
   histogram = Counter(data)
   plt.bar([x–0.4 for x in histogram.keys()],
        [v / num_points for v in histogram.values()],
        0.8,
        color='0.75')
   mu = p * n
   sigma = math.sqrt(n * p * (1–p))
   # usa un gráfico de líneas para mostrar la aproximación normal
   xs = range(min(data), max(data) + 1)
   ys = [normal_cdf(i + 0.5, mu, sigma)–normal_cdf(i–0.5, mu, sigma)
      for i in xs]
```

```
    plt.plot(xs,ys)
    plt.title("Binomial Distribution vs. Normal Approximation")
    plt.show()
```

Por ejemplo, cuando llamamos a `make_hist(0.75, 100, 10000)`, obtenemos el gráfico de la figura 6.4.

La moraleja de esta aproximación es que, si queremos conocer la probabilidad de que (supongamos) al lanzar una moneda se saquen más de 60 caras en 100 lanzamientos, se puede estimar como la probabilidad de que un Normal(50,5) es mayor que 60, que es más fácil que calcular la función CDF Binomial(100,0.5) (aunque en la mayoría de las aplicaciones probablemente utilizaríamos software estadístico que calcularía felizmente cualesquiera probabilidades deseadas).

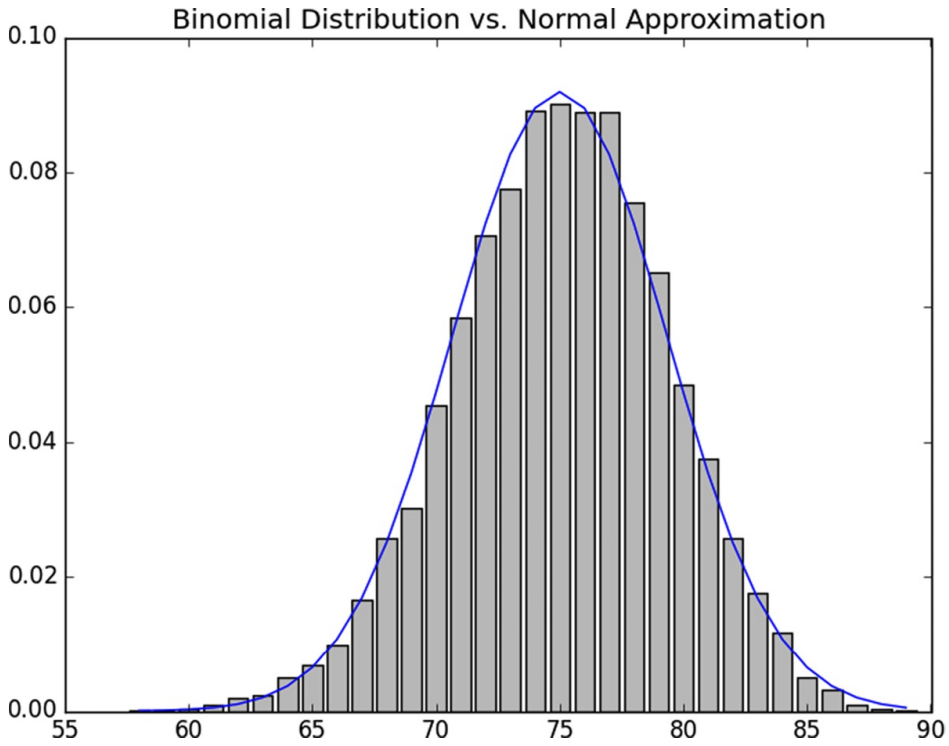


Figura 6.4. El resultado de `binomial_histogram`.

Para saber más

- scipy.stats, en https://docs.scipy.org/doc/scipy/reference/stats.html, contiene funciones PDF y CDF para la mayoría de las distribuciones de probabilidad más conocidas.
- ¿Recuerda que, al final del capítulo 5, dije que sería una buena idea estudiar un libro de texto de estadística? Pues también lo sería estudiarlo de probabilidad. El mejor que conozco que está disponible en la red es *Introduction to ProbabilityK*, en https://math.dartmouth.edu/~prob/prob/prob.pdf, de Charles M. Grinstead y J. Laurie Snell (American Mathematical Society).

---

[1] https://es.wikipedia.org/wiki/Funci%C3%B3n_error.

[2] https://es.wikipedia.org/wiki/B%C3%BAsqueda_binaria.

7 Hipótesis e inferencia

Es signo de una persona realmente inteligente el sentirse conmovida por las estadísticas.

—George Bernard Shaw

¿Qué haremos con toda esta estadística y toda esta teoría de la probabilidad? La parte científica de la ciencia de datos implica habitualmente la formación y comprobación de hipótesis sobre nuestros datos y sobre los procesos que los generan.

Comprobación de hipótesis estadísticas

A menudo, como científicos de datos, querremos probar si una determinada hipótesis es probable que sea cierta. Para nuestros fines, las hipótesis son aseveraciones como “esta moneda está equilibrada”, o “los científicos de datos prefieren Python a R”, o “es más probable que la gente salga de la página sin haber leído el contenido si aparece un irritante anuncio intercalado con un botón de cerrar diminuto y difícil de encontrar”, que se pueden traducir en estadísticas sobre datos. Bajo diferentes supuestos, se pueden considerar esas estadísticas como observaciones de variables aleatorias de distribuciones conocidas, lo que nos permite hacer afirmaciones sobre la probabilidad de que esos supuestos se cumplan.

En una configuración clásica, tenemos una hipótesis nula, H_0, que representa una cierta posición predeterminada, y otra hipótesis alternativa, H_1, con la que nos gustaría compararla. Utilizamos la estadística para decidir si podemos rechazar H_0 como falsa o no. Probablemente esto tiene más sentido con un ejemplo.

Ejemplo: Lanzar una moneda

Imaginemos que tenemos una moneda y queremos comprobar si es justa. Haremos la suposición de que la moneda tiene una cierta probabilidad *p* de sacar cara, de modo que nuestra hipótesis nula es que la moneda es justa (es decir, que $p = 0{,}5$). Comprobaremos esto frente a la hipótesis alternativa $p \neq 0{,}5$.

En particular, nuestra prueba implicará lanzar la moneda un número *n* de veces y contar el número de caras que salen, *X*. Cada lanzamiento de la moneda es un ensayo de Bernoulli, lo que significa que *X* es una variable aleatoria Binomial(*n*,*p*), la cual (como ya vimos en el capítulo 6) podemos aproximar utilizando la distribución normal:

```
from typing import Tuple
import math
def normal_approximation_to_binomial(n: int, p: float) -> Tuple[float, float]:
   """Returns mu and sigma corresponding to a Binomial(n, p)"""
   mu = p * n
   sigma = math.sqrt(p * (1-p) * n)
   return mu, sigma
```

Siempre que una variable aleatoria siga una distribución normal, podemos utilizar `normal_cdf` para averiguar la probabilidad de que su valor realizado esté dentro o fuera de un determinado intervalo:

```
from scratch.probability import normal_cdf
# La normal cdf _es_ la probabilidad de que la variable esté por debajo de un
límite
normal_probability_below = normal_cdf
# Está por encima del límite si no está por debajo
def normal_probability_above(lo: float,
 mu: float = 0,
 sigma: float = 1) -> float:
   """The probability that an N(mu, sigma) is greater than lo."""
   return 1-normal_cdf(lo, mu, sigma)
# Está en medio si es menor que hi, pero no menor que lo
def normal_probability_between(lo: float,
 hi: float,
 mu: float = 0,
 sigma: float = 1) -> float:
```

```
  """The probability that an N(mu, sigma) is between lo and hi."""
  return normal_cdf(hi, mu, sigma)–normal_cdf(lo, mu, sigma)
# Está fuera si no está en medio
def normal_probability_outside(lo: float,
 hi: float,
 mu: float = 0,
 sigma: float = 1) -> float:
  """The probability that an N(mu, sigma) is not between lo and hi."""
  return 1–normal_probability_between(lo, hi, mu, sigma)
```

También podemos hacer lo contrario: encontrar o bien la región que no esté en un extremo o el intervalo (simétrico) en torno a la media que se tiene en cuenta para un determinado nivel de probabilidad. Por ejemplo, si queremos encontrar un intervalo centrado en la media y que contenga un 60 % de probabilidad, entonces tenemos que hallar los límites en los que los extremos superior e inferior contienen cada uno un 20 % de la probabilidad (dejando el 60 %):

```
from scratch.probability import inverse_normal_cdf
def normal_upper_bound(probability: float,
               mu: float = 0,
               sigma: float = 1) -> float:
  """Returns the z for which P(Z <= z) = probability"""
  return inverse_normal_cdf(probability, mu, sigma)
def normal_lower_bound(probability: float,
               mu: float = 0,
               sigma: float = 1) -> float:
  """Returns the z for which P(Z >= z) = probability"""
  return inverse_normal_cdf(1–probability, mu, sigma)
def normal_two_sided_bounds(probability: float,
                 mu: float = 0,
                 sigma: float = 1) -> Tuple[float, float]:
  """
  Returns the symmetric (about the mean) bounds
  that contain the specified probability
  """
  tail_probability = (1–probability) / 2
  # el extremo superior tendría tail_probability por encima
  upper_bound = normal_lower_bound(tail_probability, mu, sigma)
  # el extremo inferior tendría tail_probability por debajo
  lower_bound = normal_upper_bound(tail_probability, mu, sigma)
  return lower_bound, upper_bound
```

En particular, digamos que elegimos lanzar la moneda n = 1.000 veces. Si nuestra hipótesis nula es cierta, X debería estar distribuida aproximadamente normal con una media de 500 y una desviación estándar de 15,8:

```
mu_0, sigma_0 = normal_approximation_to_binomial(1000, 0.5)
```

Tenemos que tomar una decisión sobre la significancia (lo dispuestos que estamos a cometer un error tipo 1 o "falso positivo", en el que rechazamos H_0 incluso aunque sea verdadero). Por razones perdidas en los anales de la historia, esta voluntad se suele establecer en un 5 % o en un 1 %. Elijamos un 5 %.

Consideremos la prueba que rechaza H_0 si X queda fuera de los extremos dados por:

```
# (469, 531)
lower_bound, upper_bound = normal_two_sided_bounds(0.95, mu_0, sigma_0)
```

Suponiendo que p es de verdad igual a 0,5 (es decir, H_0 es verdadero), solamente hay un 5 % de posibilidades de que observemos una X que esté fuera de este intervalo, que es exactamente la significancia que queríamos. Dicho de otro modo, si H_0 es verdadero, entonces aproximadamente 19 veces de 20 esta prueba dará el resultado correcto.

También nos interesaremos a menudo por la potencia de una prueba de hipótesis, que es la probabilidad de no cometer un error tipo 2 ("falso negativo"), en el que no rechazamos H_0 incluso aunque sea falso. Para medir esto, tenemos que especificar lo que significa exactamente que H_0 sea falso (saber simplemente que p no es 0,5 no nos da mucha información sobre la distribución de X). En particular, comprobemos lo que ocurre si p es realmente 0,55, o sea, que la moneda tiende levemente hacia la cara.

En ese caso, podemos calcular la potencia de la prueba con:

```
# extremos en 95 % basados en suponer que p es 0,5
lo, hi = normal_two_sided_bounds(0.95, mu_0, sigma_0)
# mu y sigma reales basadas en p = 0,55
mu_1, sigma_1 = normal_approximation_to_binomial(1000, 0.55)
# un error tipo 2 significa que no rechazamos la hipótesis nula
# lo que ocurrirá cuando X siga en nuestro intervalo original
type_2_probability = normal_probability_between(lo, hi, mu_1, sigma_1)
power = 1-type_2_probability                          # 0.887
```

Imaginemos, en lugar de esto, que nuestra hipótesis nula fuera que la moneda no estuviera inclinada hacia la cara, o que $p \leq 0{,}5$. En ese caso, queremos una prueba de una sola cara, que rechace la hipótesis nula cuando X es mucho mayor que 500, pero no cuando X es menor que 500. Así, una prueba de significancia del 5 % implica utilizar `normal_probability_below` para encontrar el límite bajo el que se sitúa el 95 % de la probabilidad:

```
hi = normal_upper_bound(0.95, mu_0, sigma_0)
# es 526 (<531, ya que necesitamos más probabilidad en el límite superior)
type_2_probability = normal_probability_below(hi, mu_1, sigma_1)
power = 1–type_2_probability                              # 0.936
```

Esta es una comprobación más potente, dado que ya no rechaza H_0 cuando X está por debajo de 469 (que es muy improbable que ocurra si H_1 es verdadero), y en su lugar rechaza H_0 cuando X está entre 526 y 531 (lo que tiene alguna probabilidad de ocurrir si H_1 es verdadero).

Valores p

Una forma distinta de pensar en la prueba anterior involucra a los valores *p* o *p-values*. En vez de elegir límites basándonos en un tope de probabilidad, calculamos la probabilidad (suponiendo que H_0 sea verdadero) de ver un valor al menos tan extremo como el que realmente observamos.

Para nuestra prueba de dos caras de si la moneda es justa, hacemos estos cálculos:

```
def two_sided_p_value(x: float, mu: float = 0, sigma: float = 1) -> float:
   """
   How likely are we to see a value at least as extreme as x (in either
   direction) if our values are from an N(mu, sigma)?
   """
   if x >= mu:
      # x es mayor que la media, así el extremo es todo lo que es mayor que x
      return 2 * normal_probability_above(x, mu, sigma)
   else:
```

```
    # x es menor que la media, así el extremo es todo lo que es menor que x
    return 2 * normal_probability_below(x, mu, sigma)
```

Si viéramos 530 caras, este sería el cálculo:

```
two_sided_p_value(529.5, mu_0, sigma_0) # 0.062
```

Nota: ¿Por qué hemos utilizado un valor de `529.5` en lugar de utilizar `530`? Esto es lo que se llama corrección por continuidad.[1] Refleja el hecho de que `normal_probability_between(529.5, 530.5, mu_0, sigma_0)` es una mejor estimación de la probabilidad de ver 530 caras que `normal_probability_between(530, 531, mu_0, sigma_0)`.
En consecuencia, `normal_probability_above(529.5, mu_0, sigma_0)` es una mejor estimación de la probabilidad de ver al menos 530 caras. Quizá se haya dado cuenta de que también hemos utilizado esto en el código que producía la figura 6.4.

Una forma de autoconvencernos de que es una estimación razonable es utilizando una simulación:

```
import random
extreme_value_count = 0
for _ in range(1000):
   num_heads = sum(1 if random.random() < 0.5        # Cuenta el nº de caras
   else 0
             for _ in range(1000))                   # en 1.000 lanzamientos,
   if num_heads >= 530 or num_heads <= 470:          # y cuenta con qué
                                                     frecuencia
      extreme_value_count += 1                       # el nº es ‘extremo’
# el p-value era 0,062 => ~62 valores extremos de 1.000
assert 59 < extreme_value_count < 65, f"{extreme_value_count}"
```

Como el valor *p* es mayor que nuestra significancia del 5 %, no rechazamos la hipótesis nula. Si viéramos sin embargo 532 caras, el valor *p* sería:

```
two_sided_p_value(531.5, mu_0, sigma_0)      # 0,0463
```

Que es menor que la significancia del 5 %, lo que indica que sí la rechazaríamos. Es exactamente la misma prueba que antes, solo que con una

forma diferente de enfocar las estadísticas.

De forma similar, tendríamos:

```
upper_p_value = normal_probability_above
lower_p_value = normal_probability_below
```

Para nuestra prueba de una sola cara, si viéramos 525 caras calcularíamos:

```
upper_p_value(524.5, mu_0, sigma_0)     # 0,061
```

Lo que significa que no rechazaríamos la hipótesis nula. Si viéramos 527 caras, el cálculo sería:

```
upper_p_value(526.5, mu_0, sigma_0)     # 0,047
```

Y sí rechazaríamos la hipótesis nula.

Advertencia: Asegúrese de que sus datos están más o menos normalmente distribuidos antes de utilizar `normal_probability_above` para calcular valores *p*. Los anales de la ciencia de datos mal practicada están llenos de ejemplos de personas opinando que la posibilidad de que algún evento observado se produzca aleatoriamente es una entre un millón, cuando lo que realmente quieren decir es "la posibilidad, suponiendo que los datos estén normalmente distribuidos", lo que no tiene mucho sentido si los datos no lo están.
Existen diferentes pruebas estadísticas para la normalidad, pero incluso trazar los datos es un buen comienzo.

Intervalos de confianza

Hemos estado probando hipótesis sobre el valor de la probabilidad *p* de que salga cara, que es un parámetro de la distribución desconocida "cara". Cuando es este el caso, un tercer método es construir un intervalo de confianza en torno al valor observado del parámetro.

Por ejemplo, podemos estimar la probabilidad de la moneda injusta mirando el valor medio de las variables de Bernoulli correspondientes a cada lanzamiento (1 si es cara, 0 si es cruz). Si observamos 525 caras en 1.000

lanzamientos, entonces estimamos que *p* es igual a 0,525.

¿Qué confianza podemos tener en esta estimación? Bueno, si conociéramos el valor exacto de *p*, el teorema central del límite (recuerde la sección del mismo nombre del capítulo 6) nos dice que la media de dichas variables de Bernoulli debería ser aproximadamente normal, con una media de *p* y una desviación estándar de:

```
math.sqrt(p * (1–p) / 1000)
```

Aquí no conocemos *p*, así que por eso utilizamos nuestra estimación:

```
p_hat = 525 / 1000
mu = p_hat
sigma = math.sqrt(p_hat * (1–p_hat) / 1000)      # 0,0158
```

Esto no está del todo justificado, pero la gente parece hacerlo de todas formas. Utilizando la aproximación normal, concluimos que "tenemos una confianza del 95 %" en que el siguiente intervalo contiene el verdadero parámetro *p*:

```
normal_two_sided_bounds(0.95, mu, sigma)      # [0.4940, 0.5560]
```

Nota: Esta es una afirmación sobre el intervalo, no sobre *p*. Se debería entender como la aseveración de que, si repitiéramos el experimento muchas veces, el 95 % del tiempo el parámetro "verdadero" (que es el mismo cada vez) estaría dentro del intervalo de confianza observado (que podría ser diferente cada vez).

En particular, no concluimos que la moneda sea injusta, ya que 0,5 está dentro de nuestro intervalo de confianza.

Si lo que viéramos fueran 540 caras, entonces tendríamos:

```
p_hat = 540 / 1000
mu = p_hat
sigma = math.sqrt(p_hat * (1–p_hat) / 1000)      # 0,0158
normal_two_sided_bounds(0.95, mu, sigma)         # [0.5091, 0.5709]
```

Aquí, "moneda justa" no está en el intervalo de confianza (la hipótesis de

“moneda justa” no pasa una prueba que se supone que debiera pasar el 95 % de las veces si fuera verdadera).

p-hacking o dragado de datos

Un procedimiento que rechace erróneamente la hipótesis nula solo el 5 % de las veces rechazará (por definición) erróneamente la hipótesis nula el 5 % de las veces:

```
from typing import List
def run_experiment() -> List[bool]:
   """Flips a fair coin 1000 times, True = heads, False = tails"""
   return [random.random() < 0.5 for _ in range(1000)]
def reject_fairness(experiment: List[bool]) -> bool:
   """Using the 5% significance levels"""
   num_heads = len([flip for flip in experiment if flip])
   return num_heads < 469 or num_heads > 531
random.seed(0)
experiments = [run_experiment() for _ in range(1000)]
num_rejections = len([experiment
                for experiment in experiments
                if reject_fairness(experiment)])
assert num_rejections == 46
```

Lo que esto significa es que, si pretendemos encontrar resultados “significativos”, es posible hallarlos. Probando las hipótesis suficientes sobre nuestro conjunto de datos, casi con seguridad una de ellas será significativa. Si eliminamos los valores atípicos correctos, probablemente podremos obtener el valor p inferior a 0,05 (hemos hecho algo vagamente parecido en la sección “Correlación” del capítulo 5; ¿se había dado cuenta?).

Esto a veces se denomina *p-hacking* o dragado de datos,[2] y en algunos aspectos es una consecuencia de la “infraestructura de inferencia de valores *p*”. Un buen artículo que critica este enfoque es “The Earth Is Round”[3] de Jacob Cohen.

Si queremos hacer buena ciencia, deberemos determinar nuestras hipótesis antes de mirar los datos, limpiar los datos sin tener en mente las hipótesis y

recordar que los valores *p* no son sustitutos del sentido común (un método alternativo se trata en la sección “Inferencia bayesiana”, antes de finalizar este capítulo).

Ejemplo: Realizar una prueba A/B

Una de sus principales responsabilidades en DataSciencester es la optimización de la experiencia, un eufemismo para intentar que la gente haga clic en los anuncios. Uno de los anunciantes ha desarrollado una nueva bebida energética destinada a los científicos de datos, y el vicepresidente de Publicidad quiere que elija entre el anuncio A (“¡menudo sabor!) y el anuncio B (“¡menos prejuicios!”).

Como somos científicos, decidimos realizar un experimento mostrando de forma aleatoria a los visitantes del sitio uno de los dos anuncios y haciendo un seguimiento de cuántos de ellos hacen clic sobre cada uno.

Si 990 de 1.000 espectadores del anuncio A hacen clic en él, mientras que solamente 10 de 1.000 del anuncio B hacen clic en el B, podríamos estar bastante seguros de que A es el mejor anuncio. Pero ¿qué pasa si las diferencias no son tan abismales? Aquí es cuando entra en juego la inferencia estadística.

Digamos que *NA* personas ven el anuncio A y que *nA* de ellas hacen clic en él. Podemos pensar en cada visión del anuncio como en un ensayo de Bernoulli, donde *pA* es la probabilidad de que alguien haga clic en el anuncio A. Entonces (si *NA* es grande, que lo es aquí) sabemos que *nA*/*NA* es aproximadamente una variable aleatoria normal, con una media de *pA* y una desviación estándar de $\sigma_A = \sqrt{p_A(1 - p_A)/N_A}$.

De forma similar, *nB*/*NB* es aproximadamente una variable aleatoria normal con una media de *pB* y una desviación estándar de $\sigma_B = \sqrt{p_B(1 - p_B)/N_B}$. Podemos expresar esto en código del siguiente modo:

```
def estimated_parameters(N: int, n: int) -> Tuple[float, float]:
   p = n / N
   sigma = math.sqrt(p * (1–p) / N)
   return p, sigma
```

Si suponemos que esas dos normales son independientes (lo que parece razonable, ya que los ensayos de Bernoulli individuales podrían serlo), entonces su diferencia también debería ser una normal con media *PB–PA* y desviación estándar $\sqrt{\sigma_A^2 + \sigma_B^2}$.

> **Nota:** Esto es un poco trampa. Las mates solo cuadran exactamente así si conocemos las desviaciones estándar. Aquí estamos estimándolas desde los datos, lo que significa que en realidad deberíamos estar utilizando una distribución *t*. Pero, para conjuntos de datos bastante grandes, se aproxima lo suficiente como para no suponer una gran diferencia.

Esto significa que podemos probar la hipótesis nula de que *pA* y *pB* son iguales (es decir, que *pA* - *pB* = 0) utilizando la estadística:

```
def a_b_test_statistic(N_A: int, n_A: int, N_B: int, n_B: int) -> float:
   p_A, sigma_A = estimated_parameters(N_A, n_A)
   p_B, sigma_B = estimated_parameters(N_B, n_B)
   return (p_B–p_A) / math.sqrt(sigma_A ** 2 + sigma_B ** 2)
```

Que debería ser aproximadamente una normal estándar.

Por ejemplo, si “menudo sabor” obtiene 200 clics de 1.000 visitantes y “menos prejuicios” obtiene 180 clics de 1.000 espectadores, la estadística es igual a:

```
z = a_b_test_statistic(1000, 200, 1000, 180)      # -1,14
```

La probabilidad de ver una diferencia tan grande si las medias fueran realmente iguales sería:

```
two_sided_p_value(z)      # 0,254
```

Que es tan grande que no podemos concluir que haya mucha diferencia. Por otro lado, si “menos prejuicios” solo obtuviera 150 clics, tendríamos:

```
z = a_b_test_statistic(1000, 200, 1000, 150)      # -2,94
two_sided_p_value(z)                              # 0,003
```

Lo que significa que solamente hay una probabilidad de 0,003 de que veamos una diferencia tan grande si los anuncios fueran igualmente efectivos.

Inferencia bayesiana

Los procedimientos que hemos visto han supuesto realizar afirmaciones de probabilidad sobre nuestras pruebas: por ejemplo, “solo hay un 3 % de posibilidades de que observemos una estadística tan extrema si nuestras hipótesis nulas fueran verdaderas”.

Un método distinto a la inferencia implica tratar los parámetros desconocidos como variables aleatorias. El analista (que es usted) empieza con una distribución previa para los parámetros y después utiliza los datos observados y el teorema de Bayes para obtener una distribución posterior actualizada para los parámetros. En lugar de hacer juicios de probabilidad sobre las pruebas, mejor hacer juicios de probabilidad sobre los parámetros.

Por ejemplo, cuando el parámetro desconocido es una probabilidad (como en nuestro ejemplo del lanzamiento de la moneda), a menudo empleamos una previa de la distribución beta, que coloca toda su probabilidad entre 0 y 1:

```
def B(alpha: float, beta: float) -> float:
   """A normalizing constant so that the total probability is 1"""
   return math.gamma(alpha) * math.gamma(beta) / math.gamma(alpha + beta)
def beta_pdf(x: float, alpha: float, beta: float) -> float:
   if x <= 0 or x >= 1:           # no hay peso fuera de [0, 1]
      return 0
   return x ** (alpha–1) * (1–x) ** (beta–1) / B(alpha, beta)
```

Por lo general, esta distribución centra su peso en:

```
alpha / (alpha + beta)
```

Y, cuanto más grandes sean `alpha` y `beta`, más “ajustada” es la distribución.

Por ejemplo, si `alpha` y `beta` son ambas 1, es la distribución uniforme como tal (centrada en 0,5, muy dispersa). Si `alpha` es mucho mayor que

`beta`, la mayor parte del peso está cerca de 1. Y si `alpha` es mucho menor que `beta`, la mayor parte del peso está cerca de 0. La figura 7.1 muestra varias distribuciones beta diferentes.

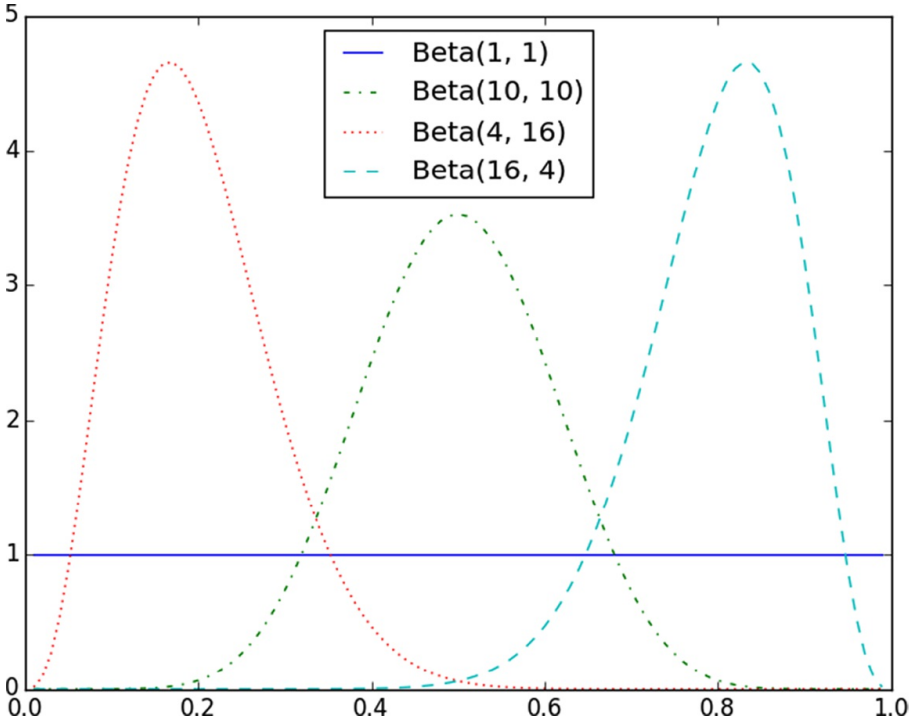


Figura 7.1. Distribuciones beta de ejemplo.

Digamos que suponemos una distribución anterior en *p*. Quizá no queramos tomar partido sobre si la moneda es justa, así que elegimos que `alpha` y `beta` sean ambas 1. O quizá tenemos clarísimo que la moneda saca cara el 55 % de las veces, por lo que elegimos que `alpha` sea igual a 55 y `beta` sea igual a 45.

Entonces lanzamos nuestra moneda un montón de veces y vemos *h* caras y *t* cruces. El teorema de Bayes (y otros cálculos matemáticos demasiado aburridos como para explicarlos aquí) nos dice que la distribución posterior para *p* es de nuevo una distribución beta, pero con los parámetros `alpha + h` y `beta + t`.

Nota: No es una coincidencia que la distribución posterior sea de nuevo una distribución beta. El número de caras viene dado por una distribución binomial,

y la beta es la previa conjugada[4] a la distribución binomial. Esto significa que, siempre que actualicemos una beta previa utilizando observaciones de la correspondiente binomial, obtendremos una posterior beta.

Digamos que lanzamos la moneda 10 veces y solo vemos 3 caras. Si empezáramos con la anterior uniforme (negándonos en cierto sentido a tomar partido sobre que la moneda sea justa o no), nuestra distribución posterior sería una Beta(4, 8), centrada en torno a 0,33. Como consideramos todas las probabilidades igualmente probables, nuestro mejor intento es próximo a la probabilidad observada.

Si empezáramos con una Beta(20, 20) (expresando la creencia de que la moneda era más o menos justa), nuestra distribución posterior sería una Beta(23, 27), centrada en torno a 0,46, indicando una creencia revisada de que quizá la moneda tiene una ligera tendencia hacia cruz.

Pero, si empezáramos con una Beta(30, 10) (expresando la creencia de que la moneda tiende a sacar cara un 75 % de las veces), nuestra distribución posterior sería una Beta(33, 17), centrada en torno a 0,66. En ese caso, seguiríamos creyendo en un desequilibrio hacia cara, pero con menos vehemencia que al principio. Estas tres diferentes posteriores se trazan en la figura 7.2.

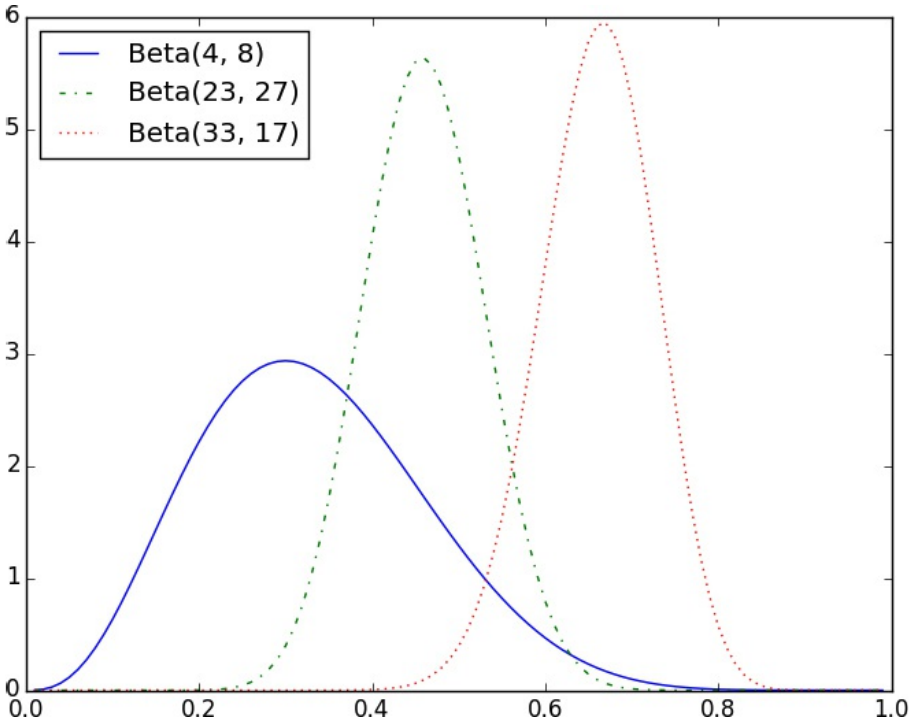


Figura 7.2. Posteriores surgiendo de distintas previas.

Si tirásemos la moneda más y más veces, la previa importaría cada vez menos hasta que al final tendríamos (casi) la misma distribución posterior sin importar la anterior con la que hubiéramos empezado.

Por ejemplo, sin importar la tendencia que nos parecía que pudiera tener la moneda en un principio, sería difícil mantener esa creencia tras ver 1.000 caras después de 2.000 lanzamientos, a menos que fuéramos unos lunáticos que eligiéramos algo parecido a una previa Beta(1000000,1).

Lo interesante es que esto nos permite realizar afirmaciones de probabilidad sobre las hipótesis: “Basándonos en la previa y en los datos observados, solo hay un 5 % de posibilidades de que la probabilidad de que la moneda saque cara esté entre el 49 % y el 51 %”. Esto es filosóficamente muy distinto a una aseveración como “si la moneda fuera justa, esperaríamos observar datos tan extremos solo el 5 % de las veces”.

Utilizar la inferencia bayesiana para probar hipótesis se considera un poco controvertido (en parte porque las matemáticas pueden llegar a ser bastante complicadas, y en parte debido a la naturaleza subjetiva de elegir una previa). No la utilizaremos más en este libro, pero es bueno conocerla.

Para saber más

- Apenas hemos arañado la superficie de lo que debería saber sobre inferencia estadística. Los libros recomendados al final del capítulo 5 entran mucho más en detalle al respecto.
- Coursera ofrece un curso de análisis de datos e inferencia estadística, en https://www.coursera.org/specializations/statistics, que trata muchos de estos temas.

---

[1] https://es.wikipedia.org/wiki/Correcci%C3%B3n_por_continuidad.

[2] https://www.nature.com/news/scientific-method-statistical-errors-1.14700.

[3] https://www.iro.umontreal.ca/~dift3913/cours/papers/cohen1994_The_earth_is_round

[4] https://www.johndcook.com/blog/conjugate_prior_diagram/.

8 Descenso de gradiente

Los que presumen de su ascendencia se jactan de lo que deben a los demás.

—Séneca

Cuando estemos haciendo ciencia de datos, muchas veces intentaremos encontrar el mejor modelo para una determinada situación. Generalmente "mejor" quiere decir algo así como "minimiza el error de sus predicciones" o "maximiza la probabilidad de los datos". En otras palabras, representará la solución a algún tipo de problema de optimización.

Esto significa que tendremos que resolver unos cuantos problemas de optimización; en particular, tendremos que hacerlo desde el principio. Nuestro enfoque para ello será una técnica denominada descenso de gradiente, que se presta a la perfección a un tratamiento iniciado desde cero. Quizá no resulte superinteresante en sí mismo, pero sí nos permitirá hacer cosas apasionantes a lo largo del libro, así que les pido un poco de paciencia.

La idea tras el descenso de gradiente

Supongamos que tenemos una función `f` cuya entrada es un vector de números reales y cuya salida es un solo número real. Una función como esta sencilla es algo así:

```
from scratch.linear_algebra import Vector, dot
def sum_of_squares(v: Vector) -> float:
   """Computes the sum of squared elements in v"""
   return dot(v, v)
```

En muchas ocasiones, tendremos que maximizar o minimizar este tipo de funciones. Es decir, tendremos que encontrar la entrada `v` que produzca el mayor (o menor) valor posible.

Para funciones como la nuestra, el gradiente (si se acuerda del cálculo que estudió en su día, es el vector de las derivadas parciales) proporciona la dirección de entrada en la que la función aumenta a mayor velocidad (si no se acuerda, créase lo que le digo, o busque en Internet).

Según esto, un método para maximizar una función es elegir un punto de inicio aleatorio, calcular el gradiente, dar un pequeño paso en la dirección del gradiente (es decir, la dirección que hace que la función aumente al máximo) y repetir con el nuevo punto de inicio. De forma similar, se puede minimizar una función dando pequeños pasos en la dirección opuesta, como muestra la figura 8.1.

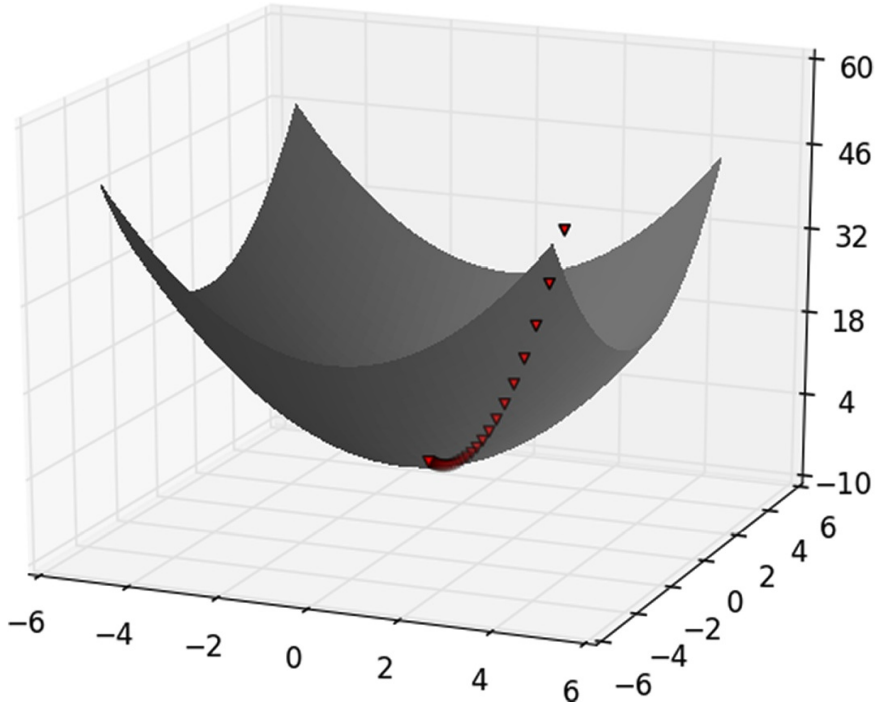


Figura 8.1. Hallar un mínimo utilizando el descenso de gradiente.

Nota: Si una función tiene un mínimo global único, es probable que este procedimiento lo encuentre. Si tiene varios mínimos (locales), quizá lo que el procedimiento "encuentre" sea el mínimo erróneo de todos ellos, en cuyo caso se podría volver a ejecutar desde distintos puntos de inicio. Si una función no tiene mínimo, entonces es posible que el procedimiento siga ejecutándose para siempre.

Estimar el gradiente

Si `f` es una función de una sola variable, su derivada en un punto `x` mide cómo cambia `f(x)` cuando le aplicamos un pequeño cambio a `x`. La derivada se define como el límite de los cocientes de diferencias:

```
from typing import Callable
def difference_quotient(f: Callable[[float], float],
                        x: float,
                        h: float) -> float:
   return (f(x + h)–f(x)) / h
```

Cuando `h` se aproxima a cero.

(Muchos aspirantes a estudiantes de cálculo se han visto obstaculizados por la definición matemática de límite, que es preciosa, pero puede resultar ciertamente intimidatoria. Aquí haremos trampas y simplemente diremos que "límite" significa lo que todos piensan que significa).

La derivada es la pendiente de la tangente en (x, $f(x)$), mientras que el cociente de diferencias es la pendiente de la línea no tan tangente que cruza ($x + h$, $f(x + h)$). A medida que h es más pequeño, la línea no tan tangente se acerca cada vez más a la línea tangente (figura 8.2).

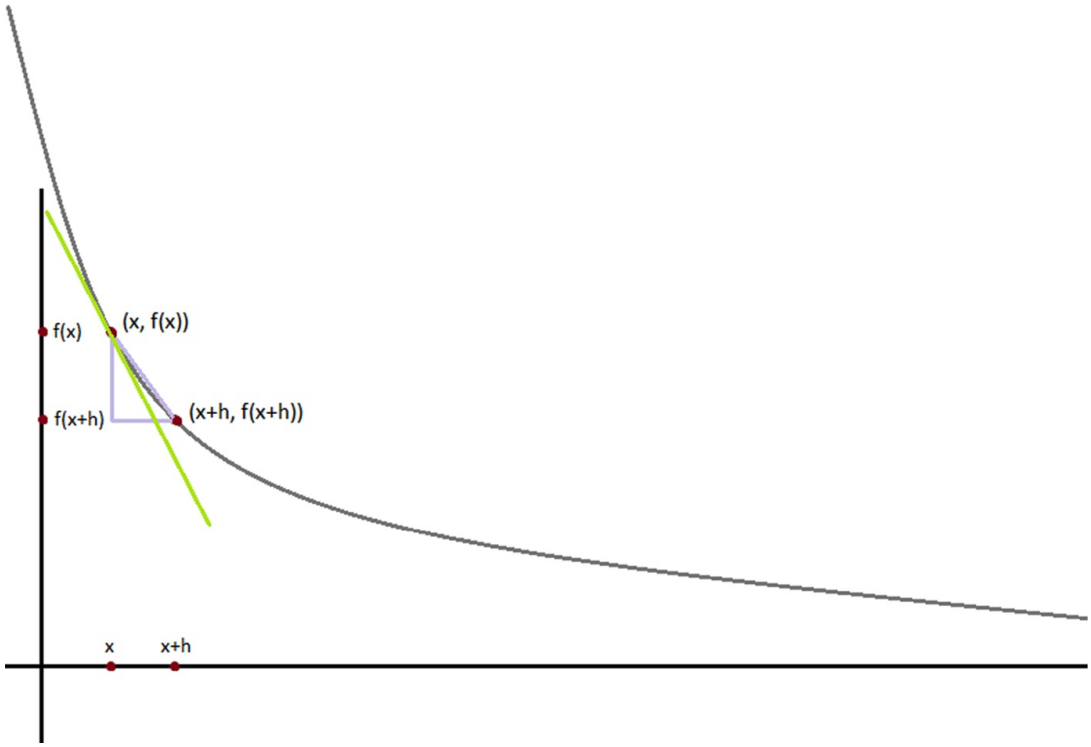


Figura 8.2. Aproximar una derivada con un cociente de diferencias.

Para muchas funciones es fácil calcular las derivadas con exactitud. Por ejemplo, la función `square`:

```
def square(x: float) -> float:
   return x * x
```

Tiene la derivada:

```
def derivative(x: float) -> float:
   return 2 * x
```

Que nos resulta sencillo comprobar calculando de manera explícita el cociente de diferencias y tomando el límite (lo que no requiere nada más que un poco de álgebra de instituto).

¿Qué pasa si no podemos (o no queremos) encontrar el gradiente? Aunque no podemos tomar límites en Python, podemos estimar derivadas evaluando el cociente de diferencias para un valor e muy pequeño. La figura 8.3 muestra los resultados de una estimación así:

```
xs = range(-10, 11)
actuals = [derivative(x) for x in xs]
estimates = [difference_quotient(square, x, h=0.001) for x in xs]
# se traza para mostrar que son básicamente lo mismo
import matplotlib.pyplot as plt
plt.title("Actual Derivatives vs. Estimates")
plt.plot(xs, actuals, ‘rx’, label=’Actual’)             # rojo x
plt.plot(xs, estimates, ‘b+’, label=’Estimate’)         # azul +
plt.legend(loc=9)
plt.show()
```

Cuando `f` es una función de muchas variables, tiene varias derivadas parciales, cada una de las cuales indica cómo cambia `f` cuando realizamos pequeñas modificaciones en tan solo una de las variables de entrada.

Calculamos su *i* derivada parcial tratándola como una función solo de su *i* variable, manteniendo fijas el resto de variables:

```
def partial_difference_quotient(f: Callable[[Vector], float],
                        v: Vector,
                        i: int,
                        h: float) -> float:
   """Returns the i-th partial difference quotient of f at v"""
   w = [v_j + (h if j == i else 0)     # suma h al i elemento de v
      for j, v_j in enumerate(v)]
   return (f(w)–f(v)) / h
```

Tras lo cual podemos estimar el gradiente del mismo modo:

```
def estimate_gradient(f: Callable[[Vector], float],
      v: Vector,
      h: float = 0.0001):
   return [partial_difference_quotient(f, v, i, h)
         for i in range(len(v))]
```

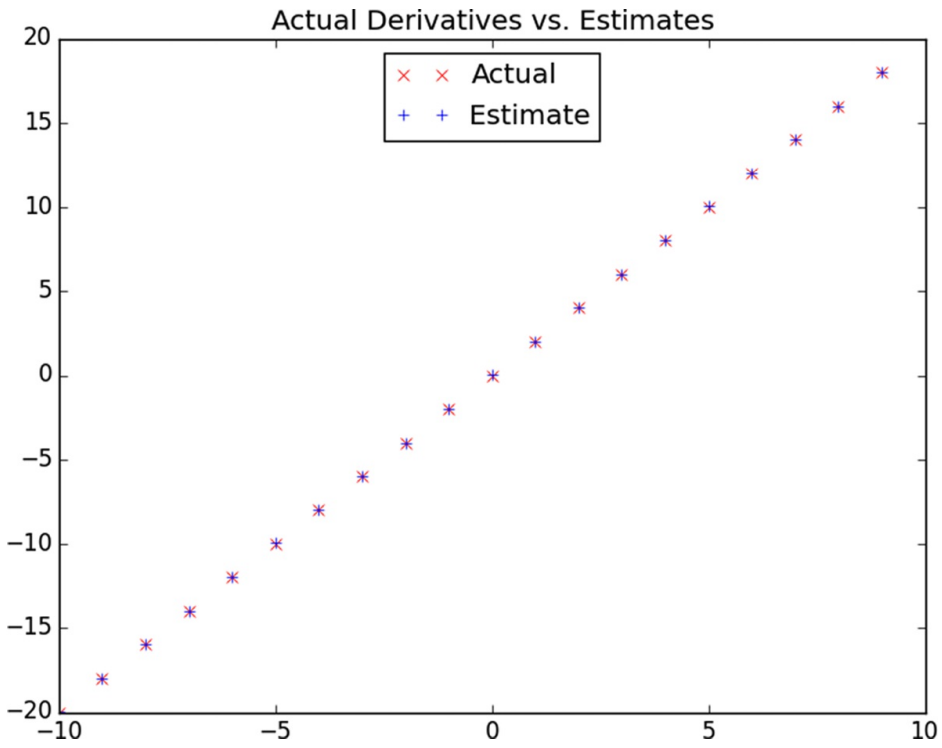


Figura 8.3. La bondad de la aproximación del cociente de diferencias.

Nota: Un inconveniente a tener en cuenta de este método de "estimación mediante cocientes de diferencias" es que resulta caro desde el punto de vista computacional. Si `v` tiene una longitud *n*, `estimate_gradient` tiene que evaluar `f` en 2*n* entradas distintas. Si estamos estimando gradientes repetidamente, estaremos trabajando demasiado. En todo lo que hagamos, emplearemos las matemáticas para calcular nuestras funciones de gradiente de manera explícita.

Utilizar el gradiente

Es fácil darse cuenta de que la función `sum_of_squares` es menor cuando su entrada `v` es un vector de ceros. Pero imaginemos que no sabíamos eso. Utilicemos los gradientes para hallar el mínimo entre todos los vectores tridimensionales. Elegiremos simplemente un punto de inicio aleatorio y después daremos pequeños pasos en la dirección opuesta al gradiente hasta alcanzar un punto en el que el gradiente sea muy pequeño:

```
import random
from scratch.linear_algebra import distance, add, scalar_multiply
def gradient_step(v: Vector, gradient: Vector, step_size: float) -> Vector:
   """Moves 'step_size' in the 'gradient' direction from 'v'"""
   assert len(v) == len(gradient)
   step = scalar_multiply(step_size, gradient)
   return add(v, step)
def sum_of_squares_gradient(v: Vector) -> Vector:
   return [2 * v_i for v_i in v]
# elige un punto de inicio aleatorio
v = [random.uniform(-10, 10) for i in range(3)]
for epoch in range(1000):
   grad = sum_of_squares_gradient(v)       # calcula el gradiente en v
   v = gradient_step(v, grad, -0.01)       # da un paso de gradiente negativo
   print(epoch, v)
assert distance(v, [0, 0, 0]) < 0.001      # v debería estar cerca de 0
```

Si ejecutamos esto, veremos que siempre termina con un valor `v` muy próximo a `[0,0,0]`. Cuántas más veces se ejecute, más se acercará.

Elegir el tamaño de paso adecuado

Aunque los motivos para alejarnos del gradiente están claros, lo que no queda claro es lo lejos que queremos llegar. Sin duda, elegir el tamaño de paso correcto es más un arte que una ciencia. Entre las opciones más conocidas están las siguientes:

- Utilizar un tamaño de paso fijo.
- Ir encogiendo el tamaño de paso gradualmente con el tiempo.
- En cada paso, elegir el tamaño de paso que minimice el valor de la función objetivo.

El último método suena muy bien, pero, en la práctica, es un cálculo muy costoso. Para simplificar las cosas, utilizaremos la mayoría de las veces un tamaño de paso fijo. El tamaño de paso que “funcione” depende del problema: demasiado pequeño, y el descenso de gradiente se mantendrá para siempre; demasiado grande, y tendremos que dar pasos enormes que podrían

lograr que la función que nos ocupa sea cada vez más grande o incluso que se quede sin definir. Por lo tanto, hay que experimentar.

Utilizar descenso de gradiente para ajustar modelos

En este libro, utilizaremos el descenso de gradiente para ajustar modelos paramétricos a los datos. Lo más habitual es que tengamos un conjunto de datos y un modelo (hipotético) para los datos que depende (de una forma diferenciada) de uno o más parámetros. También tendremos una función de pérdida que mide lo bien que se ajusta el modelo a nuestros datos (menor es mejor). Si pensamos que nuestros datos son fijos, entonces nuestra función de pérdida nos indica lo buenos o malos que son los parámetros de un modelo cualquiera. Esto significa que podemos utilizar el descenso de gradiente para encontrar los parámetros del modelo que minimicen al máximo la pérdida. Veamos un sencillo ejemplo:

```
# x va de -50 a 49, y es siempre 20 * x + 5
inputs = [(x, 20 * x + 5) for x in range(-50, 50)]
```

En este caso, conocemos los parámetros de la relación lineal entre `x` e `y`, pero imaginemos que queremos descubrirlos a partir de los datos. Utilizaremos el descenso de gradiente para hallar la pendiente y la intersección que minimizan el error cuadrático medio.

Empezaremos con una función que determina el gradiente basándose en el error a partir de un solo punto de datos:

```
def linear_gradient(x: float, y: float, theta: Vector) -> Vector:
   slope, intercept = theta
   predicted = slope * x + intercept     # La predicción del modelo.
   error = (predicted-y)                 # el error es (previsto-real).
   squared_error = error ** 2            # Minimizaremos el error cuadrático
   grad = [2 * error * x, 2 * error]     # usando su gradiente.
   return grad
```

Pensemos en lo que significa el gradiente. Imaginemos que para un cierto

`x` nuestra predicción es demasiado grande. En ese caso, el `error` es positivo. El segundo término de gradiente, `2 * error`, es positivo, lo que refleja el hecho de que pequeños incrementos harán que la predicción (ya demasiado grande) sea aún más grande, lo que provocará que el error cuadrático (para este `x`) sea aún mayor.

El primer término de gradiente, `2 * error * x`, tiene el mismo signo que `x`. Por supuesto que, si `x` es positivo, los pequeños incrementos en la pendiente harán de nuevo que la predicción (y de ahí el error) sea más grande. Si `x` es negativo, sin embargo, esos pequeños incrementos harán que la predicción (y por lo tanto el error) sea más pequeña.

Pero ese cálculo está hecho para un solo punto de datos. Para el conjunto completo miraremos el error cuadrático medio; el gradiente del error cuadrático medio no es más que la media de los gradientes individuales.

Así, esto es lo que vamos a hacer:

1. Empezar con un valor aleatorio para `theta`.
2. Calcular la media de los gradientes.
3. Ajustar `theta` en esa dirección.
4. Repetir.

Tras muchos *epochs* (como llamamos a cada pasada por el conjunto de datos), descubriríamos algo parecido a los parámetros correctos:

```
from scratch.linear_algebra import vector_mean
# Empieza con valores aleatorios para pendiente e intersección
theta = [random.uniform(-1, 1), random.uniform(-1, 1)]
learning_rate = 0.001
for epoch in range(5000):
   # Calcula la media de los gradientes
   grad = vector_mean([linear_gradient(x, y, theta) for x, y in inputs])
   # Da un paso en esa dirección
   theta = gradient_step(theta, grad, -learning_rate)
   print(epoch, theta)
slope, intercept = theta
assert 19.9 < slope < 20.1, "slope should be about 20"
assert 4.9 < intercept < 5.1, "intercept should be about 5"
```

Descenso de gradiente en minilotes y estocástico

Un inconveniente del método anterior es que hemos tenido que evaluar los gradientes en el conjunto de datos entero antes de poder dar un paso de gradiente y actualizar nuestros parámetros. En este caso estaba bien, porque nuestro conjunto de datos contenía solamente 100 pares y el cálculo del gradiente resultó barato.

Pero otros modelos tendrán con frecuencia grandes conjuntos de datos y caros cálculos de gradientes. En ese caso, nos interesará dar pasos de gradiente más a menudo.

Podemos hacerlo utilizando una técnica denominada descenso de gradiente en minilotes (o *minibatch*), en la que calculamos el gradiente (y damos un paso de gradiente) basándonos en un "minilote" muestreado del conjunto de datos principal:

```
from typing import TypeVar, List, Iterator
T = TypeVar('T')                          # nos permite escribir funciones
                                          "genéricas"
def minibatches(dataset: List[T],
                batch_size: int,
                shuffle: bool = True) -> Iterator[List[T]]:
  """Generates 'batch_size'-sized minibatches from the dataset"""
  # inicia índices 0, batch_size, 2 * batch_size, ...
  batch_starts = [start for start in range(0, len(dataset), batch_size)]
  if shuffle:                             # mezcla los lotes
  random.shuffle(batch_starts)
  for start in batch_starts:
    end = start + batch_size
    yield dataset[start:end]
```

Nota: `TypeVar (T)` nos permite crear una función "genérica", que dice que nuestro conjunto de datos puede ser una lista de cualquier tipo sencillo (`str`, `int`, `list`, lo que sea), pero, sea cual sea el tipo, los resultados serán lotes de él.

Podemos resolver nuestro problema de nuevo utilizando minilotes:

```
theta = [random.uniform(-1, 1), random.uniform(-1, 1)]
```

```
for epoch in range(1000):
   for batch in minibatches(inputs, batch_size=20):
      grad = vector_mean([linear_gradient(x, y, theta) for x, y in batch])
      theta = gradient_step(theta, grad, -learning_rate)
   print(epoch, theta)
slope, intercept = theta
assert 19.9 < slope < 20.1, "slope should be about 20"
assert 4.9 < intercept < 5.1, "intercept should be about 5"
```

Otra variante es el descenso de gradiente estocástico, en el que se dan pasos de gradiente basados en un ejemplo de entrenamiento cada vez:

```
theta = [random.uniform(-1, 1), random.uniform(-1, 1)]
for epoch in range(100):
   for x, y in inputs:
      grad = linear_gradient(x, y, theta)
      theta = gradient_step(theta, grad, -learning_rate)
   print(epoch, theta)
slope, intercept = theta
assert 19.9 < slope < 20.1, "slope should be about 20"
assert 4.9 < intercept < 5.1, “intercept should be about 5”
```

En este problema, el descenso de gradiente estocástico encuentra los parámetros óptimos en un número de *epochs* mucho menor. Pero siempre hay inconvenientes. Basar los pasos de gradiente en pequeños minilotes (o en puntos de datos sencillos) permite dar más pasos, pero el gradiente para un solo punto podría residir en una dirección muy distinta a la del gradiente para el conjunto completo de datos.

Además, si no estuviéramos creando nuestra álgebra lineal desde cero, se producirían mejoras en el rendimiento por “vectorizar” nuestros cálculos a lo largo de lotes en lugar de calcular el gradiente un punto cada vez.

A lo largo del libro, jugaremos a buscar y encontrar tamaños de lote y de paso óptimos.

Nota: La terminología para los distintos tipos de descensos de gradiente no es uniforme. El método “calcular el gradiente para el conjunto de datos completo” se suele denominar descenso de gradiente en lotes, y algunas personas dicen descenso de gradiente estocástico cuando se quieren referir a la versión de minilotes (cuya versión “un punto cada vez” es un caso especial).

Para saber más

- ¡Siga leyendo! Utilizaremos el descenso de gradiente para resolver problemas en el resto del libro.
- A estas alturas, es casi seguro que ya se ha hartado de leerme recomendándole que lea libros de texto. Si le sirve de consuelo, *Active Calculus 1.0*, en `https://scholarworks.gvsu.edu/books/10/`, de Matthew Boelkins, David Austin y Steven Schlicker (Bibliotecas de la Universidad Estatal de Grand Valley), parece más interesante que los libros de texto con los que yo aprendí.
- Sebastian Ruder tiene una entrada épica en su blog, en `http://ruder.io/optimizing-gradient-descent/index.html`, comparando el descenso de gradiente y sus distintas variantes.

9 Obtener datos

Para escribirlo, necesité tres meses; para concebirlo, tres minutos; para recoger los datos contenidos en él, toda mi vida.

—F. Scott Fitzgerald

Para ser científico de datos se necesitan datos. De hecho, como científico de datos se pasará una fracción indecorosamente grande de su tiempo adquiriendo, limpiando y transformando datos. Si no hay más remedio, puede escribirlos usted mismo (o si tiene minions a su disposición, mejor que lo hagan ellos), pero este no es un buen uso de su tiempo. En este capítulo, veremos distintas formas de obtener datos en Python y en los formatos adecuados.

stdin y stdout

Si ejecutamos nuestros *scripts* de Python en la línea de comandos, se pueden canalizar los datos a través de ellos utilizando `sys.stdin` y `sys.stdout`. Por ejemplo, esta es una secuencia de comandos que lee líneas de texto y devuelve las que coinciden con una expresión regular:

```
# egrep.py
import sys, re
# sys.argv es la lista de argumentos de línea de comandos
# sys.argv[0] es el nombre del propio programa
# sys.argv[1] será el regex especificado en la línea de comandos
regex = sys.argv[1]
# por cada línea pasada al script
for line in sys.stdin:
   # si coincide con el regex, lo graba en stdout
   if re.search(regex, line):
      sys.stdout.write(line)
```

Y aquí tenemos un fragmento de código que cuenta las líneas que recibe y devuelve el total:

```
# line_count.py
import sys
count = 0
for line in sys.stdin:
   count += 1
# print va a sys.stdout
print(count)
```

Podríamos entonces utilizar estos *scripts* para contar cuántas líneas de un archivo contienen números. En Windows emplearíamos:

```
type SomeFile.txt | python egrep.py “[0-9]” | python line_count.py
```

Mientras que en un sistema Unix se transformaría en:

```
cat SomeFile.txt | python egrep.py “[0-9]” | python line_count.py
```

El carácter | (barra vertical) es el denominado *pipe*, que significa “utilizar la salida del comando izquierdo como entrada del comando derecho”. De este modo se pueden construir *pipelines* (o tuberías) de proceso de datos.

Nota: Si utilizamos Windows, es probable que podamos omitir la parte de Python de este comando:

```
type SomeFile.txt | egrep.py “[0-9]” | line_count.py
```

Si trabajamos en un sistema Unix, hacer esto mismo requiere un par de pasos adicionales.[1] Primero, añadimos un “shebang” como primera línea del *script* `#!/usr/bin/env python`. Después, en la línea de comandos, utilizamos `chmod` x egrep.py++ para convertir el archivo en ejecutable.

De forma similar, este es un *script* que cuenta las palabras de su entrada y devuelve las más comunes:

```
# most_common_words.py
import sys
from collections import Counter
# pasa el número de palabras como primer argumento
```

```
try:
   num_words = int(sys.argv[1])
except:
   print("usage: most_common_words.py num_words")
   sys.exit(1)                              # un código de salida no cero indica
                                            error
counter = Counter(word.lower()                  # palabras en minúscula
             for line in sys.stdin
             for word in                        # divide por espacios
             line.strip().split()
             if word)                           # salta las 'palabras' vacías
for word, count in counter.most_common(num_words):
   sys.stdout.write(str(count))
   sys.stdout.write("\t")
   sys.stdout.write(word)
   sys.stdout.write("\n")
```

Después de esto, se podría hacer algo así:

```
$ cat the_bible.txt | python most_common_words.py 10
36397                    the
30031                    and
20163                    of
7154                     to
6484                     in
5856                     that
5421                     he
5226                     his
5060                     unto
4297                     shall
```

(Si utiliza Windows, emplee `type` en lugar de `cat`).

Nota: Si es un experto programador de Unix, probablemente estará familiarizado con una gran variedad de herramientas de línea de comandos (por ejemplo, `egrep`), integradas en el sistema operativo, que es preferible crear desde cero. Aun así, es bueno saber que puede si lo necesita.

Leer archivos

También es posible leer archivos y escribir en ellos de forma explícita directamente en el código. Python simplifica bastante el trabajo con archivos.

Conocimientos básicos de los archivos de texto

El primer paso para trabajar con un archivo de texto es obtener un objeto archivo utilizando `open`:

```
# ‘r’ significa solo lectura, se da por sentado si se omite
file_for_reading = open(‘reading_file.txt’, ‘r’)
file_for_reading2 = open(‘reading_file.txt’)
# ‘w’ es escribir — ¡destruirá el archivo si ya existe!
file_for_writing = open(‘writing_file.txt’, ‘w’)
# ‘a’ es añadir – para añadir al final del archivo
file_for_appending = open(‘appending_file.txt’, ‘a’)
# no olvide cerrar sus archivos al terminar
file_for_writing.close()
```

Como es fácil olvidarse de cerrar los archivos, siempre se deben utilizar con un bloque `with`, al final del cual se cerrarán automáticamente:

```
with open(filename) as f:
   data = function_that_gets_data_from(f)
# en este momento f ya se ha cerrado, así que no trate de usarlo
process(data)
```

Si hace falta leer un archivo de texto completo, basta simplemente con pasar varias veces por las líneas del archivo utilizando `for`:

```
starts_with_hash = 0
with open(‘input.txt’) as f:
   for line in f:                  # mira cada línea del archivo
      if re.match("^#",line):      # usa un regex para ver si empieza por ‘#’
         starts_with_hash += 1     # si es así, suma 1 al total
```

Todas las líneas obtenidas así terminan en un carácter de línea nueva, así que con frecuencia nos interesará limpiarlas con un `strip` antes de hacer nada con ellas.

Por ejemplo, imaginemos que tenemos un archivo lleno de direcciones de correo electrónico, una por línea, y necesitamos generar un histograma de los dominios. Las reglas para extraer dominios correctamente son algo sutiles (vea, por ejemplo, la lista de sufijos públicos `https://publicsuffix.org`), pero una buena primera aproximación es coger las partes de las direcciones de correo que vienen después del @ (lo que da la respuesta equivocada con direcciones como joel@mail.datasciencester.com, pero solo para este ejemplo estamos dispuestos a vivir con ello):

```
def get_domain(email_address: str) -> str:
   """Split on '@' and return the last piece"""
   return email_address.lower().split("@")[-1]

# un par de pruebas
assert get_domain('joelgrus@gmail.com') == 'gmail.com'
assert get_domain('joel@m.datasciencester.com') == 'm.datasciencester.com'

from collections import Counter

with open('email_addresses.txt', 'r') as f:
   domain_counts = Counter(get_domain(line.strip())
                   for line in f
                   if "@" in line)
```

Archivos delimitados

Las direcciones de correo electrónico hipotéticas que acabamos de procesar solo tenían una dirección por línea. Lo más habitual es que los archivos tengan muchos datos en cada línea. Estos archivos suelen estar separados (o delimitados) por comas o tabuladores: cada línea tiene varios campos, con una coma o un tabulador indicando el lugar en el que termina un campo y empieza el siguiente.

Esto empieza a complicarse cuando se tienen campos que incluyen comas, tabuladores y líneas nuevas (algo que ocurrirá forzosamente). Por esta razón, no conviene nunca intentar analizarlos uno mismo. Es mejor utilizar el módulo `csv` de Python (o la librería pandas, o cualquier otra diseñada para leer archivos delimitados por comas o tabuladores).

Advertencia: ¡Nunca analice usted solo un archivo delimitado por comas! ¡Se cargará los casos límite!

Si el archivo no tiene encabezados (lo que significa que probablemente nos interese cada fila como una `list`, y lo que además nos obliga a saber lo que hay en cada columna), se puede utilizar `csv.reader` para pasar varias veces por las filas, cada una de las cuales será una lista adecuadamente dividida.

Si tuviéramos por ejemplo un archivo delimitado por tabuladores de precios de acciones:

```
6/20/2014 AAPL 90.91
6/20/2014 MSFT 41.68
6/20/2014 FB  64.5
6/19/2014 AAPL 91.86
6/19/2014 MSFT 41.51
6/19/2014 FB  64.34
```

Podríamos procesar las líneas con:

```
import csv
with open('tab_delimited_stock_prices.txt') as f:
   tab_reader = csv.reader(f, delimiter='\t')
   for row in tab_reader:
      date = row[0]
      symbol = row[1]
      closing_price = float(row[2])
      process(date, symbol, closing_price)
```

Si el archivo tiene encabezados:

```
date:symbol:closing_price
6/20/2014:AAPL:90.91
6/20/2014:MSFT:41.68
6/20/2014:FB:64.5
```

Se puede omitir la fila del encabezado con una llamada inicial a `reader.next`, o bien obtener cada fila como un `dict` (con los encabezados como claves) utilizando `csv.DictReader`:

```
with open(‘colon_delimited_stock_prices.txt’) as f:
   colon_reader = csv.DictReader(f, delimiter=’:’)
   for dict_row in colon_reader:
      date = dict_row["date"]
      symbol = dict_row["symbol"]
      closing_price = float(dict_row["closing_price"])
      process(date, symbol, closing_price)
```

Aunque el archivo no tuviera encabezados, aún podríamos utilizar `DictReader` pasándole las claves como un parámetro `fieldnames`.

También se pueden escribir datos delimitados utilizando `csv.writer`:

```
todays_prices = {‘AAPL’: 90.91, ‘MSFT’: 41.68, ‘FB’: 64.5 }
with open(‘comma_delimited_stock_prices.txt’, ‘w’) as f:
   csv_writer = csv.writer(f, delimiter=’,’)
   for stock, price in todays_prices.items():
        csv_writer.writerow([stock, price])
```

`csv.writer` hará lo correcto si los campos contienen comas. Escribirlos a mano probablemente no servirá. Por ejemplo, si intentamos lo siguiente:

```
results = [["test1", "success", "Monday"],
        ["test2", "success, kind of", "Tuesday"],
        ["test3", "failure, kind of", "Wednesday"],
        ["test4", "failure, utter", "Thursday"]]
# ¡no haga esto!
with open(‘bad_csv.txt’, ‘w’) as f:
   for row in results:
      f.write(",".join(map(str,            # ¡podría contener demasiadas comas!
      row)))
      f.write("\n")                        # ¡la fila podría tener líneas
                                           nuevas!
```

Terminará con un archivo `.csv` parecido a esto:

```
test1,success,Monday
test2,success, kind of,Tuesday
test3,failure, kind of,Wednesday
test4,failure, utter,Thursday
```

Al que nadie podrá dar sentido.

Raspado web

Otra forma de conseguir datos es extrayéndolos de páginas web mediante el método del raspado web (*web scraping*). Parece que conseguir páginas web es bastante fácil; otra cosa es obtener de ellas información estructurada con sentido.

HTML y su análisis

Las páginas de la web están escritas en HTML, donde el texto (preferiblemente) se marca con elementos y sus atributos:

```
<html>
  <head>
    <title>Una página web</title>
  </head>
  <body>
    <p id="author">Joel Grus</p>
    <p id="subject">Ciencia de datos</p>
  </body>
</html>
```

En un mundo perfecto, en el que todas las páginas web estuvieran marcadas semánticamente para nuestro beneficio, podríamos extraer datos utilizando reglas como "encuentra el elemento `<p>` cuyo `id` es `subject` y devuelve el texto que contiene". Pero, en la realidad, HTML no suele estar bien formado, y no digamos bien comentado. Esto significa que necesitaremos ayuda para darle sentido.

Para extraer datos de HTML, utilizaremos la librería Beautiful Soup,[2] que construye un árbol con los distintos elementos de una página web y ofrece una sencilla interfaz para acceder a ellos. La última versión en el momento de escribir este libro es Beautiful Soup 4.6.0, que es la que utilizaremos. También vamos a emplear la librería Requests,[3] una forma mucho más interesante de hacer peticiones HTTP que nada que esté integrado en Python.

El analizador de HTML integrado en Python no es tan tolerante, es decir, que no siempre se lleva bien con código HTML que no esté perfectamente

formado. Por esa razón, instalaremos además el analizador `html5lib`.

Asegurándonos de estar en el entorno virtual correcto, instalamos las librerías:

```
python -m pip install beautifulsoup4 requests html5lib
```

Para utilizar Beautiful Soup, pasamos una cadena de texto que contiene HTML a la función `BeautifulSoup`. En nuestros ejemplos, este será el resultado de una llamada a `requests.get`:

```
from bs4 import BeautifulSoup
import requests
# Pongo el archivo HTML en GitHub. Para encajar
# la URL en el libro tuve que dividirla en dos líneas.
# Recuerde que las cadenas de texto whitespace-separated se concatenan.
url = ("https://raw.githubusercontent.com/"
     "joelgrus/data/master/getting-data.html")
html = requests.get(url).text
soup = BeautifulSoup(html, 'html5lib')
```

Tras de lo cual podemos llegar bastante lejos utilizando unos cuantos métodos sencillos.

Normalmente trabajaremos con objetos `Tag`, que corresponden a las etiquetas que representan la estructura de una página HTML.

Por ejemplo, para encontrar la primera etiqueta `<p>` (y su contenido), se puede utilizar:

```
first_paragraph = soup.find('p')      # o simplemente soup.p
```

Se puede obtener el contenido de texto de una `Tag` utilizando su propiedad `text`:

```
first_paragraph_text = soup.p.text
first_paragraph_words = soup.p.text.split()
```

Y se pueden extraer los atributos de una etiqueta tratándola como un `dict`:

```
first_paragraph_id = soup.p['id']           # da un KeyError si no hay 'id'
first_paragraph_id2 = soup.p.get('id')      # devuelve None si no hay 'id'
```

Se pueden conseguir varias etiquetas al mismo tiempo del siguiente modo:

```
all_paragraphs = soup.find_all('p')      # o simplemente soup('p')
paragraphs_with_ids = [p for p in soup('p') if p.get('id')]
```

Con frecuencia nos vendrá bien encontrar etiquetas con una determinada `class`:

```
important_paragraphs = soup('p', {'class' : 'important'})
important_paragraphs2 = soup('p', 'important')
important_paragraphs3 = [p for p in soup('p')
                  if 'important' in p.get('class', [])]
```

Y también es posible combinar estos métodos para implementar una lógica más elaborada. Por ejemplo, si queremos encontrar todos los elementos `<span>` contenidos dentro de un elemento `<div>`, podríamos hacer lo siguiente:

```
# Atención: devolverá el mismo <span> varias veces
# si está dentro de varios <div>s.
# Sea más listo si ocurre esto.
spans_inside_divs = [span
               for div in soup('div')        # por cada <div> de la página
               for span in div('span')]      # halla los <span> contenidos
```

Solo este montón de funciones ya nos permitirá hacer bastante. Si finalmente hay que hacer cosas más complicadas (o simplemente si tiene curiosidad), eche un vistazo a la documentación que encontrará en https://www.crummy.com/software/BeautifulSoup/bs4/doc/.

Sin duda alguna, los datos importantes no van a estar etiquetados, claro está, como `class="important"`. Será necesario inspeccionar con detalle el código fuente HTML, razonar sobre la lógica de selección y preocuparse de los casos límite para estar seguros de que los datos son correctos. Veamos un ejemplo.

Ejemplo: Controlar el congreso

El vicepresidente de Política de DataSciencester está preocupado por las posibles regulaciones del sector de la ciencia de datos y le pide que cuantifique lo que dice el Congreso de los Estados Unidos sobre el tema. En especial quiere que encuentre a todos los representantes que tengan notas de prensa sobre “datos”.

En el momento de la publicación de este libro, existe la página `https://www.house.gov/representatives` con enlaces a los sitios web de todos los representantes.

Si visualizamos el código, todos los enlaces tienen este aspecto:

```
<td>
   <a href="https://jayapal.house.gov">Jayapal, Pramila</a>
</td>
```

Empecemos recopilando todas las URL a las que se enlaza desde esa página:

```
from bs4 import BeautifulSoup
import requests
url = "https://www.house.gov/representatives"
text = requests.get(url).text
soup = BeautifulSoup(text, "html5lib")
all_urls = [a['href']
        for a in soup('a')
        if a.has_attr('href')]
print(len(all_urls))     # 965 para mí, demasiadas
```

Esto devuelve demasiadas URL. Si les echamos un vistazo, las que queremos empiezan con `http://` o `https://`, tienen después algún nombre y terminan con `.house.gov` o `.house.gov/`.

Es un buen momento para utilizar una expresión regular:

```
import re
# Debe empezar con http:// o https://
# Debe terminar con .house.gov o .house.gov/
regex = r"^https?://.*\.house\.gov/?$"
# ¡Escribamos algunas pruebas!
assert re.match(regex, "http://joel.house.gov")
assert re.match(regex, "https://joel.house.gov")
```

```
assert re.match(regex, "http://joel.house.gov/")
assert re.match(regex, "https://joel.house.gov/")
assert not re.match(regex, "joel.house.gov")
assert not re.match(regex, "http://joel.house.com")
assert not re.match(regex, "https://joel.house.gov/biography")
# Ahora aplicamos
good_urls = [url for url in all_urls if re.match(regex, url)]
print(len(good_urls))              # aun 862 para mi
```

Siguen siendo demasiados, ya que solamente hay 435 representantes. Si miramos la lista, hay muchos duplicados. Utilicemos `set` para deshacernos de ellos:

```
good_urls = list(set(good_urls))
print(len(good_urls))# solo 431 para mi
```

Siempre hay un par de escaños libres en la cámara, o quizá haya algún representante sin sitio web. En cualquier caso, con esto es suficiente. Al revisar los sitios, vemos que la mayoría de ellos tienen un enlace a notas de prensa. Por ejemplo:

```
html = requests.get('https://jayapal.house.gov').text
soup = BeautifulSoup(html, 'html5lib')
# Usa un conjunto porque los enlaces podrían aparecer varias veces.
links = {a['href'] for a in soup('a') if 'press releases' in a.text.lower()}
print(links)           # {'/media/press-releases'}
```

Hay que tener en cuenta que es un enlace relativo, es decir, tenemos que recordar el sitio del que se originó. Hagamos un poco de raspado:

```
from typing import Dict, Set
press_releases: Dict[str, Set[str]] = {}
for house_url in good_urls:
   html = requests.get(house_url).text
   soup = BeautifulSoup(html, 'html5lib')
   pr_links = {a['href'] for a in soup('a') if 'press releases'
                                 in a.text.lower()}
   print(f"{house_url}: {pr_links}")
   press_releases[house_url] = pr_links
```

Nota: Normalmente, no es muy correcto raspar un sitio libremente de esta forma. La mayoría de los sitios web tienen un archivo `robots.txt` que indica la frecuencia con la que se pueden extraer datos del sitio (y las rutas en las que se supone que no se debe hacer), pero, como es el Congreso, no hace falta que seamos especialmente educados.

Si los observamos según van apareciendo, veremos muchos `/media/press-releases` y `media-center/press-releases`, además de otras direcciones varias. Una de estas URL es `https://jayapal.house.gov/media/press-releases`.

Conviene recordar que nuestro objetivo es averiguar qué congresistas tienen notas de prensa que contienen "datos" (*data* en inglés). Escribiremos una función algo más general que verifique si una página de notas de prensa menciona un determinado término.

Visitando el sitio y revisando el código fuente, parece que haya un fragmento de cada nota de prensa dentro de una etiqueta `<p>`, de modo que utilizaremos esto como primer intento:

```
def paragraph_mentions(text: str, keyword: str) -> bool:
   """
   Returns True if a <p> inside the text mentions {keyword}
   """
   soup = BeautifulSoup(text, 'html5lib')
   paragraphs = [p.get_text() for p in soup('p')]
   return any(keyword.lower() in paragraph.lower()
           for paragraph in paragraphs)
```

Preparemos una prueba rápida de esto:

```
text = """<body><h1>Facebook</h1><p>Twitter</p>"""
assert paragraph_mentions(text, "twitter")     # está dentro de una <p>
assert not paragraph_mentions(text, "facebook")# no está dentro de una <p>
```

Finalmente, estamos listos para encontrar a los congresistas que buscábamos y dar sus nombres al vicepresidente:

```
for house_url, pr_links in press_releases.items():
   for pr_link in pr_links:
```

```
    url = f"{house_url}/{pr_link}"
    text = requests.get(url).text
    if paragraph_mentions(text, 'data'):
       print(f"{house_url}")
       break        # lista esta house_url
```

Al ejecutar este fragmento de código obtenemos una lista de 20 representantes. Probablemente otra persona obtenga un resultado distinto.

> **Nota:** Si revisamos las distintas páginas de "notas de prensa" (*press releases* en inglés), la mayoría están paginadas solo con 5 o 10 notas de prensa por página. Esto significa que solamente hemos recuperado las notas de prensa más recientes para cada congresista. Una solución más meticulosa habría pasado varias veces por las páginas y recuperado el texto completo de cada nota de prensa.

Utilizar API

Muchos sitios y servicios web ofrecen interfaces de programación de aplicaciones o API (*Application Programming Interfaces*), que permiten solicitar de manera explícita datos en un formato estructurado (¡lo que nos ahorra el problema de tener que rasparlos!).

JSON y XML

Como HTTP es un protocolo para transferir texto, los datos que se soliciten a través de una API web se tienen que serializar en un formato de cadena de texto. Con frecuencia esta serialización emplea la notación de objeto de JavaScript o JSON (*JavaScript Object Notation*). Los objetos de JavaScript son bastante parecidos a las clases `dict` de Python, lo que facilita la interpretación de sus representaciones de texto:

```
{ "title" : "Data Science Book",
  "author" : "Joel Grus",
  "publicationYear" : 2019,
```

```
   "topics" : [ "data", "science", "data science"] }
```

Podemos analizar JSON utilizando el módulo `json` de Python. En particular, utilizaremos su función `loads`, que deserializa una cadena de texto que representa un objeto JSON y la transforma en un objeto Python:

```
import json
serialized = """{ "title" : "Data Science Book",
             "author" : "Joel Grus",
             "publicationYear" : 2019,
             "topics" : [ "data", "science", "data science"] }"""
# analiza JSON para crear un dict de Python
deserialized = json.loads(serialized)
assert deserialized["publicationYear"] == 2019
assert "data science" in deserialized["topics"]
```

A veces, un proveedor de API te odia y solamente ofrece respuestas en XML:

```
<Book>
   <Title>Data Science Book</Title>
   <Author>Joel Grus</Author>
   <PublicationYear>2014</PublicationYear>
   <Topics>
      <Topic>data</Topic>
      <Topic>science</Topic>
      <Topic>data science</Topic>
   </Topics>
</Book>
```

Puede utilizar Beautiful Soup para obtener datos de XML de forma parecida a como lo usamos para obtener datos de HTML; revise su documentación para más detalles.

Utilizar una API no autenticada

La mayoría de las API actuales exigen que primero se autentifique uno mismo antes de poder utilizarlas. Aunque no envidiamos esta política, crea una gran cantidad de información adicional que enturbia nuestra exposición.

Según esto, empezaremos echando un vistazo a la API de GitHub,[4] con la que podemos hacer algunas cosas sencillas sin necesitar de autenticación:

```
import requests, json
github_user = "joelgrus"
endpoint = f"https://api.github.com/users/{github_user}/repos"
repos = json.loads(requests.get(endpoint).text)
```

Debo decir que `repos` es una `list` de clases `dict` de Python, que representa cada una un repositorio público en mi cuenta de GitHub (coloque con toda libertad su nombre de usuario y obtenga su propio repositorio de GitHub; porque, tiene cuenta en GitHub, ¿verdad?).

Podemos utilizar esto para averiguar en qué meses y días del año es más probable que yo cree un repositorio. El único inconveniente es que las fechas de la respuesta son cadenas de texto:

```
“created_at”: “2013-07-05T02:02:28Z”
```

Python no incluye un analizador de fechas demasiado bueno, así que tendremos que instalar uno:

```
python -m pip install python-dateutil
```

A partir del cual es probable que solo se necesite la función `dateutil.parser.parse`:

```
from collections import Counter
from dateutil.parser import parse
dates = [parse(repo["created_at"]) for repo in repos]
month_counts = Counter(date.month for date in dates)
weekday_counts = Counter(date.weekday() for date in dates)
```

De forma similar, se pueden conseguir los lenguajes de mis cinco últimos repositorios:

```
last_5_repositories = sorted(repos,
 key=lambda r: r["pushed_at"],
 reverse=True)[:5]
last_5_languages = [repo["language"]
                for repo in last_5_repositories]
```

Lo normal es que no trabajemos con API en este bajo nivel de "hacemos las solicitudes y analizamos las respuestas nosotros mismos". Uno de los beneficios de utilizar Python es que alguien ya ha creado una librería para prácticamente cualquier API a la que estemos interesados en acceder. Cuando lo han hecho bien, estas librerías pueden ahorrar buena parte del problema de averiguar los detalles más peliagudos del acceso API (pero, cuando no lo han hecho tan bien o cuando resulta que están basados en versiones difuntas de las correspondientes API, pueden provocar enormes dolores de cabeza).

No obstante, de vez en cuando habrá que desarrollar una librería de acceso API propia (o, lo más probable, depurar, porque la de otra persona no funcione), de modo que resulta positivo conocer parte de los detalles.

Encontrar API

Si nos hacen falta datos de un determinado sitio, hay que buscar en él una sección "desarrolladores", "*developers*" o "API" para obtener más información, y tratar de buscar en la web "python <nombredelsitio> api" para encontrar una librería.

Hay librerías para la API de Yelp, de Instagram, de Spotify, etc.

Si lo que queremos es una lista de API que tengan *wrappers* de Python, una que está muy bien es la de Real Python de GitHub.[5]

Y, si no se encuentra lo buscado, siempre quedará el raspado, el último refugio del científico de datos.

Ejemplo: Utilizar las API de Twitter

Twitter es una fuente fenomenal de datos para trabajar. Se puede utilizar para obtener noticias en tiempo real, para medir reacciones a acontecimientos actuales, para encontrar enlaces relacionados con determinados temas, etc. En definitiva, se puede utilizar prácticamente para cualquier cosa que se pueda imaginar, siempre y cuando se tenga acceso a sus datos, obviamente a través de sus API.

Para interactuar con las API de Twitter, vamos a emplear la librería

Twython[6] (`python -m pipinstall twython`). Existen bastantes librerías de Python para Twitter, pero con esta es con la que me ha ido mejor. ¡Le animo a que explore otras opciones!

Obtener credenciales

Para poder utilizar las API de Twitter, es necesario conseguir credenciales (para lo cual hay que tener una cuenta de Twitter, que probablemente tendrá si desea formar parte de la animada y amistosa comunidad #datascience de Twitter).

Advertencia: Como todas las instrucciones relacionadas con sitios web que no controlo, estas pueden quedar obsoletas en algún momento, pero espero que funcionen durante el tiempo suficiente (aunque ya han cambiado varias veces desde que empecé a escribir este libro, así que ¡buena suerte!).

Estos son los pasos a seguir:

1. Vaya a `https://developer.twitter.com/`.
2. Si no está ya dentro, haga clic en Sign in e introduzca su nombre de usuario de Twitter y su contraseña.
3. Haga clic en Apply para solicitar una cuenta de desarrollador.
4. Solicite acceso para uso personal.
5. Rellene la solicitud. Dispondrá de 300 palabras para explicar (en serio) por qué necesita el acceso, de modo que para pasarse del límite puede hablarles de este libro y de lo mucho que está disfrutando con su lectura.
6. Espere una cantidad de tiempo indefinida.
7. Si conoce a alguien que trabaje en Twitter, envíele un correo electrónico preguntándole si puede acelerar su solicitud. Si no, siga esperando.
8. En cuanto se la aprueben, vuelva a `https://developer.twitter.com/`, localice la sección Apps y haga clic en Create an app.
9. Rellene todos los campos necesarios (aquí también, si necesita texto

adicional para la descripción, puede hablar de este libro y de lo edificante que le está resultando).

10. Haga clic en CREATE.

Ahora su aplicación debería incluir una pestaña Keys and tokens con una sección Consumer API keys, donde se lista una "clave API" y una "clave secreta API". Tome nota de estas claves, porque las necesitará (y guárdelas bien; son como contraseñas).

> **Advertencia:** No comparta las claves, no las publique en su libro, ni las compruebe en su repositorio público de GitHub. Una solución sencilla es almacenarlas en un archivo `credenciales.json` que no se compruebe, y hacer que su código utilice `json.loads` para recuperarlas. Otra solución es almacenarlas en variables de entorno y utilizar `os.environ` para recuperarlas.

Utilizar Twython

La parte más difícil de utilizar la API de Twitter es autentificarse a uno mismo (de hecho, esto es lo más difícil en la mayoría de las API). Los proveedores de API quieren asegurarse de que está autorizado para acceder a sus datos y que no va a exceder sus límites de uso. También quieren saber quién está accediendo a sus datos.

La autentificación es un incordio. Tenemos dos métodos: uno fácil, OAuth 2, que sirve perfectamente cuando solo se desean realizar búsquedas sencillas, y otro complejo, OAuth 1, que es necesario cuando se quieren realizar acciones (por ejemplo, tuitear) o (para nuestro caso en particular) conectar con el *stream* de Twitter.

Así que nos quedamos con la forma más complicada, que trataremos de automatizar tanto como podamos.

En primer lugar, necesitamos la clave API y la clave secreta API (a veces conocida como clave de consumidor y clave secreta de consumidor, respectivamente). Obtendré la mía a partir de variables de entorno (no dude en poner las suyas si lo desea):

```
import os
```

```
# Cambie sin dudarlo sus claves directamente
CONSUMER_KEY = os.environ.get("TWITTER_CONSUMER_KEY")
CONSUMER_SECRET = os.environ.get("TWITTER_CONSUMER_SECRET")
```

Ahora podemos instanciar el cliente:

```
import webbrowser
from twython import Twython
# Obtiene cliente temporal para recuperar URL de autenticación
temp_client = Twython(CONSUMER_KEY, CONSUMER_SECRET)
temp_creds = temp_client.get_authentication_tokens()
url = temp_creds['auth_url']
# Ahora visita la URL para autorizar a la aplicación y obtener un PIN
print(f"go visit {url} and get the PIN code and paste it below")
webbrowser.open(url)
PIN_CODE = input("please enter the PIN code: ")
# Ahora usamos ese PIN_CODE para obtener los tokens reales
auth_client = Twython(CONSUMER_KEY,
                      CONSUMER_SECRET,
                      temp_creds['oauth_token'],
                      temp_creds['oauth_token_secret'])
final_step = auth_client.get_authorized_tokens(PIN_CODE)
ACCESS_TOKEN = final_step['oauth_token']
ACCESS_TOKEN_SECRET = final_step['oauth_token_secret']
# Y obtiene una nueva instancia Twython usándolos.
twitter = Twython(CONSUMER_KEY,
                  CONSUMER_SECRET,
                  ACCESS_TOKEN,
                  ACCESS_TOKEN_SECRET)
```

Truco: Quizá en este momento le interese guardar `ACCESS_TOKEN` y `ACCESS_TOKEN_SECRET` en un sitio seguro, de modo que la próxima vez no tenga que pasar por todo este jaleo de nuevo.

En cuanto tengamos una instancia Twython autenticada, podemos empezar a realizar búsquedas:

```
# Busca tuits que contengan la frase "datascience"
for status in twitter.search(q='"data science"')["statuses"]:
   user = status["user"]["screen_name"]
   text = status["text"]
   print(f"{user}: {text}\n")
```

Si ejecutamos esto, obtendremos algunos tuits como estos:

```
haithemnyc: Data scientists with the technical savvy &amp;amp; analytical chops to
derive meaning from big data are in demand. http://t.co/HsF9Q0dShP
RPubsRecent: Data Science http://t.co/6hcHUz2PHM
spleonard1: Using #dplyr in #R to work through a procrastinated assignment for
@rdpeng in @coursera data science specialization. So easy and Awesome.
```

Que no son muy interesantes, en parte porque la API Search de Twitter solo muestra el montón de resultados recientes que quiera. Cuando estamos haciendo ciencia de datos, solemos querer muchos tuits. Aquí es donde resulta de gran utilidad la API Streaming.[7] Permite conectarse al gran Firehose de Twitter (mejor dicho, a una pequeña parte). Para utilizarlo, será necesario autentificarse utilizando sus *tokens* de acceso.

Para acceder a la API Streaming con Twython, tenemos que definir una clase que herede de `TwythonStreamer` y que anule su método `on_success`, y posiblemente también su método `on_error`:

```
from twython import TwythonStreamer
# Añadir datos a una variable global es bastante pobre
# pero simplifica mucho el ejemplo
tweets = []
class MyStreamer(TwythonStreamer):
   def on_success(self, data):
      """
      What do we do when Twitter sends us data?
      Here data will be a Python dict representing a tweet.
      """
      # Solo queremos recopilar tuits en inglés
      if data.get('lang') == 'en':
         tweets.append(data)
         print(f"received tweet #{len(tweets)}")
      # Para cuando hemos recopilado bastantes
      if len(tweets) >= 100:
         self.disconnect()
   def on_error(self, status_code, data):
      print(status_code, data)
      self.disconnect()
```

`MyStreamer` conectará con el *stream* de Twitter y esperará a que Twitter le

pase datos. Cada vez que reciba datos (aquí, un tuit representado como un objeto de Python), los pasa al método `on_success`, que los añade a nuestra lista `tweets` si su idioma es inglés, y, una vez que ha recogido 1.000 tuits, se desconecta del *streamer*.

Todo lo que queda por hacer es inicializarlo y empezar a ejecutarlo:

```
stream = MyStreamer(CONSUMER_KEY, CONSUMER_SECRET,
                    ACCESS_TOKEN, ACCESS_TOKEN_SECRET)
# empieza a consumir estados públicos que contienen 'data'
stream.statuses.filter(track='data')
# pero si queremos empezar a consumir una muestra de *todos* los estados
públicos
# stream.statuses.sample()
```

Esto se ejecutará hasta que recopile 100 tuits (o hasta que encuentre un error) y se detendrá, momento en el cual podremos empezar a analizar esos tuits. Por ejemplo, podríamos encontrar los *hashtags* más habituales con:

```
top_hashtags = Counter(hashtag['text'].lower()
                    for tweet in tweets
                    for hashtag in tweet["entities"]["hashtags"])
print(top_hashtags.most_common(5))
```

Cada tuit contiene muchos datos. Puede investigar por sí mismo o buscar en la documentación de las API de Twitter.[8]

Nota: En un proyecto real, probablemente no quiera confiar en una `list` en memoria para almacenar los tuits. Lo mejor que podría hacer sería guardarlos en un archivo o en una base de datos, de modo que así dispondría de ellos permanentemente.

Para saber más

- pandas, en `http://pandas.pydata.org/`, es la librería que utilizan normalmente los científicos de datos para trabajar con datos (específicamente, para importarlos).
- Scrapy, en `http://scrapy.org/`, es una librería repleta de funciones

para crear complicados raspadores web, que hagan cosas como seguir enlaces desconocidos.

- Kaggle, en https://www.kaggle.com/datasets, alberga una gran colección de conjuntos de datos.

---

[1] https://stackoverflow.com/questions/15587877/run-a-python-script-in-terminal-without-the-python-command.

[2] https://www.crummy.com/software/BeautifulSoup/.

[3] https://docs.python-requests.org/en/latest/.

[4] https://docs.github.com/es/rest.

[5] https://github.com/realpython/list-of-python-api-wrappers.

[6] https://github.com/ryanmcgrath/twython.

[7] https://developer.twitter.com/en/docs/tutorials/consuming-streaming-data.

[8] https://developer.twitter.com/en/docs/tweets/data-dictionary/overview/tweet-object.

10 Trabajar con datos

Los expertos suelen poseer más datos que criterio.

—Colin Powell

Trabajar con datos es un arte, así como una ciencia. En general, hemos estado hablando de la parte científica, pero en este capítulo nos centraremos en el arte.

Explorar los datos

Una vez identificadas las preguntas que intentamos responder y después de haber obtenido datos, quizá se sienta tentado a meterse de lleno y empezar inmediatamente a crear modelos y obtener respuestas. Pero es necesario resistirse a este impulso. El primer paso debe ser explorar los datos.

Explorar datos unidimensionales

El caso más sencillo es tener un conjunto de datos unidimensional, que no es más que una colección de números. Por ejemplo, podría ser el número de minutos promedio al día que cada usuario se pasa en un sitio web, el número de veces que cada uno de los vídeos de tutoriales de ciencia de datos de una colección es visionado, o el número de páginas de cada uno de los libros de ciencia de datos que hay en una biblioteca.

Un primer paso obvio es calcular algunas estadísticas de resumen. Nos interesa saber cuántos puntos de datos tenemos, el menor, el mayor, la media y la desviación estándar.

Pero incluso estos datos no tienen por qué ofrecer un elevado nivel de comprensión. El siguiente paso correcto sería crear un histograma, en el que se agrupan los datos en *buckets* discretos y se cuenta cuántos puntos caen en

cada *bucket*:

```
from typing import List, Dict
from collections import Counter
import math
import matplotlib.pyplot as plt
def bucketize(point: float, bucket_size: float) -> float:
   """Floor the point to the next lower multiple of bucket_size"""
   return bucket_size * math.floor(point / bucket_size)
def make_histogram(points: List[float], bucket_size: float) -> Dict[float,
int]:
   """Buckets the points and counts how many in each bucket"""
   return Counter(bucketize(point, bucket_size) for point in points)
def plot_histogram(points: List[float], bucket_size: float, title: str = ""):
   histogram = make_histogram(points, bucket_size)
   plt.bar(histogram.keys(), histogram.values(), width=bucket_size)
   plt.title(title)
```

Por ejemplo, tengamos en cuenta los dos siguientes conjuntos de datos:

```
import random
from scratch.probability import inverse_normal_cdf
random.seed(0)
# uniforme entre -100 y 100
uniform = [200 * random.random()–100 for _ in range(10000)]
# distribución normal con media 0, desviación estándar 57
normal = [57 * inverse_normal_cdf(random.random())
        for _ in range(10000)]
```

Ambos tienen medias próximas a 0 y desviaciones estándares cercanas a 58. Sin embargo, tienen distribuciones muy distintas. La figura 10.1 muestra la distribución de `uniform`:

```
plot_histogram(uniform, 10, “Uniform Histogram”)
```

Mientras que la figura 10.2 muestra la distribución de `normal`:

```
plot_histogram(normal, 10, “Normal Histogram”)
```

En este caso, las dos distribuciones tienen `max` y `min` bastante diferentes, pero ni siquiera saber esto habría sido suficiente para entender cómo difieren.

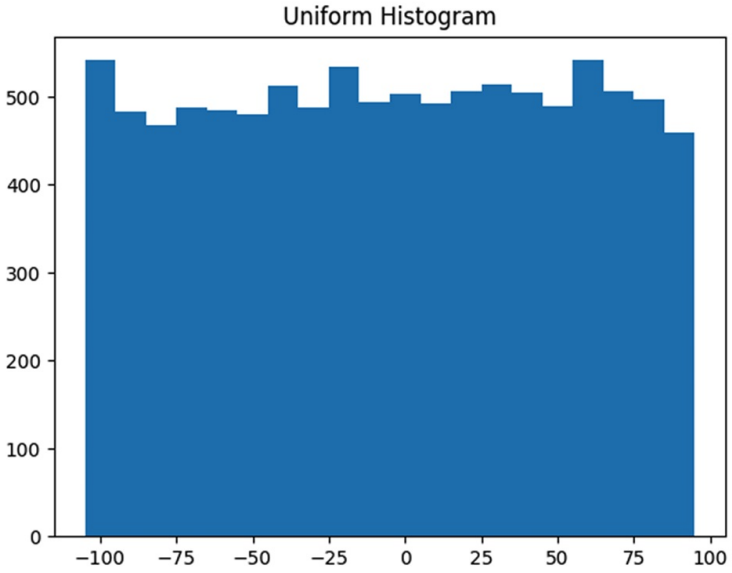


Figura 10.1. Histograma de `uniform`.

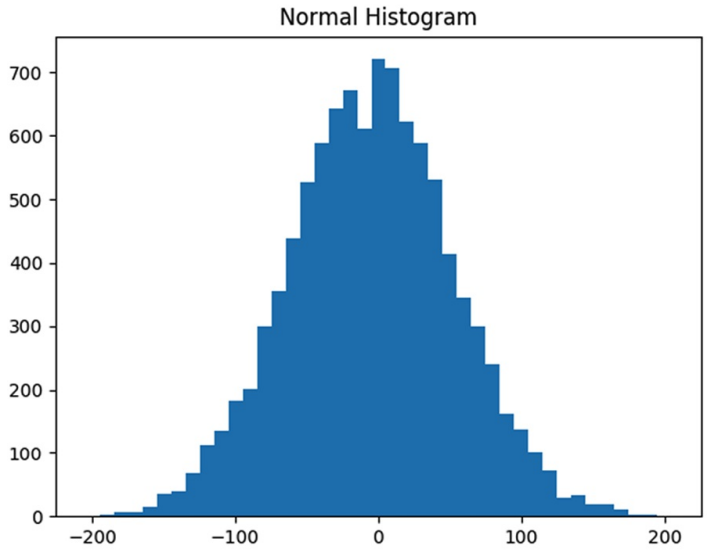


Figura 10.2. Histograma de `normal`.

Dos dimensiones

Ahora imaginemos que tenemos un conjunto de datos con dos

dimensiones. Quizá, además de los minutos diarios, tenemos años de experiencia en ciencia de datos. Por supuesto que queremos entender cada dimensión de manera individual, pero probablemente también nos interese dispersar los datos.

Por ejemplo, veamos otro conjunto de datos imaginario:

```
def random_normal() -> float:
   """Returns a random draw from a standard normal distribution"""
   return inverse_normal_cdf(random.random())
xs = [random_normal() for _ in range(1000)]
ys1 = [ x + random_normal() / 2 for x in xs]
ys2 = [-x + random_normal() / 2 for x in xs]
```

Si ejecutáramos `plot_histogram` en `ys1` e `ys2`, obtendríamos trazados de aspecto similar (de hecho, ambos están distribuidos normalmente con la misma media y desviación estándar).

Pero cada uno tiene una distribución conjunta diferente con `xs`, como puede verse en la figura 10.3:

```
plt.scatter(xs, ys1, marker='.', color='black', label='ys1')
plt.scatter(xs, ys2, marker='.', color='gray', label='ys2')
plt.xlabel('xs')
plt.ylabel('ys')
plt.legend(loc=9)
plt.title("Very Different Joint Distributions")
plt.show()
```

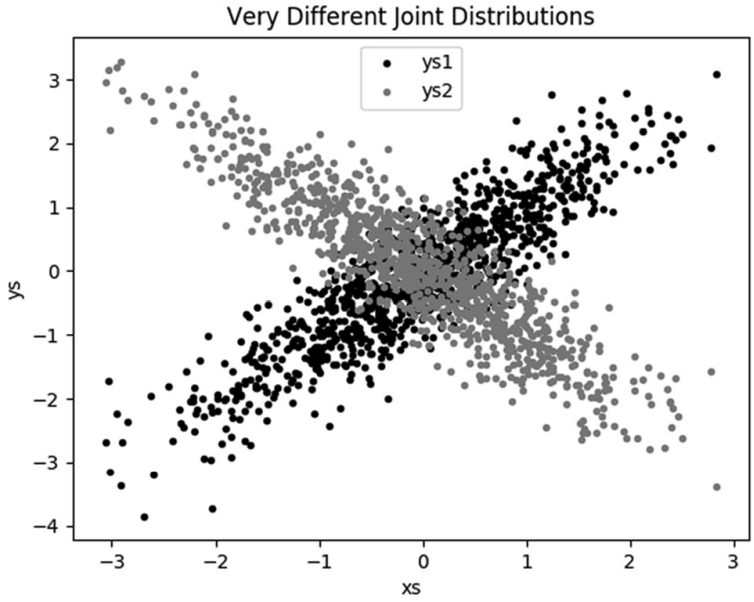


Figura 10.3. Dispersando dos `ys` distintos.

Esta diferencia también se haría patente si mirásemos las correlaciones:

```
from scratch.statistics import correlation
print(correlation(xs, ys1))      # más o menos 0.9
print(correlation(xs, ys2))      # más o menos -0.9
```

Muchas dimensiones

Si tenemos muchas dimensiones, nos interesará saber cómo se relacionan todas ellas entre sí. Una forma sencilla de averiguarlo es mirar la matriz de correlación, en la que la entrada de la fila *i* y la columna *j* es la correlación entre la dimensión *i* y la dimensión *j* de los datos:

```
from scratch.linear_algebra import Matrix, Vector, make_matrix
def correlation_matrix(data: List[Vector]) -> Matrix:
   """
   Returns the len(data) x len(data) matrix whose (i, j)-th entry
   is the correlation between data[i] and data[j]
   """
   def correlation_ij(i: int, j: int) -> float:
      return correlation(data[i], data[j])
   return make_matrix(len(data), len(data), correlation_ij)
```

Un método más visual (si no tenemos demasiadas dimensiones) es hacer una matriz de *scatterplot* o de diagrama de dispersión (figura 10.4), que muestre todos los gráficos de dispersión por pares. Para ello, utilizaremos `plt.subplots`, que nos permite crear *subplots* de nuestro gráfico. Le damos el número de filas y columnas y devuelve un objeto `figure` (que no usaremos) y un *array* bidimensional de objetos `axes` (que mostraremos en pantalla):

```
# corr_data es una lista de vectores de 100 dimensiones
num_vectors = len(corr_data)
fig, ax = plt.subplots(num_vectors, num_vectors)
for i in range(num_vectors):
   for j in range(num_vectors):
      # Dispersa column_j en el eje x frente a column_i en el eje y
      if i != j: ax[i][j].scatter(corr_data[j], corr_data[i])
      # a menos que i == j, en ese caso muestra el nombre de la serie
      else: ax[i][j].annotate("series " + str(i), (0.5, 0.5),
       xycoords='axes fraction',
       ha="center", va="center")
      # Luego oculta etiquetas de eje, salvo los diagramas izquierdo e inferior
      if i < num_vectors-1: ax[i][j].xaxis.set_visible(False)
      if j > 0: ax[i][j].yaxis.set_visible(False)
# Fija las etiquetas de ejes de abajo derecha y arriba izquierda, que están mal
# porque sus diagramas solo contienen texto
ax[-1][-1].set_xlim(ax[0][-1].get_xlim())
ax[0][0].set_ylim(ax[0][1].get_ylim())
plt.show()
```

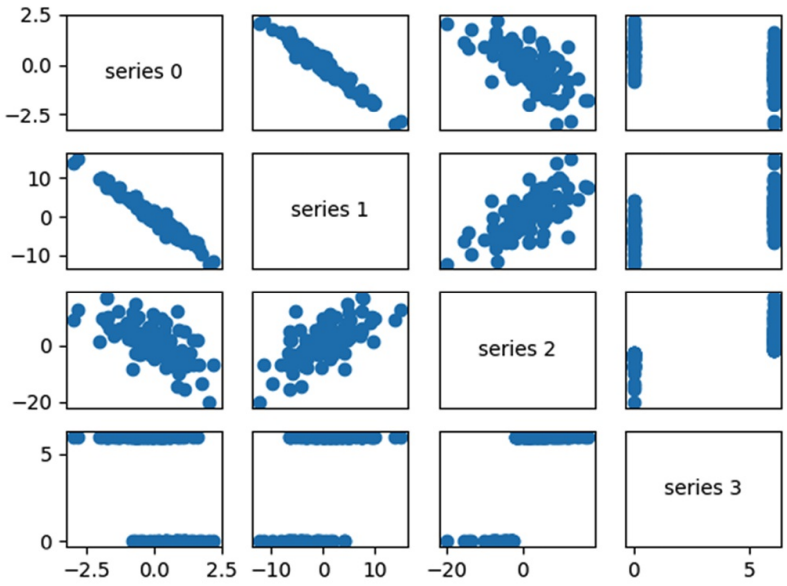


Figura 10.4. Matriz de diagrama de dispersión.

Mirando los diagramas de dispersión, podemos ver que la serie 1 está correlacionada muy negativamente con la serie 0, la serie 2 lo está positivamente con la serie 1 y la serie 3 solo toma los valores 0 y 6, correspondiendo el 0 a los valores pequeños de la serie 2 y el 6 a los valores grandes.

Esta es una forma rápida de hacerse una idea general de cuál de las variables está correlacionada (a menos que se pase horas retocando matplotlib para mostrar las cosas exactamente como las quiere, en cuyo caso no es un método rápido).

Utilizar NamedTuples

Una forma habitual de representar datos es utilizando `dict`:

```
import datetime
stock_price = {‘closing_price’: 102.06,
          ‘date’: datetime.date(2014, 8, 29),
          ‘symbol’: ‘AAPL’}
```

Hay varias razones por las que esto, sin embargo, no es lo ideal. Es una representación ligeramente ineficaz (un `dict` implica gastos extra), de modo que, si tenemos muchos precios de acciones, ocuparán más memoria de la necesaria. En general, esto es una consideración de menor importancia.

Un problema mayor es que acceder a las cosas mediante una clave `dict` tiene tendencia a producir errores. El siguiente código funcionará sin errores y solo hará lo incorrecto:

```
# huy, error
stock_price[‘cosing_price’] = 103.06
```

Finalmente, aunque podemos anotar los tipos de diccionarios uniformes:

```
prices: Dict[datetime.date, float] = {}
```

No hay un modo útil de anotar diccionarios como datos que tengan muchos tipos de valores distintos, de forma que también perdemos el poder de las comprobaciones de tipos.

Como alternativa, Python incluye una clase `namedtuple`, que es como una `tuple` pero con nombres:

```
from collections import namedtuple
StockPrice = namedtuple(‘StockPrice’, [‘symbol’, ‘date’, ‘closing_price’])
price = StockPrice(‘MSFT’, datetime.date(2018, 12, 14), 106.03)
assert price.symbol == ‘MSFT’
assert price.closing_price == 106.03
```

Como las `tuple` normales, las `namedtuple` son inmutables, lo que significa que no se pueden modificar sus valores una vez que se crean. De vez en cuando, esto se interpondrá en nuestro camino, pero en general es algo bueno.

Vemos que no hemos resuelto aún el tema de la anotación de tipo. Lo hacemos utilizando la variante con nombre, `NamedTuple`:

```
from typing import NamedTuple
class StockPrice(NamedTuple):
   symbol: str
   date: datetime.date
```

```
    closing_price: float
    def is_high_tech(self) -> bool:
        """It's a class, so we can add methods too"""
        return self.symbol in ['MSFT', 'GOOG', 'FB', 'AMZN', 'AAPL']
price = StockPrice('MSFT', datetime.date(2018, 12, 14), 106.03)
assert price.symbol == 'MSFT'
assert price.closing_price == 106.03
assert price.is_high_tech()
```

Ahora el editor nos puede ayudar, como muestra la figura 10.5.



Figura 10.5. Útil editor.

Nota: Muy poca gente utiliza así `NamedTuple`. ¡Pero deberían!

Clases de datos

Las clases de datos son (más o menos) una versión mutable de `NamedTuple` (digo "más o menos" porque las `NamedTuple` representan sus datos de manera compacta como tuples, mientras que las *dataclasses* son clases de Python normales que simplemente se encargan de generar automáticamente ciertos métodos).

Nota: Las clases de datos son nuevas en Python 3.7. Si está utilizando una versión más antigua, esta sección no le servirá.

La sintaxis es muy parecida a la de `NamedTuple`. Pero, en lugar de heredar de una clase base, utilizamos un decorador:

```
from dataclasses import dataclass
@dataclass
```

```
class StockPrice2:
   symbol: str
   date: datetime.date
   closing_price: float
   def is_high_tech(self) -> bool:
      """It's a class, so we can add methods too"""
      return self.symbol in ['MSFT', 'GOOG', 'FB', 'AMZN', 'AAPL']
price2 = StockPrice2('MSFT', datetime.date(2018, 12, 14), 106.03)
assert price2.symbol == 'MSFT'
assert price2.closing_price == 106.03
assert price2.is_high_tech()
```

Como ya hemos dicho antes, la gran diferencia es que podemos modificar los valores de la instancia de una clase de datos:

```
# división de acciones
price2.closing_price /= 2
assert price2.closing_price == 51.03
```

Si intentáramos modificar un campo de la versión `NamedTuple`, obtendríamos un `AttributeError`.

Esto nos deja también susceptibles al tipo de errores que esperamos evitar no utilizando `dict`:

```
# Es una clase regular, así que añada campos siempre que quiera.
price2.cosing_price = 75                          # huy
```

No utilizaremos clases de datos, pero es fácil que se las encuentre por ahí fuera.

Limpiar y preparar datos

Los datos del mundo real están "sucios". Muchas veces tendremos que trabajar con ellos antes de poder utilizarlos. Hemos visto ejemplos en el capítulo 9. Hay que convertir cadenas de texto en `float` o `int` antes de poder usarlos, o revisar en busca de valores perdidos, valores atípicos (*outliers*) y datos erróneos.

Anteriormente, hicimos esto justo antes de usar los datos:

```
    closing_price = float(row[2])
```

Pero probablemente induce menos errores analizar una función que podemos probar:

```
from dateutil.parser import parse
def parse_row(row: List[str]) -> StockPrice:
   symbol, date, closing_price = row
   return StockPrice(symbol=symbol,
                date=parse(date).date(),
                closing_price=float(closing_price))
# A probar la función
stock = parse_row(["MSFT", "2018-12-14", "106.03"])
assert stock.symbol == "MSFT"
assert stock.date == datetime.date(2018, 12, 14)
assert stock.closing_price == 106.03
```

¿Qué pasa si hay datos erróneos? ¿O un valor “float” que en realidad no representa un número? ¿Quizá es mejor obtener un `None` que el programa se cuelgue?

```
from typing import Optional
import re
def try_parse_row(row: List[str]) -> Optional[StockPrice]:
   symbol, date_, closing_price_ = row
   # El símbolo de acción debe estar en mayúscula
   if not re.match(r"^[A-Z]+$", symbol):
      return None
   try:
      date = parse(date_).date()
   except ValueError:
      return None
   try:
      closing_price = float(closing_price_)
   except ValueError:
      return None
   return StockPrice(symbol, date, closing_price)
# Debería devolver None por errores
assert try_parse_row(["MSFT0", "2018-12-14", "106.03"]) is None
assert try_parse_row(["MSFT", "2018-12—14", "106.03"]) is None
assert try_parse_row(["MSFT", "2018-12-14", "x"]) is None
# Pero debería devolver lo mismo que antes si los datos son correctos
assert try_parse_row(["MSFT", "2018-12-14", "106.03"]) == stock
```

Por ejemplo, si tenemos precios de acciones delimitados por comas con datos sobrantes:

```
AAPL,6/20/2014,90.91
MSFT,6/20/2014,41.68
FB,6/20/3014,64.5
AAPL,6/19/2014,91.86
MSFT,6/19/2014,n/a
FB,6/19/2014,64.34
```

Ahora podemos leer y devolver solo las filas válidas:

```
import csv
data: List[StockPrice] = []
with open("comma_delimited_stock_prices.csv") as f:
   reader = csv.reader(f)
   for row in reader:
      maybe_stock = try_parse_row(row)
      if maybe_stock is None:
         print(f"skipping invalid row: {row}")
      else:
         data.append(maybe_stock)
```

Y decidir lo que queremos hacer con las no válidas. En general, las tres opciones son deshacerse de ellas, volver al origen y tratar de arreglar los datos sobrantes/faltantes, o no hacer nada y cruzar los dedos. Si hay una sola fila errónea de millones, probablemente no pasa nada por ignorarla. Pero, si la mitad de las filas tienen datos sobrantes, es algo que conviene arreglar.

Algo correcto que podemos hacer a continuación es buscar valores atípicos (*outliers*), utilizando las técnicas vistas en el apartado “Explorar datos” del comienzo de este capítulo o investigando como corresponde. Por ejemplo, ¿se dio cuenta de que una de las fechas del archivo de acciones tenía el año 3014? No tendría por qué dar error, pero es claramente incorrecto, y se obtendrían resultados caóticos si no se detectara. A los conjuntos de datos reales les faltan puntos decimales, tienen ceros de más, errores tipográficos y otros incontables problemas que nosotros tenemos que localizar (quizá este no sea oficialmente su trabajo, pero ¿quién más lo va a hacer?).

Manipular datos

Una de las habilidades más importantes de un científico de datos es la manipulación de los mismos. Se trata de un acercamiento general más que de una técnica específica, así que simplemente veremos unos cuantos ejemplos para que se haga una idea.

Imaginemos que tenemos un montón de datos de precios de acciones con este aspecto:

```
data = [
  StockPrice(symbol='MSFT',
         date=datetime.date(2018, 12, 24),
         closing_price=106.03),
  # ...
]
```

Empecemos por plantear preguntas sobre estos datos. Por el camino intentaremos observar patrones en lo que estamos haciendo y abstraer algunas herramientas para facilitar la manipulación.

Por ejemplo, supongamos que queremos conocer el precio de cierre máximo posible para AAPL. Dividamos esto en pasos:

1. Limitarnos a las filas AAPL.
2. Coger el `closing_price` de cada fila.
3. Tomar el `max` de esos precios.

Podemos hacer las tres cosas a la vez utilizando una lista de comprensión:

```
max_aapl_price = max(stock_price.closing_price
                for stock_price in data
                if stock_price.symbol == "AAPL")
```

De forma más general, nos podría interesar conocer el precio de cierre máximo posible para cada acción de nuestro conjunto de datos. Una forma de hacer esto es:

1. Crear un `dict` para mantener controlados los precios máximos (utilizaremos un `defaultdict` que devuelve menos infinito para valores que faltan, ya que cualquier precio será mayor).

2. Iterar nuestros datos, actualizándolos.

Este es el código:

```
from collections import defaultdict
max_prices: Dict[str, float] = defaultdict(lambda: float('-inf'))
for sp in data:
   symbol, closing_price = sp.symbol, sp.closing_price
   if closing_price > max_prices[symbol]:
      max_prices[symbol] = closing_price
```

Ahora podemos empezar a pedir cosas más complicadas, como averiguar cuáles son los cambios porcentuales de un día mayor y menor de nuestro conjunto de datos. El cambio porcentual es `price_today / price_yesterday -1`, lo que significa que necesitamos un modo de asociar el precio de hoy y el de ayer. Una forma es agrupando los precios por símbolo, y después, dentro de cada grupo:

1. Ordenar los precios por fecha.
2. Utilizar `zip` para obtener pares (anterior, actual).
3. Convertir los pares en nuevas filas de "cambio porcentual".

Empecemos agrupando los precios por símbolo:

```
from typing import List
from collections import defaultdict
# Recopila los precios por símbolo
prices: Dict[str, List[StockPrice]] = defaultdict(list)
for sp in data:
   prices[sp.symbol].append(sp)
```

Como los precios son tuplas, se clasifican por sus campos en orden: primero por símbolo, después por fecha y por último por precio. Esto significa que, si tenemos varios precios, todos con el mismo símbolo, `sort` los ordenará por fecha (y después por precio, lo que no hace nada, ya que tenemos solo uno por fecha), que es lo que queremos:

```
# Ordena los precios por fecha
```

```
prices = {symbol: sorted(symbol_prices)
        for symbol, symbol_prices in prices.items()}
```

Y que podemos utilizar para calcular una secuencia de cambios por día:

```
def pct_change(yesterday: StockPrice, today: StockPrice) -> float:
   return today.closing_price / yesterday.closing_price–1
class DailyChange(NamedTuple):
   symbol: str
   date: datetime.date
   pct_change: float
def day_over_day_changes(prices: List[StockPrice]) -> List[DailyChange]:
   """
   Assumes prices are for one stock and are in order
   """
   return [DailyChange(symbol=today.symbol,
                date=today.date,
                pct_change=pct_change(yesterday, today))
         for yesterday, today in zip(prices, prices[1:])]
```

Y recopilarlos después todos:

```
all_changes = [change
           for symbol_prices in prices.values()
           for change in day_over_day_changes(symbol_prices)]
```

Momento en el cual es fácil encontrar el mayor y el menor:

```
max_change = max(all_changes, key=lambda change: change.pct_change)
# ver p. ej. http://news.cnet.com/2100-1001-202143.html
assert max_change.symbol == 'AAPL'
assert max_change.date == datetime.date(1997, 8, 6)
assert 0.33 < max_change.pct_change < 0.34
min_change = min(all_changes, key=lambda change: change.pct_change)
# ver p.ej. http://money.cnn.com/2000/09/29/markets/techwrap/
assert min_change.symbol == ,AAPL'
assert min_change.date == datetime.date(2000, 9, 29)
assert -0.52 < min_change.pct_change < -0.51
```

Ahora se puede utilizar este nuevo conjunto de datos `all_changes` para averiguar qué mes es el mejor para invertir en acciones tecnológicas. Veremos el cambio diario medio por mes:

```
changes_by_month: List[DailyChange] = {month: [] for month in range(1, 13)}
for change in all_changes:
   changes_by_month[change.date.month].append(change)
avg_daily_change = {
   month: sum(change.pct_change for change in changes) / len(changes)
   for month, changes in changes_by_month.items()
}
# Octubre es el mejor mes
assert avg_daily_change[10] == max(avg_daily_change.values())
```

Estaremos haciendo este tipo de manipulaciones a lo largo del libro, normalmente sin llamar de una forma demasiado explícita la atención sobre ellas.

Redimensionar

Muchas técnicas son sensibles a la dimensión de los datos. Por ejemplo, imaginemos que tenemos un conjunto de datos que consiste en las alturas y pesos de cientos de científicos de datos, y que estamos tratando de identificar *clusters* de tamaños de cuerpos. De forma intuitiva, nos gustaría que los agrupamientos representaran puntos uno al lado del otro, lo que significa que necesitamos una cierta noción de distancia entre puntos. Ya tenemos la función euclidiana `distance`, de modo que la forma natural de hacer esto podría ser tratar pares (altura, peso) como puntos en un espacio bidimensional. Veamos las personas que aparecen en la tabla 10.1.

Tabla 10.1. Alturas y pesos.

Persona	Altura(pulgadas)	Altura (centímetros)	Peso (libras)
A	63	160	150
B	67	170,2	160
C	70	177,8	171

Si medimos la altura en pulgadas, entonces el vecino más próximo a B es A:

```
from scratch.linear_algebra import distance
a_to_b = distance([63, 150], [67, 160])         # 10.77
a_to_c = distance([63, 150], [70, 171])         # 22.14
b_to_c = distance([67, 160], [70, 171])         # 11.40
```

Pero, si medimos la altura en centímetros, sin embargo el vecino más próximo a B es C:

```
a_to_b = distance([160, 150], [170.2, 160])       # 14.28
a_to_c = distance([160, 150], [177.8, 171])       # 27.53
b_to_c = distance([170.2, 160], [177.8, 171])     # 13.37
```

Obviamente es un problema el hecho de que cambiar las unidades pueda cambiar así los resultados. Por esta razón, cuando las dimensiones no sean comparables una con otra, redimensionaremos en ocasiones nuestros datos de forma que cada dimensión tenga media 0 y desviación estándar 1. Así nos deshacemos efectivamente de las unidades, convirtiendo cada dimensión en "desviaciones estándares de la media".

Para empezar, tendremos que calcular `mean` y `standard_deviation` para cada posición:

```
from typing import Tuple
from scratch.linear_algebra import vector_mean
from scratch.statistics import standard_deviation
def scale(data: List[Vector]) -> Tuple[Vector, Vector]:
   """returns the mean and standard deviation for each position"""
   dim = len(data[0])
   means = vector_mean(data)
   stdevs = [standard_deviation([vector[i] for vector in data])
        for i in range(dim)]
return means, stdevs
vectors = [[-3, -1, 1], [-1, 0, 1], [1, 1, 1]]
means, stdevs = scale(vectors)
assert means == [-1, 0, 1]
assert stdevs == [2, 1, 0]
```

Después podemos emplearlas para crear un nuevo conjunto de datos:

```
def rescale(data: List[Vector]) -> List[Vector]:
   """
   Rescales the input data so that each position has
   mean 0 and standard deviation 1. (Leaves a position
   as is if its standard deviation is 0.)
   """
   dim = len(data[0])
   means, stdevs = scale(data)
   # Hace una copia de cada vector
   rescaled = [v[:] for v in data]
   for v in rescaled:
      for i in range(dim):
         if stdevs[i] > 0:
            v[i] = (v[i]–means[i]) / stdevs[i]
   return rescaled
```

Por supuesto, escribimos una prueba para comprobar que `rescale` hace lo que pensamos que hace:

```
means, stdevs = scale(rescale(vectors))
assert means == [0, 0, 1]
assert stdevs == [1, 1, 0]
```

Como siempre, necesitamos aplicar nuestro criterio. Si tomáramos un enorme conjunto de datos de alturas y pesos y lo filtráramos para quedarnos solo con las personas con alturas de entre 69,5 pulgadas y 70,5 pulgadas, es bastante probable (dependiendo de la pregunta que estemos tratando de responder) que la variación restante sea simplemente ruido, y quizá no queramos poner su desviación estándar al mismo nivel que las desviaciones de otras dimensiones.

Un inciso: tqdm

Con frecuencia, acabaremos haciendo cálculos que requieren mucho tiempo. Cuando estemos haciendo esto, nos gustará saber que estamos haciendo progresos y calcular el tiempo que se supone que tendremos que esperar.

Una forma de hacerlo es con la librería `tqdm`, que genera barras de progreso personalizadas. Lo utilizaremos un poco a lo largo del libro, de modo que aprovechemos la oportunidad que se nos brinda de aprender cómo funciona.

Lo primero es instalarlo:

```
python -m pip install tqdm
```

Solo hace falta conocer unas cuantas funciones. La primera es que un iterable envuelto en `tqdm.tqdm` producirá una barra de progreso:

```
import tqdm
for i in tqdm.tqdm(range(100)):
   # funciona algo lento
   _ = [random.random() for _ in range(1000000)]
```

Que produce un resultado parecido a este:

```
56%|
```



```
| 56/100 [00:08<00:06, 6.49it/s]
```

En particular, muestra qué fracción del bucle está terminada (aunque no se puede hacer esto si se utiliza un generador), cuánto tiempo se ha estado ejecutando y cuánto tiempo más espera hacerlo.

En este caso (donde simplemente estamos envolviendo una llamada a `range`), basta con utilizar `tqdm.range`.

También se puede configurar la descripción de la barra de progreso mientras está funcionando. Para ello, hay que capturar el iterador `tqdm` en una sentencia `with`:

```
from typing import List
def primes_up_to(n: int) -> List[int]:
   primes = [2]
   with tqdm.trange(3, n) as t:
      for i in t:
         # i es primo si ningún primo menor lo divide
         i_is_prime = not any(i % p == 0 for p in primes)
         if i_is_prime:
```

```
            primes.append(i)
        t.set_description(f"{len(primes)} primes")
    return primes
my_primes = primes_up_to(100_000)
```

Esto añade una descripción como la siguiente, con un contador que se actualiza cuando se descubren nuevos primos:

```
5116 primes: 50%|
```



```
| 49529/99997 [00:03<00:03, 15905.90it/s]
```

Utilizar `tqdm` puede hacer que el código inspire desconfianza (ya que, a veces, la pantalla se redibuja pésimamente, y otras veces el bucle directamente se colgará). Si envolvemos accidentalmente un bucle `tqdm` dentro de otro, podrían ocurrir cosas raras. Lo normal es que sus beneficios superen ampliamente estos inconvenientes, así que trataremos de utilizarlo siempre que tengamos entre manos cálculos de ejecución lenta.

Reducción de dimensionalidad

En ocasiones, las dimensiones “reales” (o útiles) de los datos podrían no corresponder con las dimensiones que tenemos. Por ejemplo, veamos el conjunto de datos representado en la figura 10.6.

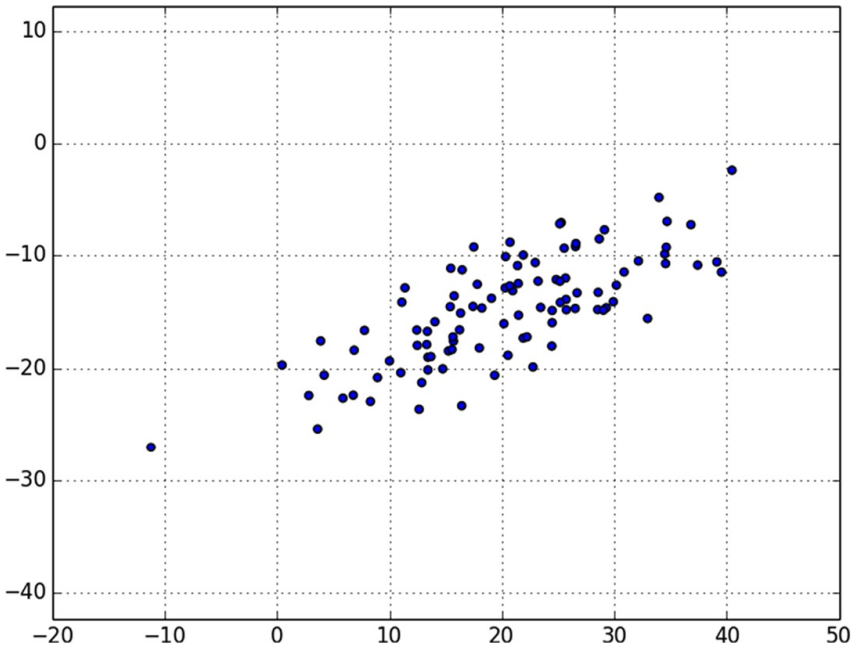


Figura 10.6. Datos con los ejes “erróneos”.

La mayor parte de la variación de los datos parece producirse en una única dimensión, que no corresponde con el eje x o y.

Cuando ocurre esto, podemos utilizar una técnica denominada análisis de componentes principales o PCA (*Principal Component Analysis*) para extraer una o más dimensiones que capturen tanto de la variación de datos como sea posible.

Nota: En la práctica, no utilizaríamos esta técnica en un conjunto de datos de tan baja dimensión. Principalmente, la reducción de dimensionalidad es útil cuando el conjunto de datos tiene un gran número de dimensiones y deseamos encontrar un pequeño subconjunto que capture la mayor parte de la variación. Lamentablemente, esta situación es difícil de ilustrar en un libro en formato de dos dimensiones.

Como primer paso a dar, tendremos que traducir los datos de modo que cada dimensión tenga media 0:

```
from scratch.linear_algebra import subtract
def de_mean(data: List[Vector]) -> List[Vector]:
   """Recenters the data to have mean 0 in every dimension"""
```

```
    mean = vector_mean(data)
    return [subtract(vector, mean) for vector in data]
```

(Si no hacemos esto, es probable que nuestras técnicas identifiquen la propia media en lugar de la variación en los datos).

La figura 10.7 muestra los datos de ejemplo tras aplicar media 0.

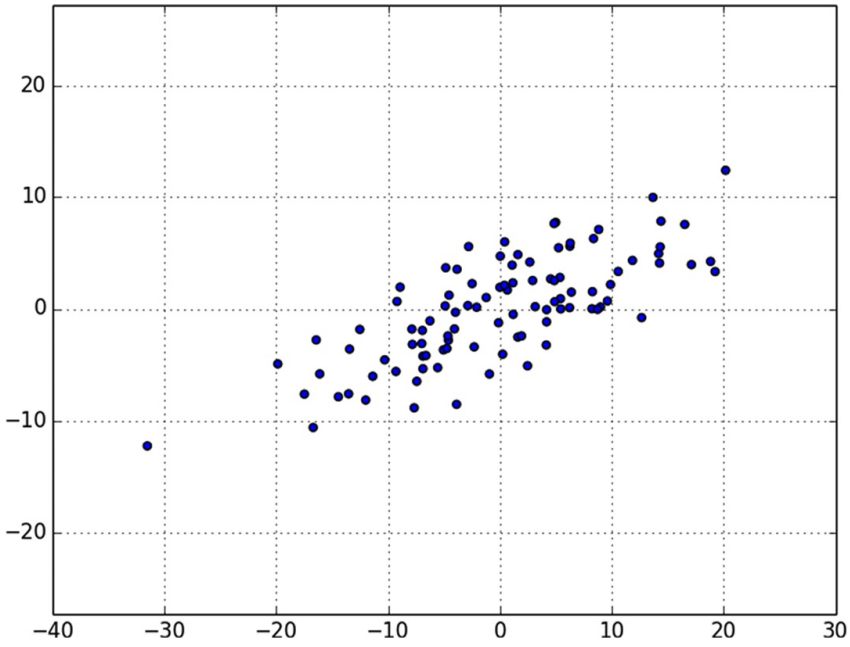


Figura 10.7. Datos tras aplicar media 0.

Ahora, dada una matriz *X* con media 0, podemos preguntar cuál es la dirección que captura la mayor varianza en los datos.

Específicamente, dada una dirección `d` (un vector de magnitud 1), cada fila `x` de la matriz extiende `dot(x, d)` en la dirección `d`. Y cada vector no cero `w` determina una dirección si lo redimensionamos para que tenga una magnitud 1:

```
from scratch.linear_algebra import magnitude
def direction(w: Vector) -> Vector:
    mag = magnitude(w)
    return [w_i / mag for w_i in w]
```

De esta forma, dado un vector no cero `w`, podemos calcular la varianza de

nuestro conjunto de datos en la dirección determinada por `w`:

```
from scratch.linear_algebra import dot
def directional_variance(data: List[Vector], w: Vector) -> float:
   """
   Returns the variance of x in the direction of w
   """
   w_dir = direction(w)
   return sum(dot(v, w_dir) ** 2 for v in data)
```

Nos gustaría encontrar la dirección que maximice esta varianza. Podemos hacerlo utilizando el descenso de gradiente, tan pronto como tengamos la función gradiente:

```
def directional_variance_gradient(data: List[Vector], w: Vector) -> Vector:
   """
   The gradient of directional variance with respect to w
   """
   w_dir = direction(w)
   return [sum(2 * dot(v, w_dir) * v[i] for v in data)
        for i in range(len(w))]
```

Y ahora el principal componente que tenemos es simplemente la dirección que maximiza la función `directional_variance`:

```
from scratch.gradient_descent import gradient_step
def first_principal_component(data: List[Vector],
      n: int = 100,
      step_size: float = 0.1) -> Vector:
   # Inicia con una suposición al azar
   guess = [1.0 for _ in data[0]]
   with tqdm.trange(n) as t:
      for _ in t:
         dv = directional_variance(data, guess)
         gradient = directional_variance_gradient(data, guess)
         guess = gradient_step(guess, gradient, step_size)
         t.set_description(f"dv: {dv:.3f}")
   return direction(guess)
```

En el conjunto de datos con media 0, esto devuelve la dirección `[0.924, 0.383]`, que parece capturar el eje principal a lo largo del cual varían nuestros

datos (figura 10.8).

Una vez localizada la dirección que es el primer componente principal, podemos proyectar en ella nuestros datos para hallar los valores de dicho componente:

```
from scratch.linear_algebra import scalar_multiply
def project(v: Vector, w: Vector) -> Vector:
   """return the projection of v onto the direction w"""
   projection_length = dot(v, w)
   return scalar_multiply(projection_length, w)
```

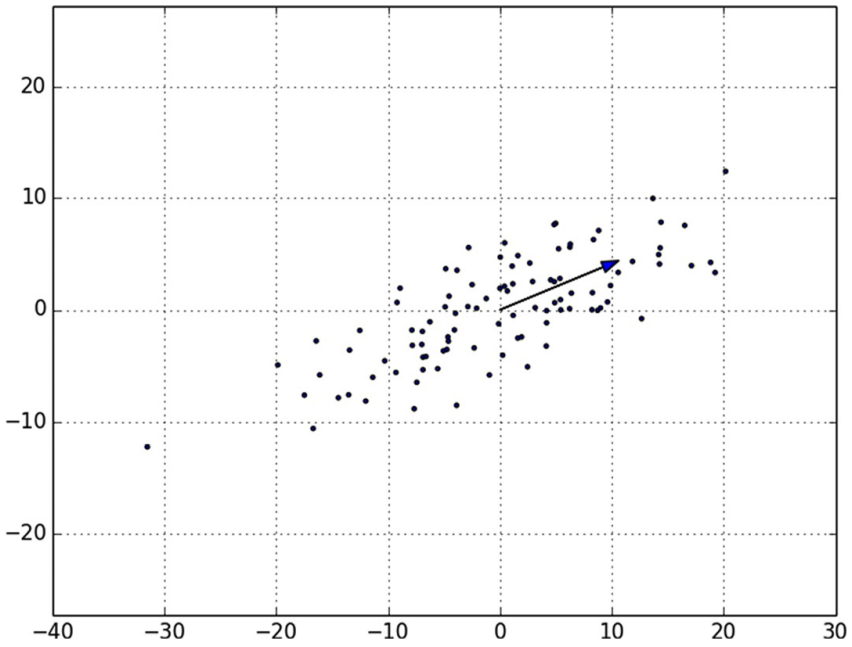


Figura 10.8. Primer componente principal.

Si queremos hallar más componentes, primero eliminamos las proyecciones de los datos:

```
from scratch.linear_algebra import subtract
def remove_projection_from_vector(v: Vector, w: Vector) -> Vector:
   """projects v onto w and subtracts the result from v"""
   return subtract(v, project(v, w))
def remove_projection(data: List[Vector], w: Vector) -> List[Vector]:
   return [remove_projection_from_vector(v, w) for v in data]
```

Como este conjunto de datos de ejemplo es solo bidimensional, tras

eliminar el primer componente, lo que queda será, efectivamente, unidimensional (figura 10.9).

En este momento, podemos hallar el siguiente componente principal repitiendo el proceso con el resultado de `remove_projection` (figura 10.10).

En un conjunto de datos de muchas dimensiones, podemos encontrar de forma iterativa tantos componentes como queramos:

```
def pca(data: List[Vector], num_components: int) -> List[Vector]:
   components: List[Vector] = []
   for _ in range(num_components):
      component = first_principal_component(data)
      components.append(component)
      data = remove_projection(data, component)
   return components
```

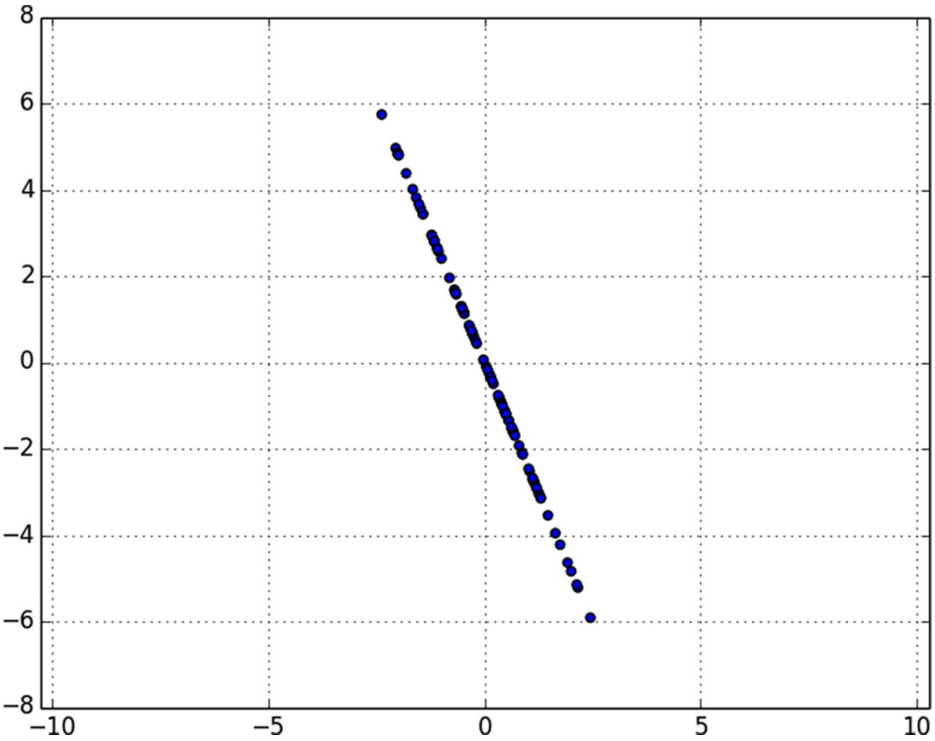


Figura 10.9. Datos tras eliminar el primer componente principal.

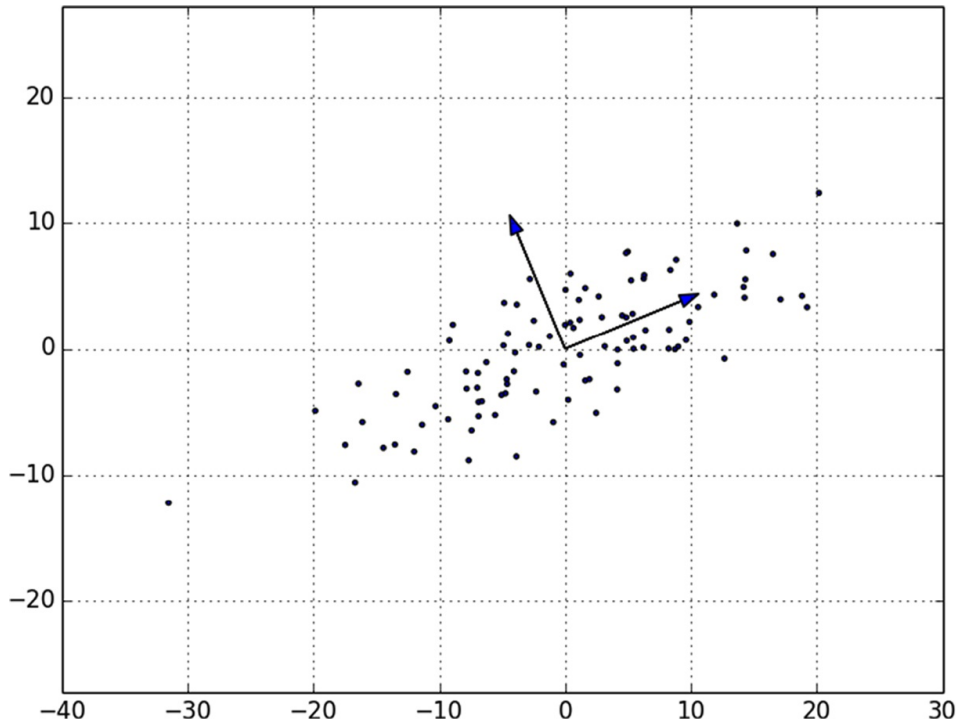


Figura 10.10. Primeros dos componentes principales.

Podemos después transformar nuestros datos en el espacio de menos dimensiones atravesado por los componentes:

```
def transform_vector(v: Vector, components: List[Vector]) -> Vector:
   return [dot(v, w) for w in components]
def transform(data: List[Vector], components: List[Vector]) -> List[Vector]:
   return [transform_vector(v, components) for v in data]
```

Esta técnica es válida por varias razones. Primero, puede permitirnos limpiar nuestros datos eliminando las dimensiones de ruido y consolidando las dimensiones altamente correlacionadas.

Segundo, tras extraer una representación de baja dimensión de nuestros datos, podemos utilizar diversas técnicas que no funcionen tan bien en datos de alta dimensión. Veremos ejemplos de dichas técnicas a lo largo del libro.

Al mismo tiempo, mientras esta técnica puede ayudarnos a crear modelos mejores, también puede hacer que esos modelos sean más difíciles de interpretar. Es fácil entender conclusiones como “cada año adicional de experiencia suma una media de 10.000 euros al salario”, pero es mucho más difícil dar sentido a “cada incremento de 0,1 en el tercer componente

principal suma una media de 10.000 euros al salario”.

Para saber más

- Como mencionamos al final del capítulo 9, pandas, en `http://pandas.pydata.org/`, es probablemente la herramienta principal de Python para limpiar, preparar, manipular y trabajar con datos. Todos los ejemplos que hicimos a mano en este capítulo podrían haberse hecho de un modo mucho más sencillo utilizando pandas. *Python for Data Analysis* (O’Reilly), de Wes McKinney, es probablemente la mejor manera de aprender pandas.
- scikit-learn tiene una amplia variedad de funciones de descomposición de matrices, en `https://scikit-learn.org/stable/modules/classes.html#module-sklearn.decomposition`, incluyendo PCA.

11 Machine learning (aprendizaje automático)

Siempre estoy dispuesto a aprender, aunque no siempre me gusta que me den lecciones.

—Winston Churchill

Mucha gente piensa que la ciencia de datos es más que nada aprendizaje automático o *machine learning*, y que los científicos de datos se pasan el día creando, entrenando y modificando modelos de *machine learning* (aunque, pensándolo bien, muchas de esas personas no saben ni siquiera lo que es esto realmente). En realidad, lo que hace la ciencia de datos es convertir problemas de empresa en problemas de datos, y recoger, comprender, limpiar y formatear datos, tras de lo cual el aprendizaje automático es casi una anécdota. Puede que lo sea, pero es interesante y esencial, y merece mucho la pena conocerlo para poder hacer ciencia de datos.

Modelos

Antes de poder hablar de *machine learning*, tenemos que hablar primero de los modelos.

¿Qué es un modelo? No es más que la especificación de una relación matemática (o probabilística) existente entre distintas variables.

Por ejemplo, si la idea es recaudar dinero para una red social, sería interesante crear un modelo de negocio (probablemente en una hoja de cálculo) que admitiera entradas como “número de usuarios”, “ingresos de publicidad por usuario” y “número de empleados” y obtuviera el beneficio anual para los siguientes años. Una receta de un libro de cocina conlleva un modelo que relaciona entradas como “número de comensales” y “hambre” con las cantidades de ingredientes necesarias. En las partidas de póker que

pueden verse en televisión, la “probabilidad de victoria” de cada jugador se estima en tiempo real basándose en un modelo que tiene en cuenta las cartas que se han levantado hasta el momento y la distribución de cartas de la baraja.

Probablemente, el modelo de negocio está basado en sencillas relaciones matemáticas: el beneficio es el ingreso menos el gasto, el ingreso es igual a las unidades vendidas multiplicadas por el precio medio, etc.

El modelo de receta se basa casi seguro en el método de prueba y error (alguien fue a una cocina y probó distintas combinaciones de ingredientes hasta que encontró una que le gustó), mientras que el modelo de póker tiene como base la teoría de la probabilidad, las reglas del póker y ciertas suposiciones razonablemente inocuas sobre el proceso aleatorio mediante el cual se reparten las cartas.

¿Qué es el machine learning?

Cada uno tiene su propia definición exacta, pero utilizaremos *machine learning* para referirnos a la creación y utilización de modelos que se aprenden a partir de los datos. En otros contextos se podría denominar modelado predictivo o minería de datos, pero nos quedaremos con aprendizaje automático o *machine learning*. Habitualmente, nuestro objetivo será utilizar datos ya existentes para desarrollar modelos que podamos emplear para predecir resultados diversos con datos nuevos, como por ejemplo:

- Si un email es *spam* o no.
- Si una transacción con tarjeta de crédito es fraudulenta.
- En qué anuncio es más probable que un comprador haga clic.
- Qué equipo de fútbol va a ganar la Champions.

Veremos modelos supervisados (en los que hay un conjunto de datos etiquetado con las respuestas correctas de las que aprender) y modelos no supervisados (que no tienen tales etiquetas). Hay otros tipos, como

semisupervisados (en los que solo parte de los datos están etiquetados), modelos *online* (en los que el modelo necesita ser continuamente ajustado con los datos más recientes) y modelos por refuerzo (en los que, tras realizar una serie de predicciones, el modelo obtiene una señal indicando lo bien que lo hizo), que no trataremos en este libro. Pero, hasta en la situación más simple, hay universos enteros de modelos que podrían describir la relación en la que estamos interesados. En la mayoría de los casos, nosotros mismos elegiremos una familia parametrizada de modelos, y después utilizaremos los datos para aprender parámetros que de algún modo son óptimos.

Por ejemplo, podríamos suponer que la altura de una persona es (más o menos) una función lineal de su peso, y utilizar después los datos para saber cuál es esa función lineal. O también podríamos creer que un árbol de decisión es una buena forma de diagnosticar cuáles son las enfermedades que tienen nuestros pacientes y utilizarlo luego para descubrir cuál sería un árbol así "óptimo". A lo largo del resto del libro, investigaremos distintas familias de modelos que podemos aprender.

Pero, antes de poder hacer esto, tenemos que comprender mejor los fundamentos del *machine learning*. En el resto de este capítulo hablaremos de algunos de estos conceptos básicos, antes de pasar a los modelos propiamente dichos.

Sobreajuste y subajuste

Un peligro habitual en *machine learning* es el sobreajuste u *overfitting* (producir un modelo que funcione bien con los datos con los que se le entrena, pero que generaliza muy mal con datos nuevos). Podría implicar descubrir ruido en los datos. También podría suponer aprender a identificar determinadas entradas en lugar de cualesquiera factores que sean realmente predictivos para el resultado deseado.

La otra cara de esto es el subajuste o *underfitting* (producir un modelo que no funcione bien ni siquiera con los datos de entrenamiento; aunque, normalmente, cuando esto ocurre, uno mismo decide que el modelo no es lo bastante bueno y sigue buscando uno mejor).

En la figura 11.1 he ajustado tres polinomios a una muestra de datos (no se preocupe por cómo lo he hecho, llegaremos a ello en capítulos posteriores).

La línea horizontal muestra el mejor polinomio ajustado de grado 0 (es decir, constante), que subajusta enormemente los datos de entrenamiento. El mejor polinomio ajustado de grado 9 (es decir, de parámetro 10) pasa exactamente por cada uno de los puntos de datos de entrenamiento, pero lo sobreajusta en gran medida; si tuviéramos que elegir algunos puntos de datos más, muy probablemente los perdería por mucho. Y la línea de grado 1 alcanza un buen equilibrio; está bastante cerca de cada punto, y (si estos datos son representativos) la línea estará probablemente cerca también de nuevos puntos de datos.

Está claro que los modelos que son demasiado complejos llevan al sobreajuste y no generalizan bien más allá de los datos con los que fueron entrenados. Así que ¿cómo nos aseguramos de que nuestros modelos no son demasiado complejos? La estrategia más básica conlleva utilizar datos diferentes para entrenar y probar el modelo.

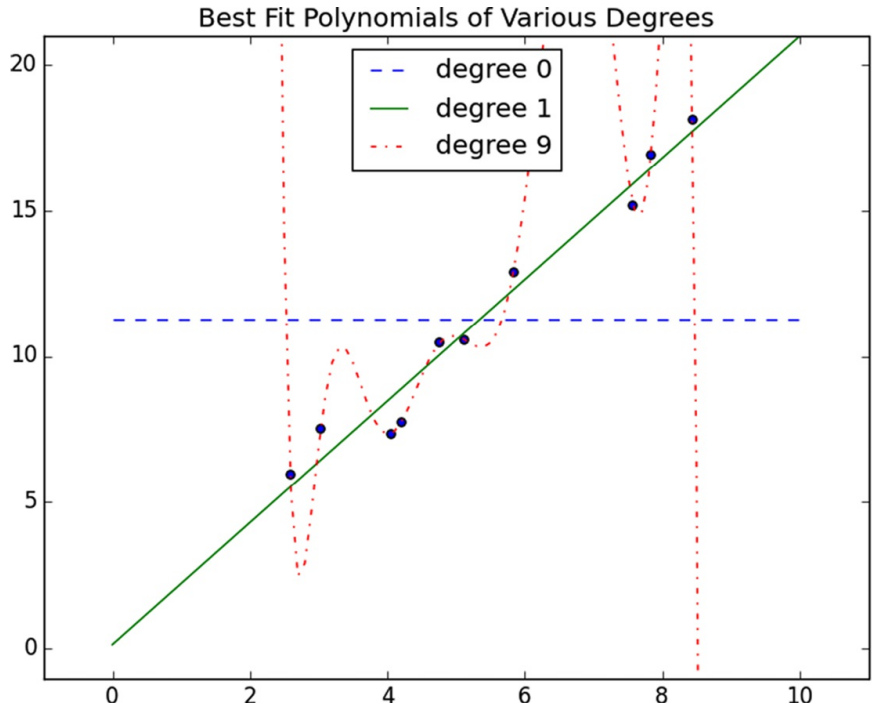


Figura 11.1. Sobreajuste y subajuste.

La forma más sencilla de hacer esto es dividir el conjunto de datos, de forma que (por ejemplo) dos tercios de él se utilicen para entrenar el modelo, después de lo cual medimos el rendimiento del modelo en el tercio restante:

```
import random
from typing import TypeVar, List, Tuple
X = TypeVar('X')              # tipo genérico para representar un punto de
                              datos
def split_data(data: List[X], prob: float) -> Tuple[List[X], List[X]]:
   """Split data into fractions [prob, 1–prob]"""
   data = data[:]                 # Hace una copia rápida
   random.shuffle(data)           # porque shuffle modifica la lista.
   cut = int(len(data) * prob)    # Usa prob para hallar un límite
   return data[:cut],             # y divide allí la lista mezclada.
   data[cut:]
data = [n for n in range(1000)]
train, test = split_data(data, 0.75)
# Las proporciones deberían ser correctas
assert len(train) == 750
assert len(test) == 250
# Y los datos originales deberían preservarse (en un cierto orden)
assert sorted(train + test) == data
```

Con frecuencia tendremos pares de variables de entrada y salida. En ese caso, debemos asegurarnos de poner juntos los valores correspondientes en los datos de entrenamiento o en los de prueba:

```
Y = TypeVar('Y')                # tipo genérico para representar variables de
                                salida
def train_test_split(xs: List[X],
                     ys: List[Y],
                     test_pct: float) -> Tuple[List[X], List[X], List[Y],
                                           List[Y]]:
   # Genera los índices y los divide
   idxs = [i for i in range(len(xs))]
   train_idxs, test_idxs = split_data(idxs, 1–test_pct)
   return ([xs[i] for i in              # x_train
   train_idxs],
      [xs[i] for i in test_idxs],       # x_test
      [ys[i] for i in train_idxs],      # y_train
      [ys[i] for i in test_idxs])       # y_test
```

Como siempre, queremos asegurarnos de que nuestro código funcione

bien:

```
xs = [x for x in range(1000)]      # xs son 1 ... 1000
ys = [2 * x for x in xs]           # cada y_i es el doble de x_i
x_train, x_test, y_train, y_test = train_test_split(xs, ys, 0.25)
# Revisa proporciones correctas
assert len(x_train) == len(y_train) == 750
assert len(x_test) == len(y_test) == 250
# Revisa puntos de datos correspondientes bien emparejados
assert all(y == 2 * x for x, y in zip(x_train, y_train))
assert all(y == 2 * x for x, y in zip(x_test, y_test))
```

Tras de lo cual se puede hacer algo como:

```
model = SomeKindOfModel()
x_train, x_test, y_train, y_test = train_test_split(xs, ys, 0.33)
model.train(x_train, y_train)
performance = model.test(x_test, y_test)
```

Si el modelo estuviera sobreajustado a los datos de entrenamiento, entonces sería de esperar que funcionara mal en los datos de prueba (que son completamente distintos). Dicho de otro modo, si funciona bien en los datos de prueba, entonces podemos estar más seguros de que está ajustado en lugar de sobreajustado. Sin embargo, hay varias maneras en las que esto puede salir mal.

La primera es si hay patrones comunes en los datos de entrenamiento y de prueba que no generalizarían a un conjunto de datos más grande.

Por ejemplo, imaginemos que el conjunto de datos que tenemos consiste en actividad de usuario, con una fila por usuario y semana. En tal caso, la mayoría de los usuarios aparecerán en los datos de entrenamiento y en los de prueba, y determinados modelos podrían aprender a identificar usuarios en lugar de descubrir relaciones que impliquen atributos. En realidad, no es una gran preocupación, aunque me ocurrió una vez.

Un problema más importante es si se utiliza la separación prueba/entrenamiento no solo para juzgar un modelo, sino también para elegir entre muchos modelos. En ese caso, aunque cada modelo individual pueda no estar sobreajustado, “elegir un modelo que funcione mejor en el conjunto de prueba” es un metaentrenamiento que hace que el conjunto de prueba funcione como un segundo conjunto de entrenamiento (por supuesto,

el modelo que funcionó mejor en el conjunto de prueba va a funcionar bien en el de entrenamiento).

En una situación como esta, deberíamos dividir los datos en tres partes: un conjunto de entrenamiento para crear modelos, un conjunto de validación para elegir entre modelos entrenados y un conjunto de prueba para juzgar el modelo final.

Exactitud

Cuando no estoy haciendo ciencia de datos, hago incursiones en medicina. En mi tiempo libre he creado una prueba barata y no invasiva que se le puede dar a un recién nacido y que predice (con más de un 98 % de exactitud) si el recién nacido desarrollará leucemia. Mi abogado me ha convencido de que la prueba no se puede patentar, de modo que compartiré aquí los detalles: predice la leucemia si y solo si el bebé se llama Luke (que suena parecido a "*leukemia*", como se dice leucemia en inglés).

Como podemos comprobar, esta prueba tiene claramente más del 98 % de precisión. Sin embargo, es increíblemente ridícula, y es también una buena forma de ilustrar la razón por la cual no solemos utilizar el término "exactitud" para medir lo bueno que es un modelo (de clasificación binaria).

Supongamos que creamos un modelo para emitir un juicio binario. ¿Es este email *spam*? ¿Deberíamos contratar a este candidato? ¿Es uno de los viajeros de este avión un terrorista en secreto?

Dado un conjunto de datos etiquetados y un modelo predictivo como este, cada punto de datos está incluido en una de cuatro categorías:

Verdadero positivo
"Este mensaje es *spam*, y hemos predicho correctamente que lo es".
Falso positivo (error tipo 1)
"Este mensaje no es *spam*, pero hemos predicho que lo es".
Falso negativo (error tipo 2)
"Este mensaje es *spam*, pero hemos predicho que no lo es".
Verdadero negativo

“Este mensaje no es *spam*, y hemos predicho correctamente que no lo es”.

A menudo representamos estas categorías como contadores de una matriz de confusión:

	Es *spam*	**No es** *spam*
Predice "es *spam*"	Verdadero positivo	Falso positivo
Predice "no es *spam*"	Falso negativo	Verdadero negativo

Veamos cómo encaja mi prueba de la leucemia en esta estructura. En estos días, aproximadamente 5 bebés de cada 1.000 se llaman Luke,[1] y la prevalencia de la leucemia a lo largo de la vida es más o menos del 1,4 %, es decir, 14 personas de cada 1.000.[2]

Si pensamos que estos factores son independientes y aplicamos mi prueba “Luke viene de leucemia” a un millón de personas, esperaríamos ver una matriz de confusión como esta:

	Leucemia	**No leucemia**	**Total**
"Luke"	70	4.930	5.000
No "Luke"	13.930	981.070	995.000
Total	14.000	986.000	1.000.000

Podemos entonces utilizar estas matrices para calcular distintas estadísticas sobre el rendimiento de modelos. Por ejemplo, exactitud se define como la fracción de las predicciones correctas:

```
def accuracy(tp: int, fp: int, fn: int, tn: int) -> float:
   correct = tp + tn
   total = tp + fp + fn + tn
   return correct / total
assert accuracy(70, 4930, 13930, 981070) == 0.98114
```

Parece un número que impresiona bastante. Pero sin duda no es una buena prueba, lo que significa que probablemente no deberíamos creer demasiado en la exactitud pura y dura.

Es habitual mirar la combinación de precisión y recuerdo. La precisión mide lo exactas que fueron nuestras predicciones positivas:

```
def precision(tp: int, fp: int, fn: int, tn: int) -> float:
   return tp / (tp + fp)
assert precision(70, 4930, 13930, 981070) == 0.014
```

Y el recuerdo mide la fracción de los positivos que identificó nuestro modelo:

```
def recall(tp: int, fp: int, fn: int, tn: int) -> float:
   return tp / (tp + fn)
assert recall(70, 4930, 13930, 981070) == 0.005
```

Ambos son números terribles, que reflejan que es un modelo espantoso.

En ocasiones, la precisión y el recuerdo se combinan en la puntuación F1 (*F1 score*), que se define como:

```
def f1_score(tp: int, fp: int, fn: int, tn: int) -> float:
   p = precision(tp, fp, fn, tn)
   r = recall(tp, fp, fn, tn)
   return 2 * p * r / (p + r)
```

Esta es la media armónica[3] de precisión y recuerdo, y se sitúa forzosamente entre ellos. Normalmente, la elección de un modelo implica un término medio entre precisión y recuerdo. Un modelo que predice “sí” en cuanto tiene la más mínima confianza en este resultado tendrá probablemente un elevado recuerdo, pero una precisión baja; un modelo que prediga “sí” solo cuando tenga toda la confianza en que será así es probable que tenga un bajo recuerdo y una elevada precisión.

También se puede pensar en esto como en un término medio entre falsos positivos y falsos negativos. Decir “sí” con demasiada frecuencia dará muchos falsos positivos, pero decir “no” proporcionará muchos falsos negativos.

Supongamos que hubiera 10 factores de riesgo para la leucemia y que, cuántos más de ellos se tuvieran, más alta sería la probabilidad de padecer la enfermedad. En ese caso podemos imaginar un continuo de pruebas: “predecir leucemia si al menos hay un factor de riesgo”, “predecir leucemia si al menos hay dos factores de riesgo”, etc. A medida que aumenta el umbral, sube la precisión de la prueba (ya que es más probable que las personas con más factores de riesgo desarrollen la enfermedad), y disminuirá el recuerdo de la prueba (ya que cada vez menos posibles sufridores de la enfermedad alcanzarán el umbral). En casos así, elegir el umbral correcto es cuestión de encontrar el término medio adecuado.

El término medio entre sesgo y varianza

Otra forma de pensar con respecto al problema del sobreajuste es como en un término medio entre sesgo y varianza. Ambas son medidas de lo que ocurriría si hubiera que volver a entrenar nuestro modelo muchas veces en distintos conjuntos de datos de entrenamiento (a partir de la misma población más grande). Por ejemplo, el modelo de grado 0 de la sección anterior “Sobreajuste y subajuste” producirá muchos errores con prácticamente cualquier conjunto de entrenamiento (dibujado a partir de la misma población), lo que significa que tiene un alto sesgo. No obstante, cualesquiera dos conjuntos de entrenamiento aleatoriamente elegidos darían modelos bastante similares (ya que deberían tener valores promedio también bastante similares). De modo que decimos que tiene una varianza baja. El alto sesgo y la baja varianza corresponden normalmente al subajuste.

Por otro lado, el modelo de grado 9 encaja perfectamente en el conjunto de entrenamiento. Tiene muy bajo sesgo, pero muy alta varianza (ya que cualesquiera dos conjuntos de entrenamiento darían probablemente lugar a modelos muy diferentes). Esto corresponde al sobreajuste.

Pensar en problemas de modelos de este modo puede permitirnos averiguar qué hacer cuando el modelo no funciona tan bien.

Si el modelo tiene un elevado sesgo (lo que significa que funciona mal incluso con los datos de entrenamiento), algo que se puede probar es añadir

más funciones. Pasar del modelo de grado 0 de “Sobreajuste y subajuste” al modelo de grado 1 supuso una gran mejora. Si nuestro modelo tiene una alta varianza, se pueden eliminar funciones de forma similar. Pero otra solución es obtener más datos (si se puede).

En la figura 11.2, ajustamos un polinomio de grado 9 a muestras de distinto tamaño. El ajuste de modelo basado en 10 puntos de datos está por todas partes, como ya vimos antes. Pero, si entrenamos con 100 puntos de datos, hay mucho menos subajuste.

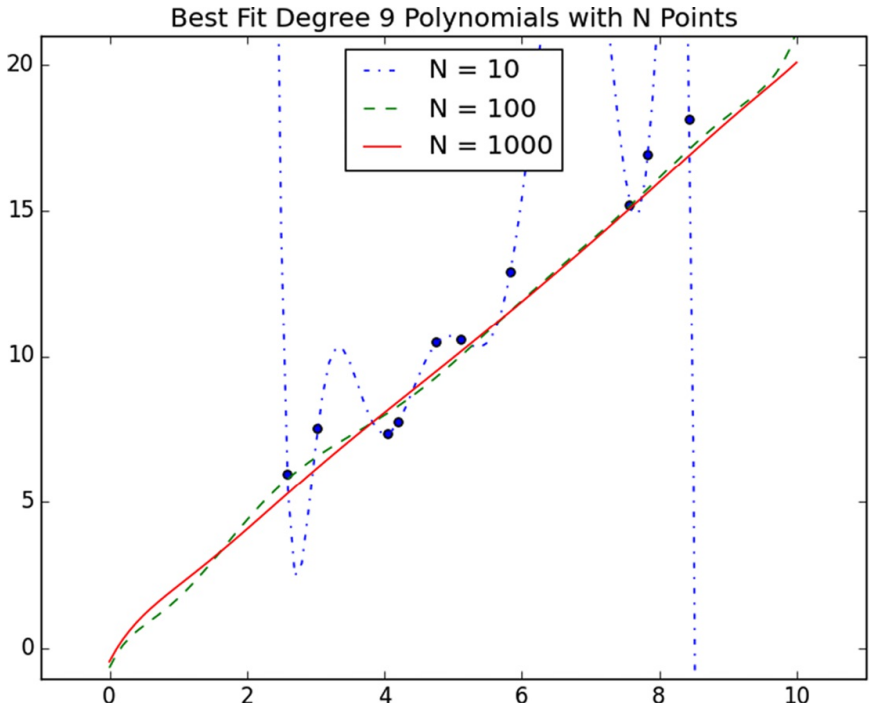


Figura 11.2. Reducir la varianza con más datos.

Y el modelo entrenado con 1.000 puntos de datos parece muy similar al modelo de grado 1. Manteniendo la complejidad del modelo constante, cuantos más datos tengamos, más difícil será que haya sobreajuste. Por otro lado, más datos no ayudarán con el sesgo. Si el modelo no utiliza suficientes funciones para capturar uniformidades en los datos, lanzarle más datos no servirá de nada.

Extracción y selección de características

Como ya se ha mencionado, cuando los datos no tienen suficientes características, es probable que el modelo subajuste; y, cuando los datos tienen demasiadas características, es fácil que sobreajuste. Pero ¿qué son las características, y de dónde proceden?

Las características son las entradas que le proporcionamos a nuestro modelo.

En el caso más sencillo, las características simplemente nos vienen dadas. Si queremos predecir el salario de alguien basándonos en sus años de experiencia, entonces los años de experiencia son la única característica que tenemos (aunque, como ya vimos en el apartado "Sobreajuste y subajuste", también se podría considerar añadir años de experiencia al cuadrado, al cubo, etc., si ello nos permitiera construir un modelo mejor).

Las cosas se ponen más interesantes a medida que los datos se van complicando. Supongamos que intentamos crear un filtro de *spam* para predecir si un email es basura o no. La mayoría de los modelos no sabrán qué hacer con un email sin depurar, que no es más que una colección de texto. Tendremos que extraer características. Por ejemplo:

- ¿Contiene el email la palabra viagra?
- ¿Cuántas veces aparece la letra d?
- ¿Cuál era el dominio del remitente?

La respuesta a la primera pregunta es simplemente sí o no, lo que codificaríamos normalmente como 1 o 0. La segunda es un número. Y la tercera es una elección entre un reducido conjunto de opciones.

Prácticamente siempre extraeremos características de nuestros datos que entran en una de estas tres categorías. Es más, los tipos de características que tenemos limitan los tipos de modelos que podemos utilizar.

- El clasificador Naive Bayes que crearemos en el capítulo 13 es adecuado para características de tipo sí o no, como la primera pregunta de la lista anterior.
- Los modelos de regresión, que estudiaremos en los capítulos 14 y 16, requieren características numéricas (que podrían incluir variables ficticias que son ceros y unos).

- Y los árboles de decisión, que veremos en el capítulo 17, pueden admitir datos numéricos o categóricos.

Aunque en el ejemplo de filtro de *spam* vimos formas de crear características, otras veces también veremos maneras de eliminarlas.

Por ejemplo, las entradas podrían ser vectores de varios cientos de números. Dependiendo de la situación, podría ser apropiado reducirlos a un puñado de dimensiones importantes (como en la sección "Reducción de la dimensionalidad" del capítulo 10) y utilizar solamente ese pequeño número de características. O podría ser adecuado utilizar una técnica (como la regularización, que veremos en la sección del mismo nombre del capítulo 15) que penaliza los modelos cuantas más características utilizan.

¿Cómo elegimos las características? Aquí entra en juego una combinación entre experiencia y conocimientos del sector. Si ha recibido muchos emails, entonces probablemente intuirá que la presencia de determinadas palabras podría ser un buen indicador de *spam*. Quizá también sea capaz de intuir que el número de letras "d" casi seguro que no es un buen indicador de *spam*. Pero, en general, siempre habrá que probar distintas cosas, lo que es parte de la diversión.

Para saber más

- ¡Siga leyendo! Los siguientes capítulos hablan de distintas familias de modelos de *machine learning*.
- El curso de *machine learning* de Coursera, en `https://www.coursera.org/learn/machine-learning`, es el MOOC original y un buen punto de partida para obtener una profunda comprensión de los fundamentos de *machine learning*.
- *The Elements of Statistical Learning*, de Jerome H. Friedman, Robert Tibshirani y Trevor Hastle (Springer), es un libro de texto un poco canónico que se puede descargar en línea gratuitamente en `https://hastie.su.domains/ElemStatLearn/printings/ESLII_prin`
  Pero queda avisado: es muy técnico en matemáticas.

[1] https://www.babycenter.com/baby-names-luke-2918.htm.

[2] https://seer.cancer.gov/statfacts/html/leuks.html.

[3] https://es.wikipedia.org/wiki/Media_arm%C3%B3nica.

12 k vecinos más cercanos

Si quieres molestar a tus vecinos, di la verdad sobre ellos.

—Pietro Aretino

Supongamos que alguien intenta predecir cómo voy a votar en las próximas elecciones. Si ese alguien no sabe nada sobre mí (y dispone de los datos), un planteamiento razonable podría ser ver cómo están pensando votar mis vecinos. Viviendo en Seattle, como yo, la intención de mis vecinos sin duda alguna es votar al candidato demócrata, lo que sugiere que "candidato demócrata" es asimismo una buena suposición para mí.

Ahora supongamos que esa persona sabe más sobre mí que solamente geografía; quizá conoce mi edad, mis ingresos, cuántos hijos tengo, etc. En la medida en que mi comportamiento se vea influido (o caracterizado) por esos conocimientos, parece probable que, de entre todas esas dimensiones, ver qué piensan los vecinos que están cerca de mí sea un indicador aún mejor que incluir a todos mis vecinos. Esta es la idea del método de clasificación de vecinos más cercanos.

El modelo

Vecinos más cercanos es uno de los modelos predictivos más sencillos que hay. No realiza suposiciones matemáticas y no exige ningún tipo de maquinaria pesada. Lo único que requiere es:

- Una cierta noción de distancia.
- La suposición de que los puntos que están cerca uno de otro son similares.

La mayor parte de las técnicas que veremos en este libro tienen en cuenta el conjunto de datos como un todo para descubrir patrones en los datos.

Vecinos más cercanos, por otra parte, descuida de forma consciente mucha información, ya que la predicción para cada nuevo punto depende únicamente del montón de puntos más cercanos a él.

Es más, probablemente vecinos más cercanos no ayudará a comprender los elementos causantes del fenómeno que se esté observando. Predecir mi voto basándose en el de mis vecinos no dice mucho sobre las causas que me hacen votar como lo hago, mientras que otro modelo alternativo que prediga mi voto basándose en (por ejemplo) mis ingresos y mi estado civil sí podría ayudar a averiguarlas.

En situaciones generales, tenemos puntos de datos y el correspondiente conjunto de etiquetas. Las etiquetas podrían ser `True` y `False`, indicando que cada entrada satisface una determinada condición, como "¿es *spam*?", "¿es venenoso?" o "¿sería divertido de ver?". O bien podrían ser categorías, como calificaciones de películas (G, PG, PG-13, R, NC-17). También podrían ser los nombres de los candidatos presidenciales o incluso lenguajes de programación favoritos.

En nuestro caso, los puntos de datos serán vectores, lo que significa que podemos utilizar la función `distance` del capítulo 4.

Supongamos que elegimos un número *k*, como 3 o 5. Entonces, cuando queremos clasificar un punto de datos nuevo, encontramos los *k* puntos más cercanos etiquetados y les dejamos votar en el nuevo resultado.

Para ello, necesitaremos una función que cuente votos. Una posibilidad es:

```
from typing import List
from collections import Counter
def raw_majority_vote(labels: List[str]) -> str:
   votes = Counter(labels)
   winner, _ = votes.most_common(1)[0]
   return winner
assert raw_majority_vote([‘a’, ‘b’, ‘c’, ‘b’]) == ‘b’
```

Pero esto no hace nada inteligente con los empates. Por ejemplo, imaginemos que estamos calificando películas y las cinco más cercanas están clasificadas como G, G, PG, PG y R. Vemos que tanto G como PG tienen dos votos. En ese caso, tenemos varias opciones:

- Elegir uno de los ganadores aleatoriamente.
- Ponderar los votos por distancia y elegir el ganador resultante.
- Reducir *k* hasta encontrar un ganador único.

Implementaremos la tercera opción:

```
def majority_vote(labels: List[str]) -> str:
   """Assumes that labels are ordered from nearest to farthest."""
   vote_counts = Counter(labels)
   winner, winner_count = vote_counts.most_common(1)[0]
   num_winners = len([count
                for count in vote_counts.values()
                if count == winner_count])
   if num_winners == 1:
      return winner                          # ganador único, así que lo devuelve
   else:
      return                                 # prueba de nuevo sin el más lejano
      majority_vote(labels[:-1])
# Empate, así que busca los primeros 4, entonces 'b'
assert majority_vote(['a', 'b', 'c', 'b', 'a']) == 'b'
```

Este método seguro que terminará funcionando, ya que en el peor de los casos lo iremos reduciendo todo hasta que quede una sola etiqueta, momento en el cual esa es la que gana.

Con esta función, es fácil crear un clasificador:

```
from typing import NamedTuple
from scratch.linear_algebra import Vector, distance
class LabeledPoint(NamedTuple):
   point: Vector
   label: str
def knn_classify(k: int,
             labeled_points: List[LabeledPoint],
             new_point: Vector) -> str:
   # Ordena los puntos etiquetados de más cercano a más lejano.
   by_distance = sorted(labeled_points,
                  key=lambda lp: distance(lp.point, new_point))
   # Halla las etiquetas para los k más cercanos
   k_nearest_labels = [lp.label for lp in by_distance[:k]]
   # y les deja votar.
   return majority_vote(k_nearest_labels)
```

Veamos cómo funciona esto.

Ejemplo: el conjunto de datos iris

El conjunto de datos iris es una de las bases de *machine learning*. Contiene un grupo de medidas para 150 flores que representan tres especies de iris. Para cada flor tenemos la longitud y la anchura del pétalo, la longitud del sépalo, además de su especie. Se puede descargar en la página `https://archive.ics.uci.edu/ml/datasets/iris`.

```
import requests
data = requests.get(
   "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
)
with open('iris.dat', 'w') as f:
   f.write(data.text)
```

Los datos están separados por comas, con los campos:

```
sepal_length, sepal_width, petal_length, petal_width, class
```

Por ejemplo, la primera fila es algo así:

```
5.1,3.5,1.4,0.2,Iris-setosa
```

En esta sección trataremos de crear un modelo que pueda predecir la clase (es decir, la especie) a partir de las cuatro primeras medidas.

Para empezar, carguemos y exploremos los datos. Nuestra función vecinos más cercanos espera un `LabeledPoint`, de modo que representemos nuestros datos de ese modo:

```
from typing import Dict
import csv
from collections import defaultdict
def parse_iris_row(row: List[str]) -> LabeledPoint:
   """
   sepal_length, sepal_width, petal_length, petal_width, class
   """
   measurements = [float(value) for value in row[:-1]]
```

```
    # la clase es p. ej. "Iris-virginica"; solo queremos "virginica"
    label = row[-1].split("-")[-1]
    return LabeledPoint(measurements, label)
with open('iris.data') as f:
    reader = csv.reader(f)
    iris_data = [parse_iris_row(row) for row in reader]
# Agrupamos también solo los puntos por especie/etiqueta para trazarlos
points_by_species: Dict[str, List[Vector]] = defaultdict(list)
for iris in iris_data:
    points_by_species[iris.label].append(iris.point)
```

Nos vendría bien trazar las medidas de forma que podamos ver cómo varían por especie. Lamentablemente, son cuatridimensionales, lo que hace que resulten difíciles de representar. Una cosa que podemos hacer es recurrir a los gráficos de dispersión para cada uno de los seis pares de medidas (figura 12.1). No explicaré todos los detalles, pero es una buena forma de ilustrar cosas más complicadas que se pueden hacer con matplotlib, por lo que vale la pena estudiarlo:

```
from matplotlib import pyplot as plt
metrics = ['sepal length', 'sepal width', 'petal length', 'petal width']
pairs = [(i, j) for i in range(4) for j in range(4) if i < j]
marks = ['+', '.', 'x']          # we have 3 classes, so 3 markers
fig, ax = plt.subplots(2, 3)
for row in range(2):
    for col in range(3):
        i, j = pairs[3 * row + col]
        ax[row][col].set_title(f"{metrics[i]} vs {metrics[j]}", fontsize=8)
        ax[row][col].set_xticks([])
        ax[row][col].set_yticks([])
        for mark, (species, points) in zip(marks, points_by_species.items()):
            xs = [point[i] for point in points]
            ys = [point[j] for point in points]
            ax[row][col].scatter(xs, ys, marker=mark, label=species)
ax[-1][-1].legend(loc='lower right', prop={'size': 6})
plt.show()
```

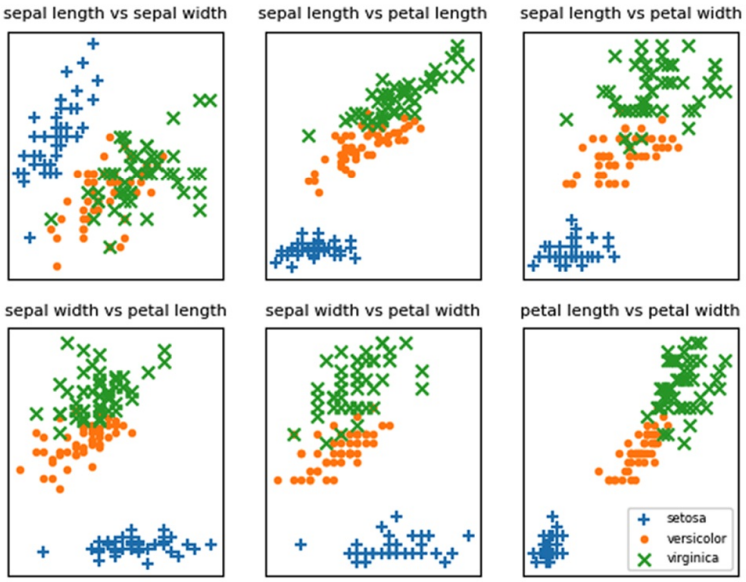


Figura 12.1. Gráficos de dispersión de iris.

Si echamos un vistazo a estos gráficos, parece que en realidad las medidas se agrupan por especie. Por ejemplo, mirando solamente la longitud y la anchura del sépalo, probablemente no se podría distinguir entre versicolor y virginica. Pero, en cuanto se añade a la mezcla la longitud y la anchura del pétalo, parece posible predecir la especie basándose en los vecinos más cercanos.

Para empezar, dividamos los datos en un conjunto de prueba y otro de entrenamiento:

```
import random
from scratch.machine_learning import split_data
random.seed(12)
iris_train, iris_test = split_data(iris_data, 0.70)
assert len(iris_train) == 0.7 * 150
assert len(iris_test) == 0.3 * 150
```

El conjunto de entrenamiento será los “vecinos” que utilizaremos para clasificar los puntos del conjunto de prueba. Simplemente tenemos que elegir un valor para *k*, el número de vecinos que consiguen votar. Demasiado pequeño (pensemos en $k = 1$), y dejaremos que los valores atípicos (*outliers*)

tengan demasiada influencia; demasiado grande (digamos *k* = 105), y simplemente predeciremos la clase más común del conjunto de datos.

En una aplicación real (y con más datos), podríamos crear un conjunto de validación diferente y emplearlo para elegir *k*. Aquí utilizaremos *k* = 5:

```
from typing import Tuple
# controla las veces que vemos (predicted, actual)
confusion_matrix: Dict[Tuple[str, str], int] = defaultdict(int)
num_correct = 0
for iris in iris_test:
   predicted = knn_classify(5, iris_train, iris.point)
   actual = iris.label
   if predicted == actual:
      num_correct += 1
   confusion_matrix[(predicted, actual)] += 1
pct_correct = num_correct / len(iris_test)
print(pct_correct, confusion_matrix)
```

En este sencillo conjunto de datos, el modelo predice casi perfectamente. Hay una única versicolor para la que predice virginica, pero por lo demás acierta de pleno.

La maldición de la dimensionalidad

El algoritmo de *k* vecinos más cercanos da problemas con muchas dimensiones gracias a la "maldición de la dimensionalidad", que se reduce al hecho de que los espacios de muchas dimensiones son inmensos. Los puntos en los espacios con muchas dimensiones tienden a no estar en absoluto cerca unos de otros. Una forma de comprobar esto es generando de forma aleatoria pares de puntos en el "cubo unitario" *d*-dimensional en diversas dimensiones, y calculando las distancias entre ellos.

A estas alturas, generar puntos aleatorios ya debería ser algo natural:

```
def random_point(dim: int) -> Vector:
   return [random.random() for _ in range(dim)]
```

Igual que lo es escribir una función para generar las distancias:

```
def random_distances(dim: int, num_pairs: int) -> List[float]:
   return [distance(random_point(dim), random_point(dim))
      for _ in range(num_pairs)]
```

Para cada dimensión de 1 a 100, calcularemos 10.000 distancias, que usaremos para calcular la distancia media entre puntos y la distancia mínima entre puntos en cada dimensión (figura 12.2):

```
import tqdm
dimensions = range(1, 101)
avg_distances = []
min_distances = []
random.seed(0)
for dim in tqdm.tqdm(dimensions, desc="Curse of Dimensionality"):
   distances = random_distances(dim, 10000)         # 10.000 pares aleatorios
   avg_distances.append(sum(distances) / 10000)     # controla la media
   min_distances.append(min(distances))             # controla la mínima
```

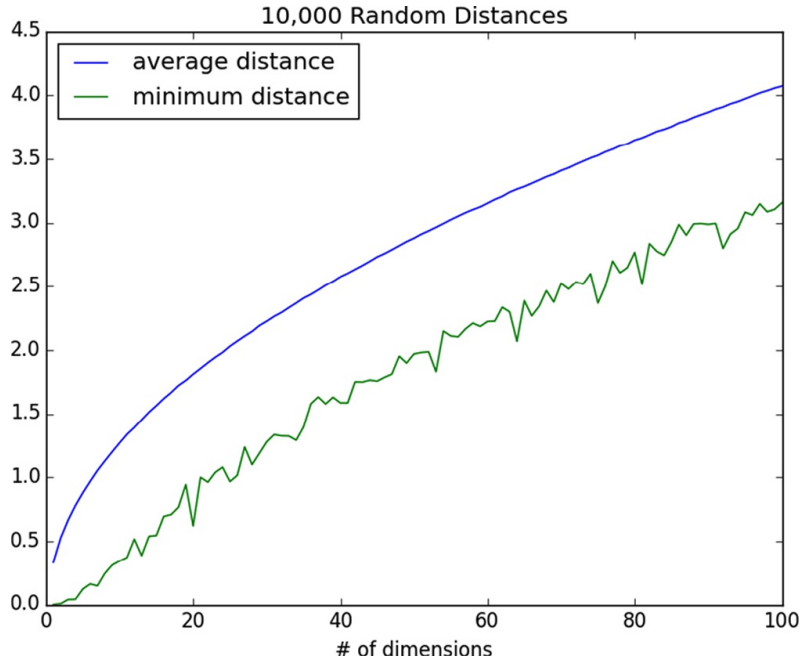


Figura 12.2. La maldición de la dimensionalidad.

A medida que aumenta el número de dimensiones, se incrementa también la distancia media entre puntos. Pero lo más problemático es la proporción

entre la distancia más cercana y la distancia media (figura 12.3):

```
min_avg_ratio = [min_dist / avg_dist
             for min_dist, avg_dist in zip(min_distances, avg_distances)]
```

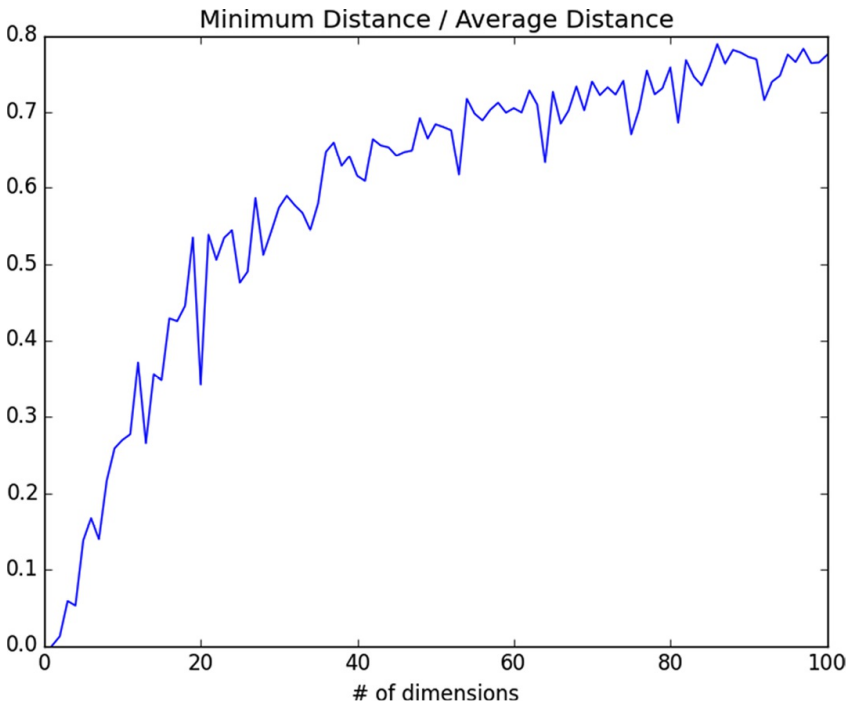


Figura 12.3. Otra vez la maldición de la dimensionalidad.

En conjuntos de datos de pocas dimensiones, los puntos más cercanos tienden a estar mucho más cerca que la media. Pero dos puntos están cerca solo si lo están en cada dimensión, y cada dimensión adicional (incluso aunque sea solo ruido) es otra oportunidad para que cada punto esté más lejos de los demás. Cuando se tienen muchas dimensiones, es probable que los puntos más próximos no estén mucho más cerca que la media, por tanto, que dos puntos estén cerca no significa mucho (a menos que haya mucha estructura en los datos que les haga comportarse como si fueran muy poco dimensionales).

Una forma diferente de pensar acerca de este problema tiene que ver con la escasez de espacios de muchas dimensiones.

Si elegimos 50 números aleatorios entre 0 y 1, probablemente obtendremos una muestra bastante buena del intervalo de la unidad (figura 12.4).

Si elegimos 50 puntos aleatorios dentro del cuadrado unitario, tendremos menos cobertura (figura 12.5).



Figura 12.4. Cincuenta puntos aleatorios en una sola dimensión.

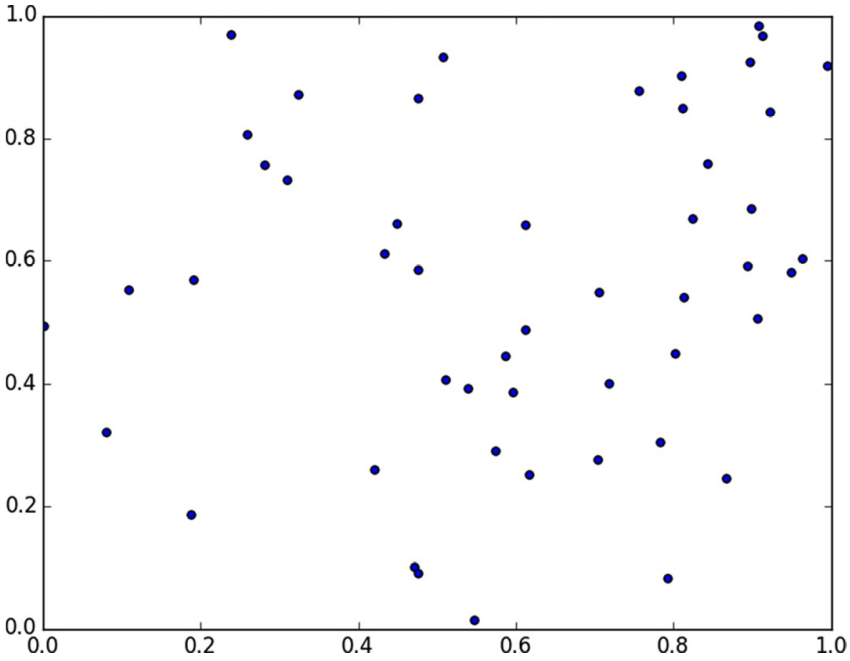


Figura 12.5. Cincuenta puntos aleatorios en dos dimensiones.

Y en tres dimensiones, todavía menos (figura 12.6).

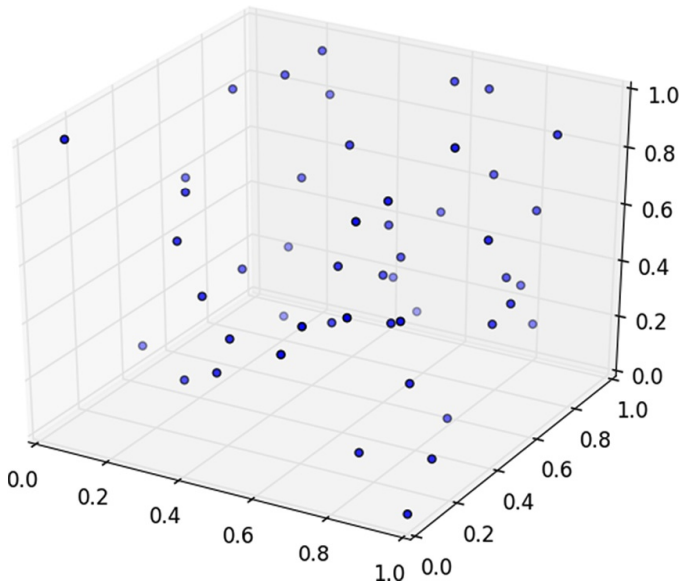


Figura 12.6. Cincuenta puntos aleatorios en tres dimensiones.

matplotlib no crea bien gráficos de cuatro dimensiones, de modo que hasta ahí llegaremos, pero ya se puede verificar que están empezando a haber grandes espacios vacíos sin puntos cerca. En más dimensiones (a menos que obtengamos más datos exponencialmente), esos grandes espacios vacíos representan regiones alejadas de todos los puntos que deseamos utilizar en nuestras predicciones.

Así que, si queremos utilizar vecinos más cercanos en muchas dimensiones, probablemente es una buena idea hacer primero algún tipo de reducción de dimensionalidad.

Para saber más

- scikit-learn tiene muchos modelos de vecinos más cercanos, en `https://scikit-learn.org/stable/modules/neighbors.html`.

13 Naive Bayes

Está bien ser ingenuo de corazón, pero no serlo de mente.

—Anatole France

Una red social no es muy buena si la gente no puede conectar. Según esto, DataSciencester tiene una característica muy popular que permite a sus miembros enviar mensajes a otros miembros. Aunque la mayoría de los miembros son ciudadanos responsables que solo envían mensajes agradables del tipo "¿cómo te va?", hay unos cuantos malandrines que envían correo basura de forma persistente sobre maneras de hacerse rico, productos farmacéuticos sin receta y programas de acreditación en ciencia de datos con ánimo de lucro. Los usuarios han empezado a quejarse, por lo que el vicepresidente de Mensajería le ha pedido que utilice la ciencia de datos para encontrar una manera de filtrar estos mensajes de *spam*.

Un filtro de spam realmente tonto

Supongamos un "universo" que consista en recibir un mensaje elegido aleatoriamente del total de posibles mensajes. Digamos que S es el evento "el mensaje es spam" y B el evento "el mensaje contiene la palabra bitcoin". El teorema de Bayes nos dice que la probabilidad de que el mensaje sea *spam*, condicionado a que contenga la palabra "bitcoin", es:

$$P(S|B) = [P(B|S)P(S)]/[P(B|S)P(S) + P(B|\neg S)P(\neg S)]$$

El numerador es la probabilidad de que un mensaje sea *spam* y contenga "bitcoin", mientras que el denominador es solo la probabilidad de que un mensaje contenga "bitcoin". Por lo tanto, se puede pensar en este cálculo como en la sencilla representación de la proporción de mensajes de bitcoin que son *spam*.

Si tenemos una gran colección de mensajes que sabemos que son *spam*, y otra gran colección de mensajes que sabemos que no son *spam*, podemos estimar fácilmente $P(B|S)$ y $P(B|\neg S)$. Si seguimos suponiendo que es igualmente probable que un determinado mensaje sea *spam* o no *spam* (de forma que $P(S) = P(\neg S) = 0{,}5$), entonces:

$$P(S|B) = P(B|S)/[P(B|S) + P(B|\neg S)$$

Por ejemplo, si el 50 % de los mensajes de *spam* tienen la palabra "bitcoin", pero solo el 1 % de los mensajes que no son *spam* la tienen, entonces la probabilidad de que cualquier email determinado que contenga la palabra "bitcoin" sea *spam* es:

$$0{,}5 / (0{,}5 + 0{,}01) = 98\ \%$$

Un filtro de spam más sofisticado

Supongamos ahora que tenemos un vocabulario formado por muchas palabras w_1, ..., w_n. Para llevar esto al reino de la teoría de la probabilidad, denominaremos X_i al evento "un mensaje contiene la palabra w_i". Vamos a imaginar que (mediante algún proceso no especificado en este momento) hemos hallado una estimación $P(X_i|S)$ para la probabilidad de que un mensaje de *spam* contenga la palabra *i*-ésima, y una estimación $P(X_i|\neg S)$ para la probabilidad de que un mensaje que no sea *spam* contenga la palabra *i*-ésima.

La clave de Naive Bayes es hacer la (gran) suposición de que las presencias (o ausencias) de cada palabra son independientes una de otra, condicionado todo ello a que un mensaje sea *spam* o no lo sea. Intuitivamente, esta suposición significa que saber si un determinado mensaje de *spam* contiene la palabra "bitcoin" no da ninguna información sobre si el mismo mensaje contiene la palabra Rolex. En términos matemáticos, esto significa que:

$$P(X_1 = x_1, ..., X_n = x_n|S) = P(X_1 = x_1|S) \text{ x } ... \text{ x } P(X_n = x_n|S)$$

Esto es una suposición extrema (hay una razón por la que esta técnica lleva el adjetivo *naive*, ingenuo, en su nombre). Imaginemos que nuestro vocabulario solo consiste en las palabras "bitcoin" y Rolex, y que la mitad de los mensajes de *spam* son para "gana bitcoin" y la otra mitad son para "auténtico Rolex". En este caso, Naive Bayes estima que un mensaje de *spam* que contiene tanto "bitcoin" como "Rolex" es:

$$P(X_1 = 1, X_2 = 1|S) = P(X_1 = 1|S)P(X_2 = 1|S) = .5 \text{ x } .5 = .25$$

Ya que hemos asumido el saber que "bitcoin" y "Rolex" nunca ocurren realmente juntas. A pesar de la falta de realismo de esta suposición, este modelo suele funcionar bien y se ha utilizado desde siempre en filtros de *spam* reales.

El mismo razonamiento del teorema de Bayes que hemos utilizado para nuestro filtro de *spam* "solo bitcoin" nos dice que podemos calcular la probabilidad de que un mensaje sea *spam* utilizando la ecuación:

$$P(S|X = x) = P(X = x|S)/[P(X = x|S) + P(X = x|\neg S)]$$

La suposición de Naive Bayes nos permite calcular cada una de las probabilidades de la derecha simplemente multiplicando las estimaciones de probabilidad individuales por cada palabra del vocabulario.

En la práctica, nos interesará evitar multiplicar muchas probabilidades para no tener un problema llamado *underflow* (o desbordamiento por defecto), en el que los ordenadores no saben manejar bien números de punto flotante que son demasiado próximos a 0. Recordando de álgebra que log (ab) = log a + log b y que exp (log x) = x, normalmente calculamos p_1 * ... * p_n como el equivalente (pero que maneja mejor el punto flotante):

$$\exp (\log (p_1) + ... + \log (p_n))$$

El único desafío que nos queda viene con las estimaciones para $P(X_i|S)$ y $P(X_i|\neg S)$, las probabilidades de que un mensaje que es *spam* (o que no es *spam*) contenga la palabra "w_i". Si tenemos un buen número de mensajes de

“entrenamiento” etiquetados como *spam* y no *spam*, un obvio primer intento es estimar $P(X_i|S)$ simplemente como la fracción de mensajes de *spam* que contienen la palabra “w_i”.

Pero esto provoca un gran problema. Supongamos que en nuestro conjunto de entrenamiento la palabra del vocabulario datos solo aparece en mensajes que no son *spam*. Entonces estimaríamos $P(\text{datos}|S) = 0$. El resultado es que nuestro clasificador Naive Bayes siempre asignaría probabilidad de *spam* 0 a cualquier mensaje que contuviera la palabra “datos”, incluso a un mensaje como “datos sobre bitcoin gratis y auténticos relojes Rolex”. Para evitar este problema, normalmente empleamos algún tipo de suavizado.

En particular, elegiremos un pseudocontador (k) y estimaremos la probabilidad de ver la palabra i-ésima en un mensaje de *spam* como:

$P(X_i|S)$ = (k + número de spams que contienen w_i)/(2k + número de spams)

Hacemos lo mismo para $P(X_i|\neg S)$. Es decir, cuando calculemos las probabilidades de *spam* para la palabra i-ésima, supondremos que también vimos k mensajes adicionales que no son *spam* y que contienen la palabra y k mensajes adicionales que no son *spam* y que no contienen la palabra.

Por ejemplo, si datos aparece en 0/98 mensajes de *spam*, y si k es 1, estimamos $P(\text{datos}|S)$ como 1/100 = 0,01, lo que permite a nuestro clasificador seguir asignando una cierta probabilidad de *spam* no cero a mensajes que contienen la palabra “datos”.

Implementación

Ahora que tenemos todas las piezas, debemos montar nuestro clasificador. Primero, creamos una sencilla función con `tokenize` para separar los mensajes en palabras. Convertiremos antes cada mensaje a minúsculas, y emplearemos después `re.findall` para extraer “palabras” formadas por

letras, números y apóstrofos. Para terminar, utilizamos `set` para obtener las palabras por separado.

```
from typing import Set
import re
def tokenize(text: str) -> Set[str]:
   text = text.lower()                            # Convierte a minúsculas,
   all_words = re.findall("[a-z0-9’]+", text)     # extrae las palabras y
   return set(all_words)                          # elimina duplicados.
assert tokenize("Data Science is science") == {"data", "science", "is"}
```

También definiremos un tipo para nuestros datos de entrenamiento:

```
from typing import NamedTuple
class Message(NamedTuple):
   text: str
   is_spam: bool
```

Como nuestro clasificador tiene que controlar los *tokens*, contadores y etiquetas en los datos de entrenamiento, le crearemos una clase. Siguiendo el convenio, denominaremos emails “*ham*” a los mensajes que no son *spam*.

El constructor tomará solo un parámetro, el pseudocontador que se utiliza al calcular las probabilidades. También inicializa un conjunto vacío de *tokens*, contadores para controlar la frecuencia con la que cada *token* es visto en mensajes de *spam* y *ham*, y contadores de la cantidad de mensajes de *spam* y *ham* en los que fue entrenado:

```
from typing import List, Tuple, Dict, Iterable
import math
from collections import defaultdict
class NaiveBayesClassifier:
   def __init__(self, k: float = 0.5) -> None:
      self.k = k              # factor suavizante
      self.tokens: Set[str] = set()
      self.token_spam_counts: Dict[str, int] = defaultdict(int)
      self.token_ham_counts: Dict[str, int] = defaultdict(int)
      self.spam_messages = self.ham_messages = 0
```

A continuación, le daremos un método para entrenarlo con un montón de

mensajes. Primero, aumentamos los contadores de `spam_messages` y `ham_messages`. Después, dividimos cada mensaje de texto con `tokenize` y por cada *token* incrementamos los `token_spam_counts` o `token_ham_counts` según el tipo de mensaje:

```
def train(self, messages: Iterable[Message]) -> None:
    for message in messages:
        # Incrementa contadores de mensajes
        if message.is_spam:
            self.spam_messages += 1
        else:
            self.ham_messages += 1
        # Incrementa contadores de palabras
        for token in tokenize(message.text):
            self.tokens.add(token)
            if message.is_spam:
                self.token_spam_counts[token] += 1
            else:
                self.token_ham_counts[token] += 1
```

Lo que queremos en última instancia es predecir *P*(spam | token). Como ya hemos visto antes, para aplicar el teorema de Bayes tenemos que conocer *P*(token | spam) y *P*(token | ham) para cada *token* del vocabulario. De modo que crearemos una función auxiliar “privada” para calcularlos:

```
def _probabilities(self, token: str) -> Tuple[float, float]:
    """returns P(token | spam) and P(token | ham)"""
    spam = self.token_spam_counts[token]
    ham = self.token_ham_counts[token]
    p_token_spam = (spam + self.k) / (self.spam_messages + 2 * self.k)
    p_token_ham = (ham + self.k) / (self.ham_messages + 2 * self.k)
    return p_token_spam, p_token_ham
```

Finalmente, estamos listos para escribir nuestro modelo `predict`. Como dijimos antes, en lugar de multiplicar muchas probabilidades pequeñas, sumaremos las probabilidades logarítmicas:

```
def predict(self, text: str) -> float:
    text_tokens = tokenize(text)
    log_prob_if_spam = log_prob_if_ham = 0.0
```

```
        # Itera por cada palabra de nuestro vocabulario
        for token in self.tokens:
            prob_if_spam, prob_if_ham = self._probabilities(token)
            # Si aparece *token* en el mensaje,
            # añade la probabilidad logarítmica de verlo
            if token in text_tokens:
                log_prob_if_spam += math.log(prob_if_spam)
                log_prob_if_ham += math.log(prob_if_ham)
            # En otro caso añade la probabilidad logarítmica de _no_ verlo,
            # que es log(1 – probabilidad de verlo)
            else:
                log_prob_if_spam += math.log(1.0–prob_if_spam)
                log_prob_if_ham += math.log(1.0–prob_if_ham)
        prob_if_spam = math.exp(log_prob_if_spam)
        prob_if_ham = math.exp(log_prob_if_ham)
        return prob_if_spam / (prob_if_spam + prob_if_ham)
```

Y ya tenemos un clasificador.

A probar nuestro modelo

Asegurémonos de que nuestro modelo funciona escribiendo para él unas unidades de prueba.

```
messages = [Message("spam rules", is_spam=True),
            Message("ham rules", is_spam=False),
            Message("hello ham", is_spam=False)]
model = NaiveBayesClassifier(k=0.5)
model.train(messages)
```

Primero, veamos si obtuvo correctamente los contadores:

```
assert model.tokens == {"spam", "ham", "rules", "hello"}
assert model.spam_messages == 1
assert model.ham_messages == 2
assert model.token_spam_counts == {"spam": 1, "rules": 1}
assert model.token_ham_counts == {"ham": 2, "rules": 1, "hello": 1}
```

Ahora hagamos una predicción. También revisaremos (laboriosamente) a mano nuestra lógica Naive Bayes, y nos aseguraremos de que obtenemos el mismo resultado:

```
text = "hello spam"
probs_if_spam = [
   (1 + 0.5) / (1 + 2 * 0.5), # "spam" (presente)
   1-(0 + 0.5) / (1 + 2 * 0.5), # "ham" (no presente)
   1-(1 + 0.5) / (1 + 2 * 0.5), # "rules" (no presente)
   (0 + 0.5) / (1 + 2 * 0.5) # "hello" (presente)
]
probs_if_ham = [
   (0 + 0.5) / (2 + 2 * 0.5), # "spam" (presente)
   1-(2 + 0.5) / (2 + 2 * 0.5), # "ham" (no presente)
   1-(1 + 0.5) / (2 + 2 * 0.5), # "rules" (no presente)
   (1 + 0.5) / (2 + 2 * 0.5), # "hello" (presente)
]
p_if_spam = math.exp(sum(math.log(p) for p in probs_if_spam))
p_if_ham = math.exp(sum(math.log(p) for p in probs_if_ham))
# Debería ser como 0,83
assert model.predict(text) == p_if_spam / (p_if_spam + p_if_ham)
```

Esta prueba funciona, de modo que parece que nuestro modelo está haciendo lo que pensamos que debe hacer. Si miramos las probabilidades reales, los dos grandes conductores son que nuestro mensaje contiene *spam* (cosa que hacía nuestro único mensaje *spam* de entrenamiento) y que no contiene *ham* (cosa que hacían nuestros dos mensajes *ham* de entrenamiento).

A continuación, probemos con datos de verdad.

Utilizar nuestro modelo

Un conjunto de datos conocido (aunque algo antiguo) es el recopilatorio público SpamAssassin.[1] Vamos a consultar los archivos con el prefijo 20021010.

Este es un fragmento de código que los descargará y descomprimirá en el directorio que elijamos (o bien puede hacerse manualmente):

```
from io import BytesIO      # Así podemos tratar bytes como archivo.
import requests             # Descargar los archivos, que
import tarfile              # están en formato .tar.bz.
BASE_URL = "https://spamassassin.apache.org/old/publiccorpus"
FILES = ["20021010_easy_ham.tar.bz2",
     "20021010_hard_ham.tar.bz2",
```

```
    "20021010_spam.tar.bz2"]
# Aquí terminarán los datos,
# en los subdirectorios /spam, /easy_ham y /hard_ham.
# Cambie esto a donde quiera los datos.
OUTPUT_DIR = 'spam_data'
for filename in FILES:
  # Usa requests para obtener el contenido del archivo en cada URL.
  content = requests.get(f"{BASE_URL}/{filename}").content
  # Envuelve los bytes en memoria para poder usarlos como "archivo".
  fin = BytesIO(content)
  # Y extrae todos los archivos al dir de salida especificado.
  with tarfile.open(fileobj=fin, mode='r:bz2') as tf:
    tf.extractall(OUTPUT_DIR)
```

Es posible que la ubicación de los archivos cambie (lo que ocurrió entre la primera y segunda edición de este libro), en cuyo caso ajuste el código adecuadamente.

Tras descargar los datos, debería tener tres carpetas: `spam`, `easy_ham` y `hard_ham`. Cada carpeta contiene muchos emails, cada uno de ellos contenido en un solo archivo. Para simplificar de verdad las cosas, solo veremos las líneas del asunto de cada email.

¿Cómo identificamos la línea del asunto? Cuando revisamos los archivos, todos parecen empezar por "Subject:" (asunto en inglés). Así que buscaremos eso:

```
import glob, re
# modifique la ruta a donde tenga los archivos
path = 'spam_data/*/*'
data: List[Message] = []
# glob.glob devuelve nombres de archivo que coinciden con la ruta comodín
for filename in glob.glob(path):
  is_spam = "ham" not in filename
  # Hay caracteres sobrantes en los emails; el errors='ignore'
  # los salta en lugar de mostrar una excepción.
  with open(filename, errors='ignore') as email_file:
    for line in email_file:
      if line.startswith("Subject:"):
        subject = line.lstrip("Subject: ")
        data.append(Message(subject, is_spam))
        break                # done with this file
```

Ahora podemos dividir los datos en datos de entrenamiento y datos de prueba, así estamos listos para crear un clasificador:

```
import random
from scratch.machine_learning import split_data
random.seed(0)      # justo así obtiene las mismas respuestas que yo
train_messages, test_messages = split_data(data, 0.75)
model = NaiveBayesClassifier()
model.train(train_messages)
```

Generemos algunas predicciones y veamos si funciona nuestro modelo:

```
from collections import Counter
predictions = [(message, model.predict(message.text))
           for message in test_messages]
# Supone que spam_probability > 0.5 corresponde a predicción de spam
# y cuenta las combinaciones de (actual is_spam, predicted is_spam)
confusion_matrix = Counter((message.is_spam, spam_probability > 0.5)
 for message, spam_probability in predictions)
print(confusion_matrix)
```

Esto proporciona 84 verdaderos positivos (*spam* clasificado como "*spam*"), 25 falsos positivos (han clasificado como "*spam*"), 703 verdaderos negativos (*ham* clasificado como "*ham*") y 44 falsos negativos (*spam* clasificado como "*ham*"), lo que significa que nuestra precisión es de 84 / (84 + 25) = 77 % y nuestro recuerdo de 84 / (84 + 44) = 65 %, que no son números malos para un modelo tan sencillo (supuestamente saldría mejor si tuviéramos en cuenta algo más que las líneas del asunto).

También podemos inspeccionar las entrañas del modelo para ver qué palabras son menos y más indicadoras de *spam*:

```
def p_spam_given_token(token: str, model: NaiveBayesClassifier) -> float:
   # Probablemente no habría que usar métodos privados, pero es por una buena
   causa.
   prob_if_spam, prob_if_ham = model._probabilities(token)
   return prob_if_spam / (prob_if_spam + prob_if_ham)
words = sorted(model.tokens, key=lambda t: p_spam_given_token(t, model))
print("spammiest_words", words[-10:])
print("hammiest_words", words[:10])
```

Entre las palabras más indicadoras de *spam* se encuentran *sale* (venta), *mortgage* (hipoteca), *money* (dinero) y *rates* (cuotas), mientras que algunas de las palabras más indicadoras de *ham* son *spambayes*, *users* (usuarios), *apt* y *perl*. Por lo tanto, esto nos da cierta confianza intuitiva en que nuestro modelo está haciendo básicamente lo que debe hacer.

¿Cómo podemos obtener un mejor rendimiento? Una forma obvia podría ser consiguiendo más datos para entrenar el modelo. También hay otras maneras diferentes de mejorar el modelo; estas son algunas posibilidades:

- Mirar el contenido del mensaje, no solo el asunto. Hay que tener cuidado con el manejo de los encabezados de los mensajes.
- Nuestro clasificador tiene en cuenta cada palabra que aparece en el conjunto de entrenamiento, incluso palabras que solo aparecen una vez. Se puede modificar para que acepte un umbral óptimo `min_count` e ignore *tokens* que no aparecen al menos esa cantidad de veces.
- El `tokenizer` no tiene la noción de palabras similares (por ejemplo, *cheap* y *cheapest*). Entonces se puede cambiar el clasificador para que admita una función opcional `stemmer`, que convierte palabras en clases de equivalencia de palabras. Por ejemplo, una función stemmer muy sencilla podría ser:

```
def drop_final_s(word):
  return re.sub("s$", "", word)
```

Crear una buena función stemmer es difícil. La gente suele utilizar el stemmer Porter.[2]

- Aunque nuestras funciones tengan la forma “el mensaje contiene la palabra w_i”, no hay razón por la que este tenga que ser el caso. En nuestra implementación, podríamos añadir funciones adicionales como “el mensaje contiene un número” creando *tokens* falsos como *contains:number* y modificando el `tokenizer` para emitirlos cuando corresponda.

Para saber más

- Los artículos de Paul Graham, “A Plan for Spam” en `http://www.paulgraham.com/spam.html` y “Better Bayesian Filtering” en (`http://www.paulgraham.com/better.html`, son interesantes y dan más información sobre las ideas que subyacen tras la creación de filtros de *spam*.
- scikit-learn, en `https://scikit-learn.org/stable/modules/naive_bayes.html`, contiene un modelo `BernoulliNB` que implementa el mismo algoritmo Naive Bayes que hemos implementado aquí, además de otras variaciones del modelo.

---

[1] `https://spamassassin.apache.org/old/publiccorpus/`.

[2] `http://tartarus.org/martin/PorterStemmer/`.

14 Regresión lineal simple

El arte, como la moral, consiste en trazar la línea en algún lugar.

—G. K. Chesterton

En el capítulo 5 utilizamos la función `correlation` para medir la fuerza de la relación lineal entre dos variables. En la mayoría de las aplicaciones, no basta con saber que existe una relación lineal como esta; queremos comprender la naturaleza de la relación. Aquí es donde empleamos la regresión lineal simple.

El modelo

Recordemos que estábamos investigando la relación entre el número de amigos de un usuario de DataSciencester y la cantidad de tiempo que el usuario se pasa en el sitio cada día. Supongamos que usted se ha convencido a sí mismo de que tener más amigos hace que la gente se pase más tiempo en el sitio, en lugar de una de las explicaciones alternativas que ya hemos tratado.

El vicepresidente de Compromiso le pide que cree un modelo que describa esta relación. Como ha descubierto ya una relación lineal bastante intensa, lo más natural es empezar por un modelo lineal.

En particular, podríamos proponer que haya dos constantes α (alfa) y β (beta) como por ejemplo:

$$y_i = \beta x_i + \alpha + \varepsilon_i$$

Donde *yi* es el número de minutos que el usuario *i* se pasa en el sitio diariamente, *xi* es el número de amigos que tiene el usuario *i*, y ε es un término de error (con suerte, pequeño) que representa el hecho de que hay

otros factores no tenidos en cuenta en este sencillo modelo.

Suponiendo que hemos determinado tales `alpha` y `beta`, hacemos predicciones fácilmente con:

```
def predict(alpha: float, beta: float, x_i: float) -> float:
   return beta * x_i + alpha
```

¿Cómo elegimos `alpha` y `beta`? Pues sea cual sea los que elijamos, nos dan un resultado previsto para cada entrada `x_i`. Como ya conocemos el resultado real `y_i`, podemos calcular el error para cada par:

```
def error(alpha: float, beta: float, x_i: float, y_i: float) -> float:
   """
   The error from predicting beta * x_i + alpha
   when the actual value is y_i
   """
   return predict(alpha, beta, x_i)–y_i
```

Lo que realmente nos gustaría saber es el error total sobre el conjunto de datos completo. Pero no queremos solamente sumar los errores (si la predicción para `x_1` es demasiado alta y para `x_2` es demasiado baja, los errores podrían anularse mutuamente).

Así que, en lugar de ello, sumamos los errores al cuadrado:

```
from scratch.linear_algebra import Vector
def sum_of_sqerrors(alpha: float, beta: float, x: Vector, y: Vector) -> float:
   return sum(error(alpha, beta, x_i, y_i) ** 2
           for x_i, y_i in zip(x, y))
```

La solución de mínimos cuadrados es elegir los `alpha` y `beta` que hagan que `sum_of_sqerrors` sea lo más pequeña posible.

Utilizando el cálculo (o la aburrida álgebra), los `alpha` y `beta` minimizadores de errores vienen dados por:

```
from typing import Tuple
from scratch.linear_algebra import Vector
from scratch.statistics import correlation, standard_deviation, mean
```

```
def least_squares_fit(x: Vector, y: Vector) -> Tuple[float, float]:
   """
   Given two vectors x and y,
   find the least-squares values of alpha and beta
   """
   beta = correlation(x, y) * standard_deviation(y) / standard_deviation(x)
   alpha = mean(y)–beta * mean(x)
   return alpha, beta
```

Sin revisar a fondo las matemáticas exactas, pensemos en la razón por la que esta podría ser una solución razonable. La elección de `alpha` dice simplemente que, cuando vemos el valor medio de la variable independiente `x`, predecimos el valor medio de la variable dependiente `y`.

La elección de `beta` significa que, cuando el valor de entrada aumenta en `standard_deviation(x)`, la predicción se incrementa entonces en `correlation(x, y) * standard_deviation(y)`. En el caso en que `x` e `y` estén perfectamente correlacionadas, un incremento de una sola desviación estándar en `x` da como resultado un aumento de una sola desviación estándar de `y` en la predicción. Cuando están perfectamente anticorrelacionadas, el aumento en `x` da como resultado una disminución en la predicción. Y, cuando la correlación es 0, `beta` es 0, lo que significa que los cambios en `x` no afectan en absoluto a la predicción.

Como es habitual, escribimos una rápida prueba para esto:

```
x = [i for i in range(-100, 110, 10)]
y = [3 * i–5 for i in x]
# Debería hallar que y = 3x–5
assert least_squares_fit(x, y) == (-5, 3)
```

Ahora es fácil aplicar esto a los datos sin *outliers* del capítulo 5:

```
from scratch.statistics import num_friends_good, daily_minutes_good
alpha, beta = least_squares_fit(num_friends_good, daily_minutes_good)
assert 22.9 < alpha < 23.0
assert 0.9 < beta < 0.905
```

Lo que nos da valores de `alpha` = 22,95 y `beta` = 0,903. Por tanto, nuestro

modelo dice que esperamos que un usuario con *n* amigos pase 22,95 + *n* * 0,903 minutos en el sitio cada día. Es decir, predecimos que un usuario sin amigos en DataSciencester aún se pasaría unos 23 minutos al día en el sitio. Por cada amigo adicional, esperamos que un usuario se pase casi un minuto más en el sitio cada día.

En la figura 14.1 trazamos la línea de predicción para hacernos una idea de lo bien que encaja el modelo en los datos observados.

Por supuesto, necesitamos una manera mejor de averiguar lo bien que hemos ajustado los datos que simplemente mirando el gráfico. Un método habitual es el coeficiente de determinación (o R^2, pronunciado R cuadrado), que mide la fracción de la variación total en la variable dependiente que es capturada por el modelo:

```
from scratch.statistics import de_mean
def total_sum_of_squares(y: Vector) -> float:
   """the total squared variation of y_i's from their mean"""
   return sum(v ** 2 for v in de_mean(y))
def r_squared(alpha: float, beta: float, x: Vector, y: Vector) -> float:
   """
   the fraction of variation in y captured by the model, which equals
   1-the fraction of variation in y not captured by the model
   """
   return 1.0-(sum_of_sqerrors(alpha, beta, x, y) /
           total_sum_of_squares(y))
rsq = r_squared(alpha, beta, num_friends_good, daily_minutes_good)
assert 0.328 < rsq < 0.330
```

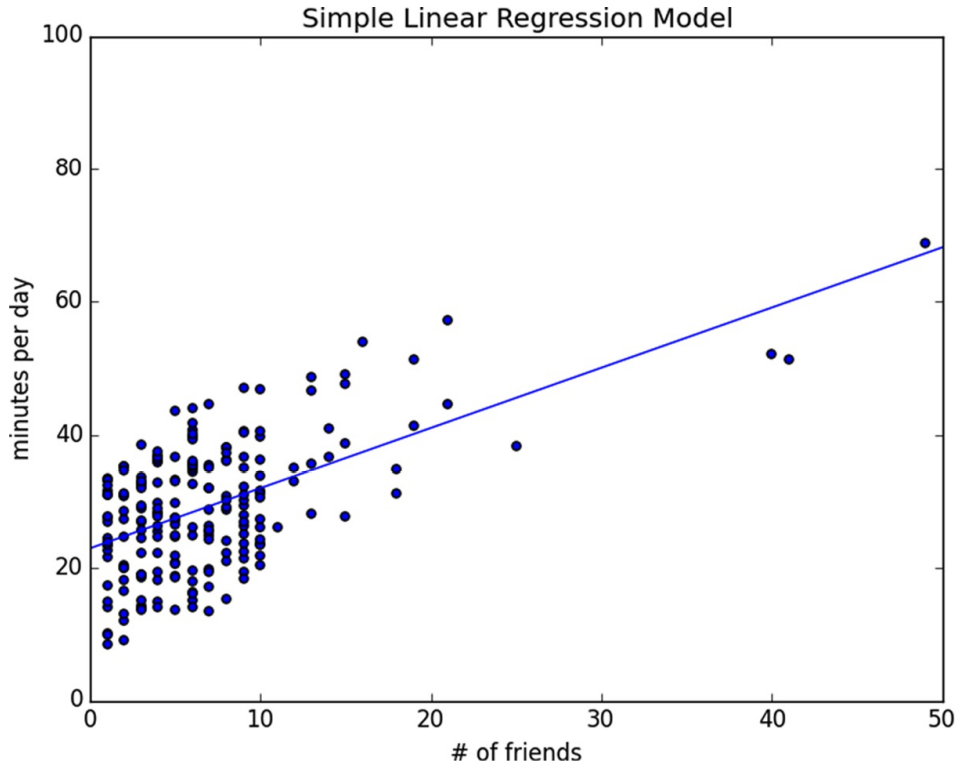


Figura 14.1. Nuestro sencillo modelo lineal.

Recordemos que elegimos los `alpha` y `beta` que minizaban la suma de los errores de predicción al cuadrado. Un modelo lineal que pudimos haber elegido es "predecir siempre `mean(y)`" (correspondiendo a `alpha` = `mean(y)` y `beta` = 0), cuya suma de errores al cuadrado es exactamente igual a su suma total de cuadrados. Esto significa un R cuadrado de 0, que indica un modelo que (obviamente, en este caso) no funciona mejor que solamente predecir la media.

Sin duda, el modelo de mínimos cuadrados debe ser al menos tan bueno como este, lo que significa que la suma de los errores al cuadrado es como mucho la suma total de cuadrados, es decir, que el R cuadrado debe ser al menos 0. Y la suma de errores al cuadrado debe ser al menos 0, lo que significa que el R cuadrado puede ser como mucho 1.

Cuánto más alto sea el número, mejor ajustará nuestro modelo los datos. Aquí calculamos un R cuadrado de 0,329, lo que nos dice que nuestro modelo solo se ajusta más o menos bien a los datos, y que sin duda hay otros factores en juego.

Utilizar descenso de gradiente

Si escribimos `theta = [alpha, beta]`, podemos también resolver esto utilizando descenso de gradiente:

```
import random
import tqdm
from scratch.gradient_descent import gradient_step
num_epochs = 10000
random.seed(0)
guess = [random.random(),                    # selecciona valor aleatorio para
random.random()]                             empezar
learning_rate = 0.00001
with tqdm.trange(num_epochs) as t:
   for _ in t:
      alpha, beta = guess
      # Derivada parcial de pérdida con respecto a alpha
      grad_a = sum(2 * error(alpha, beta, x_i, y_i)
                for x_i, y_i in zip(num_friends_good,
                    daily_minutes_good))
      # Derivada parcial de pérdida con respecto a beta
      grad_b = sum(2 * error(alpha, beta, x_i, y_i) * x_i
                for x_i, y_i in zip(num_friends_good,
                    daily_minutes_good))
      # Calcula pérdida para mantener la descripción tqdm
      loss = sum_of_sqerrors(alpha, beta,
num_friends_good, daily_minutes_good)
      t.set_description(f"loss: {loss:.3f}")
      # Por último, actualiza la conjetura
      guess = gradient_step(guess, [grad_a, grad_b], -learning_rate)
# Deberíamos obtener resultados similares:
alpha, beta = guess
assert 22.9 < alpha < 23.0
assert 0.9 < beta < 0.905
```

Si ejecutamos esto obtendremos los mismos valores para `alpha` y `beta` que obtuvimos utilizando la fórmula exacta.

Estimación por máxima verosimilitud

¿Por qué elegir mínimos cuadrados? Una justificación tiene que ver con la estimación por máxima verosimilitud. Supongamos que tenemos una muestra de datos *v*1, ..., *vn* procedente de una distribución que depende de un cierto parámetro desconocido θ (theta):

$$p(v_1, ..., v_n \mid \theta)$$

Si no conocemos θ, podríamos sentarnos a pensar en esta cantidad como en la verosimilitud de θ dada la muestra:

$$L(\theta \mid v_1, ..., v_n)$$

Según este enfoque, el parámetro θ más admisible es el valor que maximiza esta función de verosimilitud (es decir, el valor que hace que el dato observado sea el más probable). En el caso de una distribución continua, en la que tenemos una función de distribución de probabilidad en lugar de una función de masa de probabilidad, podemos hacer lo mismo.

Volvamos a la regresión. Una suposición que se hace a menudo sobre el modelo de regresión simple es que los errores de regresión se distribuyen normalmente con media 0 y una cierta desviación estándar (conocida) σ. Si es este el caso, entonces la verosimilitud basada en ver un par (`x_i`, `y_i`) es:

$$L(\alpha, \beta \mid x_i, y_i, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-(y_i - \alpha - \beta x_i)^2 / 2\sigma^2\right)$$

La verosimilitud basada en el conjunto de datos completo es el producto de las verosimilitudes individuales, que es mayor precisamente cuando `alpha` y `beta` se eligen para minimizar la suma de errores al cuadrado. Es decir, en este caso (con estas suposiciones), minimizar la suma de errores al cuadrado es equivalente a maximizar la verosimilitud de los datos observados.

Para saber más

Siga leyendo sobre la regresión múltiple en el capítulo 15.

15 Regresión múltiple

Yo no miro un problema y pongo en él variables que no le afectan.

—Bill Parcells

Aunque la vicepresidenta está bastante impresionada con su modelo predictivo, ella cree que puede hacerlo mejor. Con ese objetivo, se ha dedicado a recoger datos adicionales: sabe cuántas horas trabaja cada día cada uno de sus usuarios y si tienen un doctorado; naturalmente, quiere utilizar estos datos para mejorar su modelo.

Por consiguiente, se le ocurre proponer un modelo lineal con más variables independientes:

$$\text{minutos} = \alpha + \beta_1 \text{amigos} + \beta_2 \text{horas de trabajo} + \beta_3 \text{doctorado} + \varepsilon$$

Obviamente, el hecho de que un usuario tenga un doctorado no es un número, pero, como ya vimos en el capítulo 11, podemos introducir una variable ficticia que es igual a 1 para usuarios con doctorado y a 0 para los que no lo tienen, y así es igual de numérica que las demás variables.

El modelo

Recordemos que en el capítulo 14 ajustamos un modelo con esta forma:

$$y_i \; \alpha + \beta x_i \; \varepsilon_i$$

Ahora supongamos que cada entrada x_i no es solamente un número, sino más bien un vector de k números $x_{i1}, ..., x_{ik}$. El modelo de regresión múltiple supone que:

$$y_i = \alpha \; \beta_1 xi_1 \; \beta_k x_{ik} \; \varepsilon_i$$

En la regresión múltiple el vector de parámetros se suele denominar β. Queremos que incluya también el término constante, cosa que podemos conseguir añadiendo una columna de unos a nuestros datos:

```
beta = [alpha, beta_1, ..., beta_k]
```

Y:

```
x_i = [1, x_i1, ..., x_ik]
```

Entonces nuestro modelo queda así:

```
from scratch.linear_algebra import dot, Vector
def predict(x: Vector, beta: Vector) -> float:
   """assumes that the first element of x is 1"""
   return dot(x, beta)
```

En este caso en particular, nuestra variable independiente `x` será una lista de vectores, cada uno de los cuales tiene este aspecto:

```
[1,     # término constante
49,     # número de amigos
4,      # horas de trabajo al día
0]      # no tiene doctorado
```

Otros supuestos del modelo de mínimos cuadrados

Hay un par de supuestos adicionales necesarios para que este modelo (y nuestra solución) tenga sentido.

El primero es que las columnas de *x* son linealmente independientes (es decir, que no hay forma de escribir cualquiera de ellas como una suma ponderada de algunas de las otras). Si este supuesto falla, es imposible estimar `beta`. Para ver esto en una situación extrema, imaginemos que tuviéramos en nuestros datos un campo adicional `num_acquaintances` que, para cada usuario, fuera exactamente igual a `num_friends`.

Después, empezando con cualquier `beta`, si sumamos cualquier número al

coeficiente `num_friends` y restamos ese mismo número al coeficiente `num_acquaintances`, las predicciones del modelo no cambiarán, lo que significa que no hay forma de encontrar el coeficiente para `num_friends` (normalmente, los incumplimientos de este supuesto no serán tan obvios).

El segundo supuesto importante es que las columnas de *x* no están todas correlacionadas con los errores ε. Si este no es el caso, nuestras estimaciones de `beta` serán sistemáticamente erróneas. Por ejemplo, en el capítulo 14 creamos un modelo que predecía que cada amigo adicional estaba asociado a 0,90 minutos diarios extra en el sitio. Supongamos que es también el caso que:

- La gente que trabaja más horas se pasa menos tiempo en el sitio.
- La gente con más amigos tiende a trabajar más horas.

Es decir, imaginemos que el modelo "real" es:

$$\text{minutos} = \alpha + \beta_1 \text{amigos} + \beta_2 \text{horas de trabajo} + \varepsilon$$

Donde β_2 es negativo, y que las horas de trabajo y los amigos están positivamente correlacionados. En ese caso, cuando minimicemos los errores del modelo de única variable:

$$\text{minutos} = \alpha + \beta_1 \text{amigos} + \varepsilon$$

Subestimaremos β_1.

Pensemos en lo que ocurriría si hiciéramos predicciones utilizando el modelo de única variable con el valor "real" de $\beta 1$ (es decir, el valor que se obtiene de minimizar los errores de lo que llamamos el modelo "real"). Las predicciones tenderían a ser excesivamente grandes para usuarios que trabajan muchas horas y un poco demasiado grandes para los que trabajan pocas horas, porque $\beta 2 < 0$ y "olvidamos" incluirlo. El hecho de que las horas de trabajo estén positivamente correlacionadas con el número de amigos significa que las predicciones tienden a ser excesivamente grandes para los usuarios con muchos amigos, y solo ligeramente demasiado grandes para

usuarios con pocos amigos.

El resultado es que podemos reducir los errores (en el modelo de única variable) disminuyendo nuestra estimación de β_1, lo que significa que la β_1 minimizadora de errores es más pequeña que el valor "real". Es decir, en este caso la solución de mínimos cuadrados de única variable tiende a subestimar β_1. Además, en general, siempre que las variables independientes estén correlacionadas con errores como este, nuestra solución de mínimos cuadrados nos dará una estimación sesgada de β_1.

Ajustar el modelo

Como hicimos en el modelo lineal simple, elegiremos `beta` para minimizar la suma de errores cuadrados. Hallar una solución exacta manualmente no es sencillo, lo que significa que tendremos que utilizar descenso de gradiente. De nuevo, nos interesará minimizar la suma de los errores cuadrados. La función de error es casi idéntica a la que empleamos en el capítulo 14, salvo porque, en lugar de esperar parámetros `[alpha, beta]`, requerirá un vector de longitud arbitraria:

```
from typing import List
def error(x: Vector, y: float, beta: Vector) -> float:
   return predict(x, beta)–y
def squared_error(x: Vector, y: float, beta: Vector) -> float:
   return error(x, y, beta) ** 2
x = [1, 2, 3]
y = 30
beta = [4, 4, 4]       # así la predicción = 4 + 8 + 12 = 24
assert error(x, y, beta) == -6
assert squared_error(x, y, beta) == 36
```

Sabiendo cálculo, es fácil calcular el gradiente:

```
def sqerror_gradient(x: Vector, y: float, beta: Vector) -> Vector:
   err = error(x, y, beta)
   return [2 * err * x_i for x_i in x]
assert sqerror_gradient(x, y, beta) == [-12, -24, -36]
```

Si no, habrá que aceptar mi palabra.

En este momento, estamos preparados para encontrar la `beta` óptima utilizando descenso de gradiente. Escribamos primero una función `least_squares_fit` que pueda funcionar con cualquier conjunto de datos:

```
import random
import tqdm
from scratch.linear_algebra import vector_mean
from scratch.gradient_descent import gradient_step
def least_squares_fit(xs: List[Vector],
                      ys: List[float],
                      learning_rate: float = 0.001,
                      num_steps: int = 1000,
                      batch_size: int = 1) -> Vector:
  """
  Find the beta that minimizes the sum of squared errors
  assuming the model y = dot(x, beta).
  """
  # Empieza con una conjetura aleatoria
  guess = [random.random() for _ in xs[0]]
  for _ in tqdm.trange(num_steps, desc="least squares fit"):
     for start in range(0, len(xs), batch_size):
        batch_xs = xs[start:start+batch_size]
        batch_ys = ys[start:start+batch_size]
        gradient = vector_mean([sqerror_gradient(x, y, guess)
for x, y in zip(batch_xs, batch_ys)])
        guess = gradient_step(guess, gradient, -learning_rate)
  return guess
```

Entonces podemos aplicarla a nuestros datos:

```
from scratch.statistics import daily_minutes_good
from scratch.gradient_descent import gradient_step
random.seed(0)
# Usé prueba y error para elegir num_iters y step_size.
# Esto funcionará un rato.
learning_rate = 0.001
beta = least_squares_fit(inputs, daily_minutes_good, learning_rate, 5000, 25)
assert 30.50 < beta[0] < 30.70                # constante
assert 0.96 < beta[1] < 1.00                  # núm amigos
assert -1.89 < beta[2] < -1.85                # horas trabajo al día
assert 0.91 < beta[3] < 0.94                  # doctorado
```

En la práctica, no estimaríamos una regresión lineal utilizando descenso de gradiente; se obtendrían los coeficientes exactos empleando técnicas de álgebra lineal que están más allá del alcance de este libro. Si lo hiciéramos, daríamos con la ecuación:

minutos = 30 , 58 + 0 , 972 amigos – 1 , 87 horas de trabajo + 0 , 923 doctorados

Que se acerca bastante a lo que hemos descubierto.

Interpretar el modelo

Hay que pensar que los coeficientes del modelo representan estimaciones del tipo “siendo todo lo demás igual” de los impactos que tiene cada factor. Siendo todo lo demás igual, cada amigo adicional corresponde con un minuto extra pasado cada día en el sitio. Siendo todo lo demás igual, cada hora adicional del día de trabajo de un usuario corresponde con unos dos minutos menos pasados cada día en el sitio. Siendo todo lo demás igual, tener un doctorado se asocia a pasar un minuto más cada día en el sitio.

Lo que no nos dice (directamente) es nada sobre las interacciones entre las variables. Es posible que el efecto de las horas de trabajo sea diferente en gente con muchos amigos que en gente con pocos. Este modelo no captura eso. Una forma de manejar esta situación es introducir una nueva variable: el producto de “amigos” y “horas de trabajo”. Esto permite sin duda al coeficiente de “horas de trabajo” aumentar (o disminuir) a medida que el número de amigos se incrementa.

También es posible que cuantos más amigos se tengan, más tiempo se pase en el sitio hasta un cierto punto, tras el cual aún más amigos hace que se pase menos tiempo en el sitio (¿quizá con demasiados amigos la experiencia es demasiado abrumadora?). Podríamos intentar capturar esto en nuestro modelo añadiendo otra variable que es el cuadrado del número de amigos.

En cuanto empezamos a añadir variables, tenemos que preocuparnos de si sus coeficientes “importan”. No hay límites en las cantidades de productos,

logaritmos, cuadrados y potencias que podemos añadir.

Bondad de ajuste

De nuevo podemos echar un vistazo al R cuadrado:

```
from scratch.simple_linear_regression import total_sum_of_squares
def multiple_r_squared(xs: List[Vector], ys: Vector, beta: Vector) -> float:
   sum_of_squared_errors = sum(error(x, y, beta) ** 2
 for x, y in zip(xs, ys))
   return 1.0-sum_of_squared_errors / total_sum_of_squares(ys)
```

Que ha aumentado ahora a 0,68:

```
assert 0.67 < multiple_r_squared(inputs, daily_minutes_good, beta) < 0.68
```

Debemos recordar, sin embargo, que añadir nuevas variables a una regresión aumentará necesariamente el R cuadrado. Después de todo, el modelo de regresión simple no es más que el caso especial del modelo de regresión múltiple, en el que los coeficientes de "horas de trabajo" y "doctorado" son ambos iguales a 0. El modelo de regresión múltiple óptimo tendrá obligadamente un error al menos tan pequeño como ese.

Debido a esto, en una regresión múltiple también tenemos que mirar los errores estándares de los coeficientes, que miden lo seguros que estamos de nuestras estimaciones de cada β_1. La regresión, como un todo, puede ajustar muy bien nuestros datos, pero, si algunas de las variables independientes están correlacionadas (o son irrelevantes), sus coeficientes podrían no significar mucho.

El enfoque habitual para medir estos errores empieza con otro supuesto: que los errores ε_i son variables aleatorias normales e independientes con media 0 y una cierta desviación estándar (desconocida) σ. En ese caso, nosotros (o, lo más probable, nuestro software estadístico) podemos utilizar álgebra lineal para encontrar el error estándar de cada coeficiente. Cuanto más grande sea, menos seguro está nuestro modelo de ese coeficiente. Lamentablemente, no estamos preparados para este tipo de álgebra lineal

desde cero.

Digresión: el bootstrap

Supongamos que tenemos una muestra de *n* puntos de datos, generados por una cierta distribución (desconocida para nosotros):

```
data = get_sample(num_points=n)
```

En el capítulo 5, escribimos una función que podía calcular la `median` de la muestra, que podemos utilizar como estimación de la mediana de la propia distribución.

Pero ¿hasta qué punto podemos estar seguros de nuestra estimación? Si todos los puntos de datos de la muestra están muy cerca de 100, entonces parece probable que la mediana real esté cerca de 100. Si aproximadamente la mitad de los puntos de datos de la muestra está cerca de 0 y la otra mitad está cerca de 200, entonces no podemos estar tan seguros de la mediana.

Si pudiéramos obtener repetidamente nuevas muestras, podríamos calcular las medianas de todas esas muestras y mirar su distribución. Pero con frecuencia no es posible. En ese caso, sí se puede aplicar *bootstrap* a nuevos conjuntos de datos seleccionando *n* puntos de datos con reemplazo de nuestros datos. Después, podemos calcular las medianas de esos conjuntos de datos sintéticos:

```
from typing import TypeVar, Callable
X = TypeVar('X')               # Tipo genérico para datos
Stat = TypeVar('Stat')         # Tipo genérico para "estadística"
def bootstrap_sample(data: List[X]) -> List[X]:
   """randomly samples len(data) elements with replacement"""
   return [random.choice(data) for _ in data]
def bootstrap_statistic(data: List[X],
                  stats_fn: Callable[[List[X]], Stat],
                  num_samples: int) -> List[Stat]:
   """evaluates stats_fn on num_samples bootstrap samples from data"""
   return [stats_fn(bootstrap_sample(data)) for _ in range(num_samples)]
```

Por ejemplo, consideremos los dos conjuntos de datos siguientes:

```
# 101 puntos todos muy cerca de 100
close_to_100 = [99.5 + random.random() for _ in range(101)]
# 101 puntos, 50 de ellos cerca de 0, 50 de ellos cerca de 200
far_from_100 = ([99.5 + random.random()] +
           [random.random() for _ in range(50)] +
           [200 + random.random() for _ in range(50)])
```

Si calculamos las `median` de los dos conjuntos de datos, ambas estarán muy cerca de 100. Sin embargo, si miramos:

```
from scratch.statistics import median, standard_deviation
medians_close = bootstrap_statistic(close_to_100, median, 100)
```

Veremos en su mayoría números realmente cercanos a 100. Pero si lo que miramos es:

```
medians_far = bootstrap_statistic(far_from_100, median, 100)
```

Veremos muchos números cercanos a 0 y otros muchos cercanos a 200.

La `standard_deviation` del primer conjunto de medianas está cerca de 0, mientras que la del segundo conjunto de medianas está cerca de 100:

```
assert standard_deviation(medians_close) < 1
assert standard_deviation(medians_far) > 90
```

(Este caso extremo sería bastante fácil de averiguar manualmente inspeccionando los datos, pero en general eso no suele ocurrir).

Errores estándares de coeficientes de regresión

Podemos seguir el mismo sistema para estimar los errores estándares de nuestros coeficientes de regresión. Tomamos repetidamente una `bootstrap_sample` de nuestros datos y estimamos la `beta` basándonos en esa muestra. Si el coeficiente correspondiente a una de las variables independientes (digamos, `num_friends`) no varía mucho a lo largo de las muestras, entonces podemos estar seguros de que nuestra estimación es

relativamente ajustada. Si el coeficiente varía mucho a lo largo de las muestras, no podemos estar tan seguros entonces de nuestra estimación.

La única sutileza es que, antes de tomar las muestras, tendremos que empaquetar con `zip` nuestros datos `x` e `y` para asegurarnos de que los valores correspondientes de las variables independientes y dependientes estén juntos en la muestra. Esto significa que `bootstrap_sample` devolverá una lista de pares (`x_i`, `y_i`), que tendremos que volver a reformular como `x_sample` e `y_sample`:

```
from typing import Tuple
import datetime
def estimate_sample_beta(pairs: List[Tuple[Vector, float]]):
   x_sample = [x for x, _ in pairs]
   y_sample = [y for _, y in pairs]
   beta = least_squares_fit(x_sample, y_sample, learning_rate, 5000, 25)
   print("bootstrap sample", beta)
   return beta
random.seed(0)          # así obtiene los mismos datos que yo
# ¡Esto tardará un par de minutos!
bootstrap_betas = bootstrap_statistic(list(zip(inputs, daily_minutes_good)),
                   estimate_sample_beta,
                   100)
```

Después, podemos estimar la desviación estándar de cada coeficiente:

```
bootstrap_standard_errors = [
   standard_deviation([beta[i] for beta in bootstrap_betas])
   for i in range(4)]
print(bootstrap_standard_errors)
# [1.272,   # término constante,     error real = 1.19
# 0.103,    # num_friends,           error real = 0.080
# 0.155,    # work_hours,            error real = 0.127
# 1.249]    # doctorado,             error real = 0.998
```

(Obtendríamos mejores estimaciones si recogiéramos más de 100 muestras y utilizáramos más de 5.000 iteraciones para estimar cada `beta`, pero no tenemos todo el día).

Podemos utilizar estas estimaciones para probar hipótesis como "¿es β_i igual a 0?". Bajo la hipótesis nula $\beta_i = 0$ (y con nuestros otros supuestos

sobre la distribución de ε_i), la estadística:

$$t_j = \widehat{\beta_j}/\widehat{\sigma_j}$$

Que es nuestra estimación de β_j dividida por nuestra estimación de su error estándar, sigue a una distribución *t* de Student con "*n* – *k* grados de libertad".

Si tuviéramos una función `students_t_cdf`, podríamos calcular valores *p* para cada coeficiente de mínimos cuadrados, que indicara la probabilidad de que observáramos un valor así si el coeficiente real fuera 0. Lamentablemente, no tenemos una función como esta (aunque si la tuviéramos no estaríamos trabajando desde cero).

No obstante, a medida que los grados de libertad son mayores, la distribución *t* se acerca cada vez más a una normal estándar. En una situación como esta, donde *n* es mucho más grande que *k*, podemos utilizar `normal_cdf` y seguir sintiéndonos bien con nosotros mismos:

```
from scratch.probability import normal_cdf
def p_value(beta_hat_j: float, sigma_hat_j: float) -> float:
   if beta_hat_j > 0:
      # si el coeficiente es positivo, tenemos que calcular el doble
      # de la probabilidad de ver un valor aún *mayor*
      return 2 * (1–normal_cdf(beta_hat_j / sigma_hat_j))
   else:
      # si no el doble de la probabilidad de ver un valor *menor*
      return 2 * normal_cdf(beta_hat_j / sigma_hat_j)
assert p_value(30.58, 1.27) < 0.001           # término constante
assert p_value(0.972, 0.103) < 0.001          # num_friends
assert p_value(-1.865, 0.155) < 0.001         # work_hours
assert p_value(0.923, 1.249) > 0.4            # doctorado
```

En una situación diferente a esta, probablemente utilizaríamos software estadístico, que sabe cómo calcular la distribución *t*, además de cómo calcular los errores estándares exactos.

Aunque la mayoría de los coeficientes tienen valores *p* muy pequeños (lo que sugiere que de hecho no son cero), el coeficiente de "doctorado" no es "apreciablemente" distinto de 0, lo que hace probable que el coeficiente de

“doctorado” sea aleatorio en lugar de significativo.

En situaciones de regresión más complicadas, se suele querer probar hipótesis más complejas sobre los datos, como “al menos una de las β_j no es cero” o “β_1 es igual a β_2 y β_3 es igual a β_4”. Esto puede hacerse con una prueba F, pero, por desgracia, eso queda fuera del alcance de este libro.

Regularización

En la práctica, a menudo se suele aplicar regresión lineal a conjuntos de datos con grandes cantidades de variables. Pero esto crea otro par de problemas. Primero, cuantas más variables se empleen, más probable es que se sobreajuste el modelo al conjunto de entrenamiento. Y segundo, cuantos más coeficientes no cero se tengan, más difícil es darles sentido. Si el objetivo es explicar algún fenómeno, un modelo disperso con tres factores podría ser más útil que un modelo ligeramente mejor con cientos.

La regularización es un método en el que añadimos al término de error una penalización, que es mayor a medida que `beta` aumenta. Entonces minimizamos el error y la penalización combinados. Cuanta más importancia le demos al término de penalización, más desalentaremos a los coeficientes grandes.

Por ejemplo, en la regresión *ridge*, añadimos una penalización proporcional a la suma de cuadrados de la `beta_i` (excepto que normalmente no penalizamos `beta_0`, el término constante):

```
# alpha es un *hiperparámetro* que controla lo dura que es la penalización.
# A veces se le llama "lambda" pero eso ya significa algo en Python.
def ridge_penalty(beta: Vector, alpha: float) -> float:
   return alpha * dot(beta[1:], beta[1:])
def squared_error_ridge(x: Vector,
                        y: float,
                        beta: Vector,
                        alpha: float) -> float:
   """estimate error plus ridge penalty on beta"""
   return error(x, y, beta) ** 2 + ridge_penalty(beta, alpha)
```

Podemos después conectar esto con el descenso de gradiente de la manera habitual:

```
from scratch.linear_algebra import add
def ridge_penalty_gradient(beta: Vector, alpha: float) -> Vector:
   """gradient of just the ridge penalty"""
   return [0.] + [2 * alpha * beta_j for beta_j in beta[1:]]
def sqerror_ridge_gradient(x: Vector,
y: float,
beta: Vector,
alpha: float) -> Vector:
   """
   the gradient corresponding to the ith squared error term
   including the ridge penalty
   """
   return add(sqerror_gradient(x, y, beta),
          ridge_penalty_gradient(beta, alpha))
```

Y después tendremos que modificar la función `least_squares_fit` para usar `sqerror_ridge_gradient` en lugar de `sqerror_gradient` (no voy a repetir aquí el código).

Con `alpha` establecido en 0, no hay penalización ninguna y obtenemos los mismos resultados que antes:

```
random.seed(0)
beta_0 = least_squares_fit_ridge(inputs,                    # alpha
daily_minutes_good, 0.0,
               learning_rate, 5000, 25)
# [30.51, 0.97, -1.85, 0.91]
assert 5 < dot(beta_0[1:], beta_0[1:]) < 6
assert 0.67 < multiple_r_squared(inputs, daily_minutes_good, beta_0) < 0.69
```

A medida que aumentamos `alpha`, la bondad de ajuste empeora, pero el tamaño de `beta` se reduce:

```
beta_0_1 = least_squares_fit_ridge(inputs,                  # alpha
daily_minutes_good, 0.1,
               learning_rate, 5000, 25)
# [30.8, 0.95, -1.83, 0.54]
assert 4 < dot(beta_0_1[1:], beta_0_1[1:]) < 5
assert 0.67 < multiple_r_squared(inputs, daily_minutes_good, beta_0_1) < 0.69
```

```
beta_1 = least_squares_fit_ridge(inputs,                    # alpha
daily_minutes_good, 1,
               learning_rate, 5000, 25)
# [30.6, 0.90, -1.68, 0.10]
assert 3 < dot(beta_1[1:], beta_1[1:]) < 4
assert 0.67 < multiple_r_squared(inputs, daily_minutes_good, beta_1) < 0.69
beta_10 = least_squares_fit_ridge(inputs,                   # alpha
daily_minutes_good,10,
               learning_rate, 5000, 25)
# [28.3, 0.67, -0.90, -0.01]
assert 1 < dot(beta_10[1:], beta_10[1:]) < 2
assert 0.5 < multiple_r_squared(inputs, daily_minutes_good, beta_10) < 0.6
```

En particular, el coeficiente de “doctorado” desaparece a medida que aumentamos la penalización, lo que está de acuerdo con nuestro anterior resultado, que no era apreciablemente distinto de 0.

Nota: Normalmente conviene redimensionar los datos con `rescale` antes de utilizar este método. Después de todo, si cambiáramos años de experiencia por siglos de experiencia, su coeficiente de mínimos cuadrados aumentaría en un factor de 100 y, de repente, resultaría mucho más penalizado, incluso aunque fuera el mismo modelo.

Otro método es la regresión *lasso*, que utiliza la penalización:

```
def lasso_penalty(beta, alpha):
return alpha * sum(abs(beta_i) for beta_i in beta[1:])
```

Mientras que la penalización *ridge* encogía en general los coeficientes, la penalización *lasso* tiende a obligar a los coeficientes a ser 0, por lo que nos sirve para aprender modelos dispersos. Lamentablemente, no admite el descenso de gradiente, con lo cual no podremos resolverlo desde cero.

Para saber más

- La regresión tiene una teoría abundante y extensa, por lo que es otro tema sobre el que podría considerar leer un libro de texto, o al menos unos cuantos artículos de la Wikipedia.

- scikit-learn tiene un módulo `linear_model`, en https://scikit-learn.org/stable/modules/linear_model.html, que ofrece un modelo `LinearRegression` similar al nuestro, además de regresión *ridge*, *lasso* y otros tipos de regularización.
- StatsModels, en https://statsmodels.org, es otro módulo de Python que contiene (entre otras cosas) modelos de regresión lineal.

16 Regresión logística

Mucha gente dice que hay una línea muy fina entre el genio y la locura. No creo que haya una línea fina, lo que realmente creo que hay es un abismo.

—Bill Bailey

En el capítulo 1 hemos visto brevemente el problema de tratar de predecir qué usuarios de DataSciencester pagaron por cuentas *premium*. En este capítulo le echaremos otro vistazo a este problema.

El problema

Tenemos un conjunto de datos anónimo de unos 200 usuarios, que contiene el salario de cada usuario, sus años de experiencia como científico de datos y si pagó por una cuenta *premium* (figura 16.1). Como es habitual con las variables categóricas, representamos la variable dependiente como 0 (sin cuenta *premium*) o 1 (con cuenta *premium*).

Como siempre, nuestros datos son una lista de filas `[experience, salary, paid_account]`. Convirtámosla al formato que necesitamos:

```
xs = [[1.0] + row[:2] for row in data]      # [1, experience, salary]
ys = [row[2] for row in data]               # paid_account
```

Un primer intento obvio es utilizar regresión lineal y hallar el mejor modelo:

$$\text{cuenta de pago} = \beta_0 + \beta_1 \text{experiencia} + \beta_2 \text{salario} + \varepsilon$$

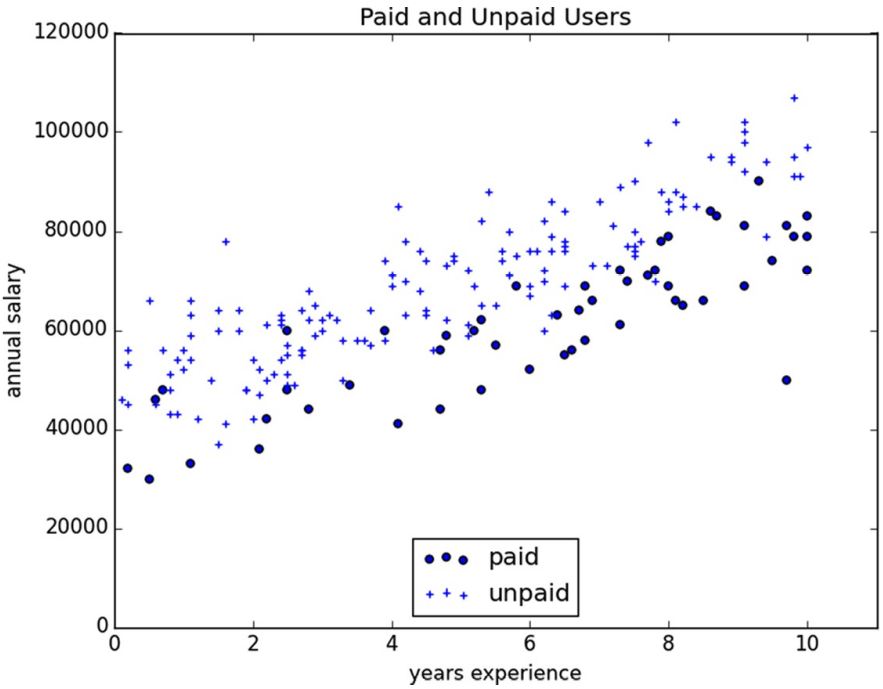


Figura 16.1. Usuarios de pago y no de pago.

Sin duda, no hay nada que nos impida crear así un modelo similar del problema. Los resultados se muestran en la figura 16.2:

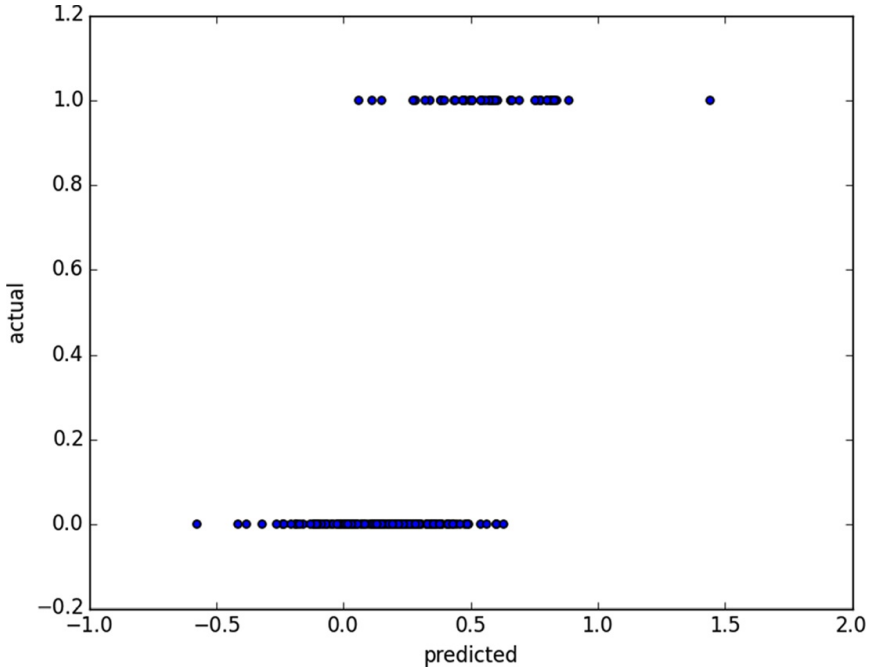


Figura 16.2. Utilizar la regresión lineal para predecir las cuentas de pago.

```
from matplotlib import pyplot as plt
from scratch.working_with_data import rescale
from scratch.multiple_regression import least_squares_fit, predict
from scratch.gradient_descent import gradient_step
learning_rate = 0.001
rescaled_xs = rescale(xs)
beta = least_squares_fit(rescaled_xs, ys, learning_rate, 1000, 1)
# [0.26, 0.43, -0.43]
predictions = [predict(x_i, beta) for x_i in rescaled_xs]
plt.scatter(predictions, ys)
plt.xlabel("predicted")
plt.ylabel("actual")
plt.show()
```

Pero este enfoque nos conduce a un par de problemas inmediatos:

- Nos gustaría que nuestras salidas previstas fueran 0 o 1, para indicar la membresía de clase. Está bien si están entre 0 y 1, ya que podemos interpretarlas como probabilidades (un resultado de 0,25 podría significar un 25 % de posibilidades de ser un miembro de pago). Pero los resultados del modelo lineal pueden ser grandes números positivos o incluso negativos, cuya interpretación no queda clara. De hecho, aquí muchas de nuestras predicciones fueron negativas.
- El modelo de regresión lineal asumía que los errores no estaban correlacionados con las columnas de *x*. Pero, en este caso, el coeficiente de regresión para `experience` es 0,43, indicando que más experiencia da lugar a una mayor probabilidad de una cuenta de pago. Esto significa que nuestro modelo da como resultado valores muy grandes para gente con mucha experiencia. Pero sabemos que los valores reales deben ser como máximo 1, lo que significa que resultados necesariamente muy grandes (y por lo tanto valores muy altos de `experience`) corresponden a valores negativos muy altos del término de error. Como es este el caso, nuestra estimación de `beta` está sesgada.

Lo que realmente nos gustaría es que valores grandes positivos de `dot(x_i, beta)` correspondieran a probabilidades cercanas a 1, y que valores grandes negativos correspondieran a probabilidades cercanas a 0. Podemos

conseguir esto aplicando otra función al resultado.

La función logística

En el caso de la regresión logística, utilizamos la función del mismo nombre, representada en la figura 16.3:

```
def logistic(x: float) -> float:
   return 1.0 / (1 + math.exp(-x))
```

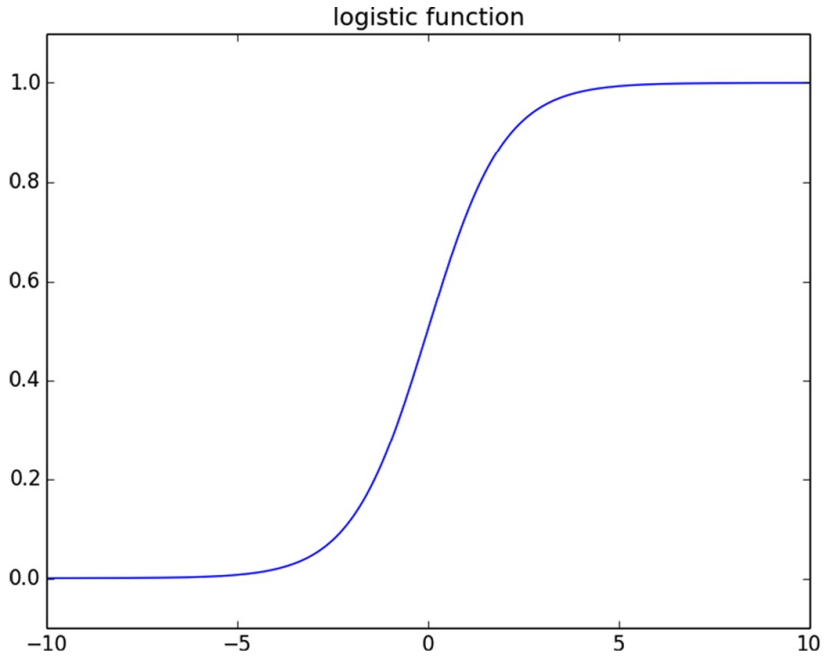


Figura 16.3. La función logística.

A medida que su entrada se hace más grande y positiva, se acerca cada vez más a 1; a medida que se hace más grande y negativa, se va acercando más a 0. Además, tiene la oportuna propiedad de que su derivada viene dada por:

```
def logistic_prime(x: float) -> float:
   y = logistic(x)
   return y * (1 – y)
```

Que utilizaremos en un momento, para ajustar un modelo:

$y_i = f(xi\beta) + \varepsilon_i$

Donde f es la función `logistic`.

Recordemos que para la regresión lineal ajustábamos el modelo minimizando la suma de errores cuadrados, lo que terminaba eligiendo la β que maximizaba la verosimilitud de los datos.

Aquí los dos no son equivalentes, de modo que utilizaremos descenso de gradiente para maximizar la verosimilitud directamente; esto significa que necesitamos calcular la función de verosimilitud y su gradiente.

Dada una cierta β, nuestro modelo dice que cada *yi* debería ser igual a 1 con probabilidad $f(x_i\beta)$ y a 0 con probabilidad 1 - $f(x_i\beta)$.

En particular, la función PDF (de densidad de probabilidad) para *yi* se puede escribir como:

$$p(y_i | x_i, \beta) = f(x_i\beta)^{y_i}(1 - f(x_i\beta))^{1 - y_i}$$

Ya que, si y_i es 0, es igual a:

1 - $f(1)$

Y, si y_i es 1, es igual a:

$f(x_i\beta)$

Resulta que es realmente más sencillo maximizar la log-verosimilitud:

$\log L(\beta \mid x_i, y_i) = y_i \log f(x_i\beta) + (1 - yi) \log (1 - f(x_i\beta))$

Como el logaritmo es una función estrictamente incremental, cualquier `beta` que maximice la log-verosimilitud también hace lo mismo con la verosimilitud, y viceversa. Como el descenso de gradiente minimiza las cosas, realmente trabajaremos con la log-verosimilitud negativa, ya que

maximizar la verosimilitud es lo mismo que minimizar su negativa:

```
import math
from scratch.linear_algebra import Vector, dot

def _negative_log_likelihood(x: Vector, y: float, beta: Vector) -> float:
  """The negative log likelihood for one data point"""
  if y == 1:
    return -math.log(logistic(dot(x, beta)))
  else:
    return -math.log(1–logistic(dot(x, beta)))
```

Si suponemos que los distintos puntos de datos son independientes uno de otro, la verosimilitud total no es más que el producto de las verosimilitudes individuales; lo que significa que la log-verosimilitud total es la suma de las log-verosimilitudes individuales:

```
from typing import List
def negative_log_likelihood(xs: List[Vector],
ys: List[float],
beta: Vector) -> float:
  return sum(_negative_log_likelihood(x, y, beta)
         for x, y in zip(xs, ys))
```

Un poco de cálculo nos da el gradiente:

```
from scratch.linear_algebra import vector_sum
def _negative_log_partial_j(x: Vector, y: float, beta: Vector, j: int) ->
float:
  """
  The jth partial derivative for one data point.
  Here i is the index of the data point.
  """
  return -(y–logistic(dot(x, beta))) * x[j]
def _negative_log_gradient(x: Vector, y: float, beta: Vector) -> Vector:
  """
  The gradient for one data point.
```

```
    """
    return [_negative_log_partial_j(x, y, beta, j)
            for j in range(len(beta))]
def negative_log_gradient(xs: List[Vector],
                          ys: List[float],
                          beta: Vector) -> Vector:
    return vector_sum([_negative_log_gradient(x, y, beta)
                       for x, y in zip(xs, ys)])
```

Momento en el cual tenemos todas las piezas que necesitamos.

Aplicar el modelo

Nos interesará dividir nuestros datos en un conjunto de entrenamiento y uno de prueba:

```
from scratch.machine_learning import train_test_split
import random
import tqdm
random.seed(0)
x_train, x_test, y_train, y_test = train_test_split(rescaled_xs, ys, 0.33)
learning_rate = 0.01
# elige un punto de partida aleatorio
beta = [random.random() for _ in range(3)]
with tqdm.trange(5000) as t:
    for epoch in t:
        gradient = negative_log_gradient(x_train, y_train, beta)
        beta = gradient_step(beta, gradient, -learning_rate)
        loss = negative_log_likelihood(x_train, y_train, beta)
        t.set_description(f"loss: {loss:.3f} beta: {beta}")
```

Y después descubrimos que `beta` es aproximadamente:

```
[-2.0, 4.7, -4.5]
```

Estos son coeficientes para los datos redimensionados con `rescale`, pero también podemos transformarlos de nuevo en los datos originales:

```
from scratch.working_with_data import scale
means, stdevs = scale(xs)
```

```
beta_unscaled = [(beta[0]
             -beta[1] * means[1] / stdevs[1]
             -beta[2] * means[2] / stdevs[2]),
             beta[1] / stdevs[1],
             beta[2] / stdevs[2]]
# [8.9, 1.6, -0.000288]
```

Lamentablemente, estos no son tan fáciles de interpretar como los coeficientes de regresión lineal. Siendo todo lo demás igual, un año de experiencia adicional suma 1,6 a la entrada de `logistic`. Siendo todo lo demás igual, 10.000 dólares extra de salario restan 2,88 a la entrada de `logistic`.

Pero el impacto del resultado depende también de las otras entradas. Si `dot(beta, x_i)` ya es grande (correspondiendo a una probabilidad cercana a 1), aumentarlo aun en una elevada cantidad no puede afectar mucho a la probabilidad. Si está cerca de 0, incrementarlo solo un poquito podría aumentar bastante la probabilidad.

Lo que podemos decir es que (siendo todo lo demás igual) es más probable que la gente con más experiencia pague por las cuentas. Y que (siendo todo lo demás igual) es menos probable que la gente con salarios más altos pague por las cuentas (esto también se hizo evidente al trazar los datos).

Bondad de ajuste

No hemos utilizado aún los datos de prueba que nos quedaban. Veamos lo que ocurre si predecimos cuenta de pago siempre que la probabilidad supere 0,5:

```
true_positives = false_positives = true_negatives = false_negatives = 0
for x_i, y_i in zip(x_test, y_test):
   prediction = logistic(dot(beta, x_i))
   if y_i == 1 and prediction >=          # TP: de pago y predecimos de pago
   0.5:
      true_positives += 1
   elif y_i == 1:                         # FN: de pago y predecimos no de pago
      false_negatives += 1
   elif prediction >= 0.5:                # FP: no de pago y predecimos de pago
```

```
    false_positives += 1
  else:                                # TN: no de pago y predecimos no de
                                       pago
    true_negatives += 1
precision = true_positives / (true_positives + false_positives)
recall = true_positives / (true_positives + false_negatives)
```

Esto ofrece una precisión del 75 % (“cuando predecimos cuenta de pago acertamos el 75 % de las veces”) y un recuerdo del 80 % (“cuando un usuario tiene una cuenta de pago predecimos cuenta de pago el 80 % de las veces”), lo que no está nada mal, teniendo en cuenta los pocos datos de que disponemos.

También podemos representar las predicciones frente a los datos reales (figura 16.4), lo que también demuestra que el modelo funciona bien:

```
predictions = [logistic(dot(beta, x_i)) for x_i in x_test]
plt.scatter(predictions, y_test, marker='+')
plt.xlabel("predicted probability")
plt.ylabel("actual outcome")
plt.title("Logistic Regression Predicted vs. Actual")
plt.show()
```

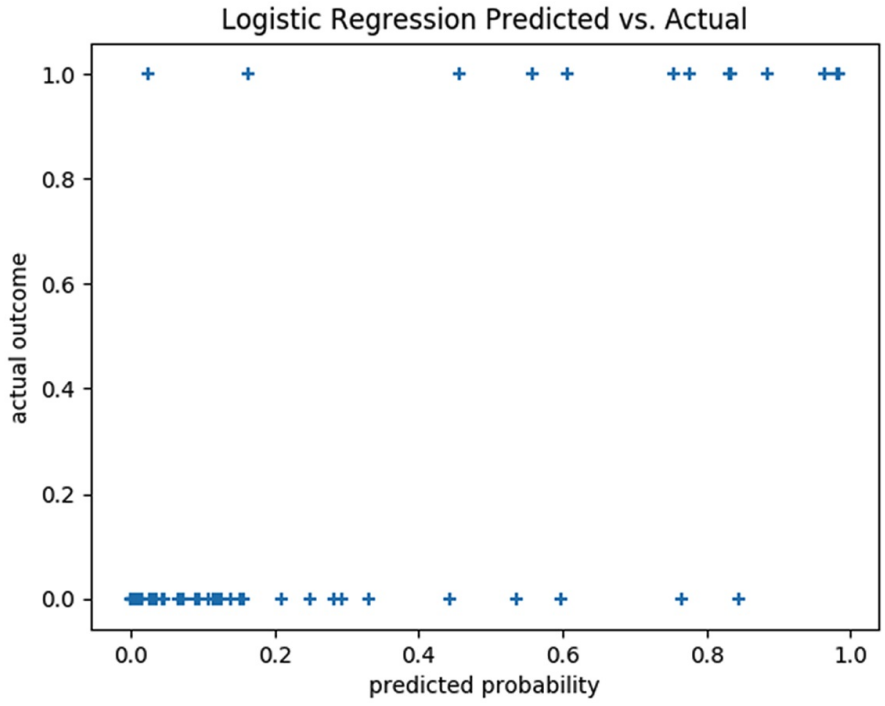


Figura 16.4. Regresión logística predicha frente a real.

Máquinas de vectores de soporte

El conjunto de puntos donde `dot(beta, x_i)` es igual a 0 es la frontera entre nuestras clases. Podemos representar esto para ver exactamente lo que está haciendo nuestro modelo (figura 16.5).

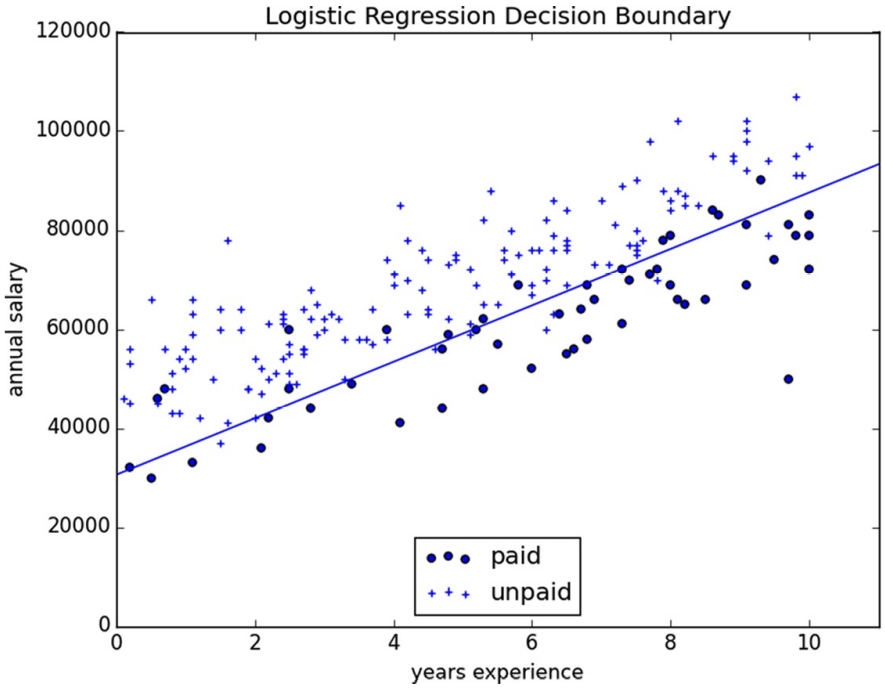


Figura 16.5. Usuarios con cuentas de pago y no de pago con frontera de decisión.

Esta frontera es un hiperplano, que divide el espacio de parámetros en dos mitades correspondientes a predice de pago y predice no de pago. Lo descubrimos como un efecto colateral de hallar el modelo logístico más probable.

Un método alternativo a la clasificación es simplemente buscar el hiperplano que "mejor" separe las clases en los datos de entrenamiento. Esta es la idea de la máquina de vectores de soporte, que localiza el hiperplano que maximiza la distancia al punto más cercano en cada clase (figura 16.6).

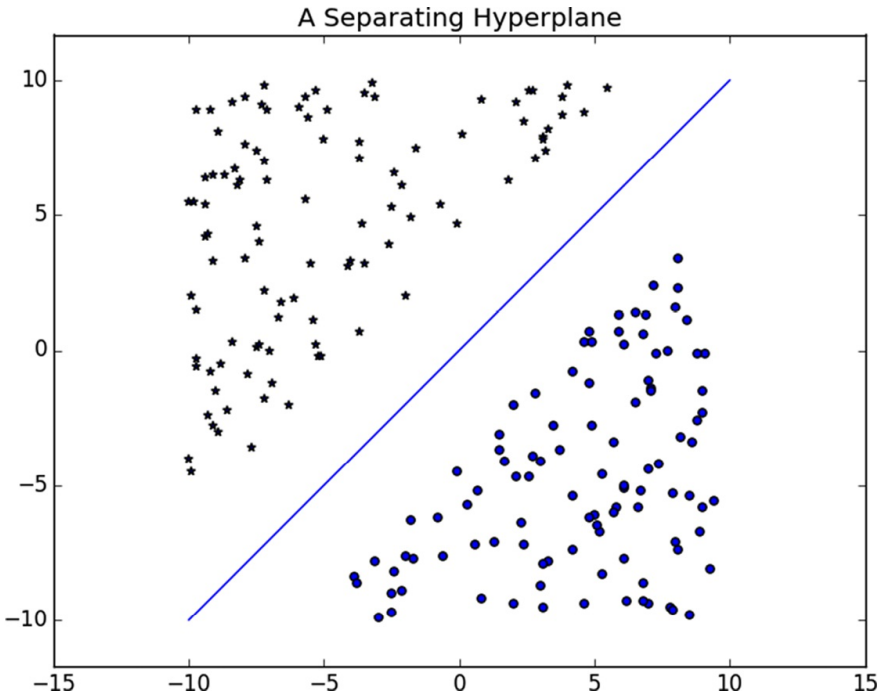


Figura 16.6. Un hiperplano de separación.

Encontrar un hiperplano como este es un problema de optimización que implica técnicas demasiado avanzadas para nosotros. Un problema distinto es que un hiperplano de separación podría no existir. En nuestro conjunto de datos "¿quién paga?", simplemente es que no hay línea que separe perfectamente los usuarios que pagan de los que no pagan.

En ocasiones, podemos sortear esto transformando los datos en un espacio de muchas dimensiones. Por ejemplo, veamos el sencillo conjunto de datos unidimensional mostrado en la figura 16.7.

Sin duda, no hay hiperplano que separe los ejemplos positivos de los negativos. Sin embargo, veamos lo que ocurre cuando mapeamos este conjunto de datos en dos dimensiones enviando el punto `x` a `(x, x**2)`. De repente es posible encontrar un hiperplano que divida los datos (figura 16.8).

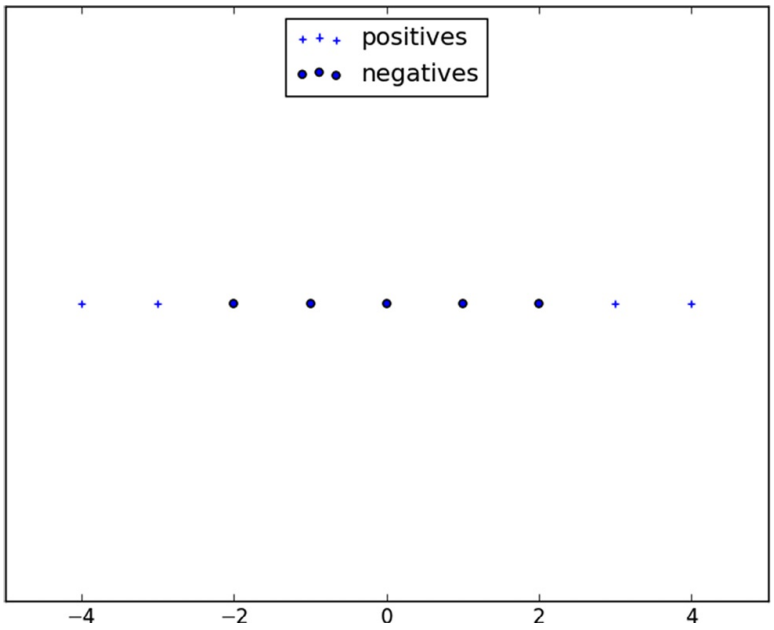


Figura 16.7. Un conjunto de datos unidimensional no separable.

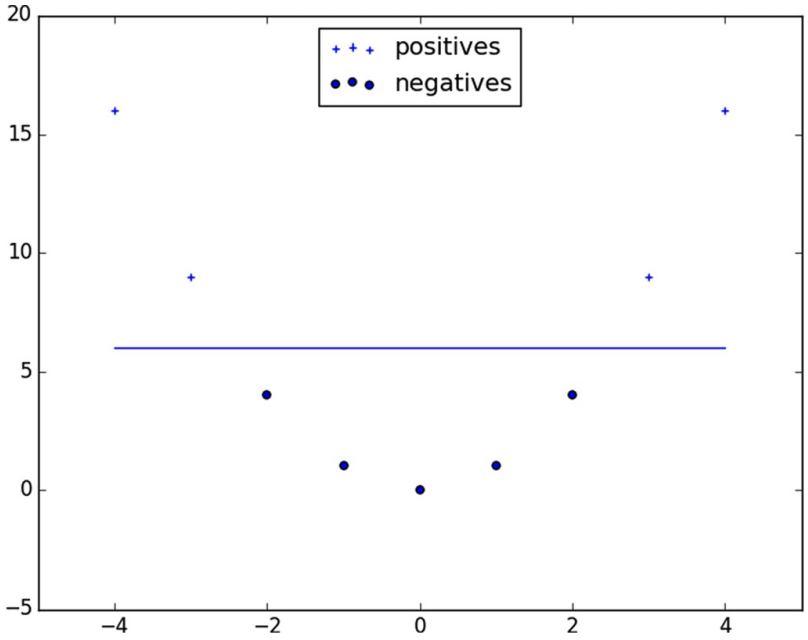


Figura 16.8. El conjunto de datos se vuelve separable con muchas dimensiones.

Esto se denomina el truco del *kernel*, porque, en lugar de mapear realmente los puntos en el espacio de muchas dimensiones (lo que podría resultar caro si hay muchos puntos y el mapeado es complicado), podemos utilizar una función "*kernel*" para calcular productos de punto en el espacio

de muchas dimensiones y utilizarlos para encontrar un hiperplano.

Es difícil (y probablemente en absoluto una buena idea) utilizar máquinas de vectores de soporte sin confiar en software de optimización especializado escrito por gente con la experiencia adecuada, de modo que tendremos que dejar aquí nuestro planteamiento.

Para saber más

- scikit-learn tiene módulos de regresión logística, en https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression, y de máquinas de vectores de soporte, en https://scikit-learn.org/stable/modules/svm.html.
- LIBSVM, en https://www.csie.ntu.edu.tw/~cjlin/libsvm/, es la implementación de máquina de soporte vectorial que realmente utiliza scikit-learn. Su sitio web tiene mucha documentación variada sobre este algoritmo.

17 Árboles de decisión

Un árbol es un misterio incomprensible.

—Jim Woodring

El vicepresidente de Talento de DataSciencester ha entrevistado a varios candidatos del sitio para un puesto de trabajo, con diversos grados de éxito. Ha recogido un conjunto de datos, que consiste en varios atributos (cualitativos) de cada candidato, además de si la entrevista con cada uno fue bien o mal. Así que plantea la siguiente pregunta: ¿se podrían utilizar estos datos para crear un modelo que identifique los candidatos que harán una buena entrevista, de forma que no tenga que perder el tiempo en esta tarea?

Parece que en esto encaja bien un árbol de decisión, otra herramienta de creación de modelos predictivos que forma parte del equipo del científico de datos.

¿Qué es un árbol de decisión?

Un árbol de decisión emplea una estructura en árbol para representar una serie de posibles rutas de ramificación y un resultado para cada ruta.

Si alguna vez ha jugado al juego de las 20 preguntas,[1] estará familiarizado con los árboles de decisión. Por ejemplo:

- "Estoy pensando en un animal".
- "¿Tiene más de cinco patas?".
- "No".
- "¿Es delicioso?".
- "No".
- "¿Aparece en el reverso de la moneda de cinco centavos australiana?".
- "Sí".

- “¿Es un equidna?”.
- “¡Sí, correcto!”.

Esta batería de preguntas corresponde a la siguiente ruta, que sería la de un árbol de decisión “adivine el animal” bastante singular (y no muy amplio) (figura 17.1):

“No más de 5 patas” → “No delicioso” → “En la moneda de 5 centavos” → “¡Equidna!”

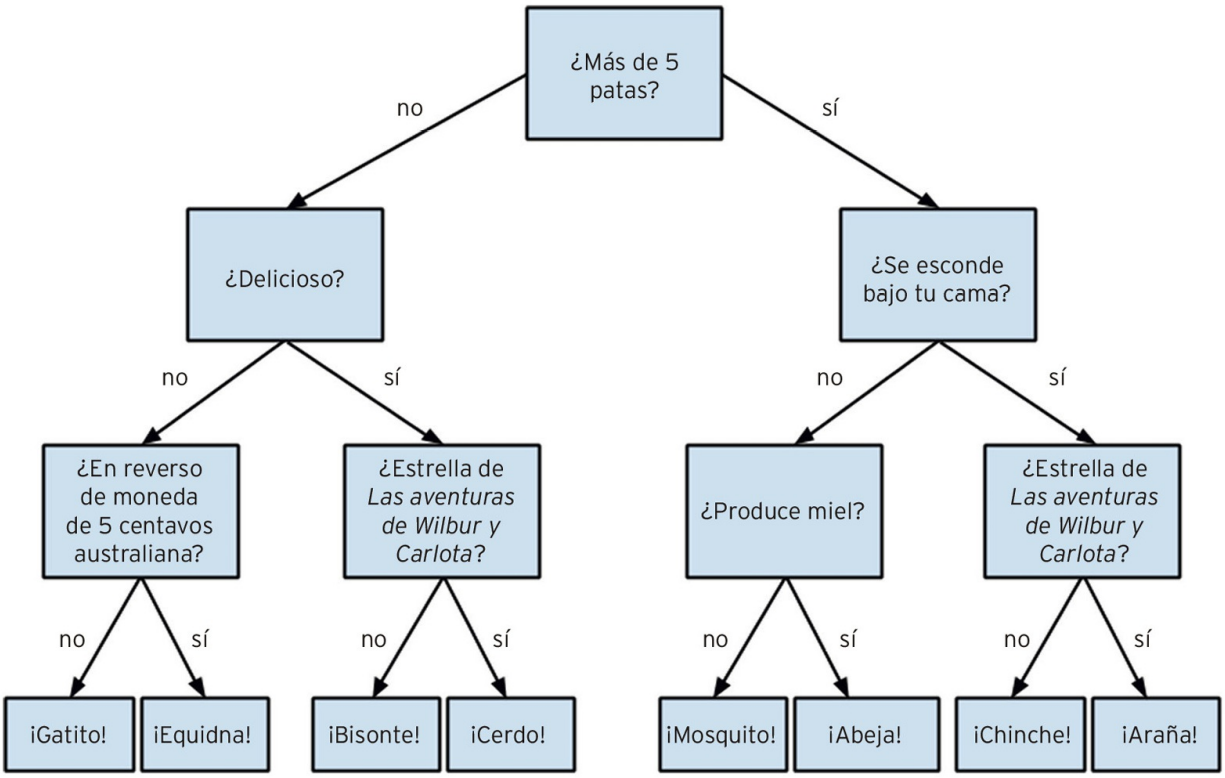


Figura 17.1. Un árbol de decisión “adivine el animal”.

Los árboles de decisión son recomendables por varias razones. Son muy fáciles de entender e interpretar, y el proceso mediante el cual alcanzan una predicción es totalmente transparente. A diferencia de los otros modelos que hemos visto hasta ahora, los árboles de decisión pueden gestionar sin problemas una mezcla de atributos numéricos (por ejemplo, número de patas) y categóricos (por ejemplo, delicioso/no delicioso) y pueden incluso

clasificar datos para los que falten atributos.

Al mismo tiempo, encontrar un árbol de decisión "óptimo" para un conjunto de datos de entrenamiento es un problema muy complicado desde el punto de vista computacional (solucionaremos esto tratando de crear un árbol bastante bueno en lugar de óptimo, aunque para conjuntos de datos grandes puede suponer mucho trabajo). Aún más importante es el hecho de que es muy fácil (y muy malo) crear árboles de decisión que estén sobreajustados a los datos de entrenamiento y que no generalicen bien con datos no visibles. Veremos formas de resolver esto.

La mayoría de la gente divide los árboles de decisión en árboles de clasificación (que producen resultados categóricos) y árboles de regresión (que producen resultados numéricos). En este capítulo, nos centraremos en los árboles de clasificación y estudiaremos el algoritmo ID3 para lograr un árbol de decisión a partir de un conjunto de datos etiquetados, lo que debería permitirnos entender cómo funcionan realmente estos algoritmos. Para simplificar las cosas, nos limitamos a problemas con resultados binarios como "¿debería contratar a este candidato?", "¿debería mostrar al visitante de este sitio web el anuncio A o el anuncio B?" o "¿me enfermaré si me como esta comida que encontré en la nevera de la oficina?".

Entropía

Para crear un árbol de decisión, necesitaremos decidir qué preguntas formular y en qué orden. En cada etapa del árbol hay algunas posibilidades que hemos eliminado y otras que no. Tras descubrir que un animal no tiene más de cinco patas, hemos eliminado la posibilidad de que sea un saltamontes. No hemos eliminado la posibilidad de que sea un pato. Cada posible pregunta divide las posibilidades restantes según su respuesta.

Lo ideal sería elegir preguntas cuyas respuestas dieran mucha información sobre lo que debería predecir nuestro árbol. Si hay una sola pregunta sí/no para la que las respuestas "sí" siempre corresponden a resultados `True` y las respuestas "no" a resultados `False` (o viceversa), sería la pregunta perfecta.

Pero, sin embargo, probablemente no sería una buena opción una pregunta sí/no para la que ninguna respuesta dé mucha información nueva sobre cómo debería ser la predicción.

Esta noción de "cantidad de información" se captura con la entropía. Es probable que haya oído ya antes este término con el significado de desorden. Aquí lo utilizamos para representar la incertidumbre asociada a los datos.

Supongamos que tenemos un conjunto S de datos, cada miembro del cual está etiquetado como perteneciente a una de un número finito de clases C_1, ..., C_n. Si todos los puntos de datos pertenecen a una sola clase, entonces no hay incertidumbre, con lo cual idealmente la entropía sería baja. Si los puntos de datos están distribuidos por igual a lo largo de las clases, sí habría mucha incertidumbre y la entropía sería alta.

En términos matemáticos, si p_i es la proporción de datos etiquetados como clase c_i, definimos la entropía como:

$$H(S) = -p_1 \log_2 p_1 - ... - p_n \log_2 p_n$$

Con el convenio (estándar) de que $0 \log 0 = 0$.

Sin preocuparnos demasiado por los detalles, cada término $-p_i \log_2 p_i$ es no negativo y está cerca de 0 precisamente cuando p_i está o bien cerca de 0 o cerca de 1 (figura 17.2).

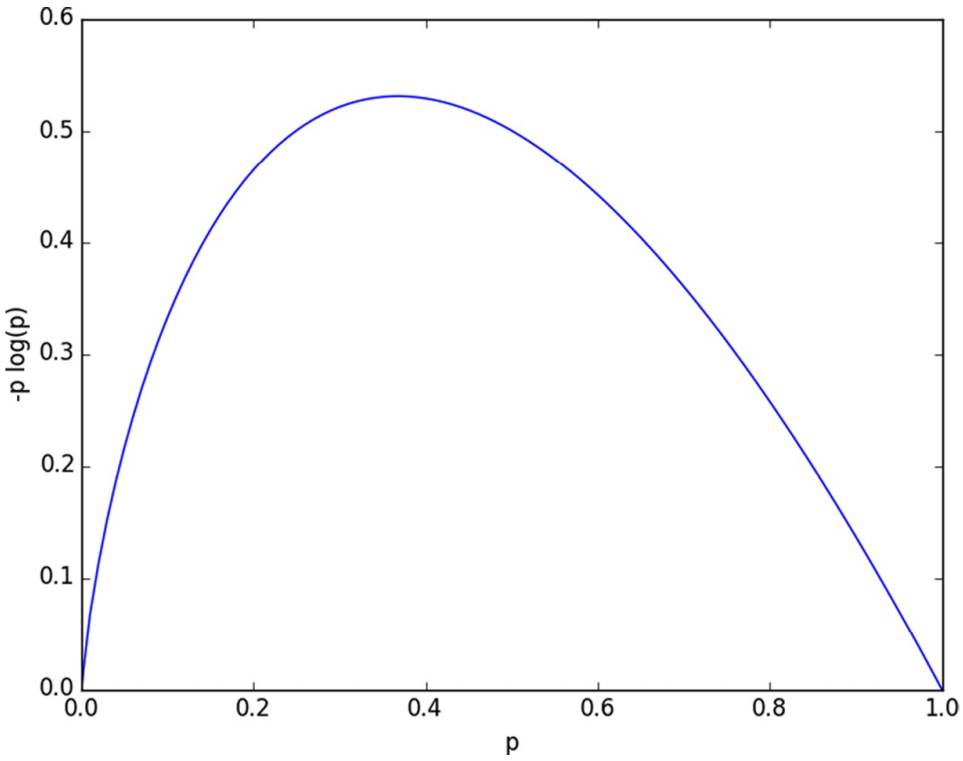


Figura 17.2. Una representación de – *p* log *p*.

Esto significa que la entropía será pequeña cuando cada p_i esté cerca de 0 o 1 (es decir, cuando la mayoría de los datos están en una sola clase), y será más grande cuando muchos de los p_i no estén cerca de 0 (es decir, cuando los datos estén repartidos a lo largo de varias clases). Este es exactamente el comportamiento que deseamos.

Es bastante sencillo desarrollar todo esto en una función:

```
from typing import List
import math
def entropy(class_probabilities: List[float]) -> float:
   """Given a list of class probabilities, compute the entropy"""
   return sum(-p * math.log(p, 2)
           for p in class_probabilities
           if p > 0)          # ignora probabilidades cero
assert entropy([1.0]) == 0
assert entropy([0.5, 0.5]) == 1
assert 0.81 < entropy([0.25, 0.75]) < 0.82
```

Nuestros datos consistirán en pares `(input, label)`, lo que significa que

tendremos que calcular nosotros mismos las probabilidades de clase. Hay que tener en cuenta que no nos preocupa realmente qué etiqueta está asociada a qué probabilidad, únicamente cuáles son las probabilidades:

```
from typing import Any
from collections import Counter
def class_probabilities(labels: List[Any]) -> List[float]:
   total_count = len(labels)
   return [count / total_count
        for count in Counter(labels).values()]
def data_entropy(labels: List[Any]) -> float:
   return entropy(class_probabilities(labels))
assert data_entropy([‘a’]) == 0
assert data_entropy([True, False]) == 1
assert data_entropy([3, 4, 4, 4]) == entropy([0.25, 0.75])
```

La entropía de una partición

Lo que hemos hecho hasta ahora es calcular la entropía (o sea, “incertidumbre”) de un conjunto único de datos etiquetados. Pero cada etapa de un árbol de decisión implica formular una pregunta cuya respuesta reparte los datos en uno o (supuestamente) varios subconjuntos. Por ejemplo, nuestra pregunta “¿tiene más de cinco patas?” divide los animales en los que tienen más de cinco patas (por ejemplo, las arañas) y los que no (por ejemplo, los equidnas).

En consecuencia, queremos tener una cierta noción de la entropía resultante de la partición de un conjunto de datos de una determinada forma. Queremos que una partición tenga entropía baja si reparte los datos en subconjuntos que tienen también entropía baja (es decir, son muy seguros), y entropía alta si contiene subconjuntos que (son grandes y) tienen asimismo entropía alta (es decir, son muy inciertos).

Por ejemplo, la pregunta de la “moneda de cinco céntimos australiana” era bastante tonta (¡aunque también bastante afortunada!), ya que dividió los animales que quedaban en ese punto en S_1 = {equidna} y S_2 = {los demás}, donde S_2 es grande y además tiene alta entropía (S_1 no tiene entropía, pero

representa una pequeña fracción de las “clases” restantes).

Matemáticamente, si dividimos nuestros datos S en subconjuntos S_1, ..., S_m conteniendo proporciones q_1, ..., q_m de los datos, calculamos entonces la entropía de la partición como una suma ponderada:

$H = q_1H(S_1) + ... + q_mH(S_m)$

Que podemos implementar como:

```
def partition_entropy(subsets: List[List[Any]]) -> float:
   """Returns the entropy from this partition of data into subsets"""
   total_count = sum(len(subset) for subset in subsets)
   return sum(data_entropy(subset) * len(subset) / total_count
           for subset in subsets)
```

Nota: Un problema con este planteamiento es que repartir según un atributo con muchos valores distintos dará como resultado una entropía muy baja debido al sobreajuste. Por ejemplo, imaginemos que trabajamos en un banco y estamos tratando de crear un árbol de decisión para predecir cuál de sus clientes es probable que no pague la hipoteca, utilizando algunos datos históricos, como el conjunto de entrenamiento de que disponemos. Vayamos aún más allá y supongamos que el conjunto de datos contiene el número de la Seguridad Social de cada cliente. Dividir según el NSS producirá subconjuntos de una sola persona, cada uno de los cuales tiene necesariamente cero entropía. Pero podemos tener la certeza de que un modelo que se base en el NSS no generaliza más allá del conjunto de entrenamiento. Por esta razón, al crear árboles de decisión deberíamos probablemente tratar de evitar (o poner en *buckets*, si corresponde) atributos con grandes cantidades de valores posibles.

Crear un árbol de decisión

El vicepresidente le ofrece los datos del entrevistado, que consisten en (según su especificación) un módulo `NamedTuple` de los atributos más importantes de cada candidato: su nivel (*level*), su lenguaje predilecto (*lang*), si es activo o no en Twitter (*tweets*), si tiene doctorado (*phd*) y si la entrevista fue positiva (*did_well*):

```
from typing import NamedTuple, Optional
class Candidate(NamedTuple):
   level: str
   lang: str
   tweets: bool
   phd: bool
   did_well: Optional[bool] = None      # permite datos sin etiquetar
             # level lang tweets phd did_well
inputs = [Candidate('Senior', 'Java', False, False, False),
        Candidate('Senior', 'Java', False, True, False),
        Candidate('Mid', 'Python', False, False, True),
        Candidate('Junior', 'Python', False, False, True),
        Candidate('Junior', 'R', True, False, True),
        Candidate('Junior', 'R', True, True, False),
        Candidate('Mid', 'R', True, True, True),
        Candidate('Senior', 'Python', False, False, False),
        Candidate('Senior', 'R', True, False, True),
        Candidate('Junior', 'Python', True, False, True),
        Candidate('Senior', 'Python', True, True, True),
        Candidate('Mid', 'Python', False, True, True),
        Candidate('Mid', 'Java', True, False, True),
        Candidate('Junior', 'Python', False, True, False)
        ]
```

Nuestro árbol consiste en nodos de decisión (que formulan una pregunta y nos dirigen de forma diferente dependiendo de la respuesta) y nodos de hoja (que nos dan una predicción). Lo crearemos utilizando el algoritmo ID3, relativamente sencillo, que funciona del siguiente modo. Digamos que nos han dado datos etiquetados y una lista de atributos para considerar las ramificaciones:

- Si los datos tienen todos la misma etiqueta, crea un nodo de hoja que predice la etiqueta y después se detiene.
- Si la lista de atributos está vacía (es decir, no hay más preguntas posibles que formular), crea un nodo de hoja que predice la etiqueta más habitual y después se detiene.
- Si no, intenta dividir los datos por cada uno de los atributos.
- Elige la bifurcación con la entropía más baja posible.
- Añade un nodo de decisión basándose en el atributo elegido.
- Vuelve a repetirlo en cada subconjunto dividido utilizando los atributos restantes.

Esto es lo que se conoce como algoritmo “voraz” porque, en cada paso, elige la mejor opción inmediata. Dado un conjunto de datos, puede haber un árbol mejor con un primer movimiento de peor aspecto. Si es así, este algoritmo no lo encontrará. No obstante, es relativamente fácil de comprender e implementar, por lo que nos ofrece un buen punto de partida para empezar a explorar los árboles de decisión.

Vayamos manualmente por cada uno de estos pasos en el conjunto de datos del entrevistado. El conjunto tiene ambas etiquetas `True` y `False`, y tenemos cuatro atributos según los cuales podemos repartir. Así que nuestro primer paso será hallar la división con la menor entropía. Empezaremos escribiendo una función que se encarga del proceso de partición:

```
from typing import Dict, TypeVar
from collections import defaultdict
T = TypeVar('T')                    # tipo genérico para entradas
def partition_by(inputs: List[T], attribute: str) -> Dict[Any, List[T]]:
   """Partition the inputs into lists based on the specified attribute."""
   partitions: Dict[Any, List[T]] = defaultdict(list)
   for input in inputs:
      key = getattr(input,                # valor del atributo especificado
      attribute)
      partitions[key].append(input)       # añade entrada a la partición
                                          correcta
   return partitions
```

Y otra que la utilice para calcular la entropía:

```
def partition_entropy_by(inputs: List[Any],
                  attribute: str,
                  label_attribute: str) -> float:
   """Compute the entropy corresponding to the given partition"""
   # partitions consiste en nuestras entradas
   partitions = partition_by(inputs, attribute)
   # pero partition_entropy solo necesita las etiquetas de clase
   labels = [[getattr(input, label_attribute) for input in partition]
           for partition in partitions.values()]
   return partition_entropy(labels)
```

Ahora solo necesitamos la partición de mínima entropía para el conjunto

de datos entero:

```
for key in ['level','lang','tweets','phd']:
   print(key, partition_entropy_by(inputs, key, 'did_well'))
assert 0.69 < partition_entropy_by(inputs, 'level', 'did_well') < 0.70
assert 0.86 < partition_entropy_by(inputs, 'lang', 'did_well') < 0.87
assert 0.78 < partition_entropy_by(inputs, 'tweets', 'did_well') < 0.79
assert 0.89 < partition_entropy_by(inputs, 'phd', 'did_well') < 0.90
```

La entropía más baja viene de dividir según `level`, de modo que nos hará falta hacer un subárbol para cada posible valor de `level`. Cada candidato `Mid` se etiqueta `True`, lo que significa que el subárbol `Mid` es simplemente un nodo de hoja que predice `True`. Para candidatos `Senior`, tenemos una mezcla de valores `True` y `False`, de modo que tenemos que dividir de nuevo:

```
senior_inputs = [input for input in inputs if input.level == 'Senior']
assert 0.4 == partition_entropy_by(senior_inputs, 'lang', 'did_well')
assert 0.0 == partition_entropy_by(senior_inputs, 'tweets', 'did_well')
assert 0.95 < partition_entropy_by(senior_inputs, 'phd', 'did_well') < 0.96
```

Esto nos demuestra que nuestra siguiente división debería realizarse según `tweets`, lo que da como resultado una partición con entropía cero. Para esos candidatos de nivel `Senior`, un "sí" en el atributo de los tuits siempre da como resultado `True`, mientras que un "no" siempre da como resultado `False`.

Por último, si hacemos lo mismo para los candidatos `Junior`, terminamos repartiendo según `phd`, tras de lo cual descubrimos que no tener doctorado siempre da como resultado `True` y tener doctorado siempre da `False`.

La figura 17.3 muestra el árbol de decisión completo.

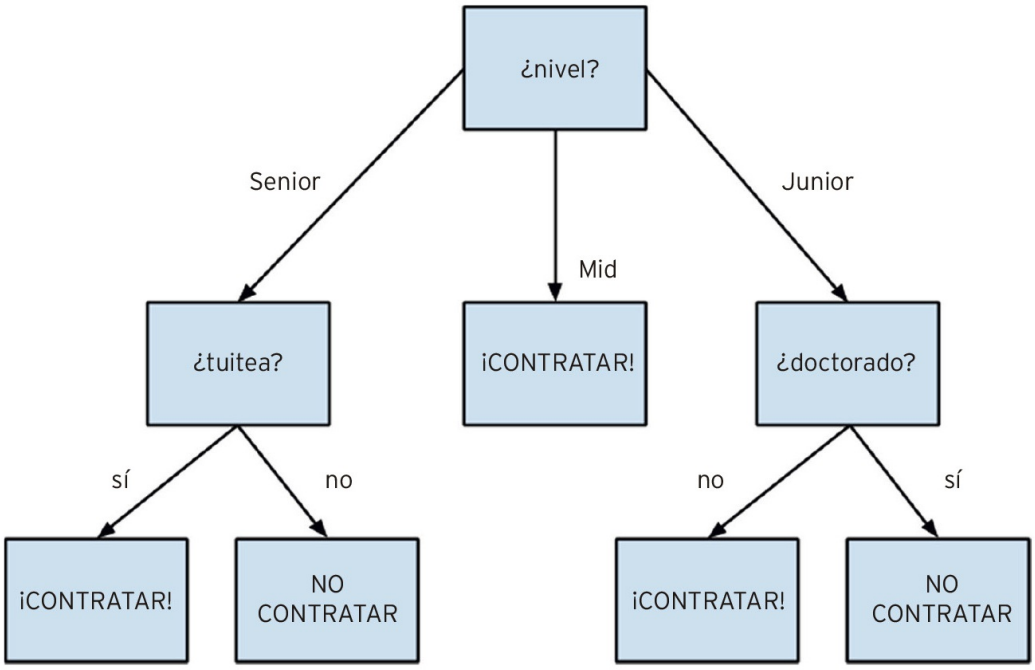


Figura 17.3. El árbol de decisión para contrataciones.

Ahora, a combinarlo todo

Ahora que ya hemos visto cómo funciona el algoritmo, nos gustaría implementarlo más en general, lo que significa que tenemos que decidir cómo queremos representar los árboles. Utilizaremos la representación más liviana posible. Definimos un árbol de dos maneras:

- Un `Leaf` (que predice un solo valor).
- Un `Split` (que contiene un atributo según el que dividir, subárboles para valores determinados de ese atributo y posiblemente un valor predeterminado que utilizar si encontramos un valor desconocido).

```
from typing import NamedTuple, Union, Any
class Leaf(NamedTuple):
   value: Any
class Split(NamedTuple):
   attribute: str
   subtrees: dict
```

```
  default_value: Any = None
DecisionTree = Union[Leaf, Split]
```

Con esta representación, nuestro árbol de contrataciones tendría este aspecto:

```
hiring_tree = Split('level', {      # primero, considera "level"
  'Junior': Split('phd', {          # si el nivel es "Junior", mira "phd"
    False: Leaf(True),              # si "phd" es False, predice True
    True: Leaf(False)               # si "phd" es True, predice False
  }),
  'Mid': Leaf(True),              # si el nivel es "Mid", predice True
  'Senior': Split('tweets', {       # si el nivel es "Senior", mira "tweets"
    False: Leaf(False),             # si "tweets" es False, predice False
    True: Leaf(True)                # si "tweets" es True, predice True
  })
})
```

Queda pendiente la cuestión de qué hacer si encontramos un valor de atributo inesperado (o faltante). ¿Qué debe hacer nuestro árbol de contrataciones si encuentra un candidato cuyo `level` es `Intern`? Gestionaremos este caso asignando al atributo `default_value` la etiqueta más común.

Dada una representación como esta, podemos clasificar una entrada con:

```
def classify(tree: DecisionTree, input: Any) -> Any:
"""classify the input using the given decision tree"""
  # Si es un nodo de hoja, devuelve su valor
  if isinstance(tree, Leaf):
    return tree.value
  # Si no este árbol consiste en un atributo según el que dividir
  # y un diccionario cuyas claves son valores de ese atributo
  # y cuyos valores son subárboles que considerar después
  subtree_key = getattr(input, tree.attribute)
  if subtree_key not in                  # Si no hay subárbol para clave,
  tree.subtrees:
    return tree.default_value            # devuelve el valor predeterminado.
  subtree =                              # Elige el subárbol adecuado
  tree.subtrees[subtree_key]
  return classify(subtree, input)        # y lo usa para clasificar la
                                         entrada.
```

Todo lo que queda por hacer es crear la representación del árbol a partir de nuestros datos de entrenamiento:

```
def build_tree_id3(inputs: List[Any],
             split_attributes: List[str],
             target_attribute: str) -> DecisionTree:
     # Cuenta etiquetas destino
     label_counts = Counter(getattr(input, target_attribute)
for input in inputs)
most_common_label = label_counts.most_common(1)[0][0]
   # Si hay una etiqueta única, la predice
   if len(label_counts) == 1:
      return Leaf(most_common_label)
   # Si no quedan atributos de división, devuelve la etiqueta mayoritaria
   if not split_attributes:
      return Leaf(most_common_label)
   # Si no divide por el mejor atributo
   def split_entropy(attribute: str) -> float:
      """Helper function for finding the best attribute"""
      return partition_entropy_by(inputs, attribute, target_attribute)
   best_attribute = min(split_attributes, key=split_entropy)
   partitions = partition_by(inputs, best_attribute)
   new_attributes = [a for a in split_attributes if a != best_attribute]
   # Crea recursivamente los subárboles
   subtrees = {attribute_value : build_tree_id3(subset,
 new_attributes,
 target_attribute)
           for attribute_value, subset in partitions.items()}
   return Split(best_attribute, subtrees, default_value=most_common_label)
```

En el árbol que creamos, cada hoja estaba enteramente formada por entradas `True` o por entradas `False`. Esto significa que el árbol predice perfectamente según el conjunto de datos de entrenamiento. Pero también podemos aplicarlo a nuevos datos que no estuvieran en el conjunto de entrenamiento:

```
tree = build_tree_id3(inputs,
                ['level', 'lang', 'tweets', 'phd'],
                'did_well')
# Debe predecir True
assert classify(tree, Candidate("Junior", "Java", True, False))
# Debe predecir False
```

```
assert not classify(tree, Candidate("Junior", "Java", True, True))
```

Y también a datos con valores inesperados:

```
# Debe predecir True
assert classify(tree, Candidate("Intern", "Java", True, True))
```

Nota: Como nuestro objetivo era principalmente mostrar cómo crear un árbol, lo construimos utilizando el conjunto de datos entero. Como siempre, si estuviéramos tratando realmente de crear un buen modelo para algo, habríamos recogido más datos y los habríamos dividido en subconjuntos de entrenamiento/validación/prueba.

Bosques aleatorios

Teniendo en cuenta lo mucho que pueden los árboles de decisión ajustarse a sus datos de entrenamiento, no resulta sorprendente que tengan tendencia a sobreajustar. Una forma de evitar esto es una técnica llamada bosques aleatorios, en la que creamos varios árboles de decisión y combinamos sus resultados. Si son árboles de clasificación, podríamos dejarlos votar; si son de regresión, podríamos promediar sus predicciones.

Nuestro proceso de creación de árboles era determinista, de modo que ¿cómo obtenemos árboles aleatorios?

Una parte del proceso implica aplicar *bootstrap* a los datos (recordemos la sección sobre *bootstrap* en el capítulo 15). En lugar de entrenar cada árbol en todas las entradas del conjunto de entrenamiento, lo entrenamos en el resultado de `bootstrap_sample (inputs)`. Como cada árbol se ha creado usando distintos datos, cada uno será diferente de los demás (un beneficio secundario es que es totalmente justo utilizar los datos no muestreados para probar cada árbol, lo que significa que uno se puede salir con la suya utilizando todos los datos como conjunto de entrenamiento si se es lo bastante listo midiendo el rendimiento). Esta técnica se conoce como agregación o empaquetado de *bootstrap*.

Una segunda fuente de aleatoriedad implica cambiar la forma que tenemos

de elegir el `best_attribute` según el cual dividir. En lugar de mirar todos los atributos restantes, primero elegimos un subconjunto aleatorio de ellos y después repartimos según el de ellos que sea mejor:

```
  # si ya hay suficientes candidatos divididos, los mira todos
  if len(split_candidates) <= self.num_split_candidates:
     sampled_split_candidates = split_candidates
  # si no elige una muestra aleatoria
  else:
     sampled_split_candidates = random.sample(split_candidates,
self.num_split_candidates)
  # ahora elige el mejor atributo solo de esos candidatos
  best_attribute = min(sampled_split_candidates, key=split_entropy)
  partitions = partition_by(inputs, best_attribute)
```

Este es un ejemplo de una técnica más amplia llamada aprendizaje combinado o *ensemble*, en la que combinamos varios estudiantes débiles (normalmente modelos de alto sesgo y baja varianza) para producir un modelo global fuerte.

Para saber más

- scikit-learn tiene muchos modelos de árbol de decisión, en https://scikit-learn.org/stable/modules/tree.html. También tiene un módulo `ensemble` en https://scikit-learn.org/stable/modules/classes.html#module-sklearn.ensemble, que incluye un `RandomForestClassifier`, además de otros métodos de aprendizaje combinado.
- XGBoost, en https://xgboost.ai/, es una librería para entrenar árboles de decisión con potenciación del gradiente que tiende a ganar muchas competiciones de *machine learning* de estilo Kaggle.
- Apenas hemos arañado la superficie de los árboles de decisión y sus algoritmos. La Wikipedia, en https://es.wikipedia.org/wiki/Aprendizaje_basado_en_%C3%A1rb es un buen punto de partida para un estudio más detallado.

[1] https://en.wikipedia.org/wiki/Twenty_questions.

18 Redes neuronales

Me gusta el sinsentido; despierta las células del cerebro.

—Dr. Seuss

Una red neuronal artificial (o red neuronal sin más) es un modelo predictivo motivado por el modo en que funciona el cerebro. Piense en el cerebro como en una colección de neuronas conectadas entre ellas. Cada neurona mira las salidas de las otras neuronas que la alimentan, hace un cálculo y después se activa (si el cálculo excede un cierto umbral) o no (si no lo excede).

Según esta explicación, las redes neuronales artificiales están formadas por neuronas artificiales, que realizan cálculos similares sobre sus entradas. Las redes neuronales pueden resolver distintos problemas, como reconocimiento de escritura y detección de caras, y se utilizan mucho en el *deep learning*, uno de los subcampos más recientes de la ciencia de datos. Sin embargo, la mayoría de las redes neuronales son "cajas negras" (es decir, inspeccionar sus detalles no permite entender mucho mejor cómo resuelven un problema). Además, pueden ser difíciles de entrenar. Para resolver la mayor parte de los problemas que uno se suele encontrar como científico de datos en ciernes, probablemente no son la mejor opción. Pero, si algún día el objetivo es construir una inteligencia artificial para crear la singularidad, entonces sí podrían ser de utilidad.

Perceptrones

La red neuronal más sencilla de todas es el perceptrón, que se aproxima a una sola neurona con *n* entradas binarias. Calcula una suma ponderada de sus entradas y se "activa" si esa suma es 0 o mayor que 0:

```
from scratch.linear_algebra import Vector, dot
```

```
def step_function(x: float) -> float:
   return 1.0 if x >= 0 else 0.0
def perceptron_output(weights: Vector, bias: float, x: Vector) -> float:
   """Returns 1 if the perceptron 'fires', 0 if not"""
   calculation = dot(weights, x) + bias
   return step_function(calculation)
```

El perceptrón simplemente distingue entre los semiespacios separados por el hiperplano de puntos `x`, para los cuales:

```
dot(weights, x) + bias == 0
```

Con pesos adecuadamente elegidos, los perceptrones pueden resolver unos cuantos problemas sencillos (véase la figura 18.1). Por ejemplo, podemos crear una puerta AND, que devuelve 1 si sus dos entradas son 1 y 0 si una de sus entradas es 0, utilizando:

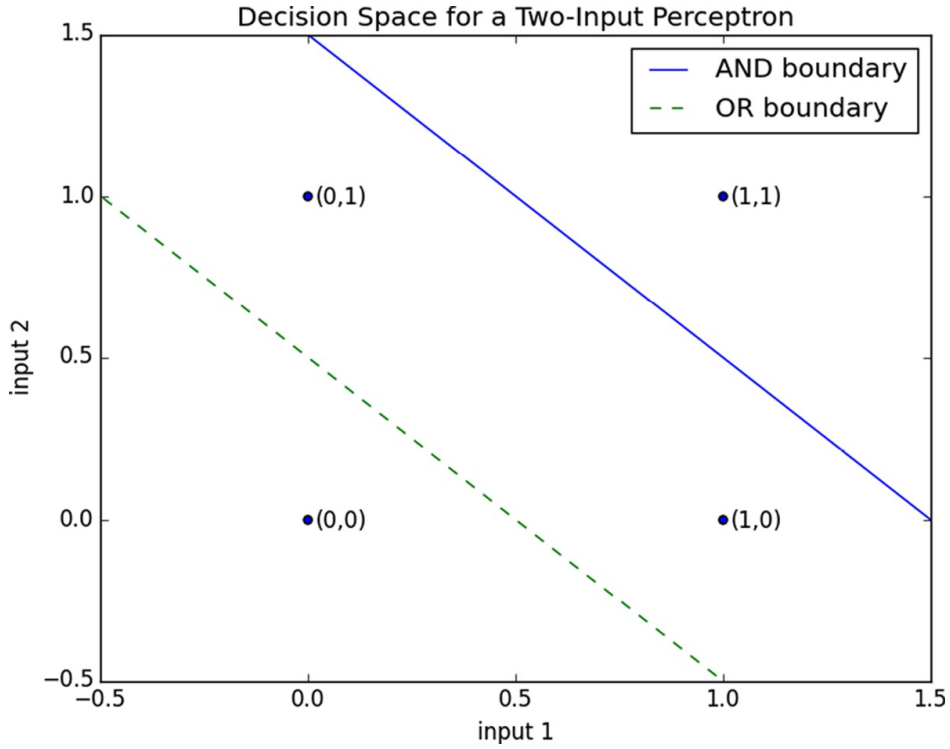


Figura 18.1. Espacio de decisión para un perceptrón de dos entradas.

```
and_weights = [2., 2]
and_bias = -3.
assert perceptron_output(and_weights, and_bias, [1, 1]) == 1
assert perceptron_output(and_weights, and_bias, [0, 1]) == 0
```

```
assert perceptron_output(and_weights, and_bias, [1, 0]) == 0
assert perceptron_output(and_weights, and_bias, [0, 0]) == 0
```

Si ambas entradas son 1, `calculation` es igual a 2 + 2 - 3 = 1, y el resultado es 1. Si solo una de las entradas es 1, `calculation` es igual a 2 + 0 - 3 = -1, y el resultado es 0. Pero, si ambas entradas son 0, `calculation` es igual a -3 y el resultado es 0.

Utilizando un razonamiento parecido, podríamos crear una puerta OR con este código:

```
or_weights = [2., 2]
or_bias = -1.
assert perceptron_output(or_weights, or_bias, [1, 1]) == 1
assert perceptron_output(or_weights, or_bias, [0, 1]) == 1
assert perceptron_output(or_weights, or_bias, [1, 0]) == 1
assert perceptron_output(or_weights, or_bias, [0, 0]) == 0
```

También podríamos crear una puerta NOT (que tiene una sola entrada y convierte 1 en 0 y 0 en 1) con:

```
not_weights = [-2.]
not_bias = 1.
assert perceptron_output(not_weights, not_bias, [0]) == 1
assert perceptron_output(not_weights, not_bias, [1]) == 0
```

Sin embargo, hay algunos problemas que simplemente no se pueden resolver con un solo perceptrón. Por ejemplo, por mucho que se intente, no se puede usar un perceptrón para crear una puerta XOR que dé como resultado 1 si exactamente una de sus entradas es 1 y 0 si no lo es. Aquí es donde empezamos a necesitar redes neuronales más complicadas.

Por supuesto, no hace falta aproximarse a una neurona para poder crear una puerta lógica:

```
and_gate = min
or_gate = max
xor_gate = lambda x, y: 0 if x == y else 1
```

Como ocurre con las neuronas de verdad, las artificiales empiezan a resultar más interesantes cuando se las empieza a conectar unas con otras.

Redes neuronales prealimentadas

La topología del cerebro es enormemente complicada, de ahí que sea habitual aproximarse a ella con una red neuronal prealimentada teórica, formada por capas discretas de neuronas, cada una de ellas conectada con la siguiente. Normalmente esto conlleva una capa de entrada (que recibe entradas y las transmite sin cambios), una o varias "capas ocultas" (cada una de las cuales consiste en neuronas que toman las salidas de la capa anterior, realizan algún tipo de cálculo y pasan el resultado a la siguiente capa) y una capa de salida (que produce los resultados finales).

Exactamente igual que en el perceptrón, cada neurona (no de entrada) tiene un peso correspondiente a cada una de sus entradas y un sesgo. Para que nuestra representación sea más sencilla, añadiremos el sesgo al final de nuestro vector de pesos y daremos a cada neurona una entrada de sesgo que siempre es igual a 1.

Igual que con el perceptrón, para cada neurona sumaremos los productos de sus entradas y sus pesos. Pero aquí, en lugar de dar como resultado `step_function` aplicado a dicho producto, obtendremos una aproximación suave de él. Lo que emplearemos es la función `sigmoid` (figura 18.2):

```
import math
def sigmoid(t: float) -> float:
   return 1 / (1 + math.exp(-t))
```